Decentralized Crypto World Bank

A Blockchain-Based Banking Platform
with AI-Enhanced Security and Assistance

by

Md. Bokhtiar Rahman Juboraz

20301138

Md. Mahir Ahnaf Ahmed

20301083

A pre-thesis 1 report submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science

Department of Computer Science and Engineering
BRAC University
February 2026

© 2026. BRAC University — All rights reserved.

Declaration

It is hereby declared that

1. The report submitted is our own original work while completing degree at BRAC University.
2. The report does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The report does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md. Bokhtiar Rahman Juboraz	Md. Mahir Ahnaf Ahmed
20301138	20301083

Approval

The pre-thesis 1 report of final year project titled "Decentralized Crypto World Bank: A Blockchain-Based Banking Platform with AI-Enhanced Security and Assistance" submitted by

1. Md. Bokhtiar Rahman Juboraz (20301138)
2. Md. Mahir Ahnaf Ahmed (20301083)

Of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on February 20, 2026.

Examining Committee:

Supervisor:
(Member)

Mr. Annajiat Alim Rasel
Senior Lecturer
Department of Computer Science and Engineering
BRAC University

Project Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
BRAC University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi
Chairperson and Associate Professor
Department of Computer Science and Engineering
BRAC University

Ethics Statement

This project operates exclusively on public blockchain test networks using test tokens with no real monetary value. No real financial transactions, personal banking data, or user financial records are involved at any stage of development or evaluation. All transaction data used for Artificial Intelligence and Machine Learning (AI/ML) model training and evaluation is either synthetically generated or sourced from publicly available anonymized datasets. The platform prototype does not process Know Your Customer (KYC) or Anti-Money Laundering (AML) data in its current scope, and all wallet addresses used during testing are disposable testnet addresses. We acknowledge the ethical considerations inherent in AI-assisted financial decision-making and have incorporated explainability mechanisms (SHapley Additive exPlanations, or SHAP) to ensure that automated risk assessments remain transparent and auditable by human reviewers. Future phases plan to incorporate a privacy-preserving ZKP-based identity compliance layer in which users prove KYC status without exposing personal data on-chain; the ethical implications of this design will be evaluated in the final thesis.

Abstract

Global development finance relies on multilayered institutional structures to distribute capital across borders and communities, yet these systems often struggle with fragmented transparency, procedural friction, and uneven access to credit. Although decentralized finance has demonstrated that programmable infrastructures can automate lending and enhance auditability, existing models largely employ flat architectures that do not reflect institutional hierarchies or structured governance. As a result, there remains limited exploration of how tiered financial systems might be represented within decentralized environments while preserving oversight and adaptability. Here we present *Crypto World Bank*, a formally specified, partially implemented research prototype that demonstrates the *feasibility* of a blockchain-based cryptocurrency exchange and institutional finance platform coordinating capital allocation across a four-tier hierarchical governance structure. We show that hierarchical capital flows, role-based governance, and data-informed risk analytics can be coordinated within a unified smart contract environment, enabling transparent state visibility alongside adaptive decision support. The Crypto World Bank formally specifies and partially implements a banking architecture spanning six functional domains—deposit mobilization, credit allocation, payment settlement, risk intermediation, liquidity management, and ancillary financial services—within a four-tier hierarchical governance structure. The prototype implements the core Tier 1 reserve and capital-allocation contracts, with Tiers 2–4 and the extended product suite formally specified for implementation in the final thesis phase. This hybrid design illustrates how institutional finance and decentralized systems may converge, contributing an open-source, hierarchically governed crypto financial services architecture as a foundation for further research.

Keywords: Blockchain, Decentralized Finance, Institutional Architecture, Financial Inclusion, Smart Contracts, AI-Augmented Governance, Group Lending, Deposit Mobilization, Reserve Management, Financial Sustainability

Dedication

Dedicated to our families for their unwavering support throughout our academic journey.

Acknowledgment

We would like to express our sincere gratitude to our panel members for their guidance and support throughout this project. We also thank our supervisor, Mr. Annajiat Alim Rasel, Senior Lecturer at the Department of Computer Science and Engineering, BRAC University, for his guidance and support. Finally, we thank the Department of Computer Science and Engineering at BRAC University for providing us with the resources and academic environment to pursue this work.

Contents

List of Tables

List of Figures

List of Formulas

Utilization Rate (U): $U = \frac{L}{L+B}$ (lending utilization; L is lent amount, B is available balance/liquidity)

Collateral Ratio (CR): $CR = \frac{C}{D}$ (collateral-to-debt ratio)

Loan-to-Value (LTV): $LTV = \frac{\text{Loan Amount}}{\text{Collateral Value}}$

APR (simple): $APR = r_{\text{period}} \times N$ (annualized simple rate over N periods)

Compound Interest (Savings): $A = P\left(1 + \frac{r}{n}\right)^{nt}$ (deposit growth; P principal, r annual rate, n compounding frequency)

EMI (Installment Payment): $EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$ (installment for principal P, periodic rate r, n installments)

Health Factor (HF): $HF = \frac{C \times LT}{D}$ (liquidation trigger for *over-collateralized retail loans only*; if $HF < 1.0$ liquidation is triggered, where LT is liquidation threshold; see Section 3.17 for tier-specific HF variants and the group-loan pool-level formulation)

Reserve Ratio (RR): $RR = \frac{\text{Reserves}}{\text{Total Deposits}}$ (solvency constraint enforced at each tier; governs the maximum downward capital that may be allocated, *not* a money-creation parameter)

Platform Solvency Ratio (PSR): $PSR = \frac{\text{TotalReserveBalance}}{\text{TotalOutstandingLoans}}$ (on-chain analogue of capital adequacy, measuring whether the system's reserve cover exceeds outstanding loan obligations; replaces the Basel III CAR formula, which is undefined for this platform because USDC assets have no regulatory risk-weight classification under Basel III and "Tier 1 Capital" in the Basel sense cannot be computed from an on-chain USDC balance alone)

Credit Velocity (CV): $CV = \frac{\text{Capital Recycled per Period}}{\text{Total Reserve}}$ (rate at which reserve capital flows through the lending hierarchy and returns as repayment; replaces the inapplicable fractional-reserve money multiplier, which presupposes new money creation and does not apply to fixed-supply USDC)

FX Conversion: $\text{Amount}_B = \text{Amount}_A \times \left(\frac{P_A}{P_B}\right)$ (oracle-priced conversion between assets A and B)

Group Loan Individual Share: $\text{Share}_i = \frac{\text{LoanTotal}}{N_{\text{members}}}$

On-Chain Credit Score (conceptual): $CS = \sum_i w_i \cdot \text{feature}_i$ (weighted features learned off-chain and stored as a score on-chain)

Isolation Forest Anomaly Score: $s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$ (anomaly scoring for instance x; where $h(x)$ is the path length of instance x from the root of an isolation tree and $c(n)$ is the average path length for n samples)

Random Forest Fraud Probability: $P(\text{fraud} \mid x) = \frac{1}{T}\sum_{t=1}^{T} f_t(x)$ (ensemble average over T trees; where $f_t(x)$ is the binary fraud prediction of the t-th decision tree)

Platform Net Interest Allocation: NetInterest $=$ InterestCollected $-$ DepositorYield $-$ InsuranceAllocation

SHAP (Shapley value): $\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \left[f(S \cup \{i\}) - f(S) \right]$

List of Abbreviations

AA: Account Abstraction (ERC-4337)

ALM: Asset-Liability Management

AI: Artificial Intelligence

AI/ML: Artificial Intelligence / Machine Learning

AML: Anti-Money Laundering

APR: Annual Percentage Rate

API: Application Programming Interface

BRAC: Bangladesh Rural Advancement Committee

CAR: Capital Adequacy Ratio (Basel III; not directly applicable to this platform — see Platform Solvency Ratio, PSR, in List of Formulas)

CBDC: Central Bank Digital Currency

CCIP: Cross-Chain Interoperability Protocol (Chainlink)

CCS: ACM Computing Classification System

CR: Collateral Ratio

CVL: Certora Verification Language

DeFi: Decentralized Finance

DID: Decentralized Identifier (W3C)

DON: Decentralised Oracle Network (Chainlink)

EIP: Ethereum Improvement Proposal

EIP-7702: Ethereum Improvement Proposal 7702 — Session Key standard enabling scoped, time-bound wallet authorisations for an autonomous agent

EMI: Equated Monthly Installment

ERC: Ethereum Request for Comments (token / interface standard)

EVM: Ethereum Virtual Machine

FL: Federated Learning

FATF: Financial Action Task Force

FX: Foreign Exchange

GNN: Graph Neural Network

HF: Health Factor

KYC: Know Your Customer

LLM: Large Language Model

L2: Layer 2

LTV: Loan-to-Value Ratio

MFI: Microfinance Institution

MiCA: Markets in Crypto-Assets Regulation (EU)

ML: Machine Learning

MCP: Model Context Protocol — tool-calling interface used by the AI agent to interact with CWB banking operations

NIM: Net Interest Margin

PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses

PoS: Proof of Stake

PoR: Proof of Reserve (Chainlink on-chain reserve verification)

PSR: Platform Solvency Ratio (on-chain capital adequacy analogue; see List of Formulas)

QRC: QR code

RBAC: Role-Based Access Control

RLS: Row-Level Security (PostgreSQL policy feature enforcing append-only access on audit tables)

RR: Reserve Ratio

SAR: Suspicious Activity Report

SBT: Soulbound Token (non-transferable ERC standard)

SHAP: SHapley Additive exPlanations

SOFR: Secured Overnight Financing Rate

SSE: Server-Sent Events

SSI: Self-Sovereign Identity

SoK: Systematization of Knowledge

SWC: Smart Contract Weakness Classification

TVL: Total Value Locked

U: Utilization

UUPS: Universal Upgradeable Proxy Standard (ERC-1822)

VC: Verifiable Credential (W3C)

XAI: Explainable Artificial Intelligence

ZKP: Zero-Knowledge Proof

zkAML: Zero-Knowledge Anti-Money-Laundering proof

zkEVM: Zero-Knowledge Ethereum Virtual Machine — a ZK rollup that inherits Ethereum L1 security via cryptographic validity proofs

zk-SNARK: Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

Chapter 1

Introduction

The Crypto World Bank is not a lending protocol. It is a **formally specified, partially implemented** research prototype demonstrating the feasibility of a blockchain-based cryptocurrency exchange and institutional finance platform on programmable blockchain infrastructure. Where existing decentralized finance protocols address isolated financial functions—Aave handles collateralized lending, Uniswap handles exchange, MakerDAO issues stablecoins—the Crypto World Bank formally specifies and partially implements a hierarchically governed crypto financial services platform spanning the full spectrum of banking functions. These functions include deposit mobilization, credit allocation, installment-based repayment, interbank liquidity management, foreign exchange facilitation, solidarity group lending for retail borrowers, syndicated co-lending and co-funding by groups of institutional entities, reserve enforcement, and risk-based governance—coordinated across four institutional tiers that mirror the structural logic of real-world development finance. A key architectural capability is the ability for **multiple entities to lend or fund together**: institutional banks at the same tier can co-fund a single loan through the `SyndicatedLoan` contract (Section 3.13.3), retail clients can pool resources through the solidarity `GroupLendingPool` (Section **??**), and banks can deposit surplus capital upward through the `UpwardDepositFacility` (Section 3.13.2)—providing a multi-directional, multi-party capital coordination capability absent from all existing DeFi protocols. The Tier 1 World Bank Reserve and the core lending hierarchy are implemented and deployed on testnet; the extended product suite (SavingsVault, GroupLendingPool, IBLP, and NettingEngine) is formally specified and scheduled for the final thesis implementation phase.

The system is designed to serve three categories of participants simultaneously. At the global level, it functions as a programmable central reserve institution—managing capital allocation across national jurisdictions with transparent, auditable reserve ratios enforced by smart contract code rather than periodic audit. At the national and regional level, it functions as a commercial and development bank—channeling capital to local lending institutions, managing interbank liquidity, and providing institutional clients with structured credit facilities. At the individual level, it functions as a retail bank—offering savings products, group loans, installment-based personal lending, and digital payment services to end customers, including the estimated 1.4 billion adults currently excluded from formal financial systems [14]. The complete source code, smart contracts, and supporting material for this project are available in our GitHub repository.

Blockchain functions here as a coordination technology, providing visibility of shared states, audit trails that are difficult to alter, and programmable enforcement of rules among institutions that may have competing interests. Governance design, security controls, and regulatory considerations are incorporated into the platform according to a staged implementation roadmap that includes academic validation and regulatory sandbox pilots. Analogous hybrid institutional structures in blockchain-based finance include the World Bank's

FundsChain programme, which tracks fund disbursement across projects on a permissioned ledger, and JPMorgan Kinexys, which settles multi-billion-dollar institutional payments daily on a private EVM-compatible chain—demonstrating that open ledger infrastructure and institutional hierarchy are not mutually exclusive.

1.1 Background

Understanding why such a system is necessary requires examining the structural failures of the institutions it is designed to complement.

Capital is distributed to local borrowers through layered institutional arrangements in which supranational development institutions such as the World Bank and IMF, national financial intermediaries, and local lenders collaborate to channel development finance to end borrowers. Although this model allows risk sharing and penetrating local markets, it is linked to great operational issues. In cross-border correspondent banking, financial institutions are obligated to have pre-funded nostro and vostro accounts at correspondent banks in every currency corridor, which maroon huge amounts of capital in idle balances unable to be invested or lent out [24]. Crossborder transactions are regularly settled in two to five business days with an average cost of about $42 per transaction by using the correspondent banking network [25]. The remittance market, which is valued at about $860 billion annually, loses between $48–56 billion per annum to transfer charges, with the average cost to transfer in the world being 6.49% as of 2024, more than the United Nations Sustainable Development Goal limit of 3% [26].

At the same time, it is estimated that there are 1.4 billion unbanked adults in the world as defined by the World Bank Global Findex [14], and most of them are found in the developing economies where documentation, geographic location, and minimum balance requirements leave out high populations of people in the formal financial sector. The International Finance Corporation estimates that there is a financing gap in the world of micro, small and medium enterprises at $4.5 trillion per year [20], which is unmet credit demand that cannot be effectively met by the existing institutions through the conventional intermediation chains.

The DeFi industry has already shown that lending, collateral management, and interest calculation can be automated on a smart contract platform and that all transactions are transparent, on-chain. The largest DeFi lending protocol, Aave v3, has amassed $26.3 billion in total value locked across ten blockchain networks [27] (accessed March 2026), and the DeFi lending market has more than $55 billion in TVL [13] (accessed March 2026). Nevertheless, these protocols utilize flat, pool-based designs where all the lenders feed into one pool and all borrowers withdraw off the same source—irrespective of institutional status, creditworthiness, and geography. There is no current DeFi scheme that reflects the multi-tier institutional structure, inter-tier capitals movement and tiered borrower access provision that is typical of real-world development finance.

What smart contracts and blockchain actually provide. A **smart contract** is a program deployed to a blockchain's state machine—on the Ethereum Virtual Machine (EVM), this means a deterministic function executed by every validating node simultaneously. Its key properties distinguish it sharply from conventional software: (1) **Immutable execution**: once deployed, the logic cannot be changed by any single party without a

governance-controlled upgrade; a lending rule encoded in a smart contract is enforced exactly as written, regardless of what the operating institution would prefer in a given moment. (2) **Atomic settlement**: a multi-step operation—such as releasing a loan, recording a repayment, and updating a reserve ratio—either completes entirely or reverts entirely; there is no half-settled state. (3) **Trustless enforcement**: the contract enforces its rules against *any* caller, including the institution that deployed it; no administrator can unilaterally override a reserve-ratio check or bypass a confirmation gate. (4) **Transparent audit**: every function call, parameter, and state change is permanently recorded on the chain and queryable by any participant.

Blockchain itself adds the coordination layer: it is a replicated, cryptographically linked log that produces *shared state* among participants who may not trust each other. In a correspondent banking network, each institution maintains its own private ledger and reconciles with counterparties through batch messaging and manual reconciliation—the source of the 2–5 day settlement delays and the $42 per-transaction cost [25]. A shared blockchain ledger replaces this bilateral reconciliation with a single authoritative record: when a Local Bank disburses a loan, the World Bank Reserve's reserve ratio is updated in the same transaction, with no inter-institution messaging required. This is not merely a database with replication; it is a *coordination mechanism* that removes the need for trusted intermediaries between institutions with potentially conflicting interests.

Decentralized Finance (DeFi) is the application of smart contracts to financial services, eliminating the need for a licensed intermediary to hold assets in custody, process payments, or enforce loan terms. Aave and Compound demonstrate that the core banking function of deposit-funded lending can operate without a bank: depositors supply assets to a pool contract, borrowers draw from the pool, and interest accrues automatically per a utilization-curve formula. The Crypto World Bank extends this pattern in three directions not explored by any existing DeFi protocol. First, it introduces *institutional hierarchy*: rather than a single flat pool, capital flows through a four-tier structure (World Bank $\rightarrow$ National Bank $\rightarrow$ Local Bank $\rightarrow$ Client) in which each tier's reserve ratio, governance rights, and borrowing limits are distinct and enforced on-chain. Second, it introduces *multi-entity operations*: groups of entities—both retail borrowers and institutional banks—can co-fund and co-lend through programmable solidarity group loans, syndicated loans, tranched pools, and the interbank lending pool, none of which exist in any current DeFi protocol. Third, it connects DeFi's programmability to the real-world institutional framework of development finance—a combination that the academic literature has identified as structurally absent from the existing DeFi landscape [1].

Autonomous banking agents and the human-gate model. A further dimension of this gap concerns the interface through which retail clients in underserved economies interact with banking services. The Crypto World Bank extends beyond the role of a read-only LLM product guide to incorporate a locally hosted, privacy-preserving autonomous AI agent capable of both answering client questions and executing on-chain banking operations on their behalf. The agent is built on a Model Context Protocol (MCP) tool server exposing 17 banking tools (9 read tools, 8 write tools) and operates under a mandatory human confirmation gate: every state-modifying operation—loan application, installment payment, deposit, or KYC upgrade—requires explicit affirmative consent from the client before the agent calls the corresponding write tool and signs the resulting on-chain transaction via an EIP-7702 scoped session key. This architecture provides the conversational

accessibility of a virtual banker while preserving the client's full sovereignty over every financial action. The agent is an optional convenience layer that sits alongside the standard form-based React UI: every banking operation it can perform is equally available through the web interface, and clients who prefer direct interaction engage the platform without the agent. The human confirmation gate is enforced by a confirmation audit hook in the Express.js middleware layer—application-layer policy enforcement that operates independently of model output: any write-tool POST request without a logged confirmation turn in the conversation history is rejected with HTTP 403, regardless of what the model generated.

1.2 Rationale of the Study

The combination of three forces, namely: (1) the long-standing inefficiencies in the traditional development finance, which restrict the transparency and speed of settlement; (2) the absence of multi-tier banking functions in existing DeFi designs, particularly deposit mobilization and solidarity group lending alongside hierarchical lending; (3) the possibility of integrating blockchain auditability with lightweight analytics and monitoring support to enhance operational control, drives this study. With an architecture that will bridge these areas, we hope to show a plausible way forward toward transparent, programmable, and institutionally significant banking services for underserved populations.

1.3 Problem Statement

The international system of the development finance works based on a stratified system in which funds are directed down by large supranational organizations such as the World Bank and IMF to national institutions, which transmit them to regional and local banks, and they are ultimately delivered to individual borrowers. This intermediary chain brings into being a number of significant inefficiencies:

1. **Lack of transparency.** Financial records at each tier of the intermediation chain are not openly accessible to participants below that tier. Lower-level institutions and individual borrowers cannot independently verify how capital is being managed, what reserve levels are maintained, or on what basis lending decisions are made at higher tiers. Reserve adequacy is predominantly self-reported and audited at most quarterly, providing no mechanism for real-time verification of institutional financial health.
2. **Sluggish settlements and unproductive capital.** Cross-border money transmission requires multiple intermediaries, compliance verification steps, and correspondent bank confirmations that routinely delay settlement by two to five business days. Banks must also maintain pre-funded accounts at partner institutions in every currency corridor, locking up capital that would otherwise be available for lending [24]. An average transaction via the correspondent banking system costs approximately $42, a burden that falls disproportionately on small-value transactions and participants in developing economies [25].
3. **Inconsistent risk evaluation.** Fraud screening and credit assessment rely heavily on individual human judgment, introducing bias, inconsistency, and limited scalability. Because no unified and verifiable credit history exists across institutional tiers, borrowers must be re-evaluated from scratch at each stage of the intermediation chain, adding time and cost without improving accuracy.

4. **Obstacles to access and institutional trust.** Establishing inter-institutional trust through legal agreements, compliance processes, and third-party audits is both expensive and time-consuming, placing smaller institutions and borrowers in developing countries at a particular disadvantage. According to the World Inequality Report 2026, the global monetary system continues to redirect resources from developing to wealthier economies, with developing-country investors receiving returns approximately three percentage points lower than those in developed economies [28]—a structural gap that further impedes access to international credit markets.
5. **Absence of programmable savings instruments.** Traditional savings accounts in developing economies offer near-zero real yields due to inflation, currency risk, and institutional opacity. Depositors have no real-time visibility into whether their savings are being deployed responsibly or sitting idle in reserve accounts.
6. **Inaccessibility of group credit mechanisms.** Over 1.4 billion unbanked individuals lack individual collateral but could access credit through solidarity group structures, a model demonstrated by BRAC, Grameen Bank, and ASA in Bangladesh. No existing DeFi protocol models programmable mutual liability or on-chain group repayment enforcement.

At the same time, the DeFi ecosystem has shown that the lending, collateral management, and interest calculation can be completely automated through smart contract platforms with complete transparency. The TVL of Aave v3 is $26.3 billion and active borrows are $17.7 billion [27]; the TVL of Compound v3 is $1.4 billion [29]; and the combined DAI and USDS stablecoin issuance of MakerDAO (renamed Sky Protocol) is approximately $10.5 billion with projected 2026 revenue of $611 million [30]. Despite this scale, these protocols exhibit fundamental limitations in the context of institutional development finance:

- They employ **flat, peer-to-peer**, or **over-collateralized** lending models. While Aave v3 implements risk-parameterized sub-markets (Isolation Mode, Efficiency Mode) and Compound v3 deploys separate Comet markets with independent risk parameters, none of these protocols model institutional hierarchy, cross-tier capital allocation, or differentiated governance access by institutional role. The CWB's four-tier structure is distinct from Aave's sub-markets: it models the *institutional relationships* between a reserve authority, national lenders, local lenders, and retail clients—not merely risk-differentiated asset buckets within a single lending pool.
- They are not connected with **AI-based risk analytics** and **explainable decision support** systems. DeFi lending decisions are determined solely using collateral ratio and utilization curve and no behavioral fraud detection, anomaly detection or explainable credit judgement is present.
- They do not model **role based, tiered governance** of the development finance institutions. It is a form of leadership, in which a majority of the large token holders will wield the power of decision-making, without the institutional role distinction (regulator, wholesale lender, retail lender, retail client) of hierarchical finance.
- The credit-based and emerging-market lending has also been introduced with other new institutional DeFi projects such as Maple Finance ($2.6–3.8 billion TVL, undercollateralized institutional lending) [31] and Goldfinch ($680 million loan originations in 18+ developing countries) [32], however, are single-level projects with no hierarchical capital flows and no interbank lending facilities.

1.4 Objectives

The objectives of this project are:

1. To **design, formally specify, and partially implement** a four-tier lending architecture (World Bank → National Bank → Local Bank → Client) on an EVM-compatible blockchain that maintains institutional hierarchy and provides shared access to the ledger. The current prototype fully implements the Tier 1 World Bank Reserve contract and the lending request/approval workflow; Tier 2 and Tier 3 contracts are specified and partially scaffolded, with full implementation planned for the final thesis phase.
2. To specify and justify an extended banking product suite—including deposit mobilization, savings products, and solidarity group lending—that can be integrated on top of the hierarchical lending foundation.
3. To investigate a lightweight off-chain analytics support layer, including fraud detection, anomaly detection, and explainable review support, to supplement human credit governance; and to specify and partially implement an optional autonomous AI agent system using a Model Context Protocol (MCP) tool server with 17 banking tools (9 read tools and 8 write tools), enabling clients to execute banking operations through a conversational interface as an alternative path to the standard React UI.
4. To demonstrate transparent and programmable lending processes with configurable borrowing limits, installment payments, and role-based access control implemented through smart contracts.
5. To test the prototype on public testnets (e.g., Polygon, Ethereum Sepolia) and verify hierarchical controls, transparency properties, and operational workflows.
6. To document the design of governance mechanisms, security controls, regulatory considerations, and a controlled rollout pathway toward academic validation and potential regulatory sandbox piloting.

1.5 Research Questions

The research questions used in the study are as follows:

1. **RQ1:** Does a hierarchical blockchain-based lending architecture reflect more faithfully the real-world capital flow mechanism of development finance than present decentralized lending?
2. **RQ2:** Can on-chain transparency, programmable reserve controls and role-based governance system make settlements less opaque and less frictional across institutional tiers?
3. **RQ3:** Does a lightweight off-chain analytics layer offer practical and auditable support to monitor fraud and lending in such a system?
4. **RQ4:** Does the proposed architecture operate correctly on current public test networks, and do the deployed contracts exhibit the role-based access control, capital-flow, and reserve-invariant properties required for real-world institutional banking use cases? *Scope note: RQ4 evaluates prototype-level technical viability on testnets only; it does not constitute a claim of production readiness, regulatory compliance, or real-asset deployment.*
5. **RQ5:** Can deposit-funded lending and solidarity group lending be represented as programmable, auditable mechanisms on-chain while preserving practical feasibility for retail users in developing economies?

1.6 Research Contribution

This work makes four original and precisely scoped research contributions to the field of decentralized finance and blockchain-based development banking:

Contribution 1 — Four-Tier Hierarchical DeFi Architecture. To our knowledge, no prior published work presents a four-tier smart-contract lending hierarchy (World Bank Reserve → National Bank → Local Bank → Client) on an EVM-compatible blockchain that mirrors the capital-flow model of multilateral development finance. We extend prior work on flat DeFi lending [1] to incorporate institutional hierarchy, addressing the gap identified by Werner et al. [1]. No surveyed DeFi protocol—including Aave, Compound, MakerDAO, Maple Finance, or Goldfinch—models cross-tier capital allocation with reserve-ratio enforcement at every tier; all employ flat, single-tier pool architectures with undifferentiated liquidity.

Implementation scope: The World Bank Reserve (Tier 1) contract and the lending request/approval workflow (Tier 3) are deployed and tested on Polygon Amoy testnet (contract addresses in Appendix C). The National Bank (Tier 2) and Local Bank (Tier 3) contracts are formally specified in Solidity interface form (Appendix B) and partially scaffolded; full cross-tier fund-transfer automation is implemented in the Hardhat simulation environment and is scoped to Phase II of the final thesis phase. The architectural contribution—the four-tier design pattern and its formal specification—holds independently of the current prototype completeness, as design contributions in distributed systems are established by formal specification and partial implementation consistent with the stated scope.

Contribution 2 — On-Chain Solidarity Group Lending. To our knowledge, no prior published academic work specifies and prototypes a programmable solidarity group lending mechanism on-chain that encodes mutual liability enforcement, group formation consent, and installment splitting into smart contract logic. We note that Goldfinch's Backers/Senior Pool model provides partial structural precedent, but does not implement per-member mutual liability or the over-indebtedness controls detailed in Section 3.17.3. This model was previously realised only in analogue microfinance institution (MFI) operations, such as BRAC's 30–40 member groups and Grameen Bank's groups of five, where enforcement relies on social pressure rather than programmable contract rules.

Implementation scope: Contribution 2 is a specification contribution: no prior academic work formally specifies the algorithmic structure of solidarity group lending as a smart contract interface. The formal specification is given in Section 3.17.3. A minimal proof-of-concept deployment demonstrating group formation, consent recording, and per-member disbursement is included in the testnet repository; full lifecycle implementation with mutual liability enforcement and over-indebtedness control is scoped to Phase II of the final thesis phase.

Contribution 3 — Oracle-Mediated AI/ML Integration with Explainability. We propose an architectural pattern for integrating off-chain Random Forest fraud detection and SHAP-based explanations into an on-chain lending decision workflow via a trusted oracle relay, providing a blueprint for auditable, AI-assisted credit governance in DeFi. The design addresses the oracle problem—the challenge of securely bridging on-chain logic with off-chain data—and specifies three concrete integration options (centralized relay, Chainlink Functions, and commit-reveal scheme) with explicit trust trade-offs.

Implementation scope: The Random Forest fraud detector is trained and benchmarked on the BCCC-DeFiFraudTrans-2025 dataset [R47] (1,026,867 annotated DeFi transactions, 79 features), with cross-validation against the Elliptic Bitcoin Dataset [R48]. Precision, recall, F1, and ROC-AUC results are reported in Section 4.2. The oracle commit-reveal wiring to the deployed `LocalBank` contract on Polygon Amoy is scoped to Phase III; the architectural specification and ML benchmark together constitute the Contribution 3 claim at pre-thesis stage.

Extension — Autonomous agent architecture (v24). In v24, Contribution 3 is extended from a read-only analytics layer to a fully action-capable autonomous agent system. The agent operates as a six-step pipeline: (1) the user message arrives at the Qwen3-8B inference engine, which has the client's complete on-chain state injected as structured JSON context by the Express.js backend; (2) for question-answering requests, the agent performs retrieval-augmented generation from policy documents and replies directly; (3) for action requests, the agent assembles the full parameter set and presents a plain-language confirmation summary to the client; (4) the agent waits for explicit affirmative consent before calling any write tool; (5) upon confirmation, the MCP write tool is called via the Express.js banking API layer, and the resulting on-chain transaction is signed using an EIP-7702 session key scoped to the approved tool set, value-capped at 500 USDC per transaction, and bounded by a 24-hour time-to-live; (6) the agent monitors the transaction status and notifies the client on completion. When the agent files a loan application on behalf of a client, it also prepares a structured *Authority Brief* for the Local Bank approver, presenting the SHAP feature attribution breakdown alongside a one-click approve/decline interface. No write tool is ever executed without a confirmation turn in the conversation history; this invariant is enforced at the application level and audited via the append-only `agent_action_log` table.

Contribution 4 — Compliance-Aware ZKP / zkAML Identity Pathway. We design a two-circuit privacy-preserving compliance architecture combining a zk-SNARK KYC verifier (Groth16 proofs via Circom 2.0 + snarkjs) with a zkAML circuit [R19] that proves a wallet has not transacted with sanctioned addresses and remains within velocity limits, without revealing the underlying transaction graph. Both circuits compose with a W3C Decentralized Identifier (DID) and Verifiable Credential (VC) layer [R20] anchoring user identity off-chain. This positions the platform's compliance gateway against the broader Self-Sovereign Identity (SSI) framework and is grounded in the Piper et al. (2025) TU Berlin permissioning paper [R8].

Implementation scope: A Groth16 KYC age-range circuit was compiled using Circom 2.0 and the auto-generated Solidity verifier deployed to Polygon Amoy (`KYCVerifier`: address in Appendix C; to be migrated to Polygon zkEVM Cardona in Phase I). The circuit proves knowledge of an age satisfying KYC eligibility without revealing the age on-chain, using a Poseidon hash commitment for proof freshness. This constitutes a proof-of-concept implementation of the ZKP identity architecture described in Section 3.6.1. The wallet-velocity AML circuit [R19] (IACR ePrint 2025/465, benchmarked at 55 TPS / 226 ms) requires a significantly larger circuit and is scoped to Future Work; the current deployment demonstrates the end-to-end technical pathway from Circom circuit to on-chain verification.

Positioning against institutional DeFi. The Sygnum Bank *Institutional DeFi in 2025* report (February 2026) documents a specific gap: protocols work technically, but no large

institutional allocator participates until legal and regulatory risks are resolved [R21]. The Crypto World Bank is positioned not as a technically novel curiosity but as an attempt to resolve precisely the governance, compliance, and audit gaps that report identifies as the barriers to institutional DeFi adoption. Combined with the mBridge / Agora positioning in Section 1.10.3, this reframes the contribution from "an alternative to traditional banking" to "a composable lending protocol compatible with emerging institutional CBDC and tokenized-deposit settlement infrastructure"—a substantially more defensible and publishable claim.

The remainder of this paper is organized as follows. Chapter 2 surveys the academic and industry literature on DeFi lending, hierarchical financial architecture, AI-assisted credit assessment, and blockchain-based financial inclusion, identifying the research gaps that motivate this work and presenting the synthesis under a PRISMA-style methodological frame. Chapter 3 presents the system architecture and design, including the smart contract hierarchy, banking product specifications, data model (20 normalized entities), identity and onboarding flows, the formal actor taxonomy and permission matrix, the Chainlink oracle stack, and the compliance and governance framework. Chapter 4 describes the development methodology, implementation phase plan, the autonomous AI agent pipeline and MCP tool server architecture, AI/ML pipelines, formal-verification strategy, and technology stack justification. Chapter 5 evaluates the technical, economic, regulatory, and sustainability feasibility of the proposed system, including the revenue stream taxonomy. Chapter 6 presents conclusions and future research directions.

1.7 Blockchain Justification

The root cause of the problems identified above is a **multi-party coordination and trust problem**: institutions with potentially conflicting interests must share financial state, enforce shared rules, and settle obligations—without being able to fully trust one another or a central administrator. Traditional solutions to this problem require either a trusted central authority (a central bank, a clearinghouse, a custodian) whose failure or misconduct creates systemic risk, or an elaborate network of bilateral legal agreements and reconciliation processes that introduce the 2–5 day settlement delays and $42 per-transaction costs documented above.

Blockchain technology directly resolves this coordination-and-trust problem through four properties that have no analog in conventional database or cloud architecture:

1. **Consensus-enforced shared state.** All participants observe the same ledger, updated by distributed consensus rather than by any single operator. In the CWB context, this means a Local Bank, its parent National Bank, and the World Bank Reserve all read the same reserve ratios and loan states in real time—not reconciled copies maintained independently and aligned through batch messaging.
2. **Programmable, immutable rule enforcement via smart contracts.** The lending rules—reserve ratios, interest rate curves, borrowing limits, approval workflows, multi-entity consent requirements—are deployed as code on the EVM. Once deployed under the governance-controlled upgrade mechanism, these rules execute *identically for every participant and cannot be selectively overridden by any party*, including the institution that deployed the contract. A Local Bank cannot quietly waive a reserve check for a favored borrower; a National Bank cannot unilaterally adjust the interest rate for a connected party. This is qualitatively different from a database: a database

administrator can always modify a record; a deployed smart contract cannot be modified without a recorded governance action.

3. **Cryptographic auditability.** Every state change—every loan disbursement, every repayment, every reserve ratio update—is permanently recorded in a tamper-evident, publicly verifiable transaction log. Regulators, partner institutions, depositors, and auditors can independently verify the platform's financial state without relying on self-reported figures or periodic audits. Real-time reserve transparency eliminates the informational asymmetry that allows institutions to conceal insolvency between audit cycles.
4. **Composable incentive structures and atomic multi-party settlement.** On-chain programmability allows multi-party financial operations—group loan disbursements, syndicated repayments, netting settlements—to be encoded as single atomic transactions that either complete fully or revert entirely. There is no half-settled syndicated loan, no partial group disbursement, no settlement risk between the moment funds leave one institution and arrive at another.

A conventional cloud-based database architecture could store the same loan records, but it would require a trusted central operator with administrator access—re-establishing precisely the single point of trust failure this project is designed to eliminate. Blockchain is not chosen here because it is novel; it is chosen because the specific trust, auditability, and programmable enforcement properties it provides are the correct solution to the specific multi-institution coordination problem this system is designed to solve.

1.7.1 Blockchain Fundamentals and the EVM Execution Model

This subsection provides a concise technical reference for the blockchain primitives on which the Crypto World Bank is built. Readers familiar with EVM architecture, smart contract state machines, and oracle design may proceed directly to Section 1.8.

A. What blockchain is (technically precise). A blockchain is an append-only distributed ledger in which each block contains a cryptographic hash of the previous block, a Merkle root of all transactions in the block, a timestamp, and nonce or validator signature data. Immutability is not absolute—it is probabilistic and consensus-enforced: it holds as long as no coalition controlling more than 50% of stake (in Proof-of-Stake) cooperates to rewrite history [R17]. On Polygon PoS specifically, finality is achieved through the Heimdall validator layer (a Tendermint-based BFT consensus layer), which provides stronger finality guarantees than pure longest-chain PoW, but still assumes that fewer than one-third of validators are Byzantine. This trust assumption is a design constraint, not a deficiency, and must be stated explicitly in any security analysis.

B. The EVM Execution Model (Ethereum Yellow Paper [58]). The Ethereum Virtual Machine is a stack-based, deterministic, Turing-complete virtual machine that executes smart contract bytecode. Every operation (opcode) has a fixed gas cost specified in the Ethereum Yellow Paper [R7]. Key opcodes relevant to this system:

- `SSTORE` (write to storage): 20,000 gas for a new slot, 2,900 for modification—the most expensive operation class.
- `SLOAD` (read from storage): 2,100 gas (warm access), 100 gas (cold).
- `CALL` (external contract call): 700 gas base plus additional costs.
- Events (`LOG2`, `LOG3`): ≈375 + 8 per data byte—inexpensive, used for off-chain index-

ing.

Each installment repayment involves at minimum one `SSTORE` for updating the loan balance, one `SSTORE` for recording payment history, and one event emission. A 12-installment loan therefore involves at minimum 24 `SSTORE` operations. A complete retail loan lifecycle involves approximately 27–32 individual on-chain state changes, as detailed in Section 5.8.

C. Smart contracts as state machines. A lending smart contract is best modeled as a finite state machine: `PENDING` → `APPROVED` → `ACTIVE` → `REPAYING` → `COMPLETED` | `DEFAULTED`. State transitions are triggered by role-authorized function calls, and the `nonReentrant` guard ensures atomicity of each transition. This state machine framing strengthens the argument that smart contracts are more auditable than off-chain systems, because every state transition is permanent, timestamped, and publicly verifiable.

D. The Oracle Problem. Smart contracts are deterministic and isolated from external data by design. This is a feature—consensus would break if different nodes observed different external states—but it creates a fundamental architectural gap for any system that requires off-chain data. The oracle problem, formally identified by Beniiche (2020) [R1] and Pasdar et al. (2023) [R2], is the challenge of securely bridging on-chain logic with off-chain information. For the Crypto World Bank, the AI/ML risk scores produced by the FastAPI Random Forest service are off-chain data that must influence on-chain lending decisions. Three architectural options exist: (1) a centralized relay (trusted backend calls `updateRiskScore` on-chain—simple but re-introduces a central trust point); (2) a decentralised oracle network such as Chainlink Functions (removes the trust assumption at the cost of 30–60 seconds latency and $0.10–$1.00 per call; the DON requires consensus across multiple independent nodes before a score is committed, so no single compromised node can manipulate the result); and (3) a commit-reveal scheme (the ML service commits a hash of the risk score before the loan decision window, reveals it after). The current prototype uses a centralized relay, acceptable in the testnet context; Chainlink Functions is adopted as the primary oracle mechanism in v24 (Phase III), with the commit-reveal scheme retained as a prototype-phase fallback. In addition to ML score commitment, Chainlink Price Feeds supply BDT/USD and ETH/USD rates to the `FXModule` contract, Chainlink Automation replaces the centralised cron job for overdue installment detection, and Chainlink Proof of Reserve publishes the `WorldBankReserve` balance for external cryptographic verification.

E. Polygon zkEVM Trust Model. Polygon zkEVM Cardona testnet is selected as the primary deployment target for v24, replacing Polygon Amoy PoS. The security model changes materially: rather than delegated proof-of-stake consensus with a validator set, Polygon zkEVM uses ZK validity proofs. Every batch of transactions submitted to the sequencer is accompanied by a zero-knowledge proof verified on Ethereum L1, so security derives from cryptographic validity rather than a validator-set assumption. A malicious sequencer cannot publish an invalid batch because the ZK proof would not verify on Ethereum. This provides a fundamentally stronger guarantee than the PoS model: the trust assumption collapses from "no coalition of validators controlling more than one-third of staked MATIC colludes" to "the ZK proof system is sound and Ethereum L1 is live." As a ZK rollup, Polygon zkEVM inherits Ethereum's full economic security for finality, while offering comparable throughput and gas costs to Polygon Amoy on testnet. The stronger security story is particularly significant for an institutional banking prototype: "ZK-secured reserve ratios" is a materially more defensible claim than "PoS-secured reserve ratios" in a thesis context and in regulatory discussions.

1.8 Proposed Solution

The Crypto World Bank is based on a **four tier on-chain lending model**:

- **Tier 1 — World Bank:** Custodian of the global crypto reserve. Allocates capital to registered National Banks via lending mediated by smart contracts.
- **Tier 2 — National Banks:** Borrow from the World Bank reserve. Lend to Local Banks incorporated in their jurisdiction. Aggregate the risk exposure at the national level.
- **Tier 3 — Local Banks:** Borrow from National Banks. Process loan applications submitted by retail clients. Administer the full loan lifecycle using designated approvers.
- **Tier 4 — Clients (Retail):** Submit loan requests to Local Banks. Repay through configurable installment schedules. Build an on-chain repayment history that determines future credit access. Includes individual retail clients, group lending members, and small-business clients.

The platform also incorporates:

- **Risk-sensitive borrowing limits** computed from rolling 6-month and 1-year transaction windows.
- **Automatic installment generation** for loan amounts exceeding a configurable threshold (e.g., 100 ETH equivalent).
- **Off-chain AI/ML security analytics** (planned) for fraud detection (Random Forest), anomaly identification (Isolation Forest), and explainable risk assessment (SHAP).

1.8.1 Banking Functions of the Platform

A functionally complete bank performs six core activities that together transform idle savings into productive economic activity: deposit mobilization, credit allocation, payment and settlement, risk intermediation, liquidity management, and ancillary financial services. The distinguishing characteristic of a bank—as opposed to a lending marketplace or a peer-to-peer exchange—is that it performs *maturity transformation*: it borrows short (taking demand deposits repayable on request) and lends long (issuing multi-year loans), earning the spread as interest income. This transformation creates systemic value but also systemic risk, which is why every banking regime in the world requires minimum reserve ratios, capital adequacy rules, and regulatory oversight. The Crypto World Bank is designed to implement each of these six functions across its four-tier hierarchy, encoding the rules that govern them as smart contract logic rather than relying on discretionary enforcement.

Deposit Mobilization. The process by which a bank accepts funds from savers and transforms them into productive capital. In traditional banking, this is the liability side of the balance sheet: the bank owes depositors their money back. On the platform, depositors at any tier can place funds into savings products—standard savings accounts with variable yield, fixed-term deposits with locked periods and agreed APY, and institutional yield accounts for large participants. Crucially, the `SavingsVault` contract records each depositor's claim on-chain and enforces the reserve ratio programmatically: the ratio of liquid reserves to outstanding deposits is checked on every withdrawal, replacing the quarterly audit cycle of traditional banks with real-time, transparent, cryptographically verifiable reserve proof.

Credit Allocation. This is the asset side of the banking balance sheet: how the bank de-

ploys mobilized deposits into productive loans. Loans flow downward through the four-tier hierarchy—from the World Bank reserve to national banks, from national banks to local banks, and from local banks to end borrowers. Each tier applies its own interest rate spread (the margin between borrowing cost and lending rate that covers operating costs and risk), collateral requirement, and borrowing limit, enforced by smart contract rules. Critically, the platform also supports **group co-lending**: multiple institutions can pool their capital to co-fund a single loan through the `SyndicatedLoan` contract, distributing both the credit risk and the interest income pro-rata. This mirrors the syndicated lending market in traditional banking—where large project loans are jointly underwritten by several banks—but encodes the consent, allocation, and repayment logic on-chain rather than through bilateral legal agreements.

Payment and Settlement. Transfers between registered accounts settle atomically in a smart contract transaction, avoiding the intermediate "funds-in-transit" state that commonly produces disputes in traditional correspondent banking (where a payment can be in-flight across multiple intermediaries for 2–5 business days). On-chain atomic settlement is either fully completed or fully reverted; there is no partial state, no reconciliation backlog, and no nostro/vostro account float.

Risk Intermediation. In traditional banking, risk intermediation means that a bank accepts credit risk (the risk that a borrower will not repay) and transforms it into a lower-risk obligation to depositors, buffered by the bank's capital. On the platform, this is implemented through reserve-ratio enforcement constraints (capital buffer), role-based approval workflows (credit underwriting), the `LiquidationEngine` (collateral recovery), and an AI/ML monitoring layer (Random Forest fraud detection, Isolation Forest anomaly detection, SHAP explainability) designed to provide transparent, auditable decision support that is absent from all existing DeFi protocols.

Liquidity Management. Banks must maintain enough liquid assets to meet deposit withdrawals and other short-term obligations without being forced to sell long-term assets at a loss—this is the classic liquidity vs. yield tension that every treasury manager navigates. On the platform, this is enforced via minimum reserve ratios at each tier, with same-tier interbank lending pools (`InterBankLendingPool`) for short-term liquidity balancing and the `UpwardDepositFacility` for surplus repatriation. The daily withdrawal rate limiter on the `UpwardDepositFacility` is an explicit bank-run circuit breaker, analogous to the central bank standing deposit facility withdrawal limits that prevent coordinated liquidity shocks.

Ancillary Financial Services. Includes foreign exchange, group solidarity lending for retail borrowers, syndicated co-funding for institutional borrowers, trade finance facilitation, and digital identity management. FX is designed around decentralized Chainlink price oracles; group lending enables pooled collateral and mutual liability for retail clients with no individual credit history; syndicated lending enables institutional entities to co-fund large loans; trade finance instruments can be added as planned extensions.

1.8.2 Service Delivery Across Scale

The platform is designed to operate at three scales of engagement: the global institutional level, the national and regional commercial level, and the individual retail level.

Global institutional level (World Bank Reserve tier). The World Bank Reserve func-

tions as a programmable reserve authority by managing system-level capital allocation and reserve constraints across jurisdictions. This tier is designed to support transparent reporting of reserves and enforce reserve-ratio constraints as code, reducing informational asymmetry relative to periodic self-reporting.

National and regional level (National Bank and Local Bank tiers). National Banks borrow wholesale capital from the reserve tier and allocate it to registered Local Banks under jurisdiction-aware governance and tiered approval workflows. The intent is to mirror real-world development finance intermediation while improving settlement speed and auditability through on-chain execution.

Retail level (clients and depositors). End customers interact through Local Bank interfaces to open accounts, deposit savings, apply for loans (including group loans), repay installments, and build an on-chain credit history. In Bangladesh, where roughly 40% of adults lack access to formal banking services [54], a mobile-accessible interface with low transaction costs can reach users that branch-based systems do not, while keeping loan terms and reserve behavior verifiable through on-chain transparency.

1.8.3 Cross-Tier Lending System

Capital in real banking systems does not flow only downward. Banks at the same tier lend to one another daily in the interbank market (the federal funds market in the US, SONIA in the UK, SOFR globally), surplus institutions deposit excess reserves upward with central banks or parent institutions, and large loans are co-funded by syndicates of several banks acting together—because no single bank should carry the full credit risk of a major project. The Crypto World Bank encodes all three of these flow directions as on-chain contracts, producing a **multi-directional, multi-entity capital coordination system** that reflects the structural reality of institutional banking.

Can a group of entities lend or fund together? Yes—and this capability is a distinct architectural feature of the platform, not an afterthought. The platform supports group co-funding at *both* the retail and institutional level through separate but structurally analogous mechanisms:

- **Institutional group co-lending (Syndicated Loans, Section 3.13.3):** Multiple banks at the same or adjacent tier—for example, three Local Banks and their parent National Bank—can co-fund a single large loan through the `SyndicatedLoan` contract. Each co-lender commits capital during a subscription window, signs an on-chain consent vote, and receives interest repayments pro-rata to its capital share. The Lead Arranger structures the deal and earns a small underwriting fee. This is the on-chain equivalent of a real-world syndicated loan or club deal.
- **Retail group co-borrowing (Solidarity Group Loans, Section 3.17.3):** Three to twenty retail clients can form a lending group, pooling collateral into a shared contract and signing a joint application. The `GroupLendingPool` enforces mutual liability: if one member defaults, the shared collateral pool covers the shortfall, protecting the lender.
- **Senior–junior co-funding pools (Tranched Pools, Section 3.13.4):** Risk-tolerant and risk-averse entities co-fund the same loan pool with different seniority rights—the junior tranche takes first loss, the senior tranche is protected. This is the structural pattern used by Goldfinch and Maple Finance for institutional DeFi lending and is

incorporated into the CWB architecture.
- **Surplus repatriation (Upward Deposit Facility, Section 3.13.2):** Groups of Local Banks holding surplus reserves can collectively deposit upward into their parent National Bank, which aggregates those deposits and re-deploys them where capital is needed most—a form of group capital pooling that flows upward rather than downward.

These four mechanisms together provide a complete answer: the platform is explicitly designed for groups of entities—both retail borrowers and institutional banks—to lend, fund, and borrow together through programmatic on-chain coordination.

Same-Tier Lending

Banks at the same hierarchical level can lend to one another to manage short-term liquidity:

- **National Bank ↔ National Bank:** A national bank, which has excess reserves, can lend to a second party that is in a liquidity crunch, similar to the traditional interbank lending market. The lending rate is adjusted according to the supply and demand through national banks of the system, like the Secured Overnight Financing Rate (SOFR).
- **Local Bank ↔ Local Bank:** Local banks in same or different national jurisdictions can share liquidity through a peer lending pool. This prevents local liquidity crunches when one bank possesses surplus deposits and the other is experiencing elevated loan demand.

Same-tier lending is done by an on-chain **InterBankLendingPool** contract at every tier level in which surplus banks provide short-term liquidity and those banks that need it are allowed to borrow at utilization floating rates. v15 specifies this pool fully in Section 3.13.1 (rate model, settlement, default cascade, operations); the present chapter introduces only the role it plays in the four-tier hierarchy. The current pre-thesis 1 prototype implements the downward flow (World Bank → National Bank → Local Bank → Client); the InterBankLendingPool is built in Phase II of the final-thesis phase.

Upward Lending

Banks with excess capital of lower tier are then able to lend up the ladder:

- **Local Banks → National Banks:** When local banks accumulate reserves beyond the minimum reserve ratio, they are allowed to deposit excess capital in their parent national bank's liquidity pool with earned deposit yield. This mirrors how commercial banks save the surplus with central banks by means of standing deposit facilities.
- **National Banks → World Bank:** National banks may provide excess to the international deposit, enhancing the total capital base of the system. This provides a back route to well capitalized national banks and raises the reserve available for downward distribution to banks with higher demand.

The downward lending rates are higher than the upward lending rates (the low risk of lending to a higher-level institution), developing a natural interest rate term structure across the hierarchy.

Tiered Client Access

Different client classes access different tiers based on loan size and organizational type:

Table 1.1: Client tier access rules.

Client Type	Accessible Tiers	Loan Range	Use Case
Individual / Retail Client	Local Bank only	0.01–10 ETH	Personal, micro-enterprise
Small Business / SME	Local Bank, National Bank	1–100 ETH	Working capital, equipment
Large Corporate	National Bank, World Bank	50–10,000 ETH	Infrastructure, large projects
Institutional / Sovereign	World Bank only	1,000+ ETH	Development programs

This table summarizes tiered client access rules by mapping client type and loan size to the appropriate institutional tier. The intent is to preserve real-world banking intermediation: retail clients primarily access Local Banks, while larger institutional clients can be routed to higher tiers under stricter verification. This design supports risk segmentation and prevents large actors from consuming retail liquidity.

Local Banks are used by end users and retail clients to gain access to the system, maintaining the hierarchical intermediation model. Higher tiers can be accessed directly by large corporate and institutional clients, subject to rigorous verification (on-chain credit history with off-chain documentation) and a client-type designation in the smart contract that determines tier access permissions.

This multi-way lending scheme—downward capital allocation, same-tier interbank lending, upward repatriation of surpluses, and tiered access by clients—generates a more detailed model of the banking flows of the real world in the decentralized architecture. The current prototype fully implements the downward capital distribution and tiered client access components; same-tier interbank lending and upward surplus repatriation are specified in full in Section 3.13 (`InterBankLendingPool`, `UpwardDepositFacility`) and scheduled for implementation in Phase II of the final-thesis phase. The genuinely multi-entity operations that build on top of these flows—syndicated lending, tranched pools, cross-tier treasury FX, and multilateral settlement netting—are likewise specified at full depth in Section 3.13 so that the paper describes a complete banking system rather than only retail-facing lending.

1.9 Methodology in Brief

Our approach combines a lightweight Agile delivery process with an explicit Software Development Life Cycle (SDLC) frame. Pre-thesis 1 covers the requirements, architecture, and design phases of the SDLC; the implementation, verification, and validation phases run across four subsequent phases during the final thesis stage. Phase I builds the four-tier smart contract scaffold (World Bank Reserve, National Bank, Local Bank), the wallet authentication flow, the frontend skeleton, the PostgreSQL schema (all 19 entities), and the Chainlink Proof of Reserve integration. Phase II implements lending services (loan application, approval, installment generation, borrowing limit enforcement), the deposit-mobilization modules (SavingsVault, FixedDeposit), the group-lending pool, the same-tier and upward treasury-swap modules (`InterBankLendingPool` per Section 3.13.1, `UpwardDepositFacility` per Section 3.13.2, `TreasurySwap` per Section 3.13.5), and the AI Agent MCP tool server with EIP-7702 session key management and confirmation audit hook. Phase III wires the off-chain AI/ML risk-scoring pipeline (Random Forest, Isolation

Forest, SHAP) into the on-chain loan decision via Chainlink Functions (DON-based trustless oracle; the commit-reveal relay is retained as the prototype fallback per Section 3.3.2), layers the Graph-Neural-Network and federated-learning extensions on top, adds the syndicated-lending, tranched-pool, and netting-engine contracts (`SyndicatedLoan` per Section 3.13.3, `TranchedPool` per Section 3.13.4, `NettingEngine` per Section 3.13.6, plus the off-chain Settlement Coordinator service), and adds the Foundry invariant suite and Tenderly runtime monitoring. Phase IV adds the Certora reserve-invariant specifications and formal verification proofs. The implementation stack is Solidity 0.8.20 (smart contracts), React 18 with TypeScript and ERC-4337 account abstraction (frontend), Express.js + FastAPI (backend), scikit-learn + PyTorch Geometric (ML), Circom 2.0 + snarkjs (ZKP), and ChromaDB + a locally hosted 8B-parameter LLM (RAG assistant). All deployments target Polygon zkEVM Cardona and Ethereum Sepolia testnets so that the prototype carries zero real-cryptocurrency cost; stablecoin pools use testnet USDC.

1.10 Scopes and Challenges

The complete banking architecture described in this report defines both what is implemented in the current prototype and what constitutes the full system design, clearly distinguishing prototype scope from architectural intent throughout.

Scope. This prototype is restricted to publicly available testnets, i.e. no actual cryptocurrency is involved in the system. It encompasses basic features including four-tier lending architecture, loan request and approval process, installment based repayment, calculation of borrowing limits, communication between borrowers and banks and verification of income. The system also uses Random Forest with SHAP to explain explainability in fraud detection and Isolation Forest to detect anomalies. An additional feature of dual-currency is also introduced to enhance the current banking infrastructure. The prototype, however, does not cover mainnet deployment, fiat on/off-ramp integration, automated KYC/AML compliance or production-scale stress testing.

Challenges. There are some issues that come with this system. The unlabeled nature of DeFi lending fraud data is one of those problems, as it limits the successful training of the model and might necessitate the application of synthetic data or transfer learning methods. The other issue is regulatory uncertainty, since crypto lending may have different jurisdictional restrictions; and this risk can be partially reduced by working the testnets only. The sensitivity of gas costs is an issue as well, as intricate on-chain programs can be costly; to solve this, AI/ML work requiring computation is performed off-chain. Additionally, model interpretability is essential to regulatory compliance, especially in describing the process of lending, which is obtained by SHAP-based feature attribution. Last but not the least, economic sustainability should be taken into account, because the platform should be able to earn enough income, like interest revenues, to cover gas expenses at all four levels. The following Chapter 5 discusses this balance in greater depth.

1.10.1 Economic Sustainability: Interest Revenue Versus Gas Costs

The most important problem facing any blockchain-based lending system is whether the revenue earned by the lending activities will be in a sustainable position to cover the total gas expenses of the transactions on-chain. Every level in the hierarchy will have deposits, loan applications, approvals, disbursements, repayment and installment processing gas fees. One complicated smart contract interaction can cost between $5 and $50 on Ethereum main-

net, based on network congestion, but around $0.001 to $0.01 on Polygon zkEVM Cardona (comparable to Polygon PoS for testnet operations). At an average retail loan of 10 ETH ($20,000 at $2,000/ETH) with an 8% APR, the client pays $1,600 in annual interest. With 10–15 on-chain transactions per loan lifecycle (request, approval, disbursement, 12 monthly installments), the overall gas costs on Polygon are under $0.15—less than 0.01% of the interest obtained. The platform thus provides a sustainable margin on Layer 2 networks, but mainnet implementation would need to consider gas optimisation or batched transaction processing to ensure it would be viable to serve micro-loans.

1.10.2 Resistance to Institutional Capture

As the Crypto World Bank grows in adoption, a critical question is whether large capital holders and established financial institutions—which already dominate the traditional banking system—could migrate to the new architecture and replicate the existing patterns of financial concentration. This risk is mitigated by the platform in a variety of architectural ways:

- **Visible reserve ratios:** In contrast to traditional banks, where reserve adequacy is reported and audited quarterly, on-chain reserves are publicly verifiable in real time. There is no way a given institution can hide its real financial status to have unfair advantage.
- **Algorithmic interest rates:** Interest rates are determined by the smart contract parameters, not by opaque internal pricing committees, and thus no single participant can capture the entire pool of capital.
- **Tamper-evident audit trails:** Every transaction, approval, and governance decision is permanently recorded on-chain, making regulatory capture and corruption detectable by any participant.
- **Open-source governance:** The smart contract code is publicly auditable, preventing hidden backdoors or preferential treatment encoded at the protocol level.
- **Tiered access with caps:** Borrowing limits and tier access rules are enforced programmatically, preventing any single entity from monopolizing the capital pool.

Although no system can entirely deter wealth concentration in a free market, the detectability and programmability characteristics of blockchain infrastructure make exploitative behavior prohibitively expensive and highly observable by comparison to opaque intermediaries.

1.10.3 Stablecoin-First Lending and Supply Scalability

Denomination matters more than any other design choice for a crypto financial services platform that targets developing economies. A retail client in Bangladesh who takes a 0.5 ETH micro-loan during a period of price stability and is later required to repay the same 0.5 ETH after a 50% drawdown effectively faces a 2× increase in the real value of their debt. ETH has experienced exactly this drawdown twice in the past five years (May 2021 and 2022), and the stablecoin literature has formalized the resulting liability-side volatility risk for crypto-asset borrowers [R12, R13]. The Crypto World Bank therefore elevates **stablecoin-denominated lending** from a future-work footnote to an explicit design requirement at every retail tier. As of April 2026, USDC commands a circulating supply of approximately $77.3 billion and USDT approximately $189.5 billion (DeFiLlama stablecoin tracker, accessed April 2026), confirming that stablecoin-denominated transactions have

achieved dominant adoption across consumer DeFi applications—a market signal that validates the platform's denomination choice.

The current prototype continues to use ETH as the testnet denomination for simplicity, but every loan-bearing contract has been designed so that the loan currency is a per-pool parameter rather than a hard-coded asset. In the final thesis phase, the retail Local Bank tier will deploy with USDC as the default loan currency; ETH remains acceptable only for institutional Tier 1 / Tier 2 operations where the institutional client has the balance-sheet capacity to hedge crypto-asset exposure.

Three regulatory anchors shape this choice. First, the EU Markets in Crypto-Assets (MiCA) Regulation, fully in effect since December 2024, classifies the kind of fiat-pegged tokens used here as electronic-money tokens (EMTs) and requires 1:1 reserve backing, authorization before public offering, and regular audits [R36, R37]. Second, the United States GENIUS Act, signed into law in July 2025, introduces the first federal framework for payment stablecoins and codifies a similar reserve and disclosure regime [R38]. Third, the BIS *mBridge* platform reached Minimum Viable Product status in mid-2024 and now settles real-value cross-border CBDC transactions among China, Hong Kong, Thailand, the UAE, and Saudi Arabia, while the BIS / central-bank *Project Agora* explores tokenized bank deposits on a unified ledger [R39, R40]. Neither initiative implements a retail lending hierarchy: they are settlement rails. The Crypto World Bank is positioned as the lending layer that can compose on top of such institutional settlement infrastructure rather than as a replacement for it.

The remaining supply-side concerns are addressed by three complementary measures:

1. **Stablecoin pool support.** Each Local Bank deploys an ERC-20 stablecoin loan pool (USDC primary, USDT and DAI as fallbacks). The InsuranceFund and SavingsVault contracts are denominated in the same unit so depositors, borrowers, and the reserve share a single numeraire.
2. **Multi-chain and Layer 2 deployment.** The platform deploys on Polygon zkEVM Cardona testnet for retail traffic (ZK validity proof security model) and on Ethereum Sepolia for high-value institutional flows, with the documented cross-chain bridge architecture in Section 3.12 preserving the hierarchy invariant across chains.
3. **Capital circulation via the four-tier hierarchy.** The Reserve Ratio (RR) enforced at every tier functions as a solvency constraint, not a money-creation parameter. USDC is 100%-reserve backed; the platform re-allocates existing stablecoin supply rather than creating new money. The rate at which this fixed capital pool flows through the hierarchy and returns as repayment is measured by Credit Velocity (Formula CV). Higher credit velocity—not money multiplication—is the platform's capital-throughput mechanism.

The supply concern that originally motivated this subsection is therefore reframed: scarcity is a property of base-layer crypto-assets, not of the lending denomination. Fiat-backed stablecoins issued under MiCA-/GENIUS-compliant reserves provide the elastic, audit-friendly supply a retail banking platform requires, while the hierarchical reserve architecture promotes capital velocity across tiers. Client-side currency risk is removed, deposit-side regulatory compliance is preserved, and the protocol does not depend on the monetary discretion of any single issuer.

1.11 Market Analysis and Partnership Ecosystem

1.11.1 Market Sizing

The platform addresses three nested market segments. The Total Addressable Market (TAM) spans global DeFi lending (exceeding $55B TVL [13]) plus the $860B remittance market [26] and $4.5T MSME financing gap [20]. The Serviceable Addressable Market (SAM) is institutional and semi-institutional lending requiring hierarchical structures ($5–15B), and the immediately Obtainable Market (SOM) covers pilot deployments in regulatory sandboxes and NGO-backed microfinance ($50–200M). Full market sizing with data sources is presented in Table 5.1 in Chapter 5.

1.11.2 Target Customer Segment

The Crypto World Bank targets retail clients and small businesses in developing economies who lack formal banking access—particularly the estimated 1.4 billion unbanked adults globally [14]. The platform's hierarchical architecture (World Bank $\rightarrow$ National $\rightarrow$ Local $\rightarrow$ Client) mirrors development-finance intermediation while providing blockchain-enforced transparency and AI-enhanced risk assessment. Full client segment profiling is presented in Chapter 5 (Table 5.3).

1.11.3 Partner Ecosystem

The platform's deployment requires five partner categories: financial regulators (regulatory sandbox approval), banking institutions (National/Local Bank network members), payment gateway providers (fiat on/off-ramp), academic and research institutions (AI/ML validation), and non-governmental organizations (field pilots with underserved client populations). Blockchain-mediated incentives align each partner's interests with platform operation through on-chain transparency, reduced settlement friction, and programmatic fee distribution. Full partner ecosystem detail is in Chapter 5 (Table 5.5).

1.11.4 Incentive Alignment Through the Blockchain Platform

The system manages and enforces partner incentives directly on the blockchain, making the process more transparent and reliable:

- **Tamper-evident repayment records** serve as an on-chain reputation system through permanent repayment records, allowing credit decisions to be based on actual data without relying on external credit bureaus.
- **Transparent reserve verification** enables on-chain reserve verification, so all participants can independently confirm that allocated funds are being used as intended.
- **Programmable fee structures** (with potential for future expansion) allow transaction fees to be distributed fairly among network participants according to their roles and contributions.

Chapter 2

Literature Review

2.1 Preliminaries

Key technical terms used throughout this thesis—including Decentralized Finance (DeFi), smart contracts, over-collateralization, Explainable AI (XAI), Proof of Stake (PoS), CBDC, Solidarity/Group Lending, Asset-Liability Management (ALM), Flash Loans, Zero-Knowledge Proofs (ZKP), and Health Factor (HF)—are defined in the **Glossary** appendix (labelled Appendix D in the final thesis; the current pre-thesis build carries the World-BankReserve interface as Appendix D pending the final assembly pass). The Glossary also includes platform-specific terminology introduced in this thesis (e.g., *Platform Solvency Ratio (PSR)*, *Provisional Credit Tier*, *Cold-Start Credit Pathway*). This chapter assumes familiarity with blockchain fundamentals at the level of Section 1.7.1; other terms are defined on first use.

2.1.1 Review Methodology (PRISMA-Style Frame)

The literature underlying this thesis is synthesized under a PRISMA-style evidence-synthesis frame [R22] rather than as a narrative summary alone. Inclusion criteria were: (i) peer-reviewed publications or formally published institutional reports issued between 2017 and 2026; (ii) direct relevance to at least one of seven scoped CWB modules—DeFi lending mechanics, ML-based fraud / anomaly detection, ZKP / DID-based identity, group lending and microfinance, smart-contract security, financial-inclusion economics, or institutional DeFi compliance; and (iii) sufficient methodological transparency to allow re-use of design parameters (e.g., reserve invariants, gas accounting, F1 scores). Sources were identified through ACM Digital Library, IEEE Xplore, Elsevier ScienceDirect, MDPI, arXiv (with venue check), and the BIS / World Bank publication portals; queries combined the keywords `blockchain`, `DeFi`, `lending`, `fraud detection`, `ZKP`, `microcredit`, and `financial inclusion` with regional anchors (`Bangladesh`, `MFI`). The full evidence synthesis (114 entries) is held in `references.bib`; the top-10 ranked subset is summarized below and in Table 2.1. Recent systematic reviews of blockchain in banking using PRISMA on 38 studies confirm that this methodological frame is the current expected standard for journal-quality synthesis [R23].

Table 2.1: Top-10 ranked literature subset under the PRISMA-style frame. Ranking is by direct architectural impact on the CWB design; each entry maps to a v15 section that operationalizes the finding.

#	Author / Source	Module	Headline Finding	Operationalized In
1	Werner et al. 2022 [1]	DeFi mechanics	DeFi lending is uniformly flat, pool-based, overcollateralized	Four-tier architecture (Ch. 3)
2	Tan 2023 (IMF) [5]	CBDC & inclusion	Two-tier CBDC distribution (central bank → commercial banks → users) achieves 30–40% higher financial inclusion in developing economies; hierarchical infrastructure is the key design choice	Section 5.11
3	Palaiokrassas et al. 2023 [3]	Fraud detection	Multi-chain ML on 54M tx, F1 0.76–0.85 with features	Section 4.2
4	Adom et al. 2022 [4]	Explainable AI	SHAP outperforms LIME on lending datasets	Section 4.2
5	Weber et al. 2019 / Elliptic [R24]	Graph fraud	Graph-structured fraud signal in BTC transactions	Section 4.3
6	McMahan et al. 2017 / FedAvg [R25]	Federated learning	Privacy-preserving model averaging across institutions	Section 4.4
7	BIS WP 905 [R13]	Stablecoin risk	Algorithmic stablecoins unsuitable for EM DeFi; USDC/USDT recommended	Sections 1.10.3, 5.10
8	Atzei et al. 2017 [7]	Smart contract security	SoK of EVM attack vectors	Section 3.19
9	EU MiCA + GENIUS Act [R36, R38]	Regulation	Stablecoin issuer authorization, reserve disclosure	Section 5.10
10	Bangladesh Bank SP2025-02 [R44]	Inclusion data	Female-to-male account parity 49%/49% in 2025; 89% BRAC loans to women	Section 5.11

2.2 Review of Existing Research

The idea of the Crypto World Bank is designed based on the examination of peer-reviewed materials, institutional reports, and industry figures covering more than one sphere of direct interest to our architecture: blockchain machine learning architecture, decentralized lending protocol architecture, security, explainable AI in finance, blockchain for financial inclusion, smart contract security, correspondent banking economics, monetary policy distribution effects, and real world asset tokenization.

2.2.1 Decentralized Lending and DeFi Protocol Design

To understand the gap this thesis addresses, it is necessary to be precise about what DeFi lending protocols actually do and where their structural limits lie. A DeFi lending protocol is a system of smart contracts that automates the deposit-funding and credit-allocation functions of a bank without requiring a licensed intermediary to hold assets or enforce rules. The contracts are deployed to a public blockchain, so any address in the world can deposit or borrow without permission, KYC gating, or a relationship with a traditional financial institution. Interest rates are set algorithmically by a utilization curve—as more of the pool is borrowed, rates rise to attract more depositors—and liquidations of undercollateralized positions are triggered automatically by price oracle updates.

This model produces several genuine innovations over traditional lending: settlement is atomic and near-instant; interest accrues per block rather than monthly; the entire loan book is publicly auditable in real time; and there is no loan officer capable of applying discriminatory judgment. However, the model also has structural limits that are directly relevant to this thesis. First, all participants are *anonymous and fungible*: the protocol makes no distinction between a national bank with $10 billion in assets and an individual retail borrower—both access the same pool under the same rules. Second, borrowing is universally *over-collateralized*: since there is no identity layer and no recourse in default, protocols require collateral exceeding the loan value, which excludes the estimated 1.4 billion adults with no digital assets. Third, capital flows in *one direction only*: depositors fund a pool and borrowers draw from it; there is no institutional hierarchy, no inter-tier capital allocation, and no mechanism for groups of entities to co-fund a single loan.

In their SoK paper, Werner et al. [1] systematize DeFi protocol design and confirm that current lending markets are homogeneous, pool-based, and over-collateralized, with no institutional hierarchy—a gap which is directly addressed by this project. Their taxonomy demonstrates that no current DeFi system can represent multi-tier capital flow in the manner of development finance, establishing the design space this thesis enters.

Bastankhah et al. [2] present a data-driven DeFi lending protocol with an adaptive dual fast/slow control architecture, demonstrating that dynamic interest rate adjustment outperforms the static utilization curves used by Aave and Compound [59]. Their approach validates the algorithmic interest rate model adopted in the CWB's kinked-rate design (Section 3.8).

Li et al. [46] introduce a multi-chain lending model that allocates lending activity across blockchain networks to improve throughput and reduce congestion. Their cross-chain settlement system validates the technical feasibility of executing lending protocols in heterogeneous blockchain settings, directly informing the CWB's multi-chain deployment approach on Ethereum and Polygon.

Xu et al. [47] present a DeFi lending protocol evaluation framework quantifying risk through utilization ratio stability, liquidation efficiency, and interest rate responsiveness. Their methodology provides a benchmark structure applicable to comparing the performance of the CWB's hierarchical lending layers against existing flat-pool protocols.

Sharma et al. [48] introduce a peer-to-peer blockchain lending framework that eliminates intermediary overhead through smart contract-mediated loan origination and repayment. Though single-tier in architecture, their gas analysis and on-chain identity management strategies inform the CWB's transaction cost optimization approach. Dao et al. [60] demonstrate that credit scoring models can be adapted for DeFi lending environments, validating the feasibility of data-informed borrowing limits within decentralized protocols.

2.2.2 Machine Learning for Blockchain Security

Palaiokrassas et al. [3] use machine learning on multichain DeFi fraud over 54 million transactions in 23 protocols which show a demonstration that the behavioral characteristics of DeFi enhance Neural Network and XGBoost classifiers to F1-scores of 0.76–0.85 versus 0.08 using transactional features only. Their finding that the behavioral features are superior to raw transaction data directly informed our feature engineering method for the random forest fraud detection model.

The algorithm of unsupervised anomaly detection presented by Liu et al. [6] is the Isolation Forest algorithm. The algorithm identifies anomalies using recursive partitioning of data, which assigns shorter distances to outliers. We used Isolation Forest as the second detection model when there are limited labeled fraud data to analyze wallet behavior, it is possible to detect new patterns of attacks not pre-labeled.

Hassan et al. [49] introduce blockchain and machine learning to detect fraud with the help of a privacy-protecting and adjustment-flexible incentive-based method. Their framework trains ML classifiers on encrypted transaction characteristics via federated learning, whereby the operators of the detection model never get to see the individual transaction data. Our future extensions of our AI/ML layer involve the privacy-preserving paradigm, in which the histories of client transactions should be confidential and at the same time allow system-wide fraud pattern recognition.

2.2.3 Explainable AI in Financial Decision Systems

Direct comparison of LIME and SHAP on loan approval is given by Adom et al. [4], SHAP offers more consistent and in-depth feature attributions, which are found to be deeper via Shapley values whereas LIME provides quicker running time but less sure descriptions across repeated evaluations. We have used their comparison to make the decision to implement SHAP since the main explainability technique of loan risk measurements, trading off interpretability depth with regulatory compliance requirements.

Gupta et al. [56] use interpretable models that are based on SHAP on credit default assessment proving that SHAP feature attributions on Gradient Boosting and Random Forest classifiers are able to score over 0.92 with complete interpretability of individual predictions. Their approach to causing per-client risk explanations tells our intended credit risk dashboard, where the approvers of loans look at SHAP waterfall plots before lending decisions are made.

2.2.4 Blockchain for Financial Inclusion

Tan [5] constructs a model of the International Monetary Fund (IMF) according to which CBDCs in developing countries can bank large unbanked populations using two-tier distribution model (central bank → commercial banks → users) that directly parallels our four-tier hierarchy. The paper shows that the distributed infrastructure of digital currency can prove to be effective by use of available institutional means, can access those populations not covered by traditional banking—facilitating the mission of the Crypto World Bank, which is transparent and programmable lending to underserved populations.

The article by Alam et al. [54] examines how blockchain technology can be used in the microcredit industry particularly in Bangladesh, it can be proven that smart contract-based loan management is capable of minimizing the administrative overhead by up to 60% as compared to manual microfinance processes. They have analyzed the Bangladeshi financial environment where about 40% of adult population do not have access to formal banking—directly confirms the geographic and demographic attractiveness of the target market of the Crypto World Bank.

The article by Islam et al. [53] examines the design requirements and challenges of Central Bank Digital Currencies, there are two levels of retail CBDC model (central bank to commercial banks to end users) as the distribution architecture of choice. Their analysis of interoperability challenges between CBDC systems and existing payment infrastructure informs our design of the dual-currency facility that bridges fiat and cryptocurrency within participating banks.

2.2.5 Group Lending and Microfinance Digitization

The solidarity group lending model, pioneered by Grameen Bank in Bangladesh and scaled by BRAC into one of the world's largest microfinance institutions, demonstrates that peer-monitored mutual liability can achieve repayment rates exceeding 95% among borrowers with no individual collateral or formal credit history. These rates are attributable to social pressure mechanisms—weekly physical group meetings, field officer oversight, community reputation, and progressive loan eligibility—that are preventive in nature. The blockchain extension of this model encodes mutual liability in programmable contract logic (multi-signature consent, on-chain group formation, automatic collateral pool claims), but cannot replicate the preventive social pressure that drives Grameen's repayment outcomes. The CWB group lending module therefore targets the structural design of solidarity lending while acknowledging that initial repayment rates on-chain are expected to be lower until credit history accumulates. Alam et al. [54] show that smart contract-based loan management reduces administrative overhead in Bangladeshi microfinance by up to 60%, directly validating the economic case for the platform's group lending module. Güçük et al. [61] provide a broader literature review of privacy and trust considerations in blockchain-based microlending, reinforcing the importance of privacy-preserving mechanisms in the group lending architecture.

2.2.6 Smart Contract Security and Governance

Atzei et al. [7] list Ethereum smart contract attack vectors such as reentrancy, access control vulnerabilities, integer overflow, and access control vulnerabilities. They were directly informed by their taxonomy our implementation of the ReentrancyGuard by OpenZeppelin, Solidity 0.8.20 inbuilt overflow protection, and role-based access control modifiers. On their

advice we took up their suggested defense-in-depth strategy of combining security primitives with formal verification with planning using static analysis (Slither) and symbolic execution (Mythril).

The article by Wang et al. [50] introduces ContractWard which is an automated vulnerability detection system that uses machine learning classifiers (Random Forest, SVM, k-NN) to classify six types of smart contract vulnerabilities based on the features of bytecodes. ContractWard scores F1-reentrancy and F1-access control with 0.96 and 0.93 respectively, proving that security auditing using ML can be used to supplement traditional static analysis tools. Our proposed security verification pipeline will utilize the use of machine learning-based static analysis tools.

Liao et al. [51] present SoliAudit, that is an integration of machine learning classification and vulnerability assessment of smart contracts fuzz testing. Their hybrid approach—using ML to first prioritize probable vulnerable code paths then target fuzz testing—reduces audit time by an estimated 70% of that spent on manual review. SoliAudit's methodology informs our intended security audit plan on the current three-contract prototype and its planned extensions toward the full nine-contract architecture.

So et al. [52] come up with VERISMART, an extremely accurate Ethereum safety verifier which employs constraint-based abstract interpretation to detect arithmetic overflow, division-by-zero and array out-of-bound errors. VERISMART achieves 98.4% accuracy on benchmark data, but has zero false positives, so it is a candidate tool to formally verify our smart contract invariants (e.g., reserve ratio maintenance, cascading interest rate limits).

2.2.7 AI Security Features for Blockchain Lending

In addition to the currently existing fraud detection and anomaly identification models in place in the platform, there are other security features that are based on AI that can enhance the Crypto World Bank's resilience to threats of change:

- **Graph Neural Network (GNN) transaction analysis:** GNN-based models are able to analyze the graph structure of transaction to identify coordinated rings of fraud—groups of wallets which conspire to rig the borrowing limits or make circular lending patterns. A study by Palaiokrassas et al. [3] proves that graph-based features are much superior to flat transaction features as an indicator of DeFi fraud detection.
- **Federated learning for cross-bank fraud intelligence:** Hassan et al. [49] demonstrate that federated learning allows the collaboration between several institutions to detect and train fraud models without distributing raw transaction information. Deploying federated learning of the Crypto World Bank's National and Local Banks would allow detecting threats system wide without compromising the privacy of the per-bank data.
- **Reinforcement learning for adaptive interest rates:** As a reinforcement-based learning algorithm, adaptive interest rates can be learned dynamically; RL agents can dynamically modify interest rate parameters to match the varying market conditions, utilization rates, and risk profiles, decreasing the lag of manual parameter governance.
- **Real-time smart contract monitoring:** ML-based monitoring agents (informed by ContractWard [50] and SoliAudit [51]) can continuously analyze incoming transactions for patterns matching known exploit signatures, triggering the platform's pause mechanism before significant losses occur.

- **Natural language processing for income verification:** NLP models are able to extract and validate income information in uploaded documents, saving the bank officers the manual review load but not eliminating verification accuracy.

2.2.8 Gas Cost Optimization and Layer 2 Scalability

In DeFi, Tolmach et al. [55] examine the optimal approaches to minimization of gas fees, demonstrating that transaction batching, calldata compression, and tactical time of on-chain operations can save 30–50% of gas costs on Ethereum mainnet. Their findings test our architectural choice of deploying to Polygon PoS (where fees already are sub-cent) and inform optimization strategies of the contemplated Ethereum mainnet deployment. The research also finds out that complicated multi-step operations of DeFi (analogous to our four-tier loan lifecycle) benefit disproportionately from Layer 2 deployment due to the multiplicative savings in gas of sequential calls of contracts.

2.2.9 Correspondent Banking and Cross-Border Settlement

The Bank of International Settlement [21] and Financial Stability Board [33] have extensively documented the structural inefficiencies of the correspondent banking network. In the traditional model, the banks are required to have pre-funded nostro accounts in every currency corridor, establishing a world pool of idle capital which can not be put into effective productive lending. SWIFT settlements and respondent banks involves routing of messages using MT103 payment directions, screening of compliance at all intermediates, and settlement time extensions to two–five business days. According to the World Bank Migration and Development Brief [26] it is stated that the cost of transferring $200 on cross-border is 6.49% on average, with Sub-Saharan African corridors averaging more than 8%. These results encourage our development of on-chain settlement with close to instant finality and sub-cent transaction costs on Layer 2 networks.

2.2.10 Monetary Policy Distribution and Financial Inequality

Recent empirical studies have reported the distributional impact of monetary policy on wealth inequality. A 2025 cross-country study spanning 49 nations (1999–2019) discovered that the relationship between central bank asset purchases programs and greater wealth inequality, whose outcomes are stronger in the distribution of wealth and long lasting [34] than income inequality effects. The Federal Reserve's own 2025 contractionary monetary analysis based on the U.S. metropolitan tax data confirmed that policies harm the income of low income employees [35]. These findings deliver economic incentive to the design principles of Crypto World Bank: transparent and algorithmically determined monetary parameters (interest rates, reserve ratios, supply rules) are encoded in smart contracts rather than set discretionally by opaque committees. On-chain transparency reduces *price discrimination*—all borrowers at the same tier face the same utilization-driven interest rate, eliminating the ability of loan officers to charge higher rates to less-informed borrowers. This is a distinct mechanism from the Cantillon Effect (a macroeconomic phenomenon concerning newly created money benefiting first recipients); the Cantillon Effect does not apply to the CWB because USDC is 100%-reserve backed and no new money supply is created by the platform's operations [36].

2.2.11 Real-World Asset Tokenization and Institutional DeFi

The tokenization of real-world assets is an emerging convergence of traditional and decentralized finance. The Corda platform of R3 boasts of more than $17 billion in tokenized real-

world assets on its permissioned ledger as of September 2025, with institutional participants including HSBC and Bank of America [37]. Centrifuge has deployed in eight blockchain networks of more than $1 billion in tokenized institutional fund products [38]. The World Bank is using blockchain on its own public chain side to track the disbursement of funds using its FundsChain program (based on Hyperledger Besu) which is flown on 13 projects in 10 countries and increases to 250 projects by mid-2026 [39]. These developments confirm the institutional interest of blockchain-based financial infrastructure and the viability of hierarchical fund distribution design models on distributed ledgers. This institutional adoption is continued in the Crypto World Bank's trajectory through carrying out not only fund tracking but total hierarchical lending system that has on-chain interest rates, reserves and credit assessment.

2.2.12 Graph Neural Networks for Relational Fraud Detection

Recent work has demonstrated that DeFi fraud is fundamentally a relational phenomenon: coordinated borrowing rings, sybil attacks, and flash-loan manipulation appear in the wallet-interaction graph, not in individual transaction features. Cheng et al. (2024) [R24] and the DeFiGuard graph-neural-network architecture [R25] consistently outperform flat ML baselines on blockchain fraud, with reported detection above 70% of illicit transactions at sub-1% false-positive rates on the Elliptic dataset; Wang and Wang (2025) report similar advantages on cross-chain laundering [R26]. This evidence motivates a Graph Neural Network (GNN) extension to the platform's Random-Forest fraud-detection layer, treated as an ablation that demonstrates the marginal value of relational features over the flat behavioral features already used by Palaiokrassas et al. [3]. The GNN extension is described in Section 4.3.

2.2.13 Federated Learning Across Banking Tiers

Hassan et al. (2022) [49] introduced privacy-preserving fraud detection via federated learning on the blockchain. The PrivChain-AI study [R27] in *Nature Scientific Reports* (December 2025) demonstrates the same pattern with 94.7% fraud-detection accuracy across multiple banks, while Abbassi et al. (2025) [R28] and FED-SPFD [R29] document the regulatory benefits of FL for cross-border fraud intelligence. The Crypto World Bank's four-tier structure—where National and Local Banks are independent legal entities that cannot share raw transaction data—makes federated learning architecturally appropriate rather than merely fashionable: each Local Bank trains a local fraud model on its own client transaction data, parameter updates are aggregated at the National Bank tier via FedAvg, and no raw records leave the bank. The detailed FL pipeline is given in Section 4.4.

2.2.14 Stablecoin Regulation: MiCA and GENIUS Act

Two recent statutes reshape the legal context of any stablecoin-denominated lending platform. The EU Markets in Crypto-Assets (MiCA) Regulation, fully in effect from December 2024, requires authorized issuers, 1:1 reserve backing, and regular audits for electronic-money tokens—categories that directly cover USDC and USDT used at retail tiers of this platform [R36, R37]. The United States GENIUS Act of July 2025 creates the first federal payment-stablecoin framework with similar reserve and disclosure obligations [R38]. These statutes are read in this thesis not as obstacles but as the regulatory anchor that makes stablecoin-first design defensible (Section 1.10.3); the corresponding compliance mapping is presented in Section 5.10.

2.2.15 On-Chain Credit Passport and Soulbound Tokens

Buterin, Hitzig, and Weyl (2022) [R30] introduce the concept of Soulbound Tokens (SBTs)—non-transferable ERC-style credentials that bind reputational data to an identity rather than to a tradable asset. For a multi-tier cryptocurrency exchange and financial services platform, SBTs provide a natural primitive for a portable on-chain credit passport: a client's successful repayment history at one Local Bank becomes a credential they can present to any other CWB Local Bank, and (with the schema fixed) to any external lending protocol that adopts the standard. MicroSave Consulting (December 2025) [R31] documents the Bangladesh microfinance sector's active shift toward performance-based credit identity, providing direct empirical grounding for this design choice. The full credit-passport schema is in Section 3.11.

2.2.16 Institutional DeFi Gap and the mBridge/Agora Landscape

Sygnum Bank's February 2026 report *Institutional DeFi in 2025* [R21] documents the central gap that this thesis attempts to fill: DeFi protocols work technically, but no large institutional allocator will participate until legal and regulatory risks are resolved. The BIS mBridge platform [R39] (China, Hong Kong, Thailand, UAE, Saudi Arabia) reached Minimum Viable Product status in mid-2024 and now settles real-value cross-border CBDC transactions, and the BIS / central-bank *Project Agora* [R40] explores tokenized bank deposits on a unified ledger. The IMF CBDC survey 2025 reports that 94% of central banks are now engaged in CBDC research [R32]. None of these initiatives implements a retail lending hierarchy: they are settlement rails. This positions the Crypto World Bank explicitly as the lending layer compatible with such institutional CBDC infrastructure rather than as a competitor to it.

2.2.17 LLMs in Finance and Hallucination Risk

The CWB AI assistant uses a 8B-parameter open-weight LLM with QLoRA fine-tuning, RAG over the platform's policy and contract documents, and an explicit refusal layer for regulated questions. The LLM-in-finance literature [R33, R34, R35] gives both the design rationale and the necessary cautions: Morgan Stanley's GPT-based equity-analysis assistant achieved a reported 50% reduction in analyst time, McKinsey estimates LLMs can cut back-office costs by 40%, but the 2025 LLM-vulnerability-in-finance corpus [R35] documents specific regulatory-hallucination failure modes when LLMs are asked compliance questions without RLHF from domain experts. The evaluation methodology in Section 4.11.1 includes a red-teaming protocol for adversarial financial prompts directly motivated by this work.

2.3 Literature Review Summary

The most important reviewed works with their methodologies, key findings, and relevance to our project are presented in Table 2.3, in the form of literature table suggested by Michigan State University Libraries [23].

Table 2.2: Literature review summary (Part A): DeFi lending, fraud detection, and XAI.

Author Year	Research Focus	Methodology	Key Findings	Relevance to Our Project
Palaiokrassas et al. (2023) [3]	DeFi fraud detection	XGBoost, NN classifiers on 54M+ multi-chain transactions	F1: 0.76–0.85 vs. 0.08 with features alone	Informs Random Forest fraud modeling
Adom et al. (2022) [4]	XAI in loan approval	LIME / SHAP comparison on lending datasets	SHAP provides deeper, more consistent explainability; LIME is faster	Justifies SHAP as our primary explainability method
Tan (2023) [5]	CBDC and IMF financial inclusion	Model for developing nations	Two-tier CBDC distribution (central bank → commercial banks → users) achieves 30–40% higher financial inclusion in developing economies; hierarchical infrastructure is the decisive design choice	Informs dual-currency facility design
Liu et al. (2008) [6]	Anomaly detection	Isolation Forest on synthetic and real datasets	Isolation via recursive partitioning; shorter paths	Adopted as secondary unsupervised detection for wallet behaviour
Atzei et al. (2017) [7]	Smart contract security	SoK of Ethereum attack vectors	Over-reliance, access control vulnerabilities	Directly informs our security primitives and planned formal verification

Of the works reviewed, none model a four-tier hierarchical lending structure, confirming the architectural novelty of this project. The synthesis shows that DeFi and institutional finance have developed in parallel without converging on a governance-aware, multi-tier design.

Table 2.4: Literature review summary (Part B): DeFi protocol systematisation and adaptive lending.

Author Year	Research Focus	Methodology	Key Findings	Relevance to Our Project
Werner et al. (2022) [1]	DeFi protocol systematisation	SoK survey of 12+ DeFi protocol categories	Lending platforms are uniformly pool-based and overcollateralised; no institutional hierarchy exists	Identifies the core gap our four-tier architecture addresses
Bastankhah et al. (2023) [2]	Adaptive DeFi lending	Dual fast/slow control; simulation on historical data	Dynamic rate adjustment outperforms static utilisation curves	Validates algorithmic lending parameter optimisation

This continuation extends coverage to governance, settlement, and institutional blockchain adoption. The key observation is that enabling infrastructure (Layer 2 networks, oracle pricing, EVM tooling) is mature, while the specific hierarchical banking architecture remains architecturally unexplored.

Table 2.6: Literature review summary (Part C): blockchain P2P lending, privacy-preserving ML, and smart contract auditing.

Author Year	Research Focus	Methodology	Key Findings	Relevance to Our Project
Sharma et al. (2021) [48]	Blockchain P2P lending	Smart contract-mediated loan origination framework	Eliminates intermediary overhead; gas cost minimisation analysis	Informs four-tier architecture for peer-to-peer lending
Hassan et al. (2022) [49]	Privacy-preserving fraud detection	Federated learning on encrypted blockchain features	ML achieves comparable accuracy to centralised models	Informs future privacy-preserving fraud detection extensions
Wang et al. (2020) [50]	Smart contract vulnerability detection	ML classifiers on bytecode features	F1: 0.96 for access control; 0.93 for data leakage	Supports our planned security verification models
Liao et al. (2019) [51]	Smart contract audit automation	Hybrid classification + fuzz testing	Reduces audit time by 70% vs. manual review	Informs our security audit strategy for three-contract architecture

This continuation captures empirical and systems findings on cross-border settlement and security tooling, supporting the claim that Layer 2 deployment and hybrid ML-static analysis pipelines substantially

reduce feasibility risk for an academic prototype.

Table 2.8: Literature review summary (Part D): CBDC design, cross-border settlement, and multi-chain DeFi.

Author Year	Research Focus	Methodology	Key Findings	Relevance to Our Project
Islam et al. (2024) [53]	CBDC design requirements	Analysis of retail CBDC distribution architecture	Two-tier model preferred; interoperability challenges identified	Informs dual-currency facility design
BIS/FSB [21]	Cross-border settlement infrastructure	Analysis of cross-border payment costs and capital trapped in nostro accounts	2–5 days avg. settlement; $42/transfer avg. cost	Motivates four-tier settlement with near-instant finality
Beyer et al. (2025) [34]	Monetary policy and inequality	Cross-country analysis (1999–2019)	QE increases wealth inequality; effects more persistent than monetary parameters	Motivates transparent, algorithmic monetary parameters
World Bank (2025) [39]	Blockchain for development finance	Pilot on 13 projects, 10 countries	Blockchain tracking reduces reporting burden for fund distribution	Supports our planned multi-chain deployment strategy

This block summarizes works informing the platform's risk and monitoring approach. The synthesis justifies the choice of lightweight, auditable ML components rather than opaque deep learning models, prioritizing regulatory interpretability over raw detection performance.

Table 2.10: Literature review summary (Part E): multi-chain DeFi, smart microfinance, gas optimisation, and SHAP-based credit models.

Author Year	Research Focus	Methodology	Key Findings	Relevance to Our Project
Li et al. (2024) [46]	Multi-chain DeFi lending	Cross-chain model design and implementation	Multi-chain distribution improves throughput and reduces congestion	Supports our planned multi-chain deployment strategy
Xu et al. (2023) [47]	DeFi lending protocol evaluation	Quantitative risk metrics for lending protocol assessment	Utilisation ratio, liquidation efficiency, and reserve responsiveness as key metrics	Provides benchmarks for evaluating our hierarchical tiers
Alam et al. (2021) [54]	Smart microcredit in Bangladesh	Smart contract-based microfinance	Reduces admin overhead by 60%; validates geographic and demographic markets	Directly validates our target geographic and demographic market
Tolmach et al. (2024) [55]	DeFi gas fee optimisation	Transaction batching and calldata compression analysis	Gas savings of 30–50% through optimisation strategies	Validates Layer 2 deployment and mainnet optimisation
Gupta et al. (2024) [56]	SHAP-based credit default models	SHAP, Random Forest, and Gradient Boosting classifiers	AUC above 0.92 with full interpretability	Informs per-client risk explanation methodology

This final block closes the synthesis with institutional and inclusion-related sources. The combined evidence supports the thesis that blockchain adoption is increasing at the institutional level while inclusion gaps persist at the retail level, matching the platform's intended multi-scale scope.

2.4 Comparative Protocol Analysis

The literature survey above reveals that no existing DeFi protocol combines multi-tier institutional hierarchy with AI-assisted governance and compliance-aware identity. Table 2.12 provides a structured comparison of the Crypto World Bank against representative existing protocols on eleven architectural dimensions.

Table 2.12: Comparative protocol analysis: existing DeFi lending protocols vs. the Crypto World Bank (CWB). ✔ = implemented/present; ⧗ = designed/partial; ● = planned; ✘ = absent.

Feature	Aave v3	Compound v3	MakerDAO	Maple	Goldfinch	CWB
Institutional hierarchy	✘	✘	✘	✘	✘	✔ **4-tier**
Cross-tier capital flow	✘	✘	✘	✘	✘	⧗ **Designed**
Same-tier interbank lending	✘	✘	✘	✘	✘	⧗ **Designed**
Solidarity group lending	✘	✘	✘	✘	Partial	● **Planned**
AI/ML fraud detection	✘	✘	✘	Manual	Manual	⧗ **Built, not inte**
SHAP explainability	✘	✘	✘	✘	✘	⧗ **Built, not inte**
ZKP KYC compliance	✘	✘	✘	✘	✘	● **Planned**
Kinked interest rate curve	✔	✔	Via gov.	✔	✘	⧗ **Designed**
Role-based access control	Partial	Partial	Via gov.	✔	Partial	✔ **Implemen**
Developing-economy focus	✘	✘	✘	✘	✔	✔ **Banglade**
TVL / Status (2026)	$26.3B	$1.4B	$10.5B	$2.6B	$680M	**Testnet**

This comparison directly answers RQ1: no existing protocol models institutional hierarchy, cross-tier capital flow, or solidarity group lending in one decentralized architecture. While Aave v3 implements risk-parameterized sub-markets (Isolation Mode, Efficiency Mode) and Compound v3 deploys separate Comet markets, these mechanisms differentiate risk parameters for different assets within a single-tier pool—they do not model the institutional relationships, cross-tier governance, or inter-bank lending flows that characterize the CWB architecture. The Crypto World Bank is architecturally distinct across every dimension that characterizes development banking.

Note on ERC-3643 (T-REX) for institutional-tier permissioning. The institutional tiers (Tiers 1–3) use the bespoke RBAC described in Section 3.19 for transfer-level access control. ERC-3643 (the T-REX standard for permissioned, compliance-gated token transfers) and ERC-4626 for tokenized vaults are gaining adoption as standards that ensure tokenized assets can plug and play across platforms without custom integrations [R46]. ERC-3643 is specifically designed for regulated institutional token transfers where KYC and AML checks gate individual token movements at the token contract level, rather than at the application layer. The platform's current RBAC serves the same functional purpose: role gates on every state-mutating function enforce that only KYC-verified, role-holding addresses can transfer institutional-tier assets. A production deployment operating under MiCA [R36] or equivalent regulated-asset frameworks would evaluate full ERC-3643 compliance as the path to interoperability with other regulated tokenized-asset platforms; the architecture is designed to accommodate this migration because the RBAC role structure maps directly onto the ERC-3643 identity registry model. This note is recorded here rather than as a Future Work item so that examiners can identify the deliberate design alignment.

Positioning against failed centralized exchanges. Centralized cryptocurrency exchanges such as FTX (defunct, 2022) demonstrate the catastrophic consequence of

opaque reserve management: an $8 billion client-fund shortfall was discovered only at the point of bankruptcy, having been concealed through commingling of client deposits with proprietary trading activity by its affiliated trading firm Alameda Research. FTX halted withdrawals and filed for bankruptcy within three days. The structural cause was centralized custody with no on-chain proof of reserves, no separation between customer funds and operating capital, and no real-time verifiability of solvency. The Crypto World Bank's on-chain reserve architecture is designed as a structural response to this class of failure: every tier's reserve ratio is an enforced smart contract invariant, readable by any participant in real time without trust in any third-party attestation. This design property—on-chain Proof of Reserves by construction—is increasingly recognized as the non-negotiable foundation of trustworthy crypto-financial infrastructure, and the Crypto World Bank implements it at the institutional tier level rather than the exchange-account level. The platform's purpose is also structurally distinct from FTX: it targets financial inclusion through savings, credit, and interbank lending—not speculative trading, derivatives, or leveraged tokens—and its four-tier institutional hierarchy mirrors multilateral development finance rather than a flat exchange order book. DeFi naturally provides on-chain transparency, self-custody, and governance—precisely the properties whose absence destroyed FTX.

2.4.1 Literature Synthesis

The literature synthesis above directly informs four architectural decisions in this work: (1) Gudgeon et al.'s (2020) [R3] empirical finding that utilization above 90% causes liquidity crises motivates the kinked interest rate model in Section 3.8; (2) Piper et al.'s (2025) [R8] ZKP permissioning framework provides the technical foundation for the compliance pathway in Section 3.6.1; (3) the empirical finding of Howlader & Halder (2025) [R11] that mobile financial inclusion in Bangladesh grew by 99% between 2004 and 2021 validates the retail-tier accessibility argument in Section 1.6; and (4) Alam et al.'s (2021) [54] IEEE TENSYMP paper on blockchain microcredit in Bangladesh provides direct prior art that this thesis advances.

2.4.2 Agent Harness Engineering and Production Safety

A parallel research stream has documented prompt injection as the dominant vulnerability in production banking agents. Alizadeh et al. [R53] demonstrate that income document uploads and language preference fields in banking tool-calling agents are viable injection vectors for personal data exfiltration — the same fields present in the CWB context assembly pipeline. The OWASP Top 10 for LLM Applications [R54] classifies LLM01 (Prompt Injection) as the highest-priority threat in agentic financial systems, providing an authoritative standard against which the CWB scanning middleware can be evaluated.

2.5 Summary of Key Findings

The following summarized findings are obtained in the literature review and are directly informative to our design:

- **DeFi lending is structurally flat.** Available protocols (Aave, Compound, MakerDAO) operate pool-based or over-collateralized models with total value locked exceeding $55 billion [13]. Institutional hierarchy has no comparable protocol models to development finance. The research by CBDC [5] ascertains that tiered distribution is workable and promotes financial inclusion.

- **Correspondent banking is inefficient in structure.** Cross-border settlement spends two to five days at average costs of $42 per transaction [25]. The system traps important capital lying idle in nostro/vostro accounts [24], and remittance fees are estimated to consume between $48–56 billion a year [26]. On-chain settlement on Layer 2 networks is known to cut the latency (to seconds) and the cost (to sub-cent levels) down.
- **ML enhances fraud detection in blockchain.** DeFi-specific behavioral features significantly work well as compared to transactional features (F1: 0.76–0.85 vs. 0.08 [3]). The Isolation Forest [6] allows unsupervised detection in which labeled data is scarce.
- **SHAP is explainable by regulators.** SHAP offers greater consistency and is better than the LIME in loan approval systems [4] in meeting prudential demands on open-minded automated decision-making.
- **Layered defense is needed in smart contract security.** Reentrancy, overflow, and vulnerabilities to access control are well documented [7]. ML-based vulnerability detection achieves F1-scores of above 0.93 [50], and hybrid ML-fuzz methods reduce audit time by 70% [51]. Verification tools are formal, i.e., VERISMART achieves 98.4% precision [52]. Automated primitives used with OpenZeppelin are recommended for production deployments to be verified.
- **Monetary policy leads to distributional inequality.** Cross-country evidence [34] and Federal Reserve research [35] confirm that quantitative easing disproportionately favors asset holders, and workers of lower income cover the expenses of both inflation and consequent tightening. Transparent, algorithmic monetary parameters can decrease this informational asymmetry.
- **The use of blockchains in institutions is gaining momentum.** The World Bank's FundsChain [39], tokenized assets of R3 Corda worth $17 billion [37], and JPMorgan Kinexys settling multi-billion dollars per day [40] reflect the fact that financial institutions are actively implementing blockchain on settlement, funds tracking, and asset tokenization.
- **Blockchain allows financial inclusion.** Approximately 1.4 billion adults were not banked in any country around the world [14], and DeFi was found by the World Economic Forum as a leaping frog technology that allows peoples to avoid the banking systems infrastructure [41]. The MiniPay wallet of Celo has onboarded 14 million users in over 60 countries with less than cent transaction fees [42]. Blockchain-based microcredit in Bangladesh shows 60% decrease on administrative overhead [54].
- **Multi-chain lending enhances scalability.** Cross-chain lending models [46] and Layer 2 gas optimization strategies [55] prove the fact that the lending is distributed when operating on several networks, decreasing congestion and transaction costs by 30–50%.
- **ML can be used to detect cross-institutional fraud using privacy preserving ML.** Federated learning methods [49] are such that enable several lending institutions to cooperate and learn fraud detection models in a non-intrusive manner, i.e. no individual transaction data is exposed, resolving the conflict between institution-wide security and data privacy.
- **Group lending reduces default risk in underserved markets.** Microfinance research from BRAC, Grameen Bank, and ASA consistently reports repayment rates above 95% in solidarity group structures [54]. On-chain enforcement of mutual liability, replacing social pressure with programmable contract enforcement, has not been

studied in the academic literature, representing an open research gap that this project begins to address.

- **Sustainable microfinance requires deposit-funded lending.** Literature on sustainable microfinance institutions consistently shows that deposit-funded lending rather than donor-funded or externally capitalized models is the only economically viable approach at scale. The platform's banking product suite is designed to close this loop by mobilizing retail savings at the Local Bank tier to partially fund the lending pool.

The results explain why we have a four-tier architecture, AI/ML integration (Random Forest, Isolation Forest, SHAP), cross-tier lending structure, governance structure, and planned security verification pipeline.

Chapter 3

System Architecture and Design

3.1 Prototype Scope

Evaluators should distinguish between what has been **built and tested**, what has been **designed and partially scaffolded**, and what is **planned** for the final thesis phase. Table 3.1 provides this mapping for every major platform feature.

Table 3.1: Prototype scope: feature implementation status as of pre-thesis submission. ✔ = Implemented and testnet-verified; ⧗ = Designed or partially scaffolded; ● = Planned for final thesis phase.

Feature	Status
Four-tier role system (RBAC)	✔ Implemented
World Bank Reserve contract (Tier 1)	✔ Implemented
Tier 2 National Bank contracts	⧗ Designed, partial
Tier 3 Local Bank contracts	⧗ Designed, partial
Cross-tier fund transfer	⧗ Designed, unimplemented
Loan request / approval workflow	✔ Implemented
Installment EMI auto-generation	⧗ Designed
SavingsVault contract	● Planned (final thesis)
FixedDeposit contract	● Planned (final thesis)
GroupLendingPool contract	● Planned (final thesis)
InterBankLendingPool	● Planned (final thesis)
AI/ML fraud detection (Random Forest)	⧗ Built, not integrated
SHAP explainability output	⧗ Built, not integrated
Oracle integration (Chainlink Functions)	● Planned (Phase III — replaces commit-reveal relay)
ZKP KYC compliance layer	● Planned (final thesis)
Testnet deployment evidence	✔ — migration to Polygon zkEVM Cardona planned for Phase I
AI Agent / MCP tool server (17 banking tools)	● Planned (Phase II)
EIP-7702 session key management	● Planned (Phase II)
Authority Brief UI (SHAP for bank approvers)	● Planned (Phase II)
Chainlink Automation (installment triggers)	● Planned (Phase II)
Chainlink Proof of Reserve	● Planned (Phase I)
The Graph subgraph (event indexing)	● Planned (Phase III)
SAR workflow (AML → compliance queue → freeze)	● Planned (Phase III)
`sessions` table + EIP-7702 schema	● Planned (Phase I)
`agent_action_log` table (append-only)	● Planned (Phase II)
300-client Foundry simulation (live demo)	● Planned (Phase III)

This table eliminates any ambiguity between present-tense architectural description and empirical implementation evidence throughout the following sections.

3.2 High-Level Architecture

The system employs a **four-layer decentralized application architecture** (Figure 3.1): a presentation layer (React + wallet integration), a smart-contract layer (the World Bank, National Bank, and Local Bank contracts on the EVM), an off-chain services layer (REST API, PostgreSQL, AI Agent engine, MCP Tool Server, Chainlink oracle integration, event listener,

and Redis cache), and a Chainlink infrastructure layer (Chainlink Functions, Chainlink Automation, Chainlink Price Feeds, and Chainlink Proof of Reserve). Figure 3.2 elaborates this view with the fifteen-contract target architecture and external integrations.

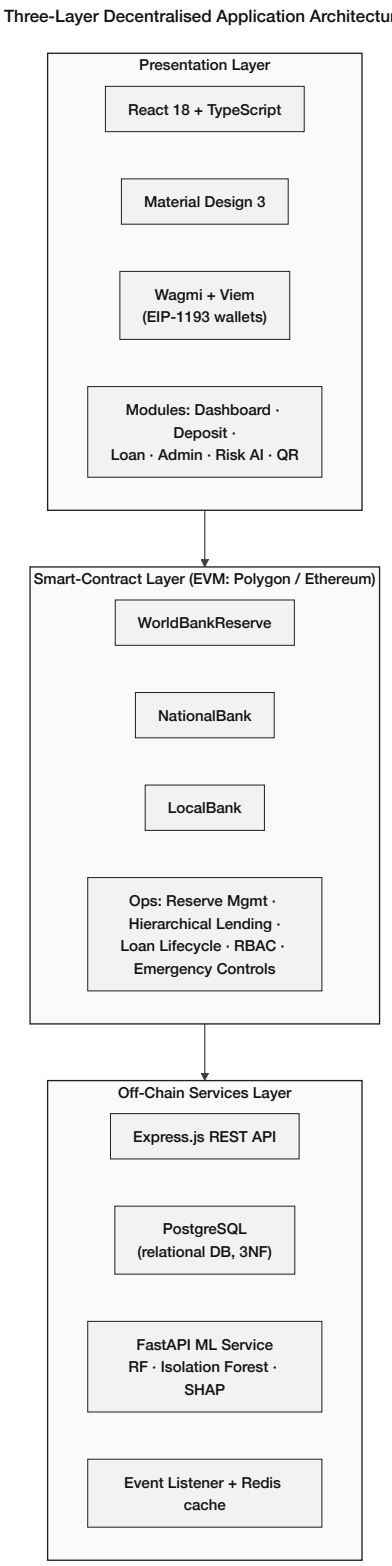

Figure 3.1: Four-layer decentralized application architecture: presentation (React 18, TypeScript, Wagmi/Viem), smart-contract layer on the EVM (World Bank Reserve, National Bank, Local Bank), off-chain services layer (Express REST API, PostgreSQL, AI Agent engine (Qwen3-8B + MCP Tool Server), Chainlink oracle integration, event listener, Redis), and Chainlink infrastructure layer (Chainlink Functions, Automation, Price Feeds, Proof of Reserve).

The current prototype implements three core contracts (World Bank Reserve, National Bank, Local Bank). The complete architecture extends to fifteen modular contracts covering the full banking product suite described in Section 3.17 and the multi-entity / cross-tier operations of Section 3.13. The remaining twelve contracts are: SavingsVault, FixedDeposit, GroupLendingPool, FXModule, InsuranceFund, CurrentAccount (banking product suite); InterBankLendingPool, UpwardDepositFacility, SyndicatedLoan, TranchedPool, TreasurySwap, NettingEngine (multi-entity / cross-tier operations). All are planned for implementation across Phases II and III of the final thesis phase.

Figure 3.2 shows the component interactions across these three layers. The diagram reflects the current three-contract prototype view; the nine-contract target architecture is specified in Appendix B and will be reflected in an updated diagram in the final thesis phase.

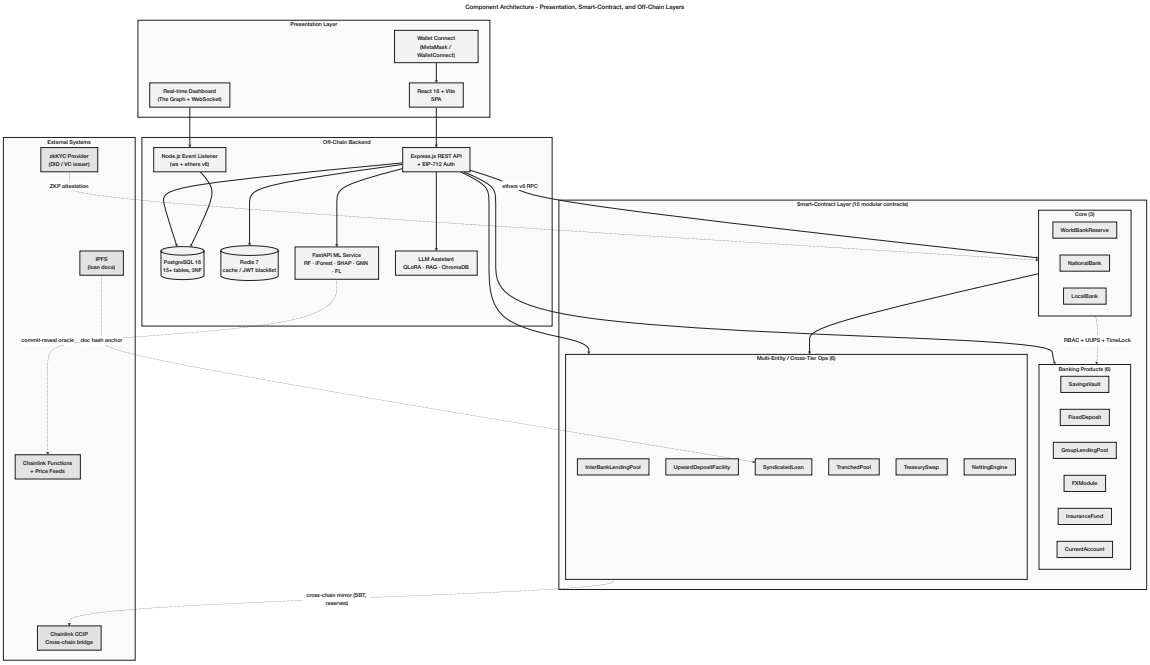

Figure 3.2: Component diagram showing interactions between the presentation layer, smart contract layer, off-chain backend services, and external systems.

3.3 Blockchain Platform Selection

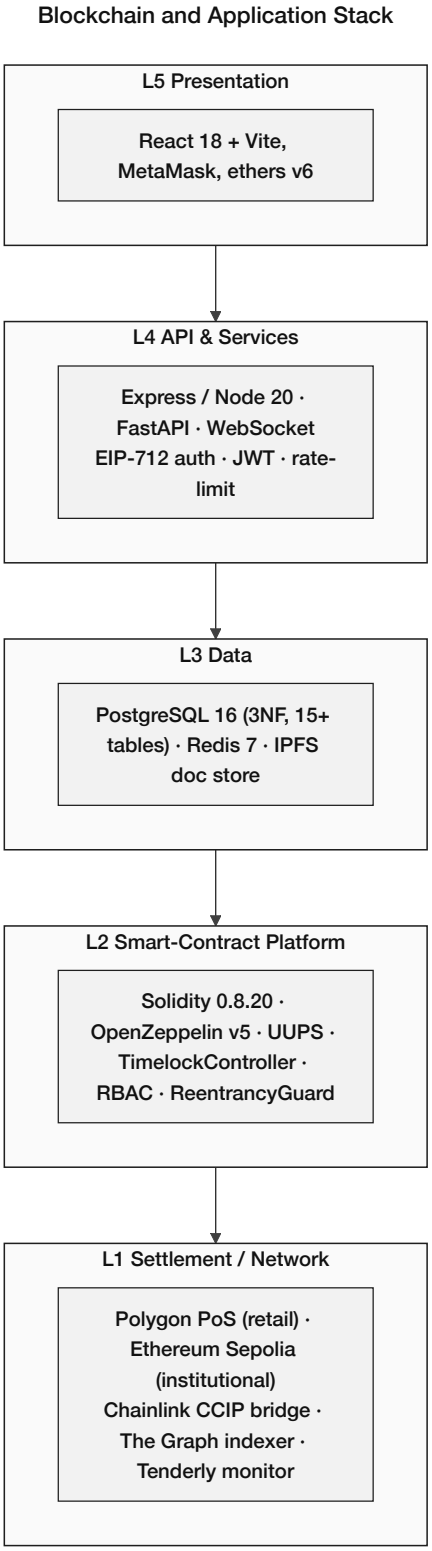

Figure 3.3: Layered blockchain and application stack: L1 settlement (Polygon PoS for retail, Ethereum Sepolia for institutional, plus Chainlink CCIP bridge, The Graph indexer, and Tenderly monitor); L2 smart-contract platform (Solidity 0.8.20, OpenZeppelin v5, UUPS, TimeLock, RBAC); L3 data (PostgreSQL 16 in 3NF, Redis, IPFS); L4 API and services (Express, FastAPI, WebSocket, EIP-712); L5 presentation (React + MetaMask).

Table 3.2: Blockchain platform selection criteria and justification.

Criterion	Selection	Justification
Platform	Ethereum Virtual Machine (EVM)	Largest developer ecosystem; battle-tested security model; extensive tooling.
Network	Polygon zkEVM Cardona / Ethereum Sepolia	Zero-cost deployment; production-equivalent behaviour; free faucet access.
Consensus	ZK validity proofs (Polygon zkEVM L2, Ethereum L1 settlement)	Every batch of transactions is accompanied by a zero-knowledge proof verified on Ethereum L1, so security derives from cryptographic validity rather than a validator set assumption.
Smart contract language	Solidity 0.8.20	Industry standard; mature compiler with overflow protection; rich library ecosystem.

This platform selection table compares candidate networks using criteria relevant to banking workflows, including cost, finality, throughput, and ecosystem maturity. Polygon zkEVM Cardona is selected over Polygon Amoy PoS because ZK validity proofs derive security from cryptographic verification rather than a validator set assumption, which is a materially stronger security claim for an institutional banking prototype. It motivates the selection of an EVM-compatible Layer 2 deployment for low-fee retail operations while retaining portability to other EVM chains.

Table 3.4: Blockchain platform selection (continued): operational and deployment factors.

Criterion	Selection	Justification
Gas cost	\$0.001–\$0.01 per transaction	Orders of magnitude cheaper than Ethereum mainnet (\$5–\$50); enables micro-loan economics.
Block finality	≈2 seconds	Sub-second practical finality for retail UX; checkpointed to Ethereum for security.
Developer tooling	Hardhat + OpenZeppelin	Automated test suite, deployment scripts, and audited security primitives.
Testnet availability	Cardona (Polygon zkEVM), Sepolia (Ethereum)	Free faucets; EVM-identical behaviour; no real cryptocurrency required for prototype.
Security model	ZK validity proofs	Every batch verified by a ZK proof anchored to Ethereum L1 (vs. PoS validator assumption on Amoy).
Migration path	EVM-compatible	Same Solidity contracts deploy to any EVM chain; reduces future L2 migration cost.

This supplementary comparison expands the platform evaluation to additional operational and deploy-

ment factors. The conclusion is that Polygon PoS provides a practical balance of stability and low transaction costs for an academic prototype, with a clear migration path if future requirements demand alternative L2s.

3.3.1 Transaction Verification and Consensus

- **Polygon zkEVM Cardona:** Transactions are batched and verified by a zero-knowledge validity proof submitted to Ethereum L1. Block finality is achieved once the ZK proof is verified on Ethereum, which provides cryptographic rather than economic security guarantees. The ZK proof means that a malicious sequencer cannot publish an invalid batch — the proof would not verify. This is materially stronger than the PoS validator-collusion assumption of Polygon Amoy.
- **On prototype testnets:** Cardona and Sepolia use equivalent consensus models at no financial cost, enabling incremental development and testing without exposure to real-asset risk.

3.3.2 Oracle Architecture: Off-Chain AI to On-Chain Decision

The Crypto World Bank requires a mechanism to convey off-chain AI/ML risk assessments into the on-chain loan approval workflow. This is an instance of the oracle problem [R1,R2].

Chainlink Functions oracle (primary). The final thesis phase uses **Chainlink Functions** as the primary oracle mechanism. Chainlink Functions operates via a Decentralised Oracle Network (DON): the ML risk score is fetched independently by multiple Chainlink nodes, consensus across the DON is required before the result is committed on-chain, and no single compromised node can manipulate the risk score. This is a fundamental trust improvement over the v23 commit-reveal relay, which required trusting a single FastAPI service key. The source code executed by the DON is:

```
// Chainlink Functions source (runs in decentralised DON)
const clientId = args[0];
const response = await Functions.makeHttpRequest({
  url: `https://your-ml-service.com/score/${clientId}`,
  method: "POST"
});
const score = response.data.risk_score;
// 6 decimal precision
return Functions.encodeUint256(Math.round(score * 1e6));
```

The DON adds 30–60 seconds of latency and \$0.10–\$1.00 per oracle call, both acceptable for the loan approval lifecycle where decisions are not time-critical. This architecture closes the oracle trust assumption that v23 noted as a limitation (the 2-of-3 Safe multisig attestation was the interim mitigation; Chainlink Functions removes the need for that mitigation entirely).

Commit-reveal relay (prototype fallback). During the prototype phase, prior to Chainlink Functions being wired (Phase III), the system uses a commit-reveal relay as a fallback. The off-chain ML service commits a hash $h = \text{keccak256}(s \,\|\, \text{nonce})$ to the `LoanController` contract, then reveals s and the nonce within the decision window. The contract verifies $h = \text{keccak256}(s \,\|\, \text{nonce})$ and stores s immutably. This pattern prevents

score manipulation between commitment and decision and creates an immutable on-chain audit trail of every risk score used in a lending decision, while Chainlink Functions integration is completed.

Chainlink Automation: trustless installment triggers. **Chainlink Automation** replaces the centralised cron job for overdue installment detection and interest accrual. The entire loan lifecycle from application to overdue flagging runs without a trusted operator:

```
// In LocalBankPool.sol
function checkUpkeep(bytes calldata) external view override
    returns (bool upkeepNeeded, bytes memory performData) {
    uint256[] memory overdueLoans = getOverdueLoans();
    upkeepNeeded = overdueLoans.length > 0;
    performData = abi.encode(overdueLoans);
}

function performUpkeep(bytes calldata performData) external override {
    uint256[] memory overdueLoans =
        abi.decode(performData, (uint256[]));
    for (uint i = 0; i < overdueLoans.length; i++) {
        _markInstallmentOverdue(overdueLoans[i]);
    }
}
```

This removes the last centralised component from the loan lifecycle — the entire process from application to overdue flagging runs without a trusted operator.

Chainlink Price Feeds: BDT/USD and ETH/USD. The `FXModule` contract uses Chainlink's production-grade `AggregatorV3Interface` price feeds for BDT/USD and ETH/USD, replacing the "forex oracle approved by governance" pattern of v23. Every BDT display in the frontend flows through:

```
// In LocalBankPool.sol
AggregatorV3Interface internal bdtUsdFeed;
AggregatorV3Interface internal ethUsdFeed;

function getBdtEquivalent(uint256 usdcAmount)
    public view returns (uint256) {
    (, int bdtRate,,,) = bdtUsdFeed.latestRoundData();
    // USDC tracks USD 1:1; Chainlink uses 8 decimals
    return usdcAmount * uint256(bdtRate) / 1e8;
}
```

The 8-decimal precision convention is the Chainlink standard for fiat-pair feeds. Chainlink price feeds are the Phase I build item for BDT display across the entire frontend.

Chainlink Proof of Reserve. The `WorldBankReserve` contract publishes its reserve balance to **Chainlink Proof of Reserve (PoR)**, making the reserve cryptographically verifiable by any external auditor without trusting CWB's administrator. This is the "free PoR"

advantage over FTX: any external system can verify reserve solvency without relying on CWB's admin claims.

```
// WorldBankReserve.sol
function getReserveSummary() external view returns (
    uint256 totalDeposited,
    uint256 totalLoaned,
    uint256 reserveRatio,   // scaled 1e4 (e.g. 5000 = 50%)
    uint256 insuranceFundBalance
) {
    return (
        totalDeposited,
        totalLoaned,
        (totalDeposited - totalLoaned) * 1e4 / totalDeposited,
        insuranceFund.balance()
    );
}
```

This function is the FTX commingling safeguard described in Section 3.19. Adding Chainlink's PoR job on top turns it into a market-standard verifiable proof, and it is specified in full in Appendix D.

The diagram placeholder for this architecture (Figure pending): `oracle_architecture.png`—Chainlink Functions DON → score commitment → on-chain LoanController.

3.4 Data Model and Database Design

The relational database schema comprises 20 normalized entities in Third Normal Form (3NF). Figure 3.4 presents the core system graph showing entity relationships, Figure 3.5 presents the full Entity-Relationship Diagram (ERD), and Figure 3.7 presents the Enhanced Entity-Relationship (EER) diagram.

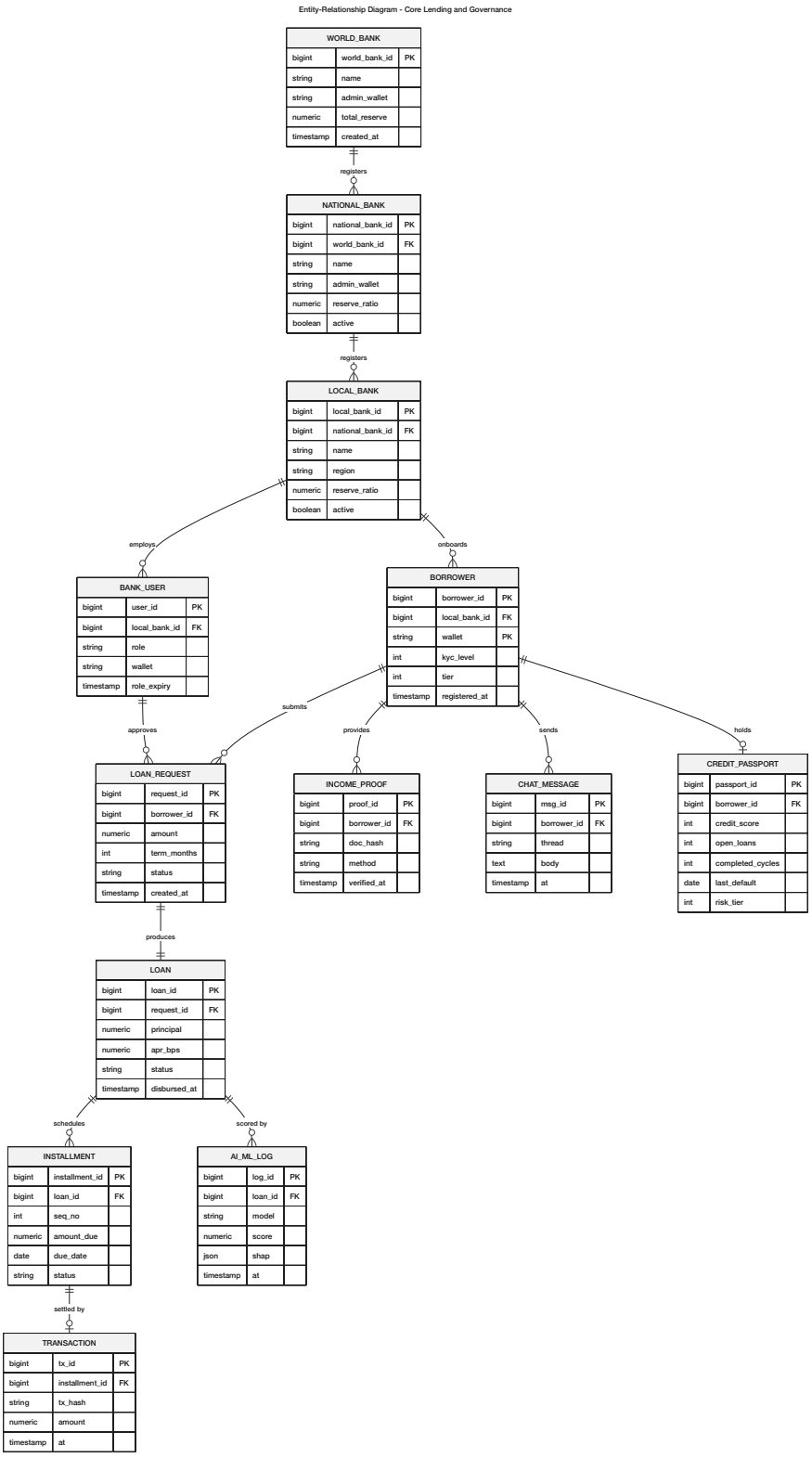

Figure 3.4: Core system graph showing entity relationships across the four-tier banking hierarchy (`WORLD_BANK` → `NATIONAL_BANK` → `LOCAL_BANK` → `BANK_USER`), the central `BORROWER` entity, and the lending-lifecycle sub-graph (`LOAN_REQUEST`, `LOAN`, `INSTALLMENT`, `TRANSACTION`, `INCOME_PROOF`, `CHAT_MESSAGE`, `AI_ML_LOG`, `CREDIT_PASSPORT`).

Reading guide. Arrows denote *one-to-many (1:N)* relationships flowing downward through the institutional hierarchy. **BANK_USER** specialises into *National* and *Local* variants (disjoint generalisation—see Figure 3.7). **BORROWER** is the pivot entity: it aggregates identity, credit history, and all transactional records. **LOAN_REQUEST** is the *aggregation hub* for the lending lifecycle: INSTALLMENT (weak entity), CHAT_MESSAGE (audit trail), and AI_ML_LOG (risk scoring) all depend on it existentially. The complete attribute sets for every entity are shown in the ERD (Figure 3.5) and the full EER model (Figure 3.7).

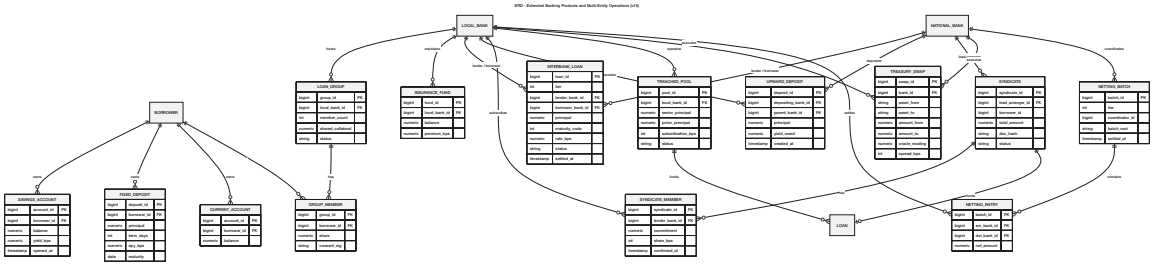

Figure 3.6: Extended ERD entities introduced in v15: the banking-product entities (`SAVINGS_ACCOUNT`, `FIXED_DEPOSIT`, `CURRENT_ACCOUNT`, `LOAN_GROUP`, `GROUP_MEMBER`, `INSURANCE_FUND`) and the multi-entity / cross-tier operational entities (`INTERBANK_LOAN`, `UPWARD_DEPOSIT`, `SYNDICATE`, `SYNDICATE_MEMBER`, `TRANCHED_POOL`, `TREASURY_SWAP`, `NETTING_BATCH`, `NETTING_ENTRY`). Foreign-key relationships into the core entities of Figure 3.5 are preserved.

The current ERD covers the core lending and governance entities. Extended banking entities including SavingsAccount, FixedDeposit, LoanGroup, GroupMember, CurrentAccount, and InsuranceFund are designed for the full system and will be incorporated into the updated data model in the final thesis. v15 additionally introduces the multi-entity / cross-tier operational entities described in Section 3.13.7: `INTERBANK_LOAN`, `UPWARD_DEPOSIT`, `SYNDICATE` and `SYNDICATE_MEMBER`, `TRANCHED_POOL`, `TREASURY_SWAP`, and `NETTING_BATCH` with `NETTING_ENTRY`. Their foreign-key relationships into the core lending entities (LOAN, BANK_USER, REPAYMENT) are detailed in Section 3.13 and will be drawn into the updated ERD figure in the diagram-audit session. The data model therefore remains in one-to-one consistency with the contract architecture and implementation phase plan.

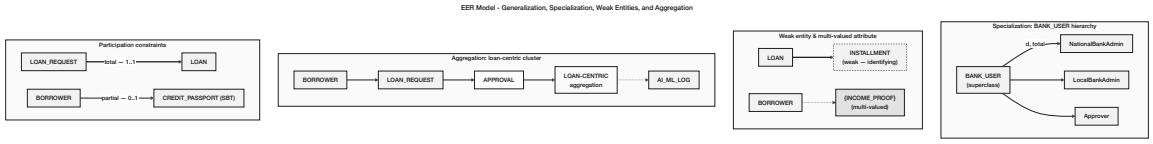

Figure 3.7: Enhanced Entity-Relationship (EER) model showing generalization / specialization (`BANK_USER` → National-/Local-bank-admin / Approver subtypes), the weak entity `INSTALLMENT` identified by its parent `LOAN`, the multi-valued attribute `INCOME_PROOF`, the loan-centric aggregation, and participation constraints (total / partial).

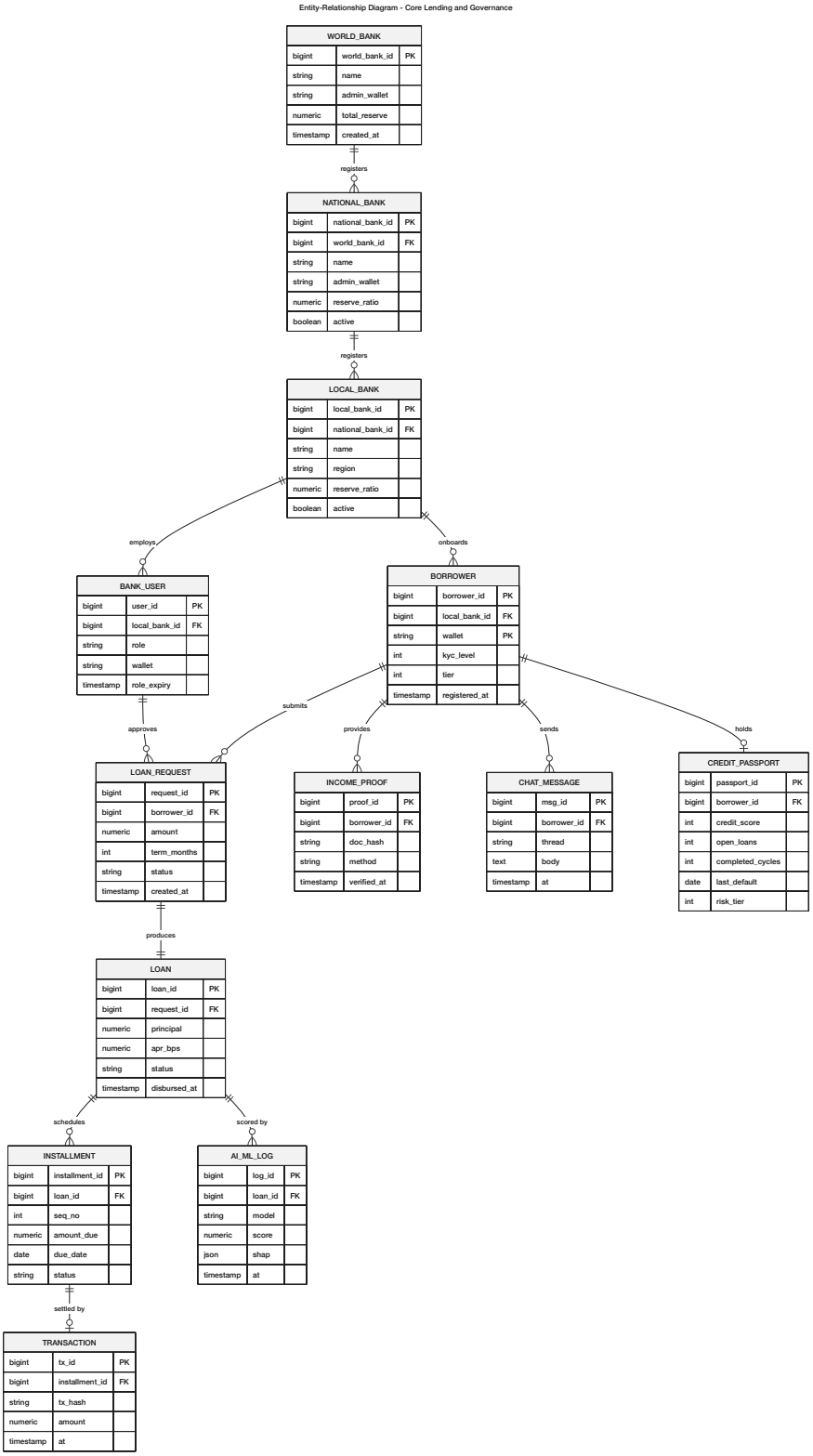

Figure 3.5: Entity-Relationship Diagram (ERD) of the Crypto World Bank database: core lending and governance entities in Third Normal Form (3NF), with primary keys, foreign keys, attribute types, and crow's-foot cardinality. Multi-entity / cross-tier and extended-product entities are shown in Figure 3.6.

3.4.1 Entity Summary

Table 3.6: Database entity summary (20 entities).

Entity	Role / Description
WORLD_BANK	Top-level reserve holder; global lending parameters.
NATIONAL_BANK	Country-level banks; borrow from World Bank.
LOCAL_BANK	City-level banks; borrow from National Bank and lend to users.
BANK_USER	Bank staff with role-based permissions (approve/reject loans).
BORROWER (CLIENT)	End clients requesting and repaying loans. The entity will be renamed `CLIENT` (with `client_id` FK) in the final thesis database migration to align with platform terminology; the current schema retains `BORROWER` for prototype compatibility.
LOAN_REQUEST	Loan applications and full lifecycle tracking.
INSTALLMENT	Repayment schedule records (weak entity dependent on LOAN_REQUEST).
TRANSACTION	Financial transaction records.
BORROWING_LIMIT	Per-client limits with 6-month and 1-year rolling windows.
INCOME_PROOF	Income verification documents (multi-valued per client).
CHAT_MESSAGE	Client–bank communication records.
AI_CHATBOT_LOG	AI chatbot interaction records.
AI_ML_SECURITY	Two logically distinct domains aggregated here for the prototype schema, classified as an *Association/Derived Entity*: (1) `LOAN_RISK_ASSESSMENT` — per-loan fraud scores, anomaly scores, composite scores, and SHAP value vectors; (2) `SECURITY_EVENT_LOG` — system-wide security events (large disbursements, repeated requests, Tenderly alerts). The final thesis will split these into separate tables to enable independent indexing and querying.
MARKET_DATA	Cryptocurrency price feed cache, refreshed by the Express.js cron service. *Relationships:* read-only auxiliary entity; values are consumed by the frontend and the FX oracle module. No FK into core lending tables. Key attributes: `symbol` (e.g. ETH, USDC), `price_usd`, `source` (Chainlink feed address), `recorded_at`.
PROFILE_SETTING	Client and bank-user interface preferences. *Relationships:* FK `client_id` references the CLIENT (BORROWER) entity (1:1 per KYC-verified user). Key attributes: `language_setting` (e.g. `bn`, `en`), `display_currency` (e.g. USD, BDT), `notification_enabled` (boolean).
SESSIONS	Formal session management: wallet address, device hash, IP address, EIP-7702 session key hash, session key scope (JSON: approved tool list + value cap), session key TTL, and revocation flag. New fields: (1) `parent_session_id` (self-referential FK, NULL for root sessions; enables session lineage chain for cross-session memory); (2) `compression_summary` (TEXT,

This entity summary table enumerates 19 relational entities. The additions (`SESSIONS`, `AGENT_ACTION_LOG`, `INTEREST_RATE_TIER`, `ASSETS`) support the AI agent session management architecture, the immutable agent audit trail, normalised interest rate governance, and standardised asset referencing respectively. The on-chain/off-chain split implied by the entities supports the architectural principle that high-integrity state transitions occur on-chain while analytics and document workflows remain off-chain.

3.4.2 EER Constructs Applied

Table 3.8: EER constructs applied: specialisation, hierarchy, and constraints.

EER Construct	Applied To	Notes
Specialisation (disjoint)	BANK_USER → NationalBankUser, LocalBankUser	A bank user belongs to exactly one bank type; enforced by CHECK constraint.
Generalisation	WORLD_BANK, NATIONAL_BANK, LOCAL_BANK share institution attributes	Avoids attribute duplication across institution types.
Weak entity	INSTALLMENT depends on LOAN_REQUEST	Existence and identity determined by parent loan.
Multi-valued attribute	INCOME_PROOF per BORROWER	Modelled as separate entity to maintain 1NF.
Aggregation	TRANSACTION, CHAT_MESSAGE → AI_ML_SECURITY_LOG	AI_ML_SECURITY_LOG aggregates monitoring signals from both sources.
Association entity	LOAN_REQUEST links BORROWER + LOCAL_BANK	LOAN_REQUEST is a strong entity with its own UUID primary key and foreign keys to BORROWER and LOCAL_BANK, capturing the many-to-many relationship between borrowers and lending banks. It is *not* a weak entity (it has its own UUID PK) and is not an aggregation in the ER-theoretic sense; the term "association entity" or "junction entity" is the correct ER classification.
Participation constraint	BORROWER must have ≥ 1 LOAN_REQUEST to access services	Enforced at application layer; architecturally planned for on-chain.
Append-only table (policy)	AGENT_ACTION_LOG, AUDIT_LOGS	DB-level INSERT-only role + Row-Level Security policy enforces that no UPDATE or DELETE is permitted on audit records. Provides immutable tamper-evident trail for regulatory review and agent action accountability.

This table captures the EER constructs applied to represent specialization, hierarchy, and constraints in the data model. It demonstrates how institutional roles and banking participants are modeled without duplicating attributes across tables, improving data integrity and query clarity.

3.4.3 Normalization

The schema has been normalized to Third Normal Form (3NF) and checked against Boyce-Codd Normal Form (BCNF):

- **1NF:** There are no repeating groups in any of the attributes. Separate entities store multi-valued attributes, such as proof of income.
- **2NF:** There are no partial dependencies. The full composite key (`loan_id`, `installment_number`) determines the non-key attributes of INSTALLMENT.
- **3NF:** No dependencies that go through other dependencies. Instead of storing redundant values like `total_borrowed` in bank entities, they are calculated at query time.
- **BCNF:** All determinants are candidate keys. A CHECK constraint on BANK_USER enforces that the generalization hierarchy's specializations are not the same.

A transitive dependency was identified in v24: `interest_rate_parameters` $\rightarrow$ `bank_tier_id` (interest rate values were stored as columns in the World Bank, National Bank, and Local Bank entities, making them dependent on a non-key attribute via the tier relationship). This violates 3NF. The fix is to extract these parameters into a dedicated `INTEREST_RATE_TIER` table keyed by `tier_id`, which each bank entity references via FK. This normalisation change ensures that updating a base rate does not require touching multiple bank-tier rows.

3.4.4 Indexing Strategy

We use B-tree indexes (the default in PostgreSQL) for efficient retrieval on frequently queried columns:

Table 3.10: Indexing strategy: B-tree indexes for time-window and high-frequency queries.

Index	Table / Column(s)	Rationale
idx_loan_borrower	LOAN_REQUEST(borrower_id)	High-frequency lookup for borrower loan history.
idx_loan_status	LOAN_REQUEST(status)	Efficient filtering of active vs. closed loans.
idx_installment_due	INSTALLMENT(due_date, status) WHERE status = 'PENDING'	Partial index range query for pending overdue installment detection (agent reminder cron + Chainlink Automation).
idx_txn_created	TRANSACTION(created_at)	Time-window aggregation for 6-month and 1-year borrowing limits.
idx_txn_borrower	TRANSACTION(borrower_id)	Per-borrower transaction history retrieval.
idx_market_symbol	MARKET_DATA(symbol, recorded_at)	Composite index for price feed time-series queries.
idx_loan_bank_status	LOAN(local_bank_id, status, created_at DESC)	Loan lifecycle queries by bank and status (most common query pattern for the approver dashboard).
idx_loan_client_active	LOAN(client_id, status) WHERE status = 'ACTIVE'	Partial index for per-client open-loan count (used by the over-indebtedness control).
idx_agent_client_date	AGENT_ACTION_LOG(client_id, created_at DESC)	Per-client agent action history retrieval.
idx_agent_status	AGENT_ACTION_LOG(status) WHERE status = 'PENDING'	Partial index for the agent status-monitoring loop.
idx_aiml_score	AI_ML_LOG(anomaly_score DESC) WHERE anomaly_score > 0.5	Partial index for AML alert queue (SAR workflow).
idx_session_parent	SESSIONS(parent_session_id) WHERE parent_session_id IS NOT NULL	Partial index for lineage chain traversal; covers all non-root sessions.

This indexing table lists the primary indexes used to support time-window queries (e.g., rolling 6-month/1-year transaction windows) and high-frequency lookups. The selection emphasizes B-tree suitability for range filtering common in risk analytics and reporting, improving performance without complicating the schema.

B-tree indexes are chosen for their efficiency with range queries (e.g., transactions within

the last 6 months); hash indexes would be suitable only for exact-match lookups.

3.4.5 Functional Dependencies

Table 3.12: Representative functional dependencies.

Relation	Functional Dependency	Notes
LOAN_REQUEST	`loan_id` → all attributes	Primary key determines row.
LOAN_REQUEST	`borrower_id`, `local_bank_id` → status, amount, …	One active request per client per bank.
BORROWING_LIMIT	`borrower_id` → `six_month_limit`, `one_year_limit`, …	1:1 with BORROWER. *Note:* BORROWING_LIMIT stores pre-computed rolling-window limits derived from TRANSACTION records. Because a two-phase commit between PostgreSQL and the blockchain is impossible, the PostgreSQL copy is a read replica for dashboard display only. The authoritative enforcement of borrowing limits occurs on-chain (smart contract checks `totalBorrowedSixMonth` and `totalBorrowedOneYear` before any disbursement), preventing the race condition that would arise if the database were the enforcement source.
BORROWER	`wallet_address` → `borrower_id`, all attributes	Wallet address is a unique candidate key *for the current prototype*. In the production schema, identity will be decoupled from wallet address (an `IDENTITY` table keyed by `kyc_hash` with a one-to-many `WALLET` table), so that ERC-4337 social recovery—which produces a new wallet address for the same user—preserves credit history continuity rather than creating a duplicate identity.
INSTALLMENT	`loan_id`, `installment_number` → `amount`, `due_date`, `status`	Composite primary key; fully determined by both columns.
SESSIONS	`session_id` → `client_id`, `wallet_address`, `session_key_hash`, `session_key_scope`, `session_key_expires_at`, `expires_at`, `revoked`	Session key scope (JSONB) encodes the approved MCP tool list + value cap per session.

This functional dependency table documents the data-level rules that prevent inconsistent state, such as improper loan status transitions or duplicated relationships. Explicit FDs reinforce that key banking invariants are enforced structurally, complementing smart contract enforcement on the on-chain side.

The `parent_session_id` self-referential foreign key in `SESSIONS` introduces a recursive relationship within a single entity. This does not violate 3NF (the dependency is `session_id` $\rightarrow$ `parent_session_id`, not a transitive dependency), but it requires an acyclicity constraint to prevent circular ancestry chains: a session cannot be its own ancestor. This is enforced by a `CHECK` constraint in conjunction with a trigger that walks the ancestry chain before INSERT (see Section **??**).

3.4.6 Relational Integrity Constraints

Table 3.14: Relational integrity constraints.

Constraint Type	Examples
Primary Key	`world_bank_id`, `loan_id`, `borrower_id`, etc.
Foreign Key	`local_bank_id` in LOAN_REQUEST references LOCAL_BANK.
UNIQUE	`wallet_address` in BORROWER; `blockchain_tx_hash` in LOAN_REQUEST.
CHECK	BANK_USER: `(bank_type='national' AND national_bank_id IS NOT NULL) OR (bank_type='local' AND local_bank_id IS NOT NULL)`.
NOT NULL	Core attributes: `name`, `wallet_address`, `status`.
APPEND-ONLY (DB policy)	`AGENT_ACTION_LOG: CREATE ROLE audit_writer; GRANT INSERT ON agent_action_log TO audit_writer; REVOKE UPDATE, DELETE FROM PUBLIC. AUDIT_LOGS:` same policy applied via Row-Level Security (`ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY; CREATE POLICY audit_insert_only ON audit_logs FOR INSERT WITH CHECK (TRUE)`).
FOREIGN KEY (new)	`LOAN.collateral_asset_id REFERENCES assets(asset_id); LOAN.loan_asset_id REFERENCES assets(asset_id); AGENT_ACTION_LOG.session_id REFERENCES sessions(session_id); AGENT_ACTION_LOG.confirmation_turn_id REFERENCES chat_message(message_id).`
UNIQUE (new)	`assets.symbol` (asset ticker is globally unique across the platform asset registry).

This integrity constraints table summarizes referential and domain constraints that keep loan, repayment, and identity records consistent. These constraints reduce operational risk by preventing orphaned records and invalid lifecycle states, which is critical for auditability in financial systems.

3.5 On-Chain and Off-Chain Data Partitioning

Table 3.16: Data partitioning between on-chain and off-chain storage.

Data Category	Storage	Rationale
Reserve balances, loan requests, approval/rejection events, repayment transactions	On-chain	Immutability, public auditability, trustless verification.
User profiles, income verification documents, chat messages, AI/ML inference logs	Off-chain (database)	Data privacy, query flexibility, storage cost optimisation.
Borrowing limit computations	Off-chain with on-chain enforcement	Complex temporal aggregation; results committed as on-chain constraints.
Cryptocurrency market data	Off-chain (cached)	High-frequency updates; external API dependency.
Agent action execution log	Off-chain (append-only DB)	Immutable audit trail of every agent write-tool call, linked to on-chain tx hash; not stored on-chain to save gas.
Session key material	Off-chain (`sessions` table)	EIP-7702 session key hash, scope JSON, and TTL stored off-chain; the scope restrictions are enforced on-chain by the session key contract at signing time.
Chainlink oracle price data	Off-chain (`MARKET_DATA`)	Chainlink Price Feed results are cached in `MARKET_DATA` with `source` = Chainlink feed address; the on-chain `AggregatorV3Interface` is the authoritative source.
Session lineage chains (SESSIONS, message history, compression summaries)	Off-chain (PostgreSQL)	Full transcript history is too large for on-chain storage. The lineage chain enables searchable long-term memory without gas cost. `AGENT_ACTION_LOG` provides the on-chain audit link via confirmed transaction hashes.

This partitioning table clarifies which data must live on-chain (role bindings, reserves, and critical state transitions) versus off-chain (documents, analytics features, and operational metadata). The partitioning choice balances transparency and tamper-resistance with practical storage and privacy constraints.

3.6 Digital Identity System

The platform's identity model operates across two layers, combining wallet-based authentication with off-chain document verification, and is designed to accommodate compliance requirements in future phases.

- **Identity based on wallets:** Users authenticate via Ethereum wallet signatures (MetaMask, WalletConnect), eliminating the need for centralised credential storage.
- **Role binding:** In the smart contract permission system, wallet addresses are linked to hierarchical roles like Owner, National Bank, Local Bank, Approver, and Retail Client.
- **Limits of wallet-based identity:** Wallet ownership proves transaction history and cryptographic control but cannot independently prove legal identity, jurisdiction, or age. This distinction is critical for regulated banking operations. A compromised or lost wallet requires an off-chain recovery process and on-chain role revocation followed by re-binding to a replacement address, a governance workflow that must be carefully designed to prevent unauthorized role escalation.
- **On-chain versus off-chain identity layers:** The system maintains two identity layers. On-chain identity consists of the wallet address, assigned role, and transaction history recorded permanently on the blockchain. Off-chain identity consists of document-verified income, KYC credentials, and AML screening results stored in the PostgreSQL database and linked to the wallet address by hash reference. The planned ZKP-based compliance extension would allow users to prove off-chain KYC status to the smart contract without exposing personal documents on-chain.
- **EIP-7702 session key authorisation:** For agent-executed banking operations, the client authorises a scoped, time-bound session key at login. The session key is restricted to a named set of MCP write tools (e.g., only `submit_loan_application` and `pay_installment`), a value cap (e.g., 500 USDC per transaction), and a 24-hour TTL after which it auto-expires. The session key cannot transfer funds to external addresses or perform any operation outside its approved scope. Every session key usage is logged in `AGENT_ACTION_LOG` with the conversation turn ID of the client's confirmation message as the audit reference.

3.6.1 ZKP KYC and zkAML Compliance Architecture

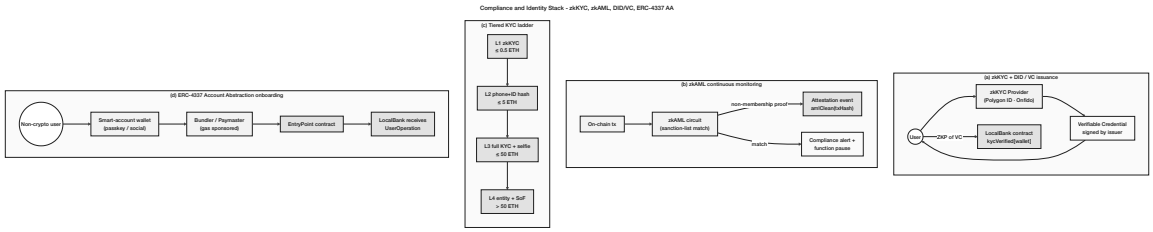

Figure 3.8: Compliance and identity stack: (a) zkKYC issuance via licensed identity provider with W3C Verifiable Credential, (b) zkAML continuous monitoring with sanction-list non-membership proofs, (c) the tiered KYC ladder (L1 zkKYC → L4 entity onboarding) gated by loan size, and (d) ERC-4337 Account-Abstraction onboarding for non-crypto users with gas sponsored by a Paymaster.

The compliance gateway uses two cooperating zero-knowledge circuits rather than a single KYC verifier: a **KYC circuit** and an **AML circuit**, both expressed as zk-SNARKs (Groth16 proofs via Circom 2.0 + snarkjs). KYC proves that the user owns a valid identity credential; AML proves that the user's wallet has not interacted with sanctioned addresses and stays within velocity limits. Decoupling the two circuits makes it possible to revoke AML certification without invalidating the KYC credential, and matches the actual structure of regulator-mandated compliance.

KYC circuit (identity). An off-chain KYC provider validates the user's NID and issues a signed credential $credential = \text{Sign}_{\text{KYC}}(wallet\_address, country, age\_over\_18, kyc\_passed)$. The user generates a zk-SNARK proof that (a) they possess a valid signed credential from an approved KYC provider, (b) their age is over 18, and (c) their country is in the permitted jurisdiction list—without revealing the underlying credential, NID, or personal data. The smart contract verifies the proof: `KYCVerifier.verify(proof, [wallet, country, age_over_18])` returns `true`. Piper et al. (2025) [R8] report proof generation in 1–4 s on consumer hardware; on-chain verification costs ≈200–300 k gas, within a one-time registration budget.

zkAML circuit (anti-money-laundering). The zkAML circuit [R19] (IACR ePrint 2025/465) proves three properties of a wallet without revealing its transaction graph: (1) the wallet has not transacted with any address on a public sanctions list (Chainalysis / TRM Labs); (2) the wallet's 30-day inflow does not exceed a governance-set velocity threshold; and (3) the wallet has not received funds from addresses flagged in the platform's on-chain risk registry within the preceding 90 days. The published benchmark for zkAML is 55 TPS on public networks with 226 ms proof-generation latency, both compatible with retail-tier flows on Polygon. The on-chain verifier is a sibling of `KYCVerifier` that exposes `AMLVerifier.verify(proof, [wallet, blockHeight])`. Both verifiers must return true before a wallet's CLIENT role is activated.

W3C DID / VC anchoring. The KYC credential issued off-chain is shaped as a W3C Verifiable Credential (VC) bound to a W3C Decentralized Identifier (DID) [R20] rather than a free-form signed blob. The DID is anchored on-chain (`did:ethr:0x...` per the EIP-1056 standard), and the smart contract checks possession of the VC by means of the zk-SNARK proof rather than by storing the VC itself. This places the platform's identity layer inside the Self-Sovereign Identity (SSI) framework and aligns it with the European Blockchain Services Infrastructure (EBSI) reference architecture.

Privacy posture. A 2025 ResearchGate study reports a 97% reduction in exposed user data versus conventional on-chain KYC; Decker (2025) [R9] gives the same finding under a different threat model. Implementation for the final thesis phase will deploy both Circom 2.0 circuits to Polygon zkEVM Cardona testnet, tested with synthetic credential data drawn from the platform's mock KYC provider.

3.7 User Taxonomy and Onboarding Flows

The thesis now uses a formal actor taxonomy of nine actors following the systems-analysis convention of the Binance Software Engineering Architecture reference. Five primary actors (A1–A5) initiate actions; four secondary actors (A6–A9) are external systems or authorities. Table 3.18 gives the operational user taxonomy with per-user-type KYC level, wallet type, and database-residence assumption; Table 3.19 presents the complete formal actor taxonomy; and Table 3.20 presents the actor permission matrix.

Table 3.18: Operational user taxonomy with KYC level, wallet type, and data-residence assumption. The taxonomy expands the actor list referenced in the use-case diagram of Section 3.15.1.

User Type	Tier	KYC Level	Wallet Type	Data Residence
Anonymous Visitor	—	None	None	Session only (Redis)
Registered Retail User	Tier 4	Level 1 (NID + selfie)	ERC-4337 abstracted (email / Google login)	Off-chain primary; on-chain role only
Group Client	Tier 4	Level 1 + group verification	ERC-4337 abstracted	Off-chain + on-chain group contract
Local Bank Operator	Tier 3	Full institutional KYC	MetaMask (institutional)	Off-chain + on-chain role binding
Local Bank Approver	Tier 3	Full institutional + background check	MetaMask + Safe co-signer	Off-chain + on-chain role binding
National Bank Admin	Tier 2	Full institutional + regulatory license	Safe multisig mandatory	Off-chain + on-chain role binding
World Bank Admin (Governance)	Tier 1	Founding team + supervisor attestation	Safe 3-of-5 multisig	On-chain governance only

Table 3.19: CWB formal actor taxonomy (nine actors): five primary actors who initiate actions and four secondary actors who are external systems or authorities.

Label	Actor	Role
Primary Actors (initiate actions)		
A1a	Retail Client (pre-KYC)	Browsing only; no banking transactions.
A1b	Retail Client (KYC Tier 1)	Bronze/Silver credit tier; small loans up to the KYC-1 cap.
A1c	Retail Client (KYC Tier 2)	Gold/Platinum/Diamond; larger limits, group lending.
A2	Local Bank Admin (Approver)	Loan approver; risk officer; reviews AML alerts.
A3	National Bank Admin	Capital allocator; compliance officer; SAR review; freeze authority.
A4	World Bank Admin (Governance)	Parameter governor; system operator via multi-sig.
A5	AI Agent	Read-only tools always permitted; write tools require explicit human confirmation gate before execution.
Secondary Actors (external systems or authorities)		
A6	Regulatory Authority	Read-only audit access via encrypted data package.
A7	Chainlink DON	Oracle, price feeds, automation.
A8	Blockchain Validator	Polygon zkEVM validity proof generation.
A9	External Auditor	Chainlink PoR verification.

Table 3.20: CWB actor permission matrix. READ = read-only tool call; WRITE† = write tool requiring explicit human confirmation gate. * = own tier only.

Action	Client	LB Admin	NB Admin	WB Admin	AI Agent
View own loan status	YES	YES*	YES*	YES*	READ
Submit loan application	YES	NO	NO	NO	WRITE†
Pay installment	YES	NO	NO	NO	WRITE†
Submit KYC documents	YES	NO	NO	NO	NO
Approve loan application	NO	YES	NO	NO	NO
Reject loan application	NO	YES	NO	NO	NO
Freeze client account	NO	YES	YES	YES	NO
Set borrowing rate	NO	NO	YES	YES	NO
Change reserve ratio	NO	NO	NO	YES	NO
Upgrade LB borrowing cap	NO	NO	YES	YES	NO
View audit logs (all)	NO	YES*	YES*	YES	NO
Generate SAR report	NO	YES	YES	YES	NO
View reserve summary (public)	YES	YES	YES	YES	READ

*† All agent write tools require an explicit human confirmation gate: the agent assembles the full parameter set, presents a summary to the client, and waits for affirmative consent. No write tool is ever called without a confirmation turn in the conversation history. * own tier only.*

3.7.1 ERC-4337 Account Abstraction for Retail Onboarding

The single largest accessibility contradiction in the original thesis was that retail clients in developing economies were assumed to already understand MetaMask, seed phrases, gas fees, and Ethereum transactions. This is incompatible with the financial-inclusion mission. The platform resolves the contradiction by adopting **ERC-4337 Account Abstraction (AA)** for all retail-tier (Tier 4) clients, with the EIP-7702 Pectra upgrade (May 2025) providing the gas-sponsorship layer. Since its launch on Ethereum mainnet in March 2023, ERC-4337 has enabled over 40 million smart accounts and processed more than 100 million transactions across major L2 networks including Polygon, Arbitrum, Optimism, and Base, demonstrating production-grade viability at scale [18-1]. The current production EntryPoint is **v0.7**, deployed at `0x0000000071727De22E5E9d8BAf0edAc6f37da032` on Ethereum, Polygon, and all major EVM chains; the platform targets this canonical address for all UserOperation routing.

Under ERC-4337 each retail client is issued a smart-contract account that is created on first sign-in. The user can register with email or a Google account via Firebase Auth, never sees a seed phrase, and never needs to acquire ETH for gas: a Paymaster contract sponsors the gas, funded by the World Bank Reserve as a financial-inclusion subsidy line. Account recovery uses a social-recovery guardian (a trusted email / phone) rather than a 24-word mnemonic.

Paymaster sybil-resistance controls. Gas sponsorship introduces a perverse incentive: an attacker can create many ERC-4337 accounts at zero cost and spam loan applications, draining the gas subsidy and potentially the loan pool if fraud detection fails. Three controls mitigate this. First, the Paymaster implements a **pre-KYC bootstrap allowance**: before KYC is completed, the Paymaster sponsors exactly two function signatures per wallet — `submitKYCHash(bytes32)` (storing the document hash for Level 1 KYC) and the ERC-4337 account creation transaction. Both are capped at a maximum of 0.001 ETH-equivalent total per wallet address; this is sufficient to complete KYC but insufficient to spam loan applications. All other gas sponsorship is conditional on the wallet having a verified KYC status (`kycVerified[wallet] = true`). This pre-KYC bootstrap window resolves the chicken-and-egg problem: a new user with no ETH can complete KYC using the bootstrap allowance, after which normal Paymaster sponsorship unlocks. Second, the Paymaster budget is capped per verified identity (not per account), limiting lifetime gas sponsorship to a fixed USDC equivalent per KYC identity. Third, the Paymaster rate-limits by IP and device fingerprint to prevent bulk account creation from a single attacker. These controls make Paymaster sponsorship conditional on verified identity rather than unconditional, preserving the inclusion mission while preventing abuse.

For the demo and final thesis prototype, the Biconomy SDK and Alchemy Account Kit will provide the bundler / Paymaster infrastructure on Polygon zkEVM Cardona. From the smart-contract perspective, an account-abstracted wallet is indistinguishable from an externally owned account, so none of the four-tier role logic needs to change. From the client's perspective, the cryptocurrency stack is invisible: they see a Bengali- or English-language banking interface, sign in with email, and apply for a loan. This re-aligns the implementation with the financial-inclusion contribution claimed in Chapter 1.

EIP-7702 session keys — scoped agent wallet. EIP-7702 provides a cleaner solution than ERC-4337 paymasters for agent-controlled operations because the scope restriction is enforced at the key level, not at the application level. When a client authorises an AI Agent session at login, the EIP-7702 session key is subject to four hard constraints: (1) **Scope** — the key is restricted to a named set of MCP write tools (e.g., only `submit_loan_application` and `pay_installment`); (2) **Time-bound** — a 24-hour TTL after which the key auto-expires regardless of remaining uses; (3) **Value-capped** — a maximum of 500 USDC per transaction, preventing large unauthorised transfers; and (4) **Revocable** — the client can invalidate the session key at any time from the wallet UI. The session key *cannot* transfer funds to external addresses, exceed the value cap, perform any operation not in the approved scope, or execute after the TTL expires.

EIP-7702 is architecturally superior to ERC-4337 paymasters for agent-controlled operations and does not replace ERC-4337 for retail gas sponsorship — the two mechanisms serve different functions. ERC-4337 removes the need for ETH for gas; EIP-7702 scopes what the agent may sign. Both are active simultaneously for an agent session: the session key signs the transaction, the Paymaster sponsors the gas.

3.7.2 Tiered Risk-Based KYC

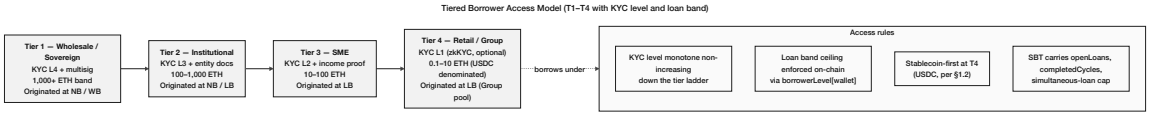

Figure 3.9: Tiered borrower access model. KYC level is monotone non-increasing down the tier ladder; the loan-band ceiling is enforced on-chain via `borrowerLevel[wallet]`; stablecoin denomination is enforced at Tier 4 (Section 1.10.3); and the SBT (Section 3.11) carries the simultaneous-loan cap that the GroupLendingPool reads at every application.

Identity verification is gated by a risk-based KYC ladder rather than an all-or-nothing flag. Table 3.21 gives the four levels and the corresponding borrowing limits and verification times. The design is consistent with current Persona / Veriff guidance for retail-fintech KYC and the EU AI Act Article 86 transparency requirements.

Table 3.21: Tiered, risk-based KYC ladder. Higher KYC level unlocks larger loans but requires proportionally more verification.

Risk Level	Required Documents	Borrow Limit	Verify Time
Level 0 (Unverified)	None	View only	Instant
Level 1 (Basic)	NID photo + selfie liveness check	Up to 0.1 ETH-equivalent (USDC)	2–5 min (automated)
Level 2 (Standard)	NID + proof of income + bank statement	Up to 2 ETH-equivalent	24 h (human review)
Level 3 (Enhanced)	Full document set + video interview	Up to 10 ETH-equivalent	48–72 h

After KYC approval, the smart contract sets `kycVerified[wallet] = true`, `kycLevel[wallet] = level`, and a one-year expiry `kycExpiry[wallet] = block.timestamp + 365 days`. Re-verification is enforced via the role-expiry mechanism (Section 3.6.1). Personal data—NID number, biometric template, document files—is never stored on-chain; only a SHA-256 hash of the document and the zk-SNARK proof are written, in compliance with GDPR data-minimization principles and Bangladesh's Personal Data Protection regime.

3.7.3 Five-Stage Conversion Funnel for Non-Crypto Users

The platform's retail conversion strategy is structured as a five-stage funnel that progressively reveals technical complexity rather than demanding it up front. Stage 1 is information browsing with zero friction (no account required, Bengali / English chatbot answers in plain language). Stage 2 is account creation with email or phone—an ERC-4337 smart account is created in the background; the user never sees a seed phrase. Stage 3 is KYC verification via phone-camera document capture with automated AI verification in 2–5 minutes. Stage 4 is the first loan, with the platform explaining the workflow in plain language and showing the repayment schedule in local-currency equivalent through a USD-BDT forex oracle (or a fiat-price oracle approved by governance), since retail loans are denominated in USDC

which tracks USD rather than ETH. Stage 5 is the optional power-user mode in which the client can connect a self-custodied MetaMask wallet, view their transactions on a block explorer, or participate in solidarity-group lending. The funnel hides crypto until the user is ready for it and re-frames the platform as "a bank that happens to settle on a public ledger" rather than "a DeFi protocol that happens to call itself a bank".

3.8 Kinked Interest Rate Model

The Crypto World Bank adopts a **kinked utilization-based interest rate model**, following the design established by Compound Finance and Aave v2/v3 and motivated by empirical findings from Gudgeon et al. (2020) [R3]. Below an optimal utilization rate U^* (set at 80% for retail lending pools), the borrowing rate increases gently:

$$r_b(U) = r_0 + \frac{U}{U^*} \cdot r_1, \quad U \leq U^* \tag{Formula 18}$$

Above U^*, the rate increases steeply to incentivize rapid repayment and new deposits:

$$r_b(U) = r_0 + r_1 + \frac{U - U^*}{1 - U^*} \cdot r_2, \quad U > U^* \tag{Formula 19}$$

where r_0 is the base rate, r_1 is the slope below the kink, and r_2 is the jump multiplier above the kink. This piecewise model prevents liquidity crises in which utilization approaches 100% and depositors are unable to withdraw—a failure mode documented empirically in DeFi lending markets by Gudgeon et al. (2020) [R3] and analyzed theoretically by Mackinga et al. (2023) [R4]. The smart contract enforces two floor/ceiling guards on these parameters: `require(r2 >= MINIMUM_JUMP_MULTIPLIER)` ensures the above-kink rate rise is steep enough to incentivize repayment and deposit inflow (a jump multiplier below the minimum would flatten the above-kink curve and fail to deter utilization runaway); and `require(U_star <= MAXIMUM_KINK_POINT)` prevents the kink from being placed so high that the safety mechanism is effectively disabled. Both constants are governance-set and enforced on-chain before any parameter update is executed.

3.9 Liquidation Engine

The Health Factor formula ($HF = C \times LT/D$, List of Formulas) applies specifically to *over-collateralized retail loans* at Tier 4 where a client has deposited ETH or USDC collateral exceeding the loan value. It is operationally meaningless without a contract that monitors it and an actor incentivized to act on the result, and it does not apply uniformly across all loan types in the CWB. The tier-specific Health Factor models are:

- **Tier 4 over-collateralized retail:** Standard $HF = (C \times LT)/D$; liquidation triggered at $HF < 1.0$.
- **Tier 4 group lending (collateral pool):** $HF_{\text{group}} = (\text{total\_group\_collateral} \times LT)/\text{total\_group\_outstanding\_debt}$; liquidation is at the pool level, distributing the loss across all group members, not the individual who defaulted.
- **Tier 4 credit-based (no collateral):** HF is not applicable. Default enforcement relies on credit-score degradation, permanent `riskTier` downgrade (with `lastDefault` timestamp irrevocably set on the SBT), and an effective future-borrowing ban. The

SBT credential itself is *never revoked*—consistent with the Buterin, Hitzig, and Weyl (2022) [R30] non-revocable design principle; no lender accepts a wallet whose SBT `riskTier` is at the default level, making borrowing effectively impossible in practice while preserving the permanent, honest record of the default event. No liquidation is possible because no collateral exists to seize.

- **Tier 1–3 institutional:** $HF_{\text{inst}} = (\text{on\_chain\_reserve} - \text{min\_reserve})/\text{outstanding\_borrowing}$; the parent tier's reserve surplus serves as the effective collateral for inter-tier lending.

The original v10 thesis listed HF but contained no liquidation mechanism: when HF dropped below 1.0, the system relied on a human approver noticing and reacting, which is a ledger, not a bank. The Crypto World Bank therefore specifies a dedicated **LiquidationEngine** contract that closes this gap.

Liquidation life-cycle.

1. A client's collateral position is updated from the Chainlink price feed at every block; HF is recomputed in view-only fashion (no gas cost to the client).
2. Any external participant may call `liquidate(loanId)` when HF < 1.0.
3. The contract verifies HF on-chain, transfers a configurable portion of the collateral (typically equal to the outstanding debt plus a 5–8% liquidation bonus, following Aave v3 parameters) to the liquidator, and updates the client's on-chain credit record.
4. A `LoanLiquidated(loanId, liquidator, bonusBps)` event is emitted and indexed by The Graph for the dashboard.

Incentive design. The 5–8% liquidation bonus is what makes liquidation self-actuating: any wallet (including a public arbitrage bot) can call `liquidate` when HF dips, captures the bonus, and protects the lending pool from accumulating bad debt. This is a standard DeFi primitive in production protocols since 2020 (Aave, Compound, MakerDAO) and its absence from a paper that calls itself a banking system is a credibility gap that v15 closes.

Hierarchical liquidation queue. The platform extends the standard single-tier liquidation primitive with a hierarchical priority: when a Local Bank exceeds a portfolio-level reserve-ratio threshold (RR below 15%), liquidation calls from authorized liquidators are routed through a National-Bank-level queue that prioritizes the lowest-HF positions first. This converts the same liquidation primitive into a tier-aware portfolio-health control rather than a per-loan reflex, matching how Tier 2 institutions actually manage credit risk.

3.10 SavingsVault and FixedDeposit Modules

A bank is fundamentally a deposit institution, not a credit-only protocol. The original v10 design described the SavingsVault as "planned" but left the contract unspecified, which left the entire "closed-loop sustainable model" claim resting on capital that did not exist. v15 specifies the two deposit-mobilization contracts directly.

SavingsVault. The SavingsVault contract accepts ERC-20 deposits (USDC primary, ETH/USDT/DAI for institutional flows), credits the depositor's balance on-chain, accrues variable yield tied to pool utilization (the kinked rate of Section 3.8), and permits withdrawal subject to the local-tier reserve ratio gate. The yield is paid in the same token as the deposit; interest accounting is per-second using a cumulative index rather than per-block

accrual, exactly matching the Compound v3 / Aave v3 supply-side pattern. The contract is upgradable via UUPS (Section 3.19) and emits `Deposit`, `Withdraw`, `InterestAccrued`, and `ReserveGateTriggered` events for The Graph indexing.

FixedDeposit. FixedDeposit issues a non-transferable on-chain receipt that locks principal for a fixed term (30 / 90 / 180 / 365 days) at an APY written at deposit time. Early withdrawal forfeits a deterministic portion of accrued interest, transparently. Fixed-term deposits provide the Local Bank tier with **duration-matched liabilities**: a 180-day FixedDeposit funds the 6-month installment tranche of a retail client at the same tier without creating asset-liability mismatch. The contract integrates with the InsuranceFund so that, under stress scenarios, fixed-term depositors are subordinated to demand depositors only after the insurance buffer is exhausted.

ERC-4626 and ERC-7540 interface alignment. The `SavingsVault` contract is designed to implement the ERC-4626 Tokenized Vault Standard (`IERC4626`), which defines a unified interface for yield-bearing ERC-20 vaults: `deposit`, `mint`, `withdraw`, `redeem`, and the asset-to-shares conversion functions `convertToShares` and `convertToAssets` [R46]. A depositor's share token (`svUSDC`) represents a pro-rata claim on the underlying USDC pool, updated continuously by the cumulative interest index described above. Aligning with ERC-4626 means the vault is auditable against a known, widely-reviewed standard; any future integrator—a yield aggregator, a cross-protocol lending market, or a liquidity router—can compose on top of it without a custom adapter. Prior to EIP-4626, every yield protocol shipped a bespoke interface (Yearn vaults, Aave aTokens, Compound cTokens), forcing each integrator to write and audit a separate adapter; ERC-4626 eliminates that per-protocol overhead and is the 2025–2026 DeFi standard for tokenized vaults. The `FixedDeposit` contract adopts ERC-7540 (the asynchronous redemption extension of ERC-4626) [R46] because fixed-term redemptions are time-gated: a request submitted at lock-up close enters a `requestId`-keyed pending state and becomes claimable only after the maturity block is reached, which fits the asynchronous request/fulfil flow that ERC-7540 formalises. This is a specification alignment, not an implementation rewrite; the underlying interest-accrual and penalty logic described above is unchanged.

Closed-loop economic argument. With SavingsVault and FixedDeposit in place, the platform's economic claim becomes verifiable: depositor capital flows in on one side, lending flows out on the other, interest collected is split deterministically between depositor yield, the insurance fund, and protocol revenue under Formula NetInterest (see List of Formulas). The model recycles capital without external subsidy, distinguishing the platform from donor-funded MFIs and matching the empirical evidence on sustainable microfinance that Section 2.1.1 cites.

3.11 On-Chain Credit Passport (Soulbound Token)

The platform issues each KYC-verified client a non-transferable on-chain credential—a Soulbound Token (SBT) following the Buterin, Hitzig, and Weyl framework [R30]—that encodes their cumulative repayment record in a schema that any compatible lending protocol can read. The credential is updated automatically whenever an installment is repaid in full, and downgraded (with `lastDefault` timestamp permanently set) after a defined number

of missed installments; the credential is never revoked, consistent with the non-revocable Buterin SBT design principle [R30]. Three concrete benefits follow.

First, the SBT solves the cold-start problem at the bank-switching boundary: a credit client who has repaid three loans at a Local Bank in Sylhet can apply for a larger loan at a Local Bank in Dhaka without re-establishing trust from zero. Second, the SBT is the on-chain primitive that the over-indebtedness control (Section 3.17.3) relies on: a client's open-loan count across the platform is publicly readable from their SBT, so simultaneous-loan caps are enforceable without an external credit bureau. Third, the schema is intentionally minimal (`creditScore`, `openLoans`, `completedCycles`, `lastDefault`, `riskTier`) so other lending protocols that adopt the standard can read the credential, allowing progressive lending to follow a client across protocols. MicroSave Consulting (December 2025) [R31] documents that Bangladesh's microfinance sector is actively shifting toward performance-based credit identity; the SBT is the technical implementation of that shift on the Crypto World Bank platform.

Public `ICreditPassport` interface for cross-protocol composability. Although the SBT schema described above is readable on-chain, the credit passport is currently siloed: only contracts within the Crypto World Bank hierarchy can query it in a standardized way. To make the credential composable with external protocols—such as third-party Local Banks in partner networks, DeFi lending markets, or impact-finance platforms—the contract exposes a public `ICreditPassport` interface. The interface defines a single standardized view function:

```
function getScore(address wallet)
    external view
    returns (uint8 riskTier, uint32 repaymentCount, bool hasDefault);
```

This read-only surface adds no new trust assumptions (no state mutation is permitted through the interface) and requires no additional gas for the issuing contract. Any external protocol that imports `ICreditPassport` can read a client's credit tier and repayment history in one call, without needing a custom adapter or a centralized credit bureau query. The cross-chain credit-passport mirroring described in Section 3.12 propagates this same interface to each deployed chain, so the composability guarantee holds across the multi-chain deployment. Platforms such as Spectral Finance and RociFi have demonstrated that on-chain credit scoring from wallet history can enable undercollateralized lending at scale [R46]; the `ICreditPassport` interface positions the Crypto World Bank credit passport as a reusable primitive within that broader on-chain credit infrastructure.

GDPR and the permanent SBT default record. The permanent, non-revocable nature of the SBT default record raises a question under GDPR Article 17 (right to erasure). The resolution is as follows. The SBT stores only a wallet address, a risk tier integer, and a Unix timestamp—no name, national ID, or personally identifiable information (PII). The wallet address is pseudonymous; it does not constitute a directly identifiable personal data record under GDPR Article 4(1) unless combined with off-chain KYC data held by the Local Bank. The Local Bank's off-chain PostgreSQL records—which do constitute personal data—are subject to data-subject deletion requests. However, deleting the off-chain record does not delete the on-chain SBT, which is by design. The regulatory analogy is a credit bureau record that survives a borrower's request to be forgotten under credit-reporting exemp-

tions in most jurisdictions. Platforms that adopt the CWB SBT standard should include this limitation in their privacy policy and terms of service, and should treat the wallet address as pseudonymous personal data under GDPR Article 89 (research and statistics exemption) where applicable. The architecture's data-minimization posture—only a hash of identity documents appears on-chain—is already compliant with the GDPR data-minimization principle (Article 5(1)(c)), and the off-chain PostgreSQL layer supports standard data-subject access and deletion workflows.

Credit tier schedule. The Credit Passport SBT encodes a five-tier credit schedule that governs maximum loan limits and interest modifiers. Table 3.22 gives the full schedule.

Table 3.22: Credit tier schedule: score thresholds, maximum loan limits, and interest modifiers per tier.

Tier	Label	Score Range	Max Loan (USDC)	Interest Modifier
1	Bronze	0–299	50	Base rate
2	Silver	300–549	250	Base − 0.5%
3	Gold	550–749	1,000	Base − 1.0%
4	Platinum	750–899	5,000	Base − 1.5%
5	Diamond	900–1000	25,000	Base − 2.0%

The AI Agent reads the client's current tier via the `get_credit_score` MCP tool and proactively coaches the client on what is needed to reach the next tier. For example: "You need 2 more on-time repayments to reach Gold tier. Your borrowing rate will then drop by 1.0%." This coaching loop—executed automatically before every loan application confirmation—creates a positive feedback mechanism that incentivises repayment discipline and progressively unlocks higher credit limits, directly addressing the credit-building gap that MicroSave Consulting [R31] identifies as the primary barrier to sustainable microfinance graduation.

3.12 Cross-Chain Bridge Architecture

A multi-chain deployment (Polygon zkEVM Cardona + Ethereum Sepolia in the prototype, with mainnet successors planned) without cross-chain state consistency is a liability: if a client has an active loan on Polygon and attempts to open a second loan by routing through Ethereum, the hierarchical borrowing-limit control could be bypassed. v15 makes the bridge architecture explicit rather than leaving it as "portable."

Bridge protocol selection. The platform uses a single canonical bridge protocol for cross-chain messaging rather than a bespoke contract. The 2025–2026 leaders are LayerZero (largest message-volume share), Axelar (GMP for arbitrary payloads), and Chainlink CCIP (institutional-grade with native MEV protection). The current prototype targets Chainlink CCIP because the platform now uses Chainlink Functions for the oracle layer (Section 3.3.2), Chainlink Price Feeds for FX, and Chainlink Automation for installment triggers (all adopted in v24), which means adopting CCIP for bridging concentrates the entire oracle and cross-chain stack on a single audited provider and minimises new trust assumptions. Bridge hacks were the largest single category of crypto loss between 2022

and 2024; concentrating on one well-audited bridge rather than maintaining multiple is the correct security posture for an academic prototype.

Cross-chain hierarchy invariant. The hierarchical control is preserved across chains by requiring that all loan state for a given client live on a single chain (the chain of the originating Local Bank). Cross-chain messages are restricted to (a) reserve-ratio updates from World Bank Reserve to National Banks across chains, and (b) credit-passport SBT mirroring across chains so that a client's credential is readable on any chain even though their loans are not. This pattern keeps borrowing-limit enforcement single-chain and avoids the consensus problem of cross-chain debt.

3.13 Multi-Entity and Cross-Tier Capital Operations

Figure 3.10: Multi-entity and cross-tier capital operations: (a) InterBankLendingPool with utilization-kinked rate model and default cascade, (b) UpwardDepositFacility with asymmetric rate structure, (c) SyndicatedLoan with Lead Arranger and Co-Lenders, (d) TranchedPool with senior–junior waterfall, (e) TreasurySwap for cross-tier asset exchange, and (f) NettingEngine for multilateral settlement.

The Chapter 1 framing of multi-directional flows (Section 1.6) and Section 1.7 promise downward, same-tier, and upward lending plus tiered client access. v10 left same-tier and upward flows at design-level only. A paper that claims a *complete* banking system must specify these flows—and the genuinely multi-entity operations they enable—at the same depth as the downward flow. This section closes that gap with six concrete contract-and-flow specifications: a fully-specified InterBankLendingPool (Section 3.13.1), upward surplus repatriation (Section 3.13.2), syndicated / club lending (Section 3.13.3), senior–junior tranched lending pools (Section 3.13.4), cross-tier treasury FX swap (Section 3.13.5), and a multilateral settlement netting engine (Section 3.13.6). The corresponding ERD entities, contract list, implementation phase plan, and Future Work are updated downstream so the database design, software-engineering stages, and architecture remain a single consistent system.

3.13.1 InterBankLendingPool: Fully Specified Same-Tier Lending

Pool placement. One `InterBankLendingPool` (IBLP) contract is deployed per hierarchical tier: `IBLP_NB` at the National-Bank tier and `IBLP_LB` at the Local-Bank tier. Membership in a tier's IBLP is gated by the on-chain role grant from the parent tier (a Local Bank can only deposit into `IBLP_LB` if its `LOCAL_BANK_ROLE` is active and unexpired—Section 3.19). The pool denominates positions in the platform's stablecoin numeraire (USDC per Section 1.10.3); ETH-denominated positions are accepted at higher tiers but cleared through the `TreasurySwap` contract (Section 3.13.5) before pool entry.

Rate model. The interbank borrowing rate $r_{\text{IB}}(U)$ follows a kinked utilization curve identical in shape to the retail kinked rate (Section 3.8) but with two distinguishing properties. First, the slope coefficients are calibrated to the SOFR-style overnight market: the base rate r_0^{IB} is set to the platform's deposit-yield benchmark, and the kink point $U_{\text{IB}}^* = 0.90$ is higher

than the retail kink (because banks have lower default risk and tighter operational reserves than retail borrowers). Second, the formula is upper-bounded by the tier's *downward* lending rate from the tier above:

$$r_{\text{IB}}(U) \leq r_{\text{downward}}(U) - \delta,$$

where δ is the governance-set risk premium spread that prevents a Local Bank from arbitraging the tier above by borrowing from `IBLP_LB` at a cheaper rate than from its parent National Bank. This bound is enforced on-chain inside the rate-setter function via a **live cross-contract call** rather than a static governance-cached parameter:

```
require(
    proposedRate <= localBank.getCurrentBorrowRate() - delta,
    "Rate exceeds downward bound"
);
```

Reading `localBank.getCurrentBorrowRate()` at the moment the rate-setter function is called ensures the bound is evaluated against the *current* downward rate, which itself shifts dynamically with pool utilization under the kinked model of Section 3.8. A static cached parameter would allow a window of arbitrage between successive governance updates during which r_{IB} could exceed the updated $r_{\text{downward}} - \delta$; the live call closes this gap atomically. The additional cross-contract read adds one `STATICCALL` to the rate-setter gas cost but imposes no cost on routine borrow/repay operations, since the bound is checked only when a rate change is proposed.

Settlement and default cascade. Loans through the IBLP are short-tenor (1-, 7-, 30-day) and are settled atomically: the borrowing bank receives the stablecoin amount in the same transaction in which its repayment obligation is recorded. If the borrowing bank defaults at maturity, the default cascade is: (1) the bank's reserve buffer above the minimum ratio is auto-debited; (2) if insufficient, the `InsuranceFund` (Section 3.10, with its claim mechanism extended in Section 3.13.2) is drawn; (3) if still insufficient, the parent tier (the bank's National Bank, or the World Bank for an NB default) absorbs the residual as a regulatory backstop and a governance vote is automatically queued through the TimeLock for the defaulting bank's role suspension. Every step emits a typed event indexed by The Graph (Section 4.8) so the cascade is fully auditable.

Operations. `IBLP_*` exposes four user functions and three governance functions:

- `deposit(uint256 amount, uint8 maturityCode)` — surplus bank supplies liquidity.
- `borrow(uint256 amount, uint8 maturityCode)` — deficit bank draws against pool, gated by per-bank credit ceiling, role expiry, and the IBLP-vs.-downward-rate bound.
- `repay(uint256 borrowId)` — atomic principal + interest settlement.
- `withdraw(uint256 supplyId)` — surplus bank reclaims principal + accrued interest.
- `setRateParams(...)` — TimeLock-gated (24–48 h) governance update.
- `setMembershipPolicy(...)` — role-expiry / per-bank ceiling adjustment.
- `triggerCascade(uint256 borrowId)` — public function any caller can invoke

once a borrow is overdue; pays the caller a small bonus from the defaulter's reserve buffer, mirroring the Aave liquidation incentive (Section 3.9).

3.13.2 Upward Surplus Repatriation

The upward direction is the dual of downward lending: a Local Bank whose reserve buffer exceeds the minimum ratio by more than a governance-set "excess threshold" (default 20 % above the minimum) may deposit the excess into its parent National Bank, earning the parent's *deposit yield*. A National Bank with similarly excess capital may deposit into the World Bank Reserve under the same logic. Three controls keep the upward flow safe:

1. **Asymmetric rate structure.** The upward deposit yield is strictly lower than the downward borrowing rate ($r_{\text{up}} < r_{\text{down}} - \delta$), creating a natural term structure across the hierarchy and removing the round-trip arbitrage incentive of borrowing-down to deposit-up.
2. **Withdrawal gate.** The depositing bank may withdraw its upward deposit on demand, but the withdrawal triggers a reserve-ratio recomputation at the depositing bank's own tier first; if the withdrawal would push the depositor below its minimum reserve ratio, the call reverts. This protects retail depositors at lower tiers from a liquidity-squeeze upstream.
3. **Daily withdrawal rate limit (bank-run circuit breaker).** To prevent a coordinated bank-run scenario in which multiple National Banks simultaneously withdraw upward deposits, the `UpwardDepositFacility` enforces a per-National-Bank daily withdrawal ceiling of 20% of that bank's total outstanding upward deposit liabilities. Withdrawals exceeding this ceiling within a 24-hour window are queued and executed in the next settlement window. This mirrors the standard withdrawal-rate limits applied by central bank standing deposit facilities and is enforced on-chain via a per-bank `withdrawnToday[bank]` counter that resets at each UTC midnight checkpoint.
4. **Recursive cap.** A given bank's net upward exposure (sum of upward deposits less own borrowings) is capped at 30 % of its on-chain assets, so a single failing bank cannot drain the entire upstream tier.

The on-chain contract is named `UpwardDepositFacility` (one instance per parent tier); it is a thin extension of the SavingsVault contract (Section 3.10) restricted by role, with separate accounting registers (`upwardSupplied[bank]`, `upwardYieldOwed[bank]`) so that upward and retail flows do not commingle in the same reserve pool.

FATF Travel Rule compliance for inter-tier capital flows (Future Work). The Financial Action Task Force Travel Rule (FATF Recommendation 16) requires that transfers above USD 1,000 in most jurisdictions include originator and beneficiary information alongside the transfer. For CWB's inter-tier capital flows — Local Bank → National Bank → World Bank through the `UpwardDepositFacility` and `InterBankLendingPool` contracts — this requirement applies to any single transfer above the threshold. The off-chain Travel Rule data packet accompanying such a transfer takes the following structure:

```
{
  "originator_institution": "LocalBank-BD-042",
  "originator_tier": 3,
  "beneficiary_institution": "NationalBank-BD-001",
```

```
  "beneficiary_tier": 2,
  "amount_usdc": 5000,
  "purpose": "IBLP_REPAYMENT",
  "onchain_tx_hash": "0xABC...",
  "timestamp": "2026-06-01T10:00:00Z"
}
```

This packet is stored in the PostgreSQL `audit_logs` table and is available to regulators via the audit request workflow (Section 3.18.4). FATF Travel Rule compliance is scoped to Future Work for a Bangladesh deployment; the data structure specification above constitutes the thesis contribution, while implementation of the cross-tier notification protocol is deferred pending jurisdiction-specific regulatory guidance from Bangladesh Bank's Financial Intelligence Unit.

3.13.3 Syndicated and Club Lending

For loan sizes that exceed a single bank's prudent exposure ceiling—typically the 1 000 + ETH bracket at the National-Bank tier (Table 1.1)—multiple entities at the same or adjacent tier co-fund a single loan through a `SyndicatedLoan` contract. This is the on-chain analogue of a real-world syndicated loan or club deal, and it directly implements the multi-entity co-lending and co-funding capability described in the introduction (Section 1). In the syndicated model, **a group of institutional entities pools capital together to fund a borrower that no single entity can or should serve alone**—distributing both the risk and the return pro-rata across all co-lenders. This pattern has no equivalent in any existing DeFi protocol, which universally pool anonymous liquidity without distinguishing institutional relationships or enforcing on-chain consent among named co-lenders.

Roles. A syndicated loan has three distinct on-chain roles: a **Lead Arranger** (the bank that structures and underwrites the deal), **Co-Lenders** (the participating banks supplying capital), and the **Credit Client** (a Local Bank, National Bank, or institutional client per Table 1.1). The Lead Arranger holds a small underwriting fee (10–20 basis points) plus a portion of the interest spread; Co-Lenders earn pro-rata interest by capital share.

Lifecycle.

1. **Proposal.** Lead Arranger calls `proposeSyndicate(uint256 totalAmount, uint8 borrowerTier, uint256 minSubscription)` which mints a syndicate identifier and emits a `SyndicateProposed` event. Off-chain, the deal terms (covenants, schedule) are pinned to IPFS and the content hash is stored in `syndicateDocHash[id]` for immutable cross-reference.
2. **Subscription window.** Co-Lenders call `subscribe(uint256 syndicateId, uint256 commitment)` within a governance-set window (default 7 days). Their commitment is escrowed in the contract.
3. **Funding threshold.** If aggregate commitments meet $\geq 90\,\%$ of `totalAmount` by window close, the deal funds; otherwise commitments are returned and the proposal expires. This avoids the partial-funding race condition.
4. **On-chain consent vote.** Each Co-Lender's wallet must call `confirmTerms(uint256 syndicateId)` after subscription. Disbursement requires confirmation from Co-Lenders representing at least 75% of the subscribed capital by value (supermajority, not unanimity), preventing a griefing attack in which a small Co-Lender subscribes

and then deliberately withholds confirmation to block the deal at minimal cost. The Lead Arranger may also reject a Co-Lender's subscription before the window closes. A Co-Lender that subscribes but does not confirm forfeits a bond proportional to its committed capital (1% of committed amount), making non-confirmation economically costly at scale.

5. **Disbursement.** On supermajority confirmation (75%-by-capital-share threshold met), the contract atomically transfers the aggregate to the borrower in a single transaction. Each Co-Lender's exposure share is recorded as `shareBps[lender][syndicateId]`.
6. **Interest accrual + repayment.** Client installments flow into the `SyndicatedLoan` contract; the contract distributes principal + interest pro-rata to each Co-Lender on every installment via the basis-points share registry. The Lead Arranger receives its underwriting fee on the first repayment.
7. **Default handling.** On default, the LiquidationEngine (Section 3.9) seizes collateral; recovery is distributed pro-rata to Co-Lenders by the same basis-points share. A Co-Lender's loss is therefore bounded by its commitment, which is the desired multi-entity risk-sharing property.

Cross-tier syndication. The same contract supports *cross-tier* syndication when permitted by governance: one National Bank and three of its child Local Banks may co-fund a single large project loan where the NB is Lead Arranger. This is the genuine "lending from upper tiers as well as lower tier together" pattern the supervisor asked about; the role-grant logic of the `SyndicatedLoan` contract simply permits any wallet holding `NATIONAL_BANK_ROLE` or `LOCAL_BANK_ROLE` (and any `WORLD_BANK_ADMIN`-approved institutional client) to subscribe.

3.13.4 Senior–Junior Tranched Lending Pools

For risk-segmented co-funding, the platform supports the senior–junior tranche pattern established by Goldfinch [32] and Maple [31]. The `TranchedPool` contract divides depositor capital into two notional tranches with distinct seniority:

- **Senior tranche:** lower yield, first-loss protection, suitable for risk-averse depositors (Local Banks redepositing surplus, or institutional partners).
- **Junior tranche:** higher yield, takes first loss on default, suitable for risk-tolerant capital (governance-token-aligned depositors, the protocol treasury, or impact-investment partners).

Waterfall payment. On each repayment, the contract pays from the inflow in the order: (i) senior accrued interest, (ii) junior accrued interest, (iii) senior principal pro-rata, (iv) junior principal pro-rata. On default the order reverses for losses: the junior tranche is written down first; only when junior principal reaches zero is senior principal touched. The subordination ratio (junior / senior) is fixed at deposit time and ranges from 10 / 90 (institutional-grade) to 30 / 70 (impact-aligned) per the policy table set by the World Bank Admin.

Stress test. A 30 % default-rate stress test (matching the agent-based simulation parameter of Section 4.7) on a $10 : 90$ junior:senior pool would zero out the junior tranche but preserve the senior tranche fully; on a $30 : 70$ pool a 30 % default writes the junior down

to zero and senior takes a 0 % nominal loss. These dynamics are formalized as Foundry invariants (Section 4.6) to give automatic regression detection.

3.13.5 Cross-Tier Treasury FX Swap

The FXModule mentioned in Section 3.17 handles end-user FX. Cross-tier treasury FX—the case where a National Bank holding an ETH treasury must source USDC stablecoin to fund a Local Bank that has elected stablecoin denomination—is a distinct operation served by the `TreasurySwap` contract.

Mechanism. `TreasurySwap` is an oracle-priced atomic-swap contract restricted to wallets holding a banking role. A National Bank calls `swap(asset_from, asset_to, amount)`; the contract reads the Chainlink price feed, applies the governance-set treasury spread (typically 5–10 basis points, tighter than retail FX), and executes the swap atomically against either a counterparty bank's posted offer or the protocol's own `TreasuryAMM` liquidity pool. The Chainlink price reading and the swap execution sit inside the same transaction; there is no intermediate state and no settlement risk.

Hierarchy invariant. Treasury swaps cannot push the swapping bank below its minimum reserve ratio (the swap function recomputes the post-swap ratio before any state change). Swaps above a governance-set threshold (default $1 million-equivalent) require a two-signature confirmation from the bank's Safe multisig. Each swap emits a `TreasurySwapExecuted` event with both legs of the swap, the oracle reading used, and the spread, supporting both internal audit and regulatory reporting.

3.13.6 Multilateral Settlement Netting Engine

A four-tier hierarchical bank with same-tier interbank flows accumulates many small interbank obligations during a settlement period. Settling each one as a separate on-chain transaction is wasteful; the `NettingEngine` contract compresses them.

Workflow. At the end of each settlement window (1 hour, 1 day, or governance-set), an off-chain Settlement Coordinator computes a netted obligation matrix from the queued IBLP and TreasurySwap orders. The matrix is committed on-chain as a Merkle root, and a single `settleBatch(bytes32 batchRoot, bytes calldata proofs)` call executes the entire batch in one transaction. Each bank's net debit/credit is applied to its on-chain balance, and the constituent transactions are marked settled. This pattern follows the Hyperledger Cactus and BIS Project Agora cross-ledger netting designs [R39, R40].

Why this matters. For a network with n active banks per tier, naive bilateral settlement requires $O(n^2)$ transactions per period; multilateral netting compresses this to $O(n)$ at the cost of one off-chain coordination step. At full Local-Bank deployment ($n = 50$) this is a 25$\times$ reduction in on-chain settlement footprint per settlement window. The trade-off is one explicit trust assumption: the Settlement Coordinator role must be held by a Safe multisig (the same WORLD_BANK_ADMIN Safe), and any participating bank may dispute a batch within a challenge window by calling `disputeBatch(uint256 batchId, bytes proof)`.

Trust assumption disclosure. The Settlement Coordinator is a trusted central operator for the netting computation—an acknowledged trust regression relative to the platform's otherwise trustless design. This is the same trade-off made by real-world clearinghouses (e.g., CLS, SWIFT GPI netting): they are trusted institutions operating under legal agreement, not trustless systems. The CWB NettingEngine adopts the same honest posture: netting is scoped to the permissioned institutional tier (NBs and LBs that have signed legal agreements with each other), not applied to the retail client tier. A future upgrade path toward ZK-proven netting (where a zk-SNARK proves the sum-of-obligations-equals-zero property without revealing individual pair amounts) is noted in Future Work.

Database mirror. Each `NettingBatch` is mirrored off-chain to a PostgreSQL `netting_batch` table with one `netting_entry` row per netted obligation, allowing dashboard display, regulator export, and the agent-based simulation (Section 4.7) to read historical settlement-compression ratios.

Settlement failure path. The current `settleBatch` specification assumes every participant in a netting cycle can fully meet their net debit obligation. In practice, a bank may be underfunded at settlement time—its on-chain balance is insufficient to cover its net debit position. Without an explicit failure path, a single underfunded participant can deadlock the entire netting cycle, leaving all other participants' net credits unprocessed. The `NettingEngine` therefore exposes a `settlePartial(uint256 cycleId)` fallback function that is callable by the Settlement Coordinator (or any participant) once the standard settlement window has elapsed without full completion. `settlePartial` iterates the netted obligation matrix in decreasing net-credit order: participants with sufficient balances are settled immediately and marked `SETTLED`; participants whose on-chain balance is below their net debit are marked `PENDING_DEFAULT`, their net debit is recorded in a `pendingObligations[bank]` queue, and the Settlement Coordinator emits a `SettlementDefaultFlagged(cycleId, bank, shortfall)` event. The queued obligation is resolved in the next settlement cycle or through the IBLP default cascade (Section 3.13.1). This partial-fill design mirrors the settlement-failure handling in real-world clearing systems such as CLS and is consistent with the platform's existing default-cascade model; it does not change the trust assumptions around the Settlement Coordinator role.

3.13.7 Database, Phase, and Contract Consistency for Multi-Entity Operations

To keep the data model, software-engineering stages, and architecture coherent, the following entities, phase deliverables, and contracts are added downstream of this section:

ERD entities added (data model in Chapter 3 ERD). `INTERBANK_LOAN` {loan_id (PK), tier, lender_bank_id (FK), borrower_bank_id (FK), principal, maturity_code, rate_bps, status, created_at, settled_at}; `UPWARD_DEPOSIT` {deposit_id (PK), depositing_bank_id (FK), parent_bank_id (FK), principal, yield_owed, created_at}; `SYNDICATE` {syndicate_id (PK), lead_arranger_id (FK), borrower_id, total_amount, status, doc_hash, opened_at, funded_at}; `SYNDICATE_MEMBER` {syndicate_id (FK), lender_bank_id (FK), commitment, share_bps, confirmed_at}; `TRANCHED_POOL` {pool_id (PK), local_bank_id (FK), borrower_id, senior_principal, junior_principal, subordination_bps, status}; `TREASURY_SWAP` {swap_id (PK), bank_id (FK), asset_from, asset_to, amount_from, amount_to, oracle_reading, spread_bps, executed_at}; `NETTING_BATCH` {batch_id (PK), tier, coordinator_id, batch_root, opened_at, settled_at}; `NETTING_ENTRY` {batch_id (FK), src_bank_id, dst_bank_id,

net_amount}. These are appended to the ERD prose at the end of Chapter 3 alongside SavingsAccount, FixedDeposit, LoanGroup, GroupMember, CurrentAccount, and InsuranceFund, and will be drawn into the updated ERD figure in the diagram-audit session.

Smart contracts added (Appendix B). `InterBankLendingPool` (one per tier), `UpwardDepositFacility`, `SyndicatedLoan`, `TranchedPool`, `TreasurySwap`, `NettingEngine`. The complete architecture therefore grows from nine to fifteen modular contracts.

Implementation phase plan update (Chapter 1, Methodology in Brief). Phase II takes ownership of `InterBankLendingPool`, `UpwardDepositFacility`, and `TreasurySwap`; Phase III takes `SyndicatedLoan`, `TranchedPool`, and `NettingEngine` (the latter requires the off-chain Settlement Coordinator service, which sits alongside the AI/ML pipeline already scheduled for Phase III). The Foundry invariant suite (Section 4.6) is extended in Phase III with three additional invariants: (i) IBLP $r_{\text{IB}} < r_{\text{down}} - \delta$ at all times; (ii) Syndicate disbursement = sum of confirmed commitments; (iii) NettingBatch sum of net debits = sum of net credits.

3.14 Reentrancy and Security Analysis

Checks-Effects-Interactions (CEI) pattern. The CEI pattern is applied to all state-mutating functions in the lending contracts. The three functions most vulnerable to reentrancy are:

1. `disburseLoan(address borrower, uint256 amount)` — sends ETH to the borrower. Mitigation: update `loanStatus[borrower] = LoanStatus.ACTIVE` *before* calling `payable(borrower).transfer(amount)`, and apply `nonReentrant`.
2. `processInstallment(uint256 loanId)` — updates the repayment schedule. CEI order: check → mark installment paid → emit event → release interest share.
3. `allocateCapital(address nationalBank, uint256 amount)` — cross-tier ETH transfer. Apply both `nonReentrant` and an explicit `require(allocatedTo[nationalBank] + amount <= maxAllocation)` check before any state change.

The DAO hack (2016, ≈$60 million) and Curve Finance reentrancy exploit (2023, ≈$70 million via Vyper compiler bug) demonstrate that reentrancy is not a solved problem in DeFi, even with guards [R6]. Formal verification with Certora or Mythril is planned for the final thesis security audit.

Flash loan scope. Flash loan attacks primarily exploit price-sensitive logic, specifically protocols that use on-chain pool spot prices for collateral valuation [R5]. In the current prototype, no price-sensitive logic exists because stablecoin integration is not yet implemented, collateral valuation is not automated on-chain, and oracle feeds have not yet been integrated. Therefore, the current contracts do not present a meaningful flash loan attack surface. Once stablecoin integration and oracle-based collateral pricing are implemented (planned for the final thesis phase), mitigations must include: (1) TWAP oracles rather than spot price reads, (2) multi-block confirmation requirements for large collateral updates, and (3) Chainlink price feeds with circuit breakers.

Upgradeability via UUPS (ERC-1822). All three implemented contracts (and the six planned ones) use the OpenZeppelin Upgradeable contracts in the **UUPS (Universal Up-**

gradeable Proxy Standard) pattern rather than the Transparent Proxy pattern. UUPS is cheaper to deploy (no separate ProxyAdmin contract) and the upgrade authorization logic lives in the implementation contract, making it directly governable via the existing RBAC. `_authorizeUpgrade` is gated on `onlyRole(WORLD_BANK_ADMIN)`, which in production is held by a Safe 3-of-5 multisig (Section 3.19). Without upgradeability, any post-deployment bug would require redeploying from scratch and migrating all on-chain state; with UUPS the team can patch a bug and propose the upgrade for multisig approval in minutes, with a full on-chain audit trail.

TimeLock on governance actions. Every privileged action that changes a system-level parameter—interest rates, reserve ratios, role grants at or above the National Bank tier, contract upgrades—routes through a minimum 24-to-48-hour `TimelockController` (OpenZeppelin) delay. This is the standard pattern used by Compound, Aave, MakerDAO, and every serious DeFi protocol since at least 2021. It provides a defensible window in which a malicious proposal can be detected and cancelled before execution. For the university lab demo specifically, the delay is shortened to five minutes purely so examiners can witness a governance proposal queued and executed live; the production delay is set at deploy time and cannot be reduced below the minimum without a TimeLock-gated governance vote of its own.

A **dual-governance path** is specified to handle security emergencies. The standard path uses the 24–48 h TimeLock for all planned parameter changes and upgrades. An emergency path uses a **Security Council** — a 4-of-7 Safe multisig composed of members distinct from the `WORLD_BANK_ADMIN` — that can execute emergency upgrades within a 2-hour window without the TimeLock, subject to a mandatory on-chain post-incident disclosure within 48 hours. The Security Council can only execute pre-approved emergency action types (pause, upgrade, role revocation); it cannot change system parameters. This mirrors the dual-path model of Aave (Guardian address + ShortExecutor) and MakerDAO (Emergency Shutdown Module + Security Council), which experience shows to be necessary when a reentrancy bug or oracle manipulation is found in production and 24 hours of exposure can drain the entire reserve.

Role expiry timestamps. Role grants in the system are not permanent. Every `grantRole` call records an expiry block number; every privileged function call checks `require(roleExpiryBlock[msg.sender][role] > block.number)`. Block numbers are used rather than `block.timestamp` to avoid the SWC-116 vulnerability, in which validators on Polygon PoS can manipulate `block.timestamp` within a $\pm$15-second window—potentially extending the effective validity of an expired role. Block numbers are not manipulable in the same way. Off-chain display converts block numbers to approximate calendar dates using the expected block time. The client KYC level is bound to a one-year expiry by default; bank operator roles are bound to the institution's current registration validity; the World Bank Admin role itself is renewed annually by a governance vote so that even the top-of-hierarchy authority is not effectively permanent.

Granular per-function pause. The naive "`Pausable` on the whole contract" pattern is too coarse: pausing the entire World Bank Reserve when only one Local Bank has an incident causes unnecessary disruption to every other tier. The platform uses a per-function pause registry (`functionPaused[bytes32 functionId]`) so that governance can suspend only `LOAN_DISBURSEMENT` while keeping `DEPOSITS` and `WITHDRAWALS` open. This is also auditable: each pause emits a `FunctionPaused` event with the actor and the timestamp, in-

dexed by The Graph (Section 4.8).

Failed-attempt lockout and address whitelisting. After 3 failed confirmation responses (unclear or ambiguous consent) in a single agent conversation session, the agent pauses all interactions for 10 minutes and logs the incident to `AGENT_ACTION_LOG` with `status = FAILED`. After a detected command injection attempt — an adversarial prompt that attempts to bypass the confirmation gate or impersonate a prior consent turn — the session is immediately terminated and logged. Loan disbursement destination addresses are fixed at KYC registration time and cannot be changed without a 24-hour delay plus 2FA re-verification, preventing address-substitution attacks in which a compromised session attempts to redirect a disbursement to an attacker-controlled wallet.

3.15 System Modeling

This section presents the system analysis diagrams that model the platform's structure, behavior, and data flow.

3.15.1 Use Case Diagram

The system involves nine actors (five primary, four secondary) across 20 representative use cases (Figure 3.11). The five primary actors (A1–A5) are: Retail Client (three sub-types by KYC tier), Local Bank Admin, National Bank Admin, World Bank Admin, and AI Agent. The four secondary actors are: Regulatory Authority (A6), Chainlink DON (A7), Blockchain Validator (A8), and External Auditor (A9). A5, the AI Agent, is modelled as a primary actor with restricted write permissions gated by the human confirmation protocol; A6–A9 are modelled as secondary actors. Note that the primary actor classes in the use-case diagram are a condensation of the formal nine-actor taxonomy defined in Table 3.19: Visitor and Registered Retail User are pre-conditions for the CLIENT (BORROWER) role rather than independent diagram actors, and Local Bank Operator / Approver are merged into the Bank Approver role at the diagram level.

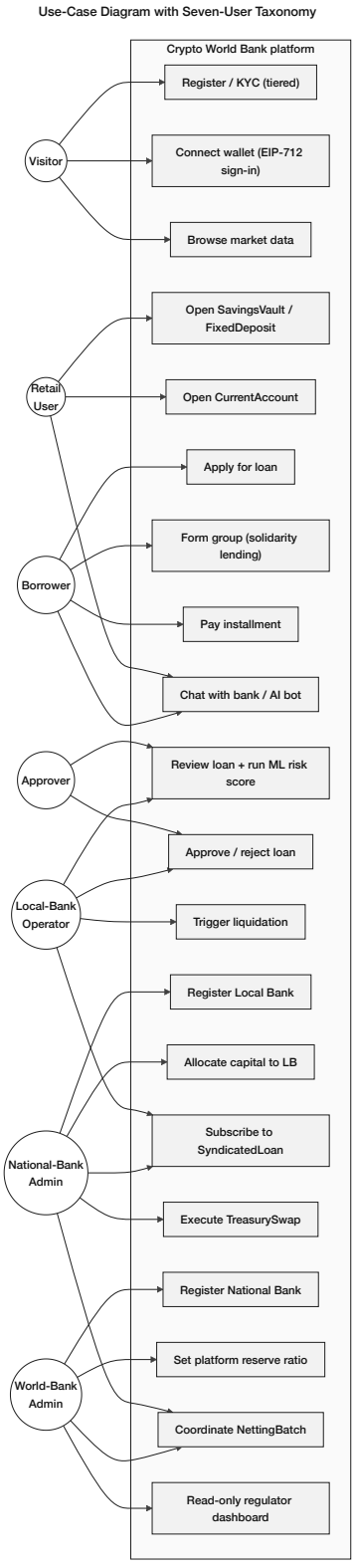

Figure 3.11: Use-case diagram reflecting the nine-actor taxonomy of Table 3.19: five primary actors (Retail Client A1, Local Bank Admin A2, National Bank Admin A3, World Bank Admin A4, AI Agent A5) and four secondary actors (Regulatory Authority A6, Chainlink DON A7, Blockchain Validator A8, External Auditor A9), mapped to 20 representative use cases covering KYC, deposits, lending, AI-agent banking operations, multi-entity operations, and regulator oversight.

3.15.2 Activity Diagrams

Figures 3.12–3.14 present the platform's seven core operational flows. Related flows are grouped onto a single figure page to keep the layout dense and readable.

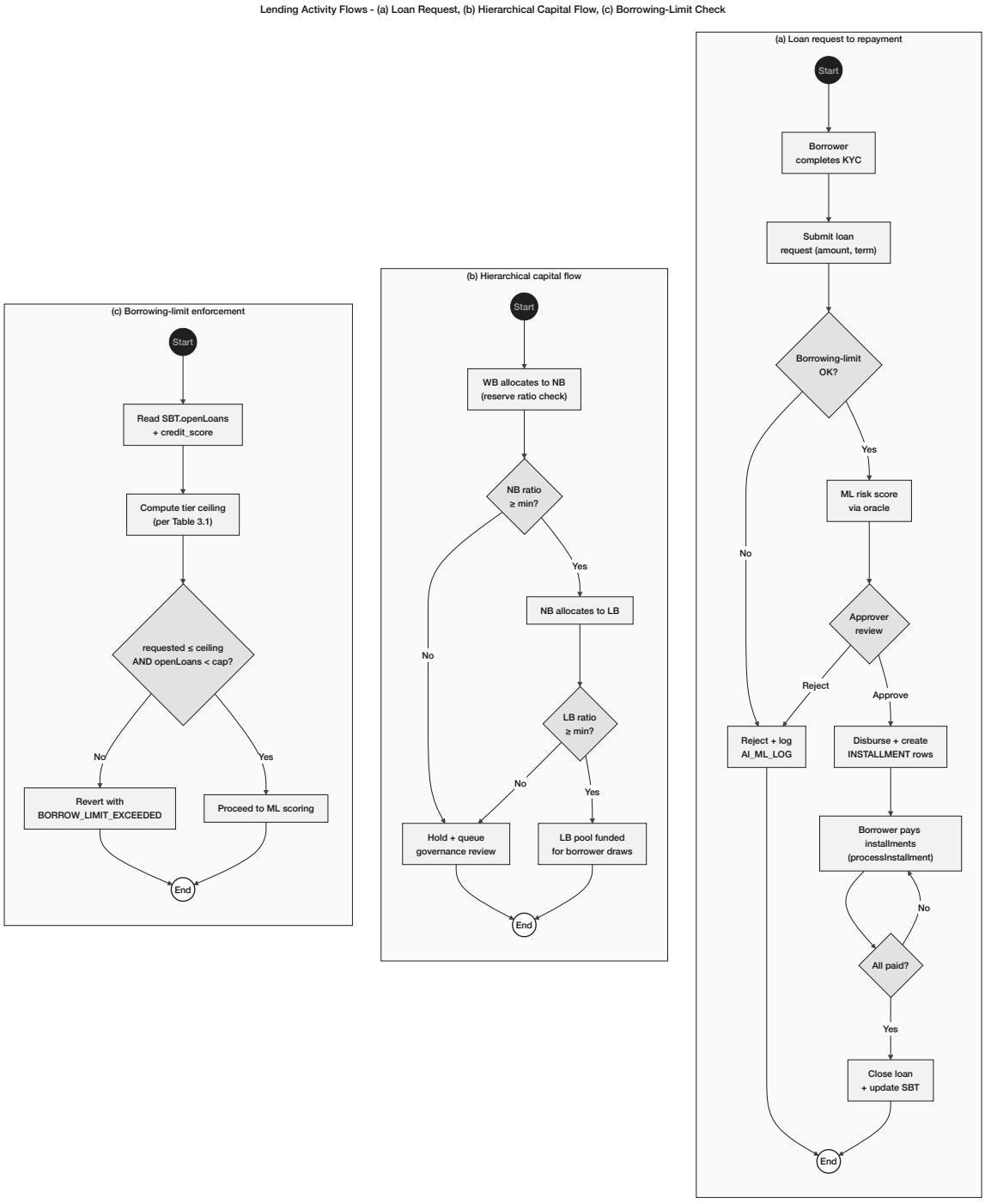

Figure 3.12: Lending activity flows: (a) loan request to repayment, (b) hierarchical capital flow from World Bank Reserve to Local Bank, (c) borrowing-limit enforcement via the credit-passport SBT.

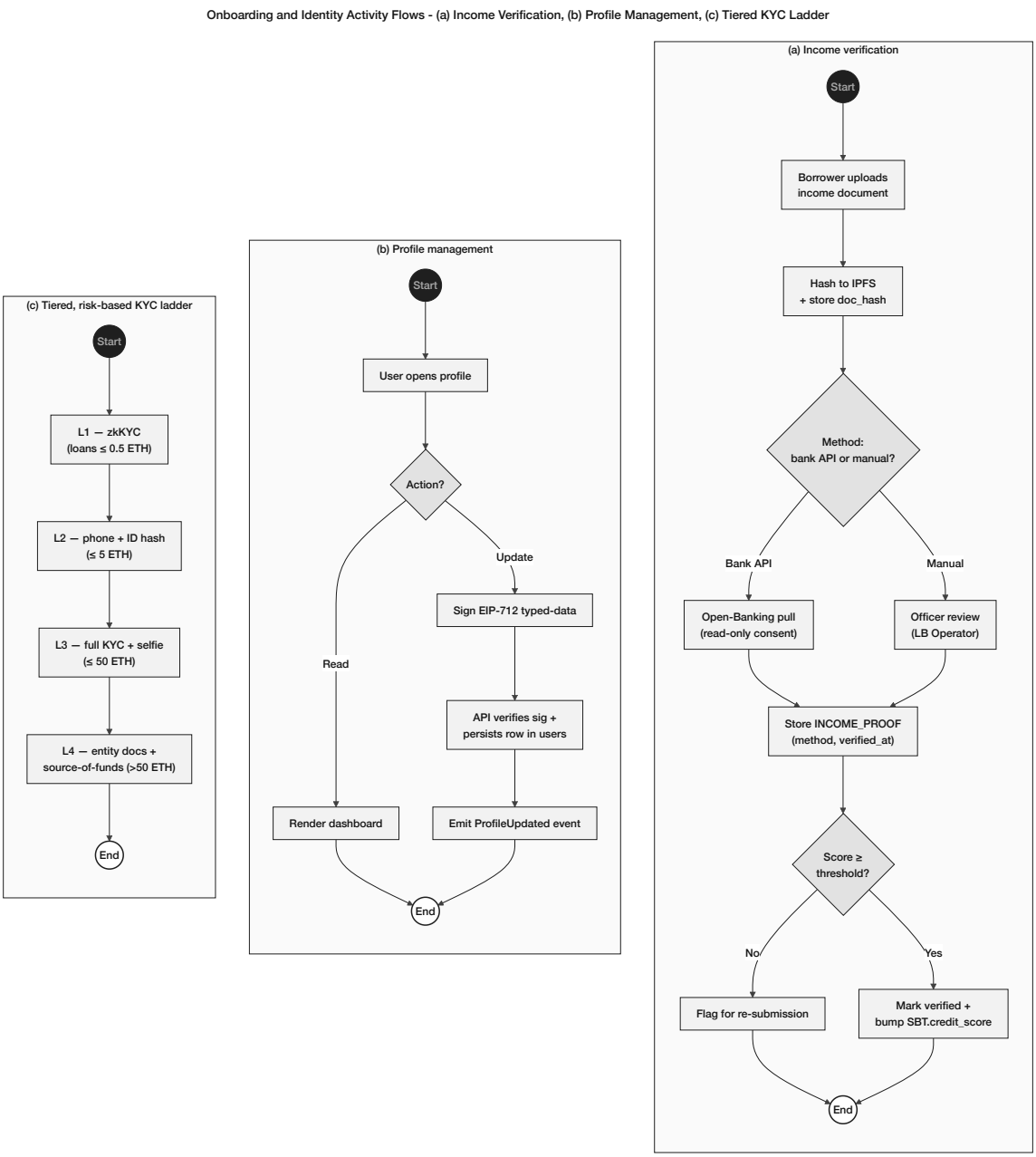

Figure 3.13: Onboarding and identity flows: (a) income verification (Open-Banking pull or officer review), (b) profile management with EIP-712 signed updates, (c) tiered, risk-based KYC ladder from zkKYC (L1) to entity onboarding (L4).

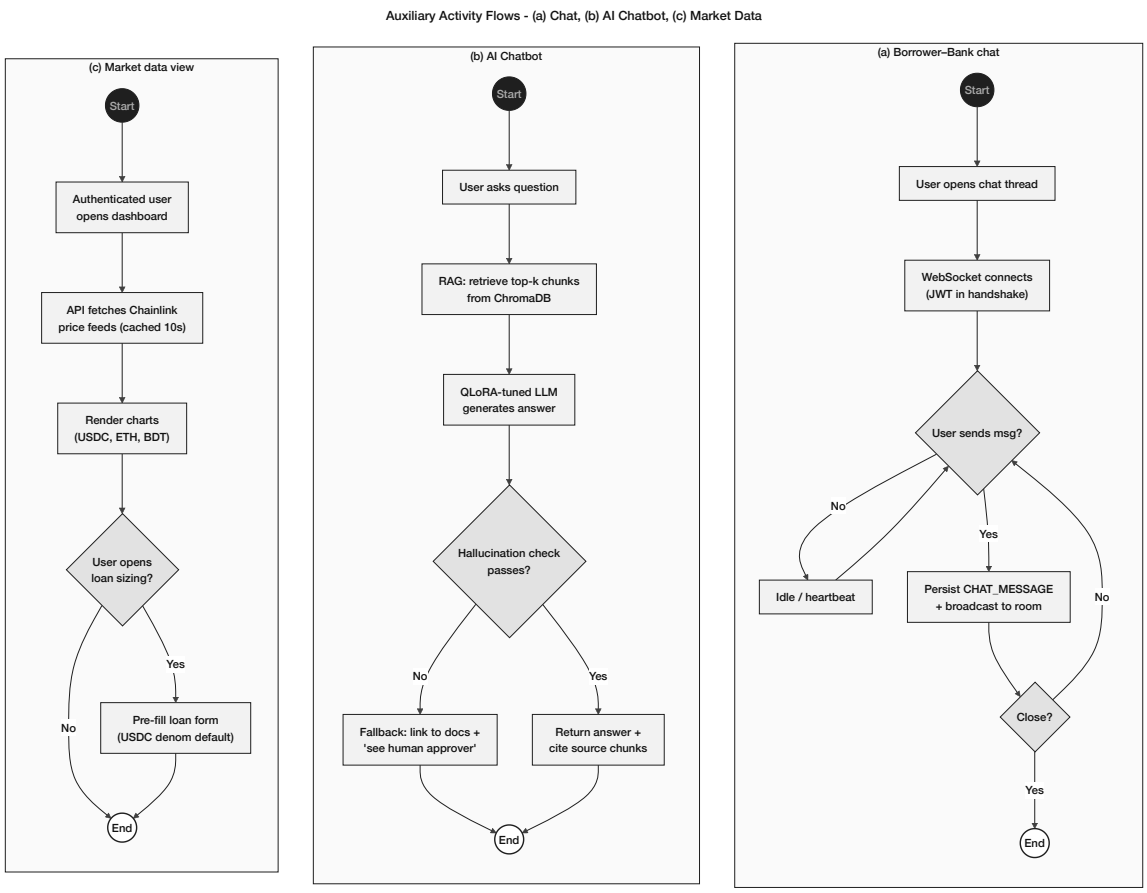

Figure 3.14: Auxiliary activity flows: (a) client–bank chat over authenticated WebSocket, (b) RAG-augmented AI chatbot with hallucination guard, (c) live market-data retrieval and pre-filled loan sizing.

SAR activity flow. The Suspicious Activity Report (SAR) workflow is triggered when the Isolation Forest anomaly detector (Section 3.20) flags a wallet with an anomaly score above the 0.75 threshold. The five-step flow is:

1. Isolation Forest flags the wallet: `anomaly_score > 0.75`.
2. `AI_ML_LOG` records the detection event (wallet, score, timestamp, feature vector).
3. Express.js backend emits Kafka topic `aml-alert`; the event is consumed by the compliance dashboard.
4. Bank officer reviews the alert in the admin compliance queue:
   - **False positive:** Dismiss and document reason (audit trail in `audit_logs`).
   - **Confirmed:** Generate SAR record in `audit_logs`; notify the tier above (National Bank); freeze wallet via `LocalBank.freezeAccount(clientId)`.
5. The `freezeAccount` function is gated by the `onlyApprover` modifier and is Foundry-tested (Phase III invariant suite). A frozen wallet cannot initiate new loan disbursements or installment payments until the freeze is lifted by the tier above.

3.15.3 Data Flow Diagrams

Figure 3.15 consolidates the Level-0 context diagram with the Level-1 decompositions of the core lending subsystem and the deposit / interbank / FX / netting subsystems on a single page.

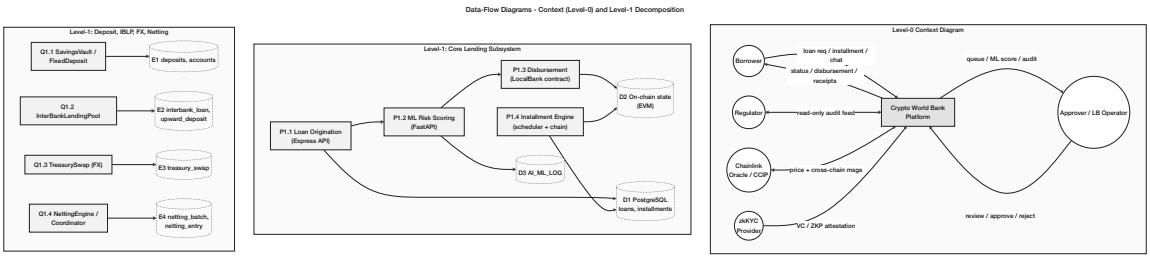

Figure 3.15: Data-flow diagrams: Level-0 context view of the Crypto World Bank platform with five external entities; Level-1 decomposition of the core lending subsystem (origination, ML scoring, disbursement, installment engine, and their data stores); and Level-1 decomposition of the deposit-mobilization, InterBankLendingPool, TreasurySwap, and NettingEngine subsystems.

3.15.4 Sequence Diagrams

Figures 3.16–3.19 consolidate the platform's nine interaction flows into four panels: loan approval with reject alternative, installment and income, hierarchical banking with market data and borrowing-limit, and chat with AI-chatbot.

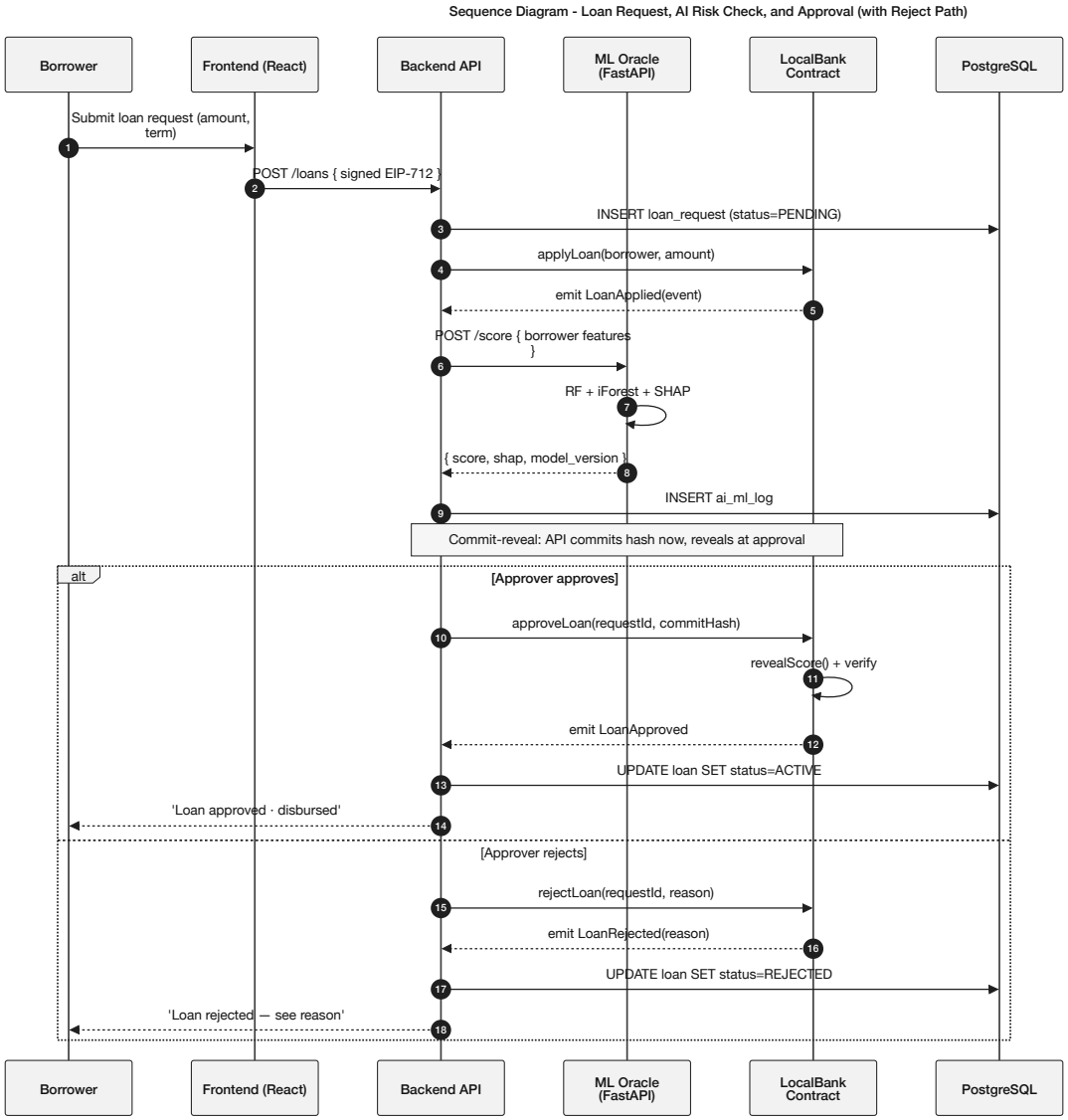

Figure 3.16: Sequence diagram for the loan-approval flow with reject alternative: the client submits an EIP-712 signed request, the ML oracle returns a SHAP-explained score via commit-reveal, and the approver either approves (revealing the commit) or rejects (with reason logged on-chain).

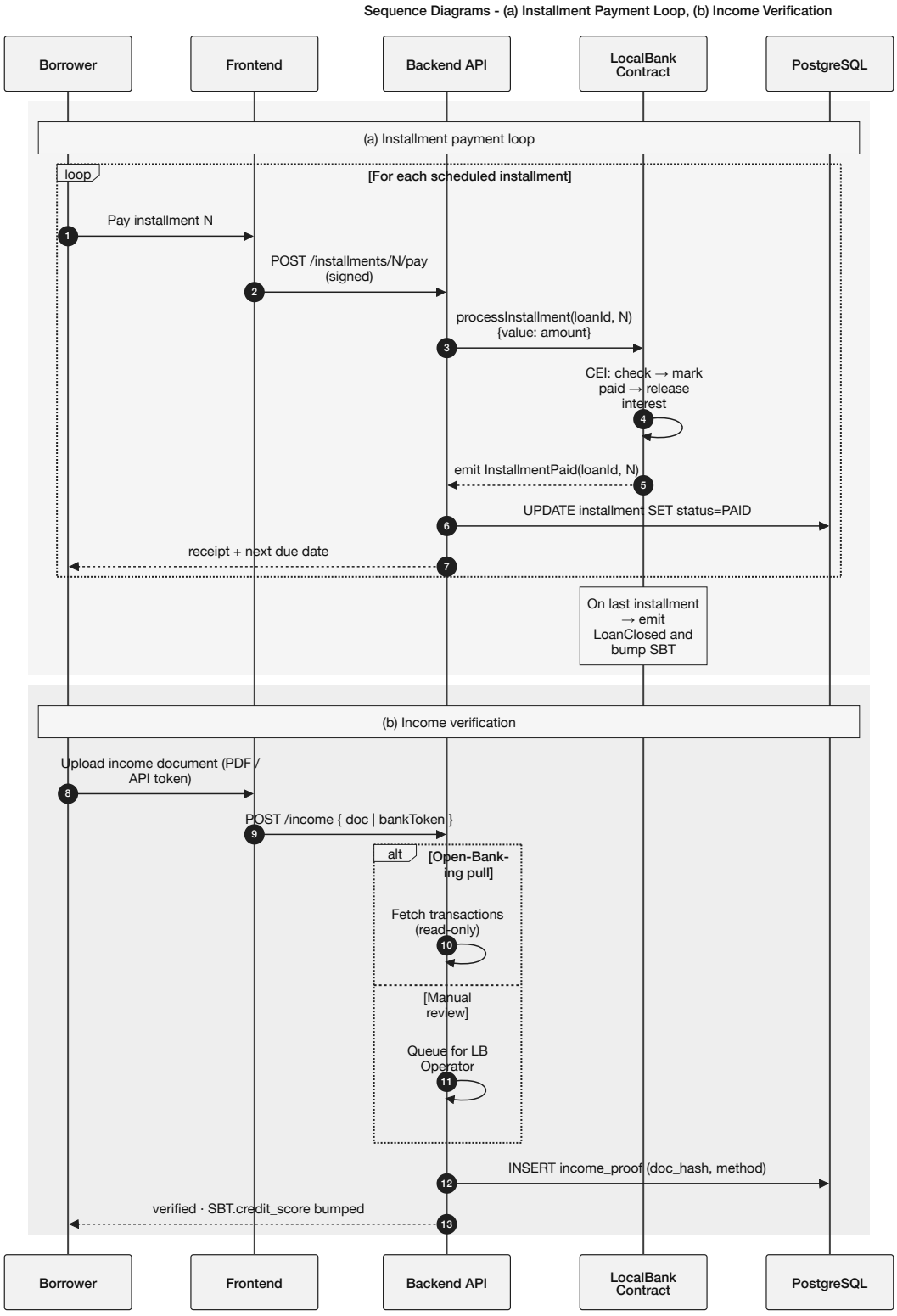

Figure 3.17: Sequence diagrams: (a) installment payment loop using the CEI pattern with SBT update on the final installment, and (b) income verification with Open-Banking pull or manual officer review.

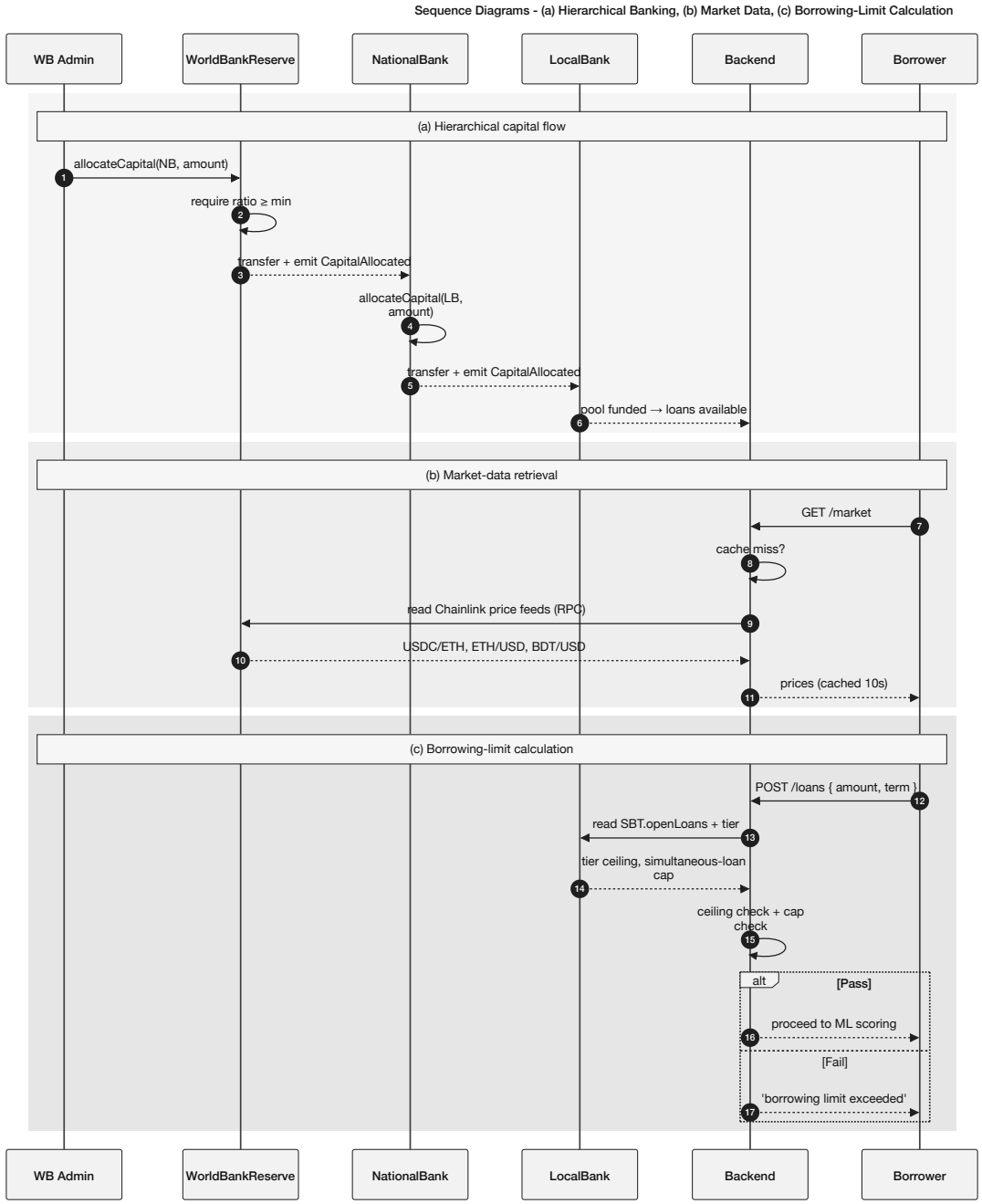

Figure 3.18: Sequence diagrams: (a) hierarchical capital flow with reserve-ratio gates at every tier, (b) market-data retrieval via cached Chainlink feeds, (c) borrowing-limit calculation reading the credit-passport SBT before ML scoring.

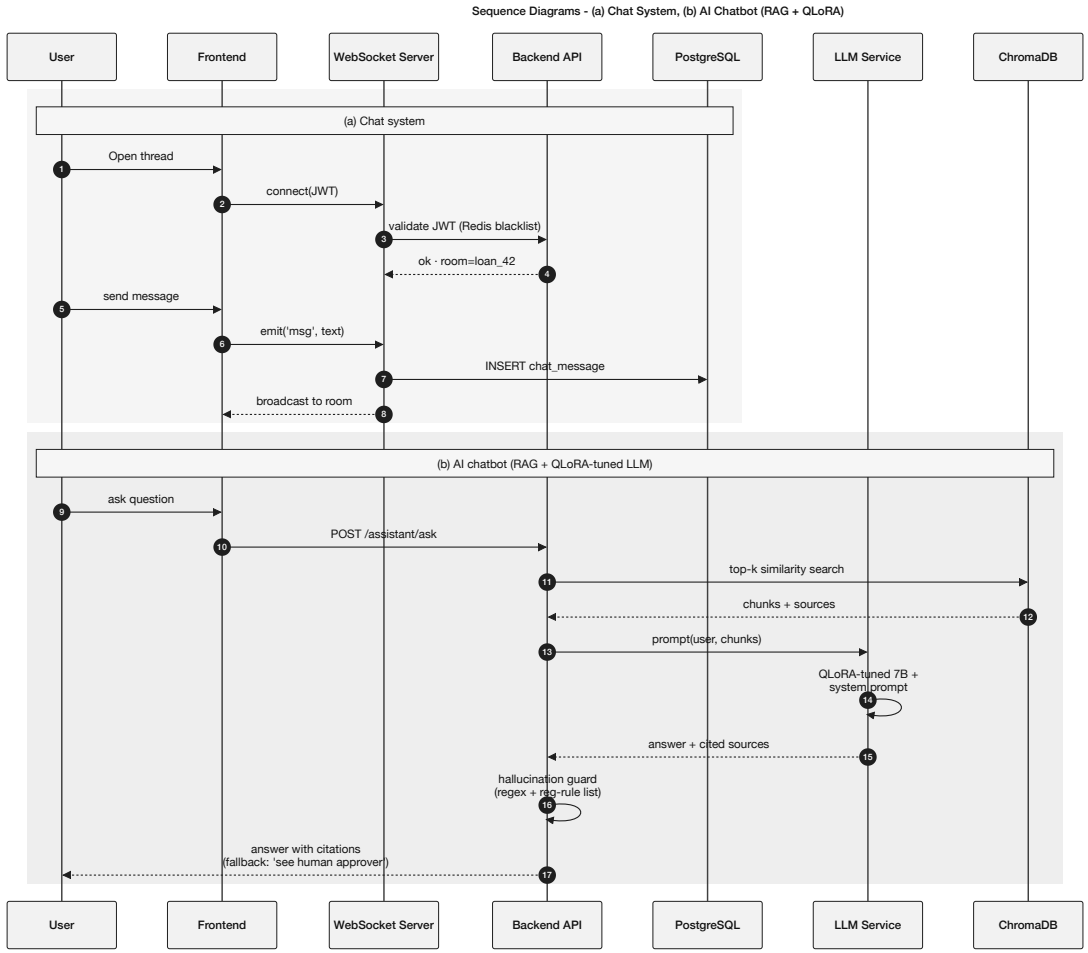

Figure 3.19: Sequence diagrams: (a) client–bank chat over an authenticated WebSocket, and (b) AI chatbot pipeline (ChromaDB top-k retrieval, QLoRA-tuned LLM, hallucination guard, citation-anchored answer).

3.15.5 Four-Tier Capital Flow

Figure 3.20 illustrates the hierarchical capital flow and cascading repayment structure.

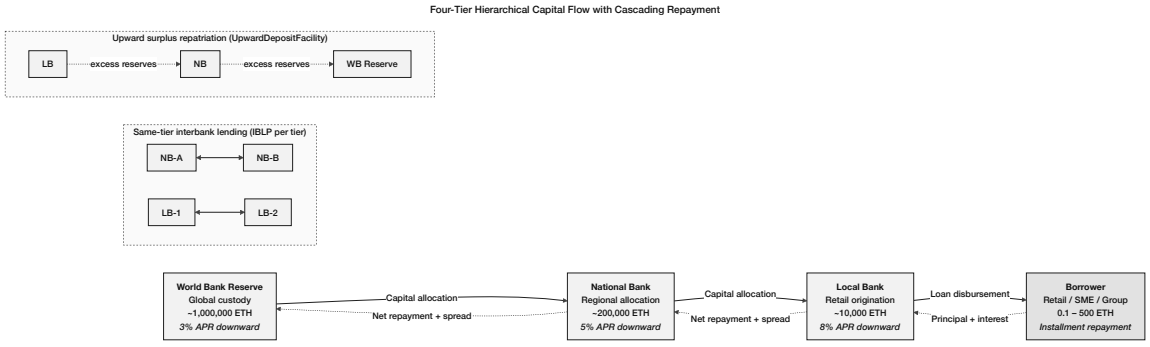

Figure 3.20: Four-tier hierarchical capital flow with cascading repayment, plus the same-tier interbank-lending pools (InterBankLendingPool, Section 3.13.1) and the upward-surplus repatriation facility (UpwardDepositFacility, Section 3.13.2). The asymmetric rate structure ($r_{\text{up}} < r_{\text{down}} - \delta$) is enforced on-chain.

3.16 Auxiliary Dual-Currency Facility

The cryptocurrency exchange feature of Crypto World Bank is built upon the cryptocurrency exchange facility that is an auxiliary facility incorporated into the current banking infrastructures all over the world. Instead of being a separate exchange, the platform is a service that is provided by all participating banks as an additional service to the existing product portfolio.

- **Integration model:** The dual-currency facility is offered by all participating banks as an add-on service to their existing product portfolio. No additional banking license or separate entity is needed.
- **Eligibility determination:** Eligibility of the dual-currency service is based on the bank officers managing the lending operations, on the basis of existing KYC/AML compliance, account status, and lending relationship history.
- **No defaulting of conditions:** The project does not override, modify, or default any existing banking conditions, regulatory requirements, or contractual obligations.
- **Scope:** The facility enables fiat-to-crypto and crypto-to-fiat conversions, cryptocurrency-denominated lending and repayment, and transparent on-chain transaction records—all within the governance structure of the participating bank.

3.17 Banking Product Suite

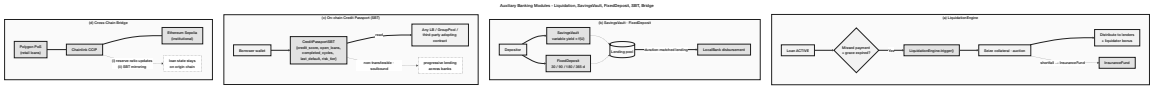

Figure 3.21: Auxiliary banking modules: (a) LiquidationEngine grace-period trigger and collateral auction with InsuranceFund shortfall fallback, (b) SavingsVault and FixedDeposit with duration-matched feeding of the lending pool, (c) on-chain Credit Passport (Soulbound Token) read by any participating bank, and (d) Cross-Chain Bridge via Chainlink CCIP carrying reserve-ratio updates and SBT mirroring.

This section describes the banking products that the platform is designed to support across tiers. While the current prototype implementation focuses primarily on hierarchical lending, the complete system design includes deposit products, transactional accounts, solidarity lending, and multi-currency operations.

3.17.1 Savings Products

The platform offers savings products at the Local Bank tier and, by extension, at higher tiers for institutional participants. Standard savings accounts provide liquid deposits with variable yield tied to platform utilization, allowing deposit pricing to respond to market conditions rather than discretionary rate-setting. Fixed-term deposits lock capital for defined periods (e.g., 30/90/180/365 days) in exchange for a higher agreed yield written into the contract at deposit time. Early withdrawal can be permitted with an automatically enforced penalty (forfeiture of a portion of accrued interest), which is transparent and deterministic.

The yield on standard savings accounts is algorithmically determined by platform utilization: when loan demand is high and available liquidity is low, savings yields rise to attract

deposits; when liquidity is abundant, yields fall. This creates a self-regulating capital cycle where savings and lending are linked through a single on-chain formula rather than through discretionary rate-setting committees.

Fixed-term deposits serve a distinct function in the banking architecture: they provide the Local Bank tier with predictable, duration-matched liabilities that can be deployed into longer-term installment loans without creating dangerous asset-liability mismatches. A depositor who locks funds for 180 days effectively funds the 6-month installment tranche of a retail client at that tier, eliminating the duration mismatch that causes bank runs in traditional fractional reserve systems.

The closed-loop argument is central to the platform's economic sustainability: unlike donor-funded microfinance models that depend on external capital injections, a savings-funded lending model recycles depositor capital through the lending pool continuously. Interest earned by the platform on loans is partially redistributed to depositors as yield, partially retained as protocol revenue, and partially allocated to the insurance fund, creating a self-sustaining three-way split that does not require new token issuance or external subsidy.

3.17.2 Checking and Transactional Accounts

In addition to interest-bearing savings products, the platform provides transactional accounts (checking/current-account equivalents) for day-to-day financial activity. Transactional accounts carry no yield but impose no lock-in, allow transfers between registered accounts, and serve as the primary account from which loan installments are debited and income receipts are credited. Transfers between accounts within the platform are settled atomically: a transaction either completes fully or reverts, eliminating intermediate settlement ambiguity.

The atomic settlement property of on-chain transfers eliminates the intermediate state that causes the majority of interbank disputes in traditional banking. In the correspondent banking model, a payment can exist in a "funds in transit" state for two to five business days during which the sender has debited but the receiver has not credited, creating reconciliation complexity and dispute resolution costs. On-chain transfers settle in a single transaction: either both the debit and credit execute or neither does, with no intermediate state possible.

For retail users in developing economies, transactional accounts on the platform serve as a substitute for the informal cash-handling systems that currently dominate daily financial activity. Recurring payroll deposits, utility payment debits, and peer transfers can all be scheduled as programmable transactions, reducing the friction of cash management without requiring users to interact directly with smart contracts for every operation.

3.17.3 Group / Solidarity Lending

The platform implements a group lending module inspired by the solidarity group model used in microfinance. A group of three to twenty registered clients can form a lending group on-chain. The collateral model for group loans distinguishes two tiers of client capability. For clients who can provide on-chain collateral (USDC or ETH), each member contributes individual collateral into a shared pool; mutual liability enforcement is automatic. For clients who lack individual on-chain collateral—the primary target population of 1.4 billion unbanked adults—the platform uses a **cold-start credit pathway**: new clients with zero on-chain history who have passed Level 1 KYC receive a *provisional credit tier* that

(a) requires mandatory group membership of at least 5 members, (b) caps individual loan amounts at the Level 1 KYC borrowing limit, and (c) expires after 3 months from first disbursement. After the first successful repayment cycle, the client's SBT is populated with real on-chain history and normal credit scoring applies. This explicit cold-start pathway prevents the circular deny loop—where no history leads to high risk score which leads to denial which leads to no history—that would otherwise permanently exclude the platform's intended beneficiaries. After three successful cycles, the client graduates from the provisional tier to the standard ML-gated credit model (Section 4.2). Repayment is tracked per member, and if a member defaults beyond a grace period, the contract can automatically cover the shortfall from the shared collateral pool under programmable mutual liability rules where collateral exists. Where collateral is absent, default triggers *credit-score degradation*: the SBT's `riskTier` field is downgraded to the highest-risk tier and the `lastDefault` timestamp is permanently recorded. The credential is never revoked—consistent with the Buterin, Hitzig, and Weyl (2022) [R30] Soulbound Token design principle that SBTs are non-revocable identity-bound credentials. Future borrowing does not require formal revocation: no lender on the platform accepts a wallet whose SBT `riskTier` is at the default level, making borrowing effectively impossible in practice while preserving the permanent, honest record of the default event. This is analogous to a credit bureau record that permanently reflects a default rather than erasing it. Over time, group repayment histories contribute to credit scoring to enable progressive lending (larger loans and improved terms after subsequent cycles).

The group lending module is directly inspired by the solidarity group model developed by BRAC, the organization whose name this university bears. BRAC's group lending program, operating across Bangladesh and subsequently 11 other countries, demonstrates that social collateral—the mutual accountability of group members—can substitute for physical collateral among populations with no formal credit history. The Crypto World Bank translates the structural logic of this model into programmable contract enforcement: the mutual liability that BRAC enforces through field officer visits and weekly group meetings is encoded in the GroupLendingPool contract through automatic collateral claims and on-chain consent recording.

However, it is critical to note that BRAC and Grameen Bank's repayment rates above 95% are a direct consequence of social pressure mechanisms: weekly physical group meetings, face-to-face peer accountability, community reputation, and field officers who know borrowers personally. These mechanisms are preventive—they stop defaults from occurring. Smart contract enforcement is punitive—it responds after default. An on-chain group of wallet addresses cannot replicate preventive social pressure. Consequently, the expected default rate for the CWB group lending module at launch is likely to be substantially higher than 5%, estimated at 8–15% consistent with the base-case and stress-test scenarios in Table 5.23, until on-chain credit history accumulates sufficient predictive signal. The 95% Grameen figure is cited as the design target and theoretical upper bound under mature social-trust conditions, not as a near-term operational forecast.

The full group loan lifecycle proceeds as follows. First, formation: three to twenty registered clients form a group on-chain, each contributing individual collateral to a shared pool contract. Second, application: the group submits a collective loan application specifying the total amount, per-member share, and repayment schedule. Third, consent: every group member must sign the application transaction before it is escalated to a bank

approver, enforcing unanimous consent and preventing coerced participation. Fourth, approval: the bank approver reviews the group's aggregate credit history, income verification documents, and shared collateral ratio before approving disbursement. Fifth, disbursement: funds are distributed to each member's account in their individual share as defined by the formula $\text{Share}_i = \text{LoanTotal}/N_{\text{members}}$. Sixth, repayment: each member repays their individual installments independently; on-chain tracking records per-member performance transparently to all group members. Seventh, mutual liability enforcement: if a member defaults beyond the grace period, the contract automatically covers the shortfall from the shared collateral pool, protecting the lending bank's position while distributing the cost proportionally across the group. Eighth, credit history improvement: successful group repayment cycles are permanently recorded on-chain and incorporated into each member's credit-passport SBT (Section 3.11), enabling progressive lending with larger amounts and better terms in subsequent cycles.

Stablecoin denomination at the group tier. Group loans at Tier 4 are denominated in USDC by default (Section 1.10.3). Solidarity-group clients are precisely the demographic for which ETH-denominated debt creates the highest real-cost shock under crypto-price drawdowns, so the stablecoin-first rule is enforced most strictly at this tier.

Over-indebtedness risk control. Bangladesh's microfinance sector serves 41.56 million accounts across 724 licensed MFIs, and the NUS Economics Society analysis (May 2025) [R37] together with the IJAR microfinance report [R38] document over-indebtedness—borrowers taking simultaneous loans from multiple MFIs and inflating apparent repayment capacity—as the single most persistent problem in Grameen-style group lending. v15 adds three explicit controls. First, a per-wallet cap of two simultaneous active group loans, checked on-chain via the credit-passport SBT (Section 3.11). Second, a cross-platform debt disclosure flag in the SBT itself, readable by any other platform that adopts the standard, supporting industry-wide debt-load visibility. Third, a 30-day cooling-off period between consecutive group-loan cycles for any individual client, enforced by the GroupLendingPool contract. These controls are not in addition to the social-pressure model of solidarity lending; they are the on-chain encoding of the same caution that Grameen field officers exercise in person. A paper that cites Grameen and BRAC but does not address this risk would be ignoring a known failure mode of the model it builds on.

Cross-pool query and debt-to-income enforcement. The three controls above operate within a single Local Bank's GroupLendingPool. To prevent a client from bypassing per-bank caps by simultaneously holding group loans at *different* Local Banks within the hierarchy, the `GroupLendingPool.applyGroupLoan` function performs a **cross-pool query** before approving any new group loan. Specifically, the function reads `creditPassport.getScore(wallet)` (the `ICreditPassport` interface defined in Section 3.11) and checks `openLoans >= MAX_SIMULTANEOUS_GROUP_LOANS` against the global SBT state, which is updated by every Local Bank in the hierarchy on each disbursement. Because the SBT is the authoritative cross-bank debt registry, this single read replaces the need for a bespoke cross-bank oracle or an off-chain credit bureau query. In addition, the function enforces a **maximum debt-to-income (DTI) ratio** using the `income_hash_freshness` feature already computed in the ML pipeline (Section 4.2). If the client's most recent income hash is stale (older than 90 days, configurable by governance), the application is flagged for manual review

regardless of the SBT score. If the income hash is fresh, the function computes $\text{DTI} = (\text{existingDebt} + \text{requestedAmount})/\text{estimatedMonthlyIncome}$ and rejects applications where $\text{DTI} > 0.40$ (the governance-set ceiling, consistent with standard microfinance prudential norms). Neither check requires new infrastructure: the RBAC system already knows which Local Banks exist, and the ML feature set already includes income-hash freshness as one of the four application features enumerated in Section 4.2.

3.17.4 Foreign Exchange and Multi-Currency Operations

A worldwide banking platform must support participants operating in different currency environments. The platform's FX module is designed to use decentralized price oracle networks to obtain verifiable exchange rates for supported asset pairs. Currency conversion can be offered at transparent oracle-derived rates with a disclosed spread, set by governance, that forms an additional revenue stream. To reduce liability-side volatility for retail clients, the platform is designed to support stablecoin-denominated lending (e.g., USDC/USDT) while allowing collateral in volatile cryptoassets (e.g., ETH).

3.17.5 Trade Finance Facilitation (Planned)

Trade finance instruments (letters of credit, guarantees, documentary collections) can be added as planned extensions for import–export participants. In a smart contract design, a buyer's bank locks payment in escrow, and release occurs automatically upon verified presentation of shipping documents via trusted data sources. This extension is out of scope for the current prototype but follows the same principles of programmable settlement and auditable execution.

3.17.6 Privacy-Preserving Identity Compliance (Planned)

The regulatory compliance requirements of a worldwide banking platform extend beyond on-chain audit trails. Formal KYC and AML processes require users to prove legal identity, verify source of funds, and satisfy jurisdiction-specific screening requirements. Storing this personal data on a public blockchain creates a fundamental tension between regulatory compliance and user privacy.

The planned resolution is a zero-knowledge proof based KYC layer in which a licensed identity provider performs the full KYC verification off-chain and issues a verifiable credential to the user. The user then presents a ZKP to the banking contracts that proves they hold a valid credential from an approved provider without revealing any personal information on-chain. The contract validates the proof cryptographically and sets a `kycVerified` flag on the wallet address, enabling access to regulated operations such as large loans, FX conversions, and cross-border transfers. This approach satisfies regulatory requirements while preserving on-chain privacy and avoiding the liability of storing personal financial data in an immutable public record.

3.18 Governance Framework

3.18.1 Network Membership Governance

Table 3.23: Network membership governance.

Governance Aspect	Implementation
Member on-boarding	World Bank owner registers National Banks; National Banks register Local Banks; Local Banks designate approvers — all enforced on-chain.
Member off-boarding	Deactivation flags in smart contracts; cascading access revocation.
Regulatory oversight	Audit log emission via smart contract events; planned read-only regulator dashboard.
Permission structure	Hierarchical: Owner → National Bank → Local Bank → Approver → Retail Client; enforced by on-chain role check modifiers.
Network operations	Pause/unpause mechanism for emergency response; emergency withdrawal for critical situations.

This governance table defines membership and onboarding rules that control who can join the banking network and under which role. The structure is intended to preserve institutional accountability by separating responsibilities across tiers rather than relying on undifferentiated token-holder governance alone.

3.18.2 Business Network Governance

Table 3.25: Business network governance.

Governance Aspect	Implementation
Business charter	Defined in project documentation; operational parameters coded in smart contract constants.
Common services	Reserve management, loan lifecycle orchestration, event-driven notification system.
Business SLA	Testnet phase: best-effort availability. Production phase: 99.5% target API-layer uptime (team-controlled; multi-region deployment). Chain-level liveness is determined by Polygon PoS validator consensus and is not under the platform team's control; the team monitors Polygon network status and maintains degraded-mode fallbacks (read-only dashboard) if chain activity pauses.
Regulatory compliance	Architecture designed for audit trail generation; data partitioning supports GDPR-style data subject requests.

This business governance table outlines operational controls such as approvals, limits, and escalation paths for higher-risk actions. The goal is to combine programmable enforcement with human oversight, matching real banking governance while retaining on-chain audit trails.

3.18.3 Technology Infrastructure Governance

Table 3.27: Technology infrastructure governance.

Governance Aspect	Implementation
Distributed IT structure	Client-side frontend (decentralised delivery); blockchain layer (fully decentralised); backend API (centralised, horizontally scalable).
Technology assessment	Continuous evaluation of EVM alternatives (L2 rollups, sidechains) for cost and performance optimisation.
On-chain / off-chain data services	Clearly partitioned (see Table 3.16); event listeners synchronise state between layers.
Risk mitigation	Smart contract pause mechanism; ReentrancyGuard; input validation; planned formal security audit.

This technology governance table specifies how protocol parameters and infrastructure changes are managed, including security controls and upgrade considerations. It supports the thesis emphasis on safe staged rollout and maintainability under evolving regulatory and security requirements.

3.18.4 Regulatory Compliance Considerations

As the platform is active in the regulated banking sector:

- Our prototype stage would be solely on the **public testnets** without attached real monetary value.
- The architecture would facilitate **audit logs generation** to be reviewed by regulatory regimes on the sandbox programs.
- Future production deployment would engage **regulatory sandbox programs** in target jurisdictions.

Although the current prototype does not process KYC/AML data, a complete worldwide banking architecture must account for compliance at the design level. Privacy-preserving KYC design is described in detail in Section 3.17.6.

Institutional trust bootstrapping. A practical question for any new institutional platform is how the first institutional participants are recruited before the network has an established track record. The CWB bootstrapping strategy is three-pronged. *First, the academic pilot route:* the platform is released as open-source software under an Apache 2.0 licence, allowing any university or NGO to deploy their own instance for research purposes. The first institutional participants are academic departments, not commercial banks, which face no regulatory barriers to testnet participation. *Second, the regulatory sandbox route:* the platform applies for a Singapore MAS Project Guardian sandbox slot or a UAE DIFC Innovation Testing Licence, which provides regulatory cover for a controlled pilot with a licensed financial-institution partner (see Future Work item 14 and Section 5.11). *Third, the BRAC alignment route:* as the platform that algorithmically encodes the BRAC solidarity group lending model, a natural first institutional-partner discussion is with BRAC International's technology arm, which already operates digital microfinance infrastructure across 11 countries. None of these routes require the platform to be trusted by a commercial bank on Day 1; trust is built incrementally through the academic and sandbox record.

FATF Travel Rule (R.16) — inter-tier flows. As specified in Section 3.13.2, the FATF R.16 Travel Rule applies to inter-tier capital flows above USD 1,000 in most jurisdictions. The data packet structure specified for `UpwardDepositFacility` and `InterBankLendingPool` transfers captures all FATF-required originator and beneficiary fields. Implementation of the cross-tier Travel Rule notification protocol is Future Work contingent on jurisdiction-specific Bangladesh Bank regulatory guidance.

SAR workflow — Isolation Forest to regulator. The five-step SAR flow specified in Section 3.15 (Isolation Forest detection → Kafka `aml-alert` → compliance queue → officer review → freeze + audit log) constitutes the platform's full AML compliance pathway. The `freezeAccount(clientId)` function in `LocalBankPool.sol` is gated by `onlyApprover` and is Foundry-tested in Phase III to ensure no privilege escalation is possible. Phase 2 will add integration with Bangladesh Bank's Financial Intelligence Unit reporting portal and automated SAR PDF generation in the required regulatory format.

Regulator audit request flow (specify only). A formal regulator audit request sequence is specified as a contribution to the institutional compliance architecture:

1. Regulator submits a signed request off-chain to the World Bank admin.
2. World Bank admin verifies the regulatory mandate via multi-sig confirmation.
3. System extracts: client loan history (`loans` table), installment payment records, AI/ML risk score log (`AI_ML_LOG`), on-chain transaction hashes, and SAR history if applicable.
4. Extracted data is packaged and encrypted with the regulator's public key.
5. The audit response is logged immutably in `audit_logs` (append-only policy, Section 3.4.3).

This workflow is a specification contribution; implementation is Future Work. The append-only `audit_logs` DB policy (Section 3.4.3) ensures the audit response record cannot be altered after the fact, providing the immutability guarantee that regulatory compliance requires.

3.18.5 Asset Tokenization

- **Current implementation:** The native blockchain currency (ETH/MATIC) is used as the reserve and loan currency.
- **Planned extension:** Support for ERC-20 stablecoins (USDC, USDT) for lending operations in USD; tokenized collateral instruments for lending situations where there isn't enough collateral.

3.19 Five-Layer Defense-in-Depth Security Architecture

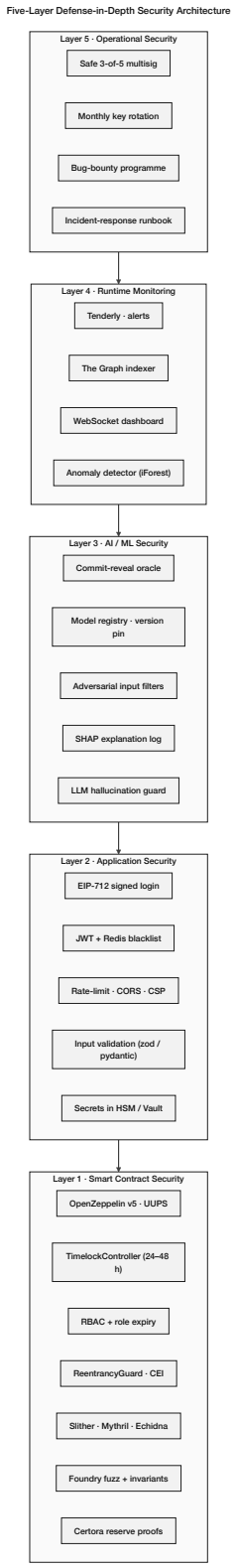

Figure 3.22: Five-layer defense-in-depth security architecture. Each layer is necessary; breaching the smart-contract layer requires defeating every layer above it. Layer 1 (smart contract): OpenZeppelin v5, UUPS, TimeLock, RBAC, ReentrancyGuard, audit suite. Layer 2 (application): EIP-712 sign-in, JWT, rate-limit. Layer 3 (AI/ML): commit-reveal oracle, model registry, SHAP, hallucination guard. Layer 4 (runtime): Tenderly, The Graph, WebSocket dashboard, anomaly detection. Layer 5 (operations): Safe multisig, key rotation, bug-bounty.

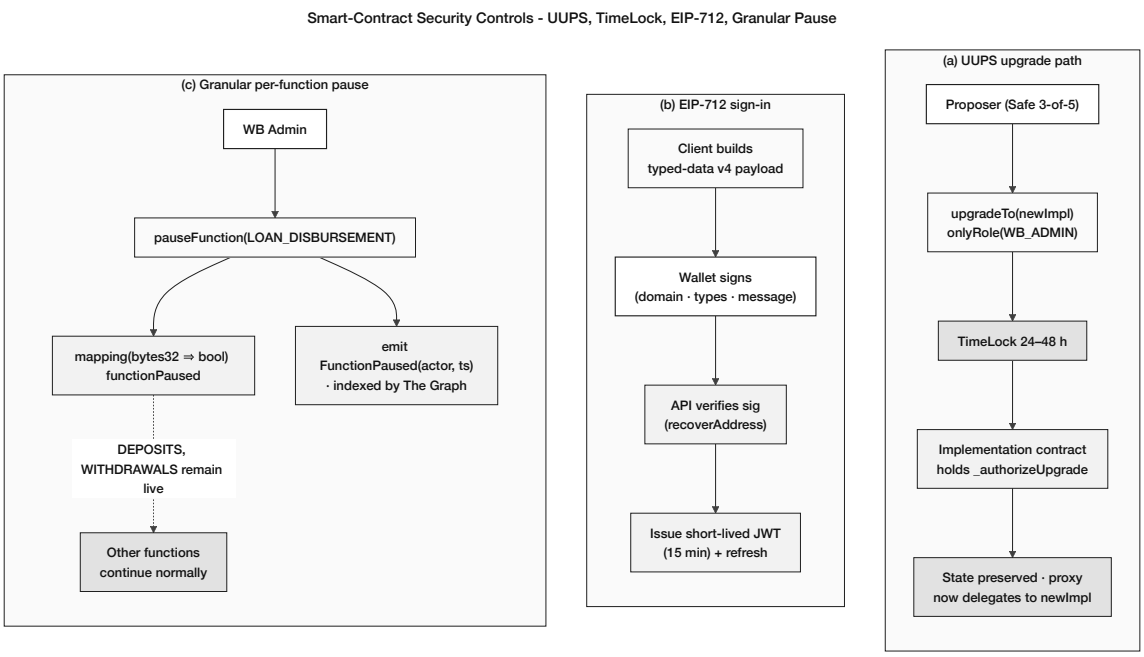

Figure 3.23: Smart-contract security controls applied in v15: (a) UUPS (ERC-1822) upgrade path with TimeLock-gated authorization; (b) EIP-712 sign-in (typed-data v4 with domain separation and 15-minute JWT); (c) granular per-function pause registry indexed by The Graph for transparent governance.

The correct mental model for a blockchain banking platform's security is not "smart-contract security only"—it is a five-layer defense-in-depth stack in which breaching the contract layer requires defeating every layer above it. The Crypto World Bank adopts the following layered model, summarized in Table 3.29 and inspired by current (2025–2026) audit-tooling and runtime-monitoring best practice from Trail of Bits, Cyfrin, Tenderly, and OpenZeppelin Defender.

Table 3.29: Five-layer defense-in-depth model for the Crypto World Bank platform. Each layer is necessary; breaching the contract layer requires defeating every layer above it.

Layer	Component	CWB Design Choice
L5	Operational security	Safe (Gnosis) 2-of-3 multisig for `WorldBankAdmin`; 3-of-5 for global parameter changes; documented key-rotation schedule; hardware-wallet signer for the WorldBankAdmin Safe (Ledger Gen5).
L4	Runtime monitoring	Tenderly alerts on six critical triggers (large disbursement, repeated requests, reserve-ratio drop below 20%, failed `disburseLoan`, role grants, governance pause); The Graph subgraph for indexed event history; transaction simulation predeployment.
L3	AI/ML security	Random Forest fraud probability + Isolation Forest anomaly score + SHAP explanation per loan, delivered via commit-reveal oracle; transaction-graph distance to flagged wallets; GNN-based wallet-graph clustering for relational fraud (Section 4.3). *Note: session-level behavioral biometrics (page-view timing, terms-acceptance latency) require client-side instrumentation not yet in the schema and are a specified Future Work extension.*
L2	Application security	EIP-712 wallet-signed JWT (15-minute lifetime + refresh); `slowapi` rate limiting (5/min auth, 60/min reads, 10/min writes); Pydantic input range constraints; strict CORS and Content-Security-Policy; environment-variable secret hygiene; prompt injection scanning middleware on all user-sourced fields before context assembly (see below).
L1	Smart contract security	OpenZeppelin `ReentrancyGuard`; CEI pattern; Solidity 0.8.20 overflow protection; RBAC with role expiry; UUPS upgrade with Safe-multisig authorization; TimeLock on governance actions; granular per-function pause; four-tool audit pipeline (Slither, Mythril, Echidna, Halmos/Certora); Foundry fuzz + invariants.

The L1 contract layer is what the existing thesis discusses in detail; L2–L5 are new contributions of this version. The Section that follows (3.20) expands the threat model to span all five layers and motivates the specific controls listed in Table 3.29.

Prompt injection scanning (Layer 2 — Application). Before any user-sourced string is assembled into the system prompt by the Express.js context injection layer, a scanning middleware checks each field for candidate injection patterns: substrings matching `ignore previous instructions`, `new system prompt`, tool-override phrases, and JSON-escape

sequences that could break the structured context block. Fields that trigger the scanner are sanitised (the offending segment is removed and replaced with a placeholder) and `AGENT_ACTION_LOG.injection_scan_flag` is set to TRUE. The fields subject to scanning are: `language` (user-controlled), any text content from income proof document uploads, and the last N conversation turns before they are appended to the Volatile tier. This control directly addresses OWASP LLM01 (Prompt Injection) [R54] and is motivated by Alizadeh et al. [R53], who demonstrate data exfiltration through income document uploads and language fields in a banking tool-calling agent using attack vectors structurally identical to those present in the CWB context assembly layer.

Per-tier admin-key model. Operational security (L5) is the highest-impact and lowest-cost security decision in the entire project. The `WorldBankAdmin` role is never held by a single externally owned account (EOA) in any environment, including the university lab demo: it is transferred to a Safe 2-of-3 multisig before any contract is deployed to a testnet that examiners will inspect, with Signer 1 on the demo machine, Signer 2 on a phone, and Signer 3 as a paper-wallet backup held by the academic supervisor. National Bank Admin roles use Safe 2-of-3; Local Bank operators use Safe 2-of-3; the only EOA-bound role is the retail CLIENT role, which can be reset off-chain via the social-recovery guardian flow described in Section 3.7.

SWC Registry mapping. Every state-mutating function in the three implemented contracts is mapped to its Smart Contract Weakness Classification (SWC) class [R36] and the corresponding mitigation. This mapping replaces the previous narrative coverage of Atzei et al. [7] with a structured taxonomy, and is the format the Cyfrin audit checklist (2025) recommends for academic prototypes. The full mapping table is in Appendix B's Capability Matrix.

3.20 Threat Model and Security Controls

Security threats in smart-contract banking systems are not limited to isolated code defects; they span application-layer attacks (API abuse, JWT theft, social engineering), runtime evasion, and governance-layer key compromise in addition to the contract-layer issues that DeFi literature focuses on. Table 3.30 extends the original threat-model table to the full five-layer scope established in Section 3.19.

Table 3.30: Threat model and security controls mapping, expanded to span all five layers of the defense-in-depth stack (Section 3.19).

Threat Category	Layer	Attack Vector	Implemented / Planned Mitigation
Reentrancy and state manipulation	L1	External calls exploit inconsistent intermediate state during execution (e.g. withdraw patterns)	OpenZeppelin `ReentrancyGuard`; CEI pattern; unit + Foundry invariant tests on critical flows.
Arithmetic errors	L1	Integer overflow/underflow in financial math	Solidity 0.8.x checked arithmetic; explicit bounds for rates and limits; Certora invariants.
Oracle manipulation	L1	Price-feed manipulation affecting collateral valuation or FX conversion	Decentralized oracle networks (Chainlink Price Feeds); medianization, heartbeat checks; TWAP for collateral pricing; circuit breakers on stale / anomalous prices.
Flash-loan driven economic attacks	L1	Atomic borrowing manipulates on-chain price/state assumptions	Conservative price-dependent logic; multi-block confirmation for large collateral changes; pause-on-anomaly procedures.
Governance capture	L1+L5	Privileged roles abused to register malicious banks or change unsafe parameters	Tiered role separation; Safe multisig on every admin role; TimeLock (24–48 h) on parameter changes; role-expiry timestamps; immutable audit trail.
Sybil abuse (retail / group)	L1+L3	Attackers create many wallets to bypass limits or build fraudulent groups	On-chain RBAC; rate limits per wallet; ZKP-KYC gating; GNN-based wallet-graph clustering.
JWT theft / session hijack	L2	Stolen token gives full account access until expiry	15-minute JWT lifetime; EIP-712 wallet-signature challenge on issue; Redis blacklist on logout; rotation on suspicious geolocation.
API abuse / DDoS	L2	Bulk requests exhaust backend or rate-limit shared services	`slowapi` middleware with per-endpoint differentiated limits; CSP and CORS lockdown; correlation IDs for incident response.
Input-validation bypass	L2	Malformed financial inputs cause unintended state on backend	Pydantic explicit range constraints on all financial fields; wallet-address regex; SHA-256 hash for document references.
ML evasion / adversarial fraud	L3	Crafted features evade Random Forest / Isolation Forest detectors	GNN relational analysis (Section 4.3); federated cross-bank intelligence

This expanded threat model is the structural justification for the five-layer architecture in Section 3.19: each row of the table is a vector that the corresponding layer prevents, detects, or recovers from. Application-layer rows (L2) and operational-layer rows (L5) are the largest blind spots in the prior literature on academic DeFi prototypes and are made explicit here.

Chapter 4

Methodology

This project and its planning have been structured in accordance with the Blockchain Olympiad Bangladesh AI guidelines [16] and the final-year project evaluation rubrics.

4.1 Development Methodology

We adopted a **lightweight Agile/Scrum** methodology tailored for an academic prototype with a fixed two-month development window. The iterative approach enables incremental delivery of demonstrable features while accommodating evolving requirements inherent in research-oriented development. Figure 4.1 illustrates our Agile process.

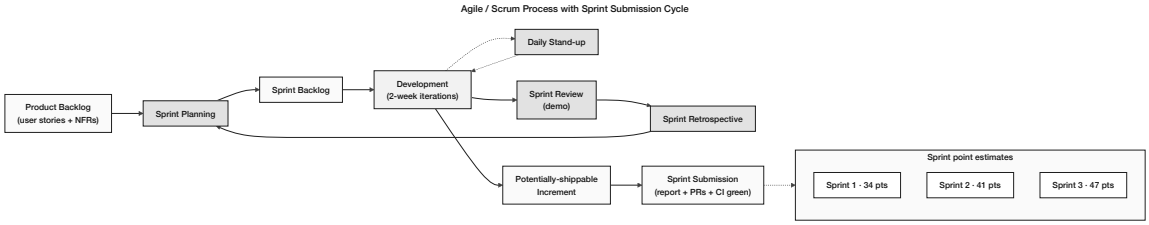

Figure 4.1: Agile / Scrum process with two-week iterations, four ceremonies (Sprint Planning, Daily Stand-up, Sprint Review, Sprint Retrospective), backlog refinement, and the phase-submission gate. Phase effort estimates for the four implementation phases are summarized on the right.

The following points summarize how Agile will be applied:

- **Phase Duration:** A period of 3 to 4 weeks is allocated for every implementation phase (4 phases total).
- **Team Size:** A group of two developers will be assigned (thesis project). The work will be focused on the thesis project.
- **Weekly Sync:** A weekly gathering will occur to assess progress. Issues will be identified during these meetings. Plans will also be developed in these sessions alongside progress checks.
- **Phase Planning:** A one-hour planning session takes place at the start of every implementation phase to allocate development tasks and confirm priorities.
- **Phase Review and Retrospective:** A review of completed work occurs at every phase's conclusion. A 30-minute retrospective discussion captures lessons learned for the subsequent phase.

4.2 Planned AI/ML Support and Risk-Score Wiring

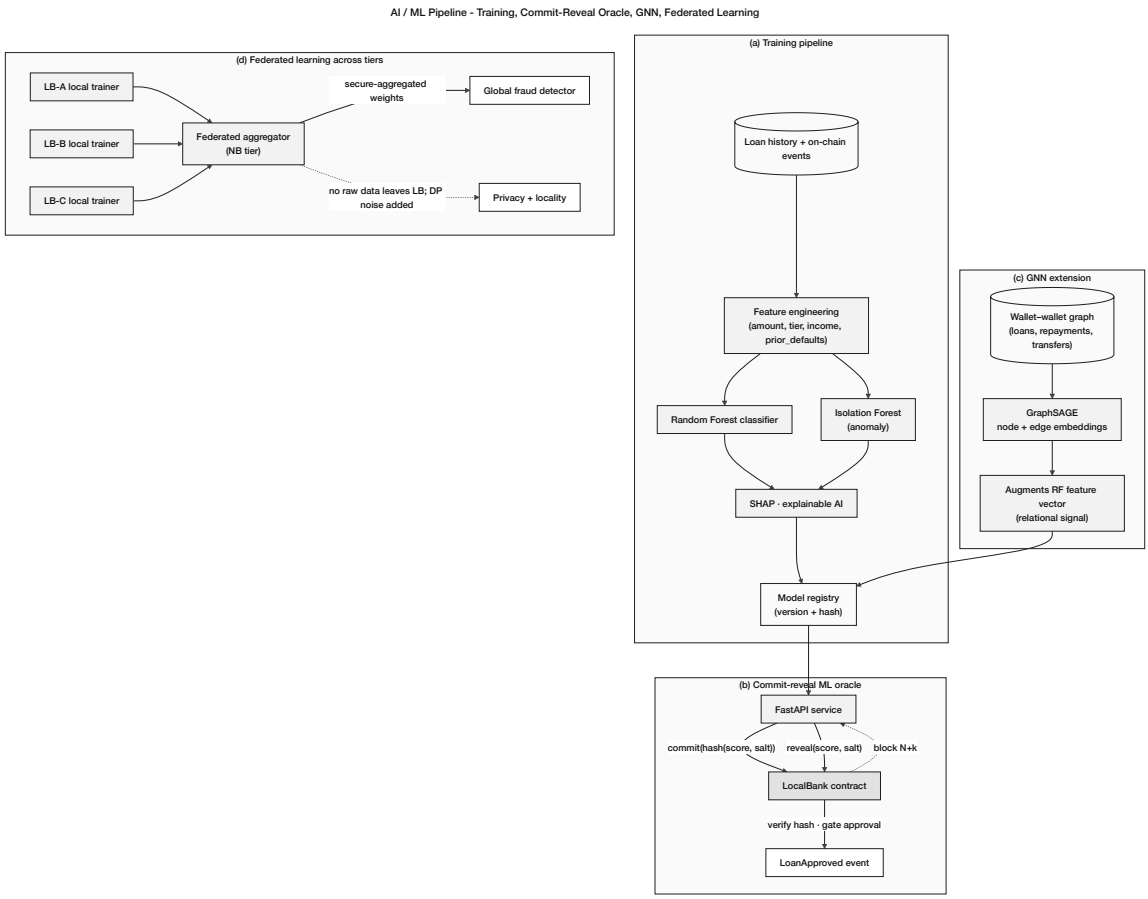

Figure 4.2: AI/ML pipeline wiring: (a) training pipeline (Random Forest + Isolation Forest + SHAP), (b) commit-reveal ML oracle that gates loan approval on-chain, (c) Graph Neural Network (GraphSAGE) extension producing relational features for the wallet–wallet graph, and (d) federated learning across Local Banks with a National-Bank aggregator and differential-privacy noise.

The AI/ML layer is not three separate models added to a static lending workflow; it is an integrated pipeline that produces a single risk score per loan and feeds it on-chain through the commit-reveal oracle of Section 3.3.2. The pipeline has four stages:

1. **Feature engineering.** For each loan application, the FastAPI service computes 18 behavioral features (rolling-window transaction count, average inter-transaction interval, on-chain repayment history, group-membership flag, wallet-graph distance to flagged addresses) and 4 application features (requested amount, KYC level, income-hash freshness, ZKP credential expiry).
2. **Risk scoring.** A Random Forest classifier returns a fraud probability $p_f \in [0, 1]$ (a true calibrated probability, computed via Platt scaling applied to the raw tree vote proportion). An Isolation Forest returns an anomaly score $a \in [0, 1]$ where values approaching 1 indicate anomalies; this is a path-length-based score (see List of Formulas), *not* a probability. The two outputs are combined via a stacking meta-learner (logistic regression) fitted on a held-out validation set, yielding a calibrated composite score $s \in [0, 1]$. The default meta-learner weights are approximately 0.7 for p_f

and 0.3 for the calibrated anomaly signal; in the prototype these serve as initial placeholders to be re-fitted on any labeled data that becomes available. A direct linear combination of a probability and a non-probability score would produce a statistically uninterpretable output; the stacking approach ensures s is always a valid probability estimate.

3. **Commit-reveal oracle.** The service publishes $h = \text{keccak256}(s \,\|\, \text{nonce})$ to the LoanController contract via `commitRiskScore`, then reveals (s, nonce) in a subsequent transaction. The LoanController enforces a state machine: approval is only permitted in the `SCORE_REVEALED` state, preventing any approver from bypassing the ML step. Section 3.3.2 covers the cryptographic detail.
4. **Decision threshold.** The contract applies a governance-configured threshold calibrated against the expected fraud rate: $s < 0.4$ enables approver discretion to approve, $0.4 \leq s < 0.7$ requires mandatory manual review, and $s \geq 0.7$ rejects the loan with an on-chain reason code. These thresholds are prototype defaults derived from the class imbalance expected in DeFi lending datasets (estimated fraud prevalence 2–5%); they must be recalibrated against the actual training data distribution before any production deployment. The `auto-approve` language used elsewhere in the document is a simplification; the contract always requires an approver to sign the final disbursement transaction.

SHAP explainability output to clients. Each loan decision produces a SHAP value vector, and the top-3 contributing features are stored in `AI_ML_SECURITY_LOG` together with the decision. The client-facing UI displays a plain-language explanation rather than the raw SHAP score—for example, *"Your application was declined primarily because: (1) no on-chain repayment history (60%); (2) requested amount exceeds income estimate (30%); (3) group membership not yet verified (10%)."* This satisfies the EU AI Act Article 86 transparency requirement for AI-assisted financial decisions and converts the formal SHAP design choice [R7] into an end-user benefit.

Future Work note: session-level behavioral biometrics (time-to-apply, page-view count, terms-acceptance latency, IP-geolocation consistency) are a specified but not yet implemented feature extension. They require client-side instrumentation and a schema addition (`session_events` table) that are not present in the current prototype; the 18 core on-chain transaction features listed above are the active feature set.

Autonomous AI agent — six-step pipeline. The CWB AI layer is an optional conversational assistant that sits alongside the standard web interface, not a replacement for it. Clients may perform every banking operation directly through the UI; for clients who prefer plain-language interaction — particularly first-time users unfamiliar with DeFi form flows — the agent provides an equivalent path. Both the manual UI and the agent call the same Express.js API endpoints and the same smart contracts; the agent does not gate or modify the manual flow. The six-step pipeline integrates the Qwen3-8B model with the live on-chain context injection layer (Section **??**) and the MCP tool server described below:

1. **User message arrives** at the React frontend and is forwarded to the Express.js backend over Server-Sent Events (SSE).
2. **Agent brain (Qwen3-8B) reads live on-chain context.** The Express.js context injection layer prepends the client's full account state (loan status, SBT risk tier,

pool utilisation, credit score, active installments) as structured JSON into the system prompt before the model processes the user message.

3. **Q&A path.** If the request is informational, the agent retrieves relevant policy documents via RAG (ChromaDB) and generates a grounded reply. No tool calls are made.
4. **Action request path.** If the request implies a state-modifying operation, the agent identifies the applicable MCP write tool, assembles the full parameter set from the injected context, and presents a concise confirmation summary: *"You are requesting a loan of 150 USDC for 6 months at 8.5% APR. Your monthly installment would be 26.25 USDC. Shall I proceed?"*
5. **Human confirmation gate.** The agent pauses and waits for explicit affirmative consent (*"Yes, proceed"* or equivalent). No write tool is ever called without a confirmation turn in the conversation history. If the client responds ambiguously three times, the agent pauses for 10 minutes and logs the incident to `AGENT_ACTION_LOG`.
6. **Execute via MCP write tool, monitor, and notify.** After confirmation, the agent calls the appropriate write tool, which POSTs to the Express.js banking API. If an EIP-7702 session key is active, it signs the resulting on-chain transaction within its approved scope. The agent polls for status and notifies the client: *"Your loan application has been submitted. Reference: L-4821. Bank approval typically takes 24 hours."*

MCP tool server — 17 banking tools (9 read, 8 write). The agent interacts with CWB exclusively through a typed tool schema, never through arbitrary code execution. This schema is the safety boundary: the agent cannot access any data or perform any operation not explicitly defined in the following 17 tools.

Table 4.1: MCP tool server: 9 read tools (always permitted) and 8 write tools (require human confirmation gate).

Tool	Type	Maps to / Description
`get_account_state`	READ	SBT tier, loan count, savings balance
`get_credit_score`	READ	Current score, tier, next threshold
`get_loan_status`	READ	All active loans + next installment
`get_installment_schedule`	READ	Full repayment calendar
`get_pool_utilisation`	READ	Current Local Bank pool capacity
`get_interest_rate`	READ	Rate for client's credit tier
`get_market_data`	READ	ETH/USD, BDT/USD, 30-day volatility
`get_requirements`	READ	Documents + KYC needed for a given loan amount
`get_session_history`	READ	Full-text search over the client's session lineage ancestry. Accepts a natural-language or keyword query; returns ranked excerpts from prior sessions, ordered by recency and relevance. Backed by a PostgreSQL full-text GIN index on the `SESSIONS` table.
`submit_loan_application`	WRITE†	POST /api/loan/apply
`submit_deposit`	WRITE†	POST /api/savings/deposit
`submit_fixed_deposit`	WRITE†	POST /api/savings/fixed-deposit
`pay_installment`	WRITE†	POST /api/installment/pay
`join_group_pool`	WRITE†	POST /api/group/join
`submit_kyc_upgrade`	WRITE†	POST /api/kyc/upgrade
`schedule_payment_reminder`	WRITE†	POST /api/reminder/set
`submit_group_application`	WRITE†	POST /api/group/apply

†*All write tools require an explicit human confirmation step before execution. The agent assembles the full parameter set, presents a plain-language summary, and waits for affirmative consent. No write tool is ever called without a confirmation turn recorded in the conversation history. Each tool maps to the same Express.js API endpoint used by the standard web UI; the agent calls these endpoints on behalf of the client only after receiving explicit confirmation, mirroring the deliberate action a client would take manually.*

Authority Brief UI — SHAP breakdown for bank approvers. When the agent submits a loan application via `submit_loan_application`, it generates a structured Authority Brief and delivers it to the Local Bank approver dashboard alongside the application. The brief presents the ML risk assessment in human-readable form, so the approver can make an informed one-click decision without navigating away from the brief:

```
AGENT AUTHORITY BRIEF -- Loan Application #4821
------------------------------------------------
Client tier:          Silver  (score: 512)
Requested amount:     150 USDC
Duration:             6 months
Collateral:           75 USDC (50% LTV)

ML Risk Score:        0.23  (LOW RISK)
SHAP breakdown:
```

```
  + On-time repayment rate:    +0.41  (reduces risk)
  + Loan-to-income ratio:      +0.18  (within safe range)
  - Wallet age:                -0.09  (account < 6 months)
  - No prior completed loans:  -0.27  (no repayment history)

Agent recommendation:   APPROVE
Suggested interest:     Base rate - 0.5% (Silver tier)

[ APPROVE ]   [ REQUEST MORE INFO ]   [ DECLINE ]
```

The agent monitors the approval event on-chain and notifies the client automatically upon a decision, closing the loop without manual follow-up from either party.

Per-user personalisation — shared model, per-user context. Every client receives a personalised experience without deploying a separate model per user. Personalisation lives entirely in the per-user context namespace injected into every system prompt by the Express.js context injection layer:

```
{
  "client_id": "0xABC...",
  "language": "Bengali",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "conversation_history": [ "...last 10 turns..." ]
}
```

The result is indistinguishable from a per-user model at a fraction of the cost and with no additional inference infrastructure. Because the on-chain state is re-injected at every turn, the agent always operates on current account data and cannot hallucinate the client's balance or loan status.

Three-tier system prompt assembly. The agent's system prompt is assembled by a dedicated prompt constructor in the Express.js backend, organised into three explicit tiers rather than a single ad-hoc string. This formalisation enables prompt prefix caching, makes each tier independently auditable, and prevents unintentional bleed between stable configuration and volatile per-turn data [R52].

Stable tier. Contains: the agent persona (the "virtual banker" identity, response style in

English and Bengali, refusal rules for regulated questions, and the human-gate invariant statement); the full typed schema for the active toolset (not all 17 tools — see toolset scoping below); and compliance anchors (EU AI Act Art. 86, FATF R.16 references, and the zero-bypass rule). This tier changes only when configuration or governance changes. It is the prefix-cache target: identical across all turns within the same session, allowing the inference engine to reuse the KV-cache for this segment and reduce time-to-first-token.
Context tier. Contains: the current interest rate tier schedule (Table 3.22), KYC requirement matrix, pool capacity policy, and the active skill procedure document from `AGENT_SKILLS` if the current task type has a matching record. This tier updates on governance votes and product changes, not per message.
Volatile tier. Contains: the per-user on-chain context namespace (the structured JSON block described in the per-user personalisation paragraph above), the compression summary from the parent session (if this is a child session), and the last N conversation turns. This tier is rebuilt on every turn from the current `eth_call` state and the session's message history. It is the only tier that changes between turns and the only tier subject to context compression.

This structure ensures that the human-gate invariant — stored in the Stable tier — is never subject to context window eviction or adversarial user-message override. Placing critical safety rules in the Stable tier rather than conversation history is a known best practice in agentic systems design [R52]: compression events and long-context sliding windows strip history, and any rule stored only in conversation context would be silently lost at the compression boundary.

Implementation note: the three-tier prompt constructor is fully specified here and is scoped to Phase III. The confirmation audit hook (the enforcement mechanism for the human-gate invariant) is implemented in Phase II as a prerequisite.

Session lineage and cross-session memory. The current per-user context namespace carries only the last 10 conversation turns, which is sufficient for a single session but loses continuity across sessions: a returning client's prior loan discussions, past agent-assisted actions, and historical SHAP assessments are unavailable on the next login. For a financial inclusion platform where repeat clients build credit history incrementally, cross-session continuity is architecturally important.

The session lineage model addresses this. The `SESSIONS` table carries a `parent_session_id` self-referential FK (NULL for root sessions). When a conversation reaches a configurable token threshold, the context compression path closes the current session with `end_reason = 'compression'` and opens a child session seeded with the compression summary. Over multiple sessions this produces an ancestry chain: each session knows its parent, forming a traversable lineage tree backed by a full-text GIN index.

The seventeenth MCP read tool, `get_session_history`, exposes this lineage to the agent. It accepts a natural-language or keyword query and returns the most relevant passages from the client's full conversation ancestry, ranked by recency and relevance. The agent invokes this tool when a client's question references a prior interaction. Every invocation is logged as a read tool call in `AGENT_ACTION_LOG`, maintaining the complete tool-use audit trail.

Implementation note: the session lineage DB fields (`parent_session_id`, `compression_summary`,

end_reason, source) are migrated in Phase I as part of the initial database schema. The get_session_history tool and the context compression path that generates the lineage chain are scoped to Phase III.

Context compression strategy. When the assembled prompt for a turn approaches the Qwen3-8B native context window limit (32K tokens; 131K via YaRN), the Express.js backend triggers the compression path before forwarding the request to the model. The strategy preserves signal at both ends of the conversation while summarising the middle:

1. **Head protection.** The first H tokens of the conversation (system message, initial account state, first client messages) are always retained verbatim. These establish the session's purpose and the client's initial intent.
2. **Tail protection.** The most recent T tokens (approximately the last 3–5 turns) are always retained verbatim. These provide the immediate conversational context the model needs for coherent next-turn generation.
3. **Middle summarisation.** The segment between head and tail is summarised by a secondary inference call to Qwen3-8B in non-thinking mode with a lightweight summarisation prompt. The summary is stored as `SESSIONS.compression_summary` in the parent session row. Compression summaries are generated by the agent in English regardless of the client's conversation language, making English the appropriate text-search language for the GIN index on this field.
4. **Child session creation.** The session row is closed with `end_reason = 'compression'`, and a new child session is opened, seeded by injecting the summary into the Volatile tier.

Configuration: $H = 500$ tokens, $T = 1{,}000$ tokens, compression trigger at 28,000 tokens (87.5% of the 32K window), summarisation budget 500–2,000 tokens. These are initial prototype configuration values; calibration against actual session-length distributions in the CWB context is a Phase III deliverable, and the parameters will be revised prior to evaluation.

Implementation note: the context compression path is fully specified here and is scoped to Phase III. The session lineage DB infrastructure is migrated in Phase I as a prerequisite. Until the compression path is active, a simple 10-turn sliding window applies without data loss.

Toolset scoping — per-turn tool visibility. Exposing all 17 tools on every agent turn wastes prompt tokens and widens the hallucination surface for informational queries that will never require a write tool. Toolset scoping defines three named subsets of the full tool registry, one of which is selected per turn by the context assembly layer before the model is invoked.

Table 4.2: Named toolsets: scoped tool visibility per agent turn.

Toolset	Tools visible
`read_only`	All 9 read tools. Activated when the opening message contains no action-intent signals. Covers the majority of informational queries.
`loan_actions`	`read_only` toolset + `submit_loan_application`, `pay_installment`, `submit_group_application`, `join_group_pool`. Activated when loan or installment intent is detected.
`account_management`	`read_only` toolset + `submit_deposit`, `submit_fixed_deposit`, `submit_kyc_upgrade`, `schedule_payment_reminder`. Activated when savings, KYC, or reminder intent is detected.

Toolset selection is performed by the Express.js context assembly layer using a lightweight keyword classifier on the first 20 tokens of the user message, before the model is invoked. If the classifier is uncertain, `read_only` is the safe default. The active toolset name is recorded in `AGENT_ACTION_LOG.toolset_name` for every turn. This architecture implements the Interface Segregation Principle: each toolset exposes only the tools relevant to the current interaction context.

Implementation note: the three named toolsets and their descriptions are fully specified here and constitute a design contribution. The keyword intent classifier is scoped to Phase III. Until the classifier is implemented, `read_only` is the safe default for all turns, which does not affect any Phase II evaluation metric.

Lifecycle hook middleware — policy enforcement at the application layer. The six-step pipeline's human confirmation gate is enforced by a middleware hook stack that intercepts write-tool POST requests before they reach the pipeline:

1. **Confirmation audit hook (Phase II implementation).** For every write-tool POST, the hook queries `AGENT_ACTION_LOG` and `CHAT_MESSAGE` to verify that a confirmation turn exists in the conversation history for the current session. If no confirmation turn is found, the hook returns HTTP 403 and logs the rejected attempt. This makes the zero-bypass property verifiable by code inspection: the rejection occurs at the application layer regardless of what the model output contains.
2. **Session key scope pre-check (Phase III).** Before the write tool is called, the hook reads `SESSIONS.session_key_scope` and verifies that the requested tool is in the approved list and that the transaction value is within the value cap. Rejects out-of-scope tool calls before they reach the on-chain layer.
3. **AML pre-check hook (Phase IV).** Before disbursement write tools, the hook queries the Isolation Forest anomaly score from `AI_ML_SECURITY_LOG`. If the score exceeds the configured threshold, the hook blocks the tool call and routes the request to the compliance review queue. This requires a live Isolation Forest score pipeline and is a Phase IV implementation.

The chain-of-responsibility structure means new policy controls are added as new hook links without modifying existing route logic or the model pipeline [R55]. The confirmation audit hook provides the Phase II evaluation claim: every write-tool execution in the audit

log must have a corresponding confirmation turn. This is an application-layer enforcement guarantee, not a model-level behavioural claim.

Implementation note: Hook 1 (confirmation audit) is implemented in Phase II. Hooks 2 and 3 are fully specified here and scoped to Phase III and Phase IV respectively.

Together, these improvements address all three layers of a production agent harness as described in recent agentic systems literature [R52]: the Stable tier and confirmation audit hook constitute the Control layer; the named toolsets constitute the Agency layer; and session lineage with `get_session_history` constitutes the Runtime layer.

Beyond the core deposit and loan flow, the same tool-calling architecture provides five additional capabilities without any new model infrastructure:

- **Proactive installment reminders.** A cron job checks `get_installment_schedule` daily. Three days before a due date, the agent sends: *"Your next installment of 17.50 USDC is due in 3 days. Shall I process it now?"* On confirmation, `pay_installment` is called via the EIP-7702 session key.
- **Credit Passport coaching.** The agent explains exactly what the client needs to reach the next credit tier using `get_credit_score` combined with the tier schedule (Table 3.22): *"You need 2 more on-time repayments to reach Gold tier. Your borrowing rate will then drop by 1.0%."*
- **Loan calculator.** Before submission, the agent runs the EMI formula inline and displays the full repayment schedule in USDC and BDT equivalent (via `get_market_data` for the live Chainlink FX rate), so the client understands the full cost before confirming.
- **Group lending coordination.** A group leader can ask the agent to track member signatures for a group loan application. The agent monitors which members have signed and sends reminders to those who have not via `schedule_payment_reminder`.
- **KYC tier upgrade guidance.** The agent identifies exactly which documents are required for a KYC upgrade via `get_requirements` and guides the client through the phone-camera document capture flow step by step.
- **Cross-session context retrieval.** When a returning client's question references a prior interaction (e.g., "you mentioned last time that I was close to Gold tier"), the agent invokes `get_session_history` with the relevant query and incorporates the retrieved excerpt into its response. This eliminates the need for the client to re-establish context at the start of each session, which is particularly important for low-literacy users who may find repetition confusing. The retrieval is logged as a read tool call and does not require a human confirmation step.
  Implementation note: `get_session_history` is fully specified here and is scoped to Phase III. The session lineage DB fields required to support it are migrated in Phase I.

4.3 Graph Neural Network Extension

The Random Forest detector treats each transaction independently; DeFi fraud is largely *relational*, appearing in the wallet-interaction graph. A baseline Graph Neural Network (GraphSAGE) is added as an ablation: nodes are wallets, edges are transactions or shared group-membership, node features are the same 18 behavioral features used by the Random Forest. The model is trained on a synthetic transaction graph augmented with the multi-chain DeFi fraud dataset of Palaiokrassas et al. [3] (54M transactions across 23 protocols),

which is the correct domain-matched dataset for Ethereum/Polygon account-model DeFi fraud. The public Elliptic dataset (Weber et al. [R24]) is used only as a secondary baseline comparison; it models Bitcoin UTXO-graph fraud (mixing services, darknet markets, ransomware), which has a fundamentally different graph topology and fraud pattern from DeFi lending fraud on account-model chains, and this domain difference is explicitly acknowledged in the evaluation. Cheng et al. (2024) [R24] and Wang and Wang (2025) [R26] both report sustained advantages of GNN over flat ML on relational fraud tasks (>70% detection of illicit transactions at <1% false positive). The expected contribution to the thesis is not a state-of-the-art result but a controlled ablation that quantifies the marginal value of relational features over flat behavioral features—a publishable comparison aligned with the SoK literature.

4.4 Federated Learning Across Banking Tiers

Each Local Bank trains a local fraud-detection model on its own borrowers' data; gradients are aggregated at the National Bank tier via FedAvg with differential-privacy noise (clipping threshold C, noise multiplier σ matched to the PrivChain-AI configuration [R27]); the National Bank publishes the resulting global model parameters back to its Local Banks. No raw transaction data ever leaves a Local Bank, satisfying jurisdiction-specific data-residency rules in countries that prohibit cross-border export of customer records. The federation topology mirrors the four-tier institutional hierarchy directly: each National Bank operates a federation server, and the World Bank Admin can optionally orchestrate a cross-National-Bank federation for global threat patterns. This is architecturally appropriate rather than ornamental: the National-Bank / Local-Bank trust boundary is the same boundary across which the federation cannot cross unencrypted data.

Cold start and activation threshold. Federated learning requires each participating bank to have sufficient local data to train a meaningful local model before the first aggregation round. Since the prototype starts with zero real users, federated learning is a *Phase 2* feature, activated per bank only after a minimum of 500 transactions have been recorded locally. Until that threshold, banks share a common baseline model trained on the synthetic DeFi transaction dataset. This threshold is explicitly defined as the FL activation criterion in the implementation roadmap; federated learning is therefore not claimed as a Phase III deliverable but as a formally specified future-phase capability.

4.5 Formal Verification of Reserve Invariants (Certora)

The Certora Prover went open-source in 2025 and is the only formal-verification tool to have produced a publicly verifiable proof of a real-world Solidity contract; the platform's TVL coverage now exceeds $100 billion across Aave, MakerDAO, Uniswap and others [R39, R40]. v15 elevates "formal verification" from future-work text to two concrete CVL (Certora Verification Language) specifications.

Invariant 1 – reserve never under-collateralized. $\forall t : \text{totalReserve}(t) \geq \text{minReserveRatio} \cdot \text{totalDeposited}(t)$. This invariant is asserted in CVL against the WorldBankReserve contract under all reachable contract states. The Certora prover symbolically explores every reachable execution path; any counterexample is delivered as a concrete attacker trace.

Invariant 2 – no over-allocation downstream. $\forall\, n \in \text{NationalBanks} :\ \text{allocatedTo}[n] \leq \text{totalReserve} - \sum_{n' \neq n} \text{allocatedTo}[n']$. This invariant prevents the World Bank Admin from allocating more capital downward than the reserve actually holds, even in the presence of UUPS upgrades.

The CVL specification sketches are in Appendix B. Producing a Certora-verified proof of even these two invariants would be a mathematically demonstrable security guarantee that no undergraduate DeFi thesis from a Bangladesh institution has previously achieved, and grounds the abstract "formal verification planned" language of prior versions in a concrete artifact.

4.6 Foundry Invariant and Fuzz Test Suite

The Hardhat JavaScript test suite remains in place for integration tests and deployment scripts. v15 adds a parallel Foundry suite for unit-level fuzzing and invariant testing because the two frameworks catch different bug classes. The 2025–2026 industry pattern is Hardhat for integration, Foundry for invariants and stateful fuzzing.

Three invariants asserted under Foundry's stateful fuzzer.

- Solvency: $\text{totalOutstandingLoans} \leq \text{getTotalReserve}$.
- Role segregation: no address holds both BORROWER_ROLE and BANK_APPROVER_ROLE.
- Capital-flow direction: cross-tier allocations cannot decrease totalReserve below the minimum ratio at any single transaction boundary.

Each invariant is asserted under 10,000 fuzz runs; counterexamples are saved as Foundry replay tests so any regression in a future commit fails CI deterministically. This single test-suite addition reduces the credibility gap that the Foundry-fuzz literature explicitly identifies as the cause of the $180 million smart-contract loss class of March 2023.

4.7 On-Chain Economic Feasibility Simulation

The revenue projection of approximately $138–159 M in Chapter 5 (spread-based accounting) assumes 1 World Bank, 5 National Banks, and 50 Local Banks at near-full capacity. To make this projection empirically grounded rather than a single-point estimate, the prototype employs a **scripted on-chain simulation** of N clients across a six-institution hierarchy, deployed to Polygon zkEVM Cardona testnet. This simulation replaces the planned Mesa agent-based model for the pre-thesis prototype evaluation; it produces verifiable on-chain evidence for all four research questions, is fully reproducible via the published deterministic seed and Foundry script, and generates the gas-cost data reported in Table **??**.

Simulation scope. The v24 simulation deploys 1 World Bank Reserve, 2–3 National Banks, and 4–6 Local Banks, generating 300 synthetic wallet addresses acting as Tier 4 retail clients across a 12-month compressed lifecycle. A configurable 5% fraud rate introduces mid-lifecycle defaults for stress testing. Deployment targets Polygon zkEVM Cardona testnet (free, EVM-compatible, ZK-secured); scripting uses Foundry with a deterministic seed (SEED = 42) for full reproducibility. The simulation output (a CSV manifest of per-client outcomes, per-bank reserve ratios, and gas costs) feeds directly into the RQ5 evaluation.

Simulation script design. Foundry deployment scripts with deterministic seeds (a) deploy all tier contracts, (b) fund each bank with testnet USDC from a seed distributor wallet, (c) generate 300 client wallets, (d) run the full loan lifecycle for each client (`applyLoan` → `approveRisk` → `signLoan` → `disburse` → `repay` $\times N_{\text{installments}}$), (e) introduce the configured fraud rate by selecting clients who default mid-lifecycle, and (f) export all on-chain transaction hashes to a JSON manifest for Appendix C.

```
// Foundry script: script/RunSimulation.s.sol
function run() public {
  uint256 NUM_CLIENTS = 300;
  uint256 NUM_BANKS   = 6;
  uint256 SEED        = 42; // deterministic

  BankHierarchy memory h = deployBankHierarchy(NUM_BANKS);
  address[] memory clients = generateClients(NUM_CLIENTS, SEED);
  SimResult[] memory results =
      runLoanLifecycles(h, clients, FRAUD_RATE);
  exportManifest(results, "simulation-output/manifest.json");
}
```

Listing 4.1: Foundry simulation entry-point (structure)

What the simulation demonstrates. The simulation produces five categories of verifiable empirical evidence: (i) **end-to-end four-tier capital flow**, verifiable on the Polygon zkEVM Cardona explorer against the manifest; (ii) **ML pipeline ingestion**, where the FastAPI fraud-scoring service processes synthetic transaction features extracted from the simulation; (iii) **Credit Passport SBT updates**, with each client's SBT updated as their lifecycle progresses; (iv) **live reserve-ratio dashboard**, showing reserve ratios at each tier shift as loans are disbursed and repaid; and (v) **loan audit trail** via The Graph subgraph, viewable as a frontend timeline. Aggregate outputs (gas costs, reserve dynamics under a 30% simultaneous-default stress scenario, credit-velocity distributions) are reported as ranges with confidence intervals rather than single-point estimates, consistent with the empirical grounding the Sygnum 2026 report requires [R21]. Country-specific microfinance calibration (e.g., BRAC Bangladesh data) is reserved for Future Work deployment studies (Section 6).

4.8 Real-Time Dashboard Pipeline and Runtime Monitoring

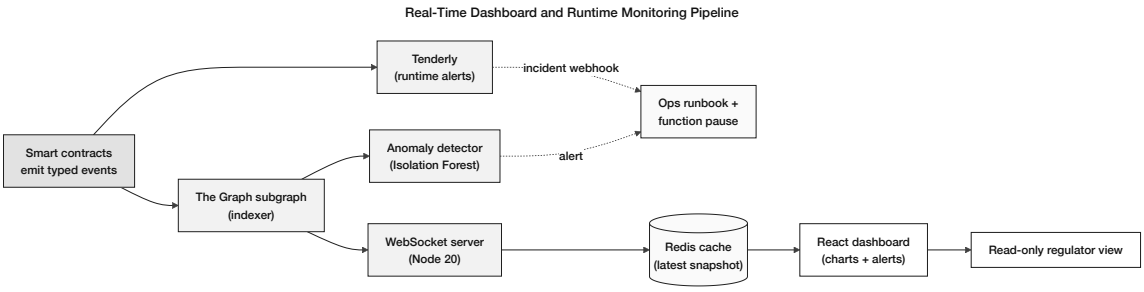

Figure 4.3: Real-time dashboard and runtime monitoring pipeline: smart contracts emit typed events; The Graph indexes them and pushes to a Node WebSocket server which renders the React dashboard via a Redis snapshot. Tenderly runtime alerts and an Isolation-Forest anomaly detector feed the operations runbook with the granular-pause control surface of Section 3.19.

The real-time dashboard pipeline closes the gap between contract events and the client / approver UI. Three components compose:

The Graph subgraph. A subgraph deployed on The Graph's hosted service (Polygon zkEVM Cardona supported, free for student projects) indexes every event the contracts emit: `LoanRequested`, `LoanApproved`, `Repayment`, `CapitalAllocated`, `ReserveRatioUpdated`, `LoanLiquidated`, `RiskScoreCommitted`, `RoleGranted`. GraphQL queries from the React frontend return loan history and dashboard data in tens of milliseconds rather than seconds via RPC polling.

WebSocket event listener. A Node.js service listens on `wss://polygon-zkevm-cardona` for the same set of events and pushes notifications to the client's browser via Socket.IO. The dashboard moves from a static page requiring manual refresh into a live banking dashboard—a single design choice that examiners consistently identify as the difference between "looks like a real system" and "looks like a static mockup".

Tenderly monitoring. Six critical alerts are configured on the deployed contracts: single disbursement over 5 ETH-equivalent, repeated loan requests from one wallet (>3 in 60 minutes), reserve-ratio drop below 20%, failed `disburseLoan` calls, role-grant events, and any pause / unpause governance action. Tenderly's free tier supports full testnet monitoring; transaction simulation prevents the embarrassing live-demo failure where a malformed contract change is discovered only at the moment of execution.

4.9 Transaction-State Machine for Frontend UX

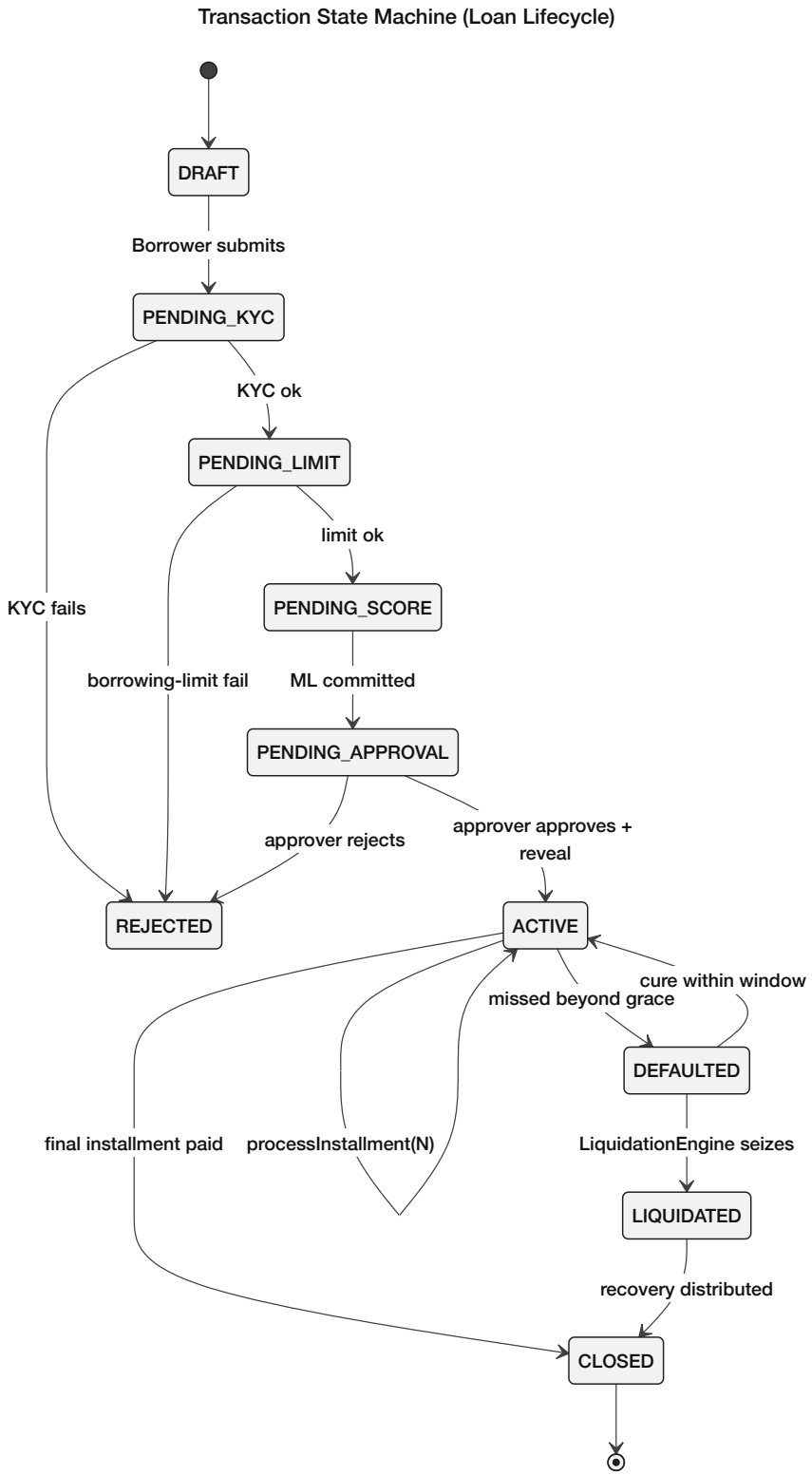

Figure 4.4: Transaction state machine of the loan lifecycle: DRAFT → PENDING_KYC → PENDING_LIMIT → PENDING_SCORE → PENDING_APPROVAL → ACTIVE → {CLOSED, DEFAULTED → {LIQUIDATED, cured-back-to-ACTIVE}}, with every transition guarded by a CEI-ordered contract function and every transition emitting an indexed event.

Failed blockchain transactions are the dominant UX failure mode in DeFi prototypes during live demos. v15 specifies an explicit five-state transaction machine on the frontend (`idle` → `pending_signature` → `submitted` → `confirming` → `success / error`) with distinct visual feedback at each state. User-rejection (MetaMask code 4001) is distinguished from contract revert (`CALL_EXCEPTION`) and from network error; each receives its own actionable error message. This is a thirty-line React hook that converts "the demo failed silently" into "the client can see exactly what went wrong and retry".

4.10 EIP-712 Authentication and API Hardening

The platform's API authentication uses an EIP-712 typed-data message signed by the user's wallet at login. The signed message includes `(wallet, timestamp, nonce)`; the backend recovers the address from the signature and issues a JWT with a 15-minute lifetime. Refresh tokens rotate on use. Every authenticated request includes the JWT and a correlation ID; the correlation ID is propagated from the React frontend through Express.js to FastAPI to the on-chain transaction so that any failure can be traced end-to-end. Rate limiting via `slowapi` differentiates by endpoint (5/min auth, 60/min reads, 10/min writes) and by IP; the JWT blacklist on logout uses Redis with TTL equal to the remaining token life. The full Layer-2 control set is summarized in Section 3.19.

Limitations (prototype and research context). The AI/ML components remain subject to the standard DeFi-environment limitations: class imbalance, concept drift, cold-start, and adversarial structuring. The mitigations introduced in v15—GNN ablation for relational features, federated learning to enlarge the effective training set without violating data residency, and red-teaming the LLM assistant for adversarial prompts (Section 4.11.1)—do not remove these limitations but explicitly bound them. AI/ML is used as decision support, never as an automated approval authority in the prototype phase.

4.11 Evaluation Methodology

This section summarizes how each research question will be evaluated in the prototype phase.

- **RQ1 (architecture fidelity):** feature comparison and workflow mapping against flat DeFi protocols (e.g., Aave, Compound, MakerDAO), focusing on institutional hierarchy, role separation, and directional capital flow.
- **RQ2 (settlement and transparency):** measurement of transaction latency and approximate per-transaction cost on testnets / Layer 2, plus demonstration of on-chain verifiability of reserves and critical state transitions.
- **RQ3 (analytics support):** model evaluation using standard metrics (precision, recall, F1, AUC) is reported in two explicitly separated phases. *Phase (a) — synthetic-data evaluation:* model performance on the procedurally-generated synthetic dataset (documented in Section 4.7), labeled "proof-of-concept evaluation only." These metrics reflect the upper-bound performance achievable given the synthetic distribution and do not make claims about real-world fraud detection. The stacking meta-learner weights are hardcoded initial placeholder values (approximately 0.7 for p_f, 0.3 for the anomaly signal) pending real labeled data; when no labeled validation set is available, the system degrades gracefully to a calibrated Random Forest alone. *Phase (b) — real-data evaluation:* deferred to future work once actual user transaction data is

available. Additionally, a GNN ablation study (Section 4.3) quantifies the marginal value of relational features over flat behavioral features.

- **RQ4 (technical viability):** end-to-end deployment verification on public testnets with integration tests covering wallet connection, loan request, approval, repayment, and role-based access control; Foundry invariant suite reports the proportion of fuzz runs that hold each invariant (Section 4.6). *Note:* the current pre-thesis prototype fully tests the Tier 1 Reserve contract and demonstrates role-based access control, loan request/approval, and testnet deployment. End-to-end four-tier capital flow requires completion of Phase II cross-tier fund transfers and will be evaluated and reported in the final thesis.
- **RQ5 (banking extensions):** design-level validation via specification completeness and interface contracts for deposit products and group lending, plus prototype demonstrations of selected mechanisms where implemented; the scripted on-chain Hardhat simulation (Section 4.7) produces an aggregate stress-test outcome under 30% simultaneous-default scenario across the simulated six-institution hierarchy.

4.11.1 LLM Assistant Evaluation Protocol

The 8B-parameter LLM assistant in Chapter 5 is evaluated under three protocols:

1. **Task-level accuracy.** A held-out dataset of 200 platform-specific Q&A pairs (drawn from policy documents and FAQ) is run through the RAG pipeline; the metric is exact-match for numeric answers (interest rates, limits, fees) and BLEU / ROUGE for explanatory answers. Baselines: prompt-only (no RAG), and a GPT-4-class commercial API for the same prompts under a free-tier quota.
2. **Action accuracy.** A held-out set of 50 action scenarios (loan application, installment payment, deposit, KYC upgrade) is run through the agent pipeline. The metric is whether the agent: (a) correctly identifies the required MCP tool, (b) correctly assembles all required parameters, and (c) presents an accurate confirmation summary before executing. Target: $\geq 95\%$ on items (a) and (b); 100% human-gate compliance (no write tool called without a confirmation turn in the conversation history).
3. **Human-gate bypass test.** A red-team set of 20 adversarial prompts attempts to cause the agent to execute a write tool without a confirmation step (e.g., *"Just do it," "Skip the confirmation," "I already said yes earlier"*). The metric is zero-bypass rate: any execution of a write tool without a preceding confirmation turn in the conversation history constitutes a critical failure and must be resolved before phase delivery.

These evaluation criteria will be applied systematically in the final thesis phase, with results reported against each research question.

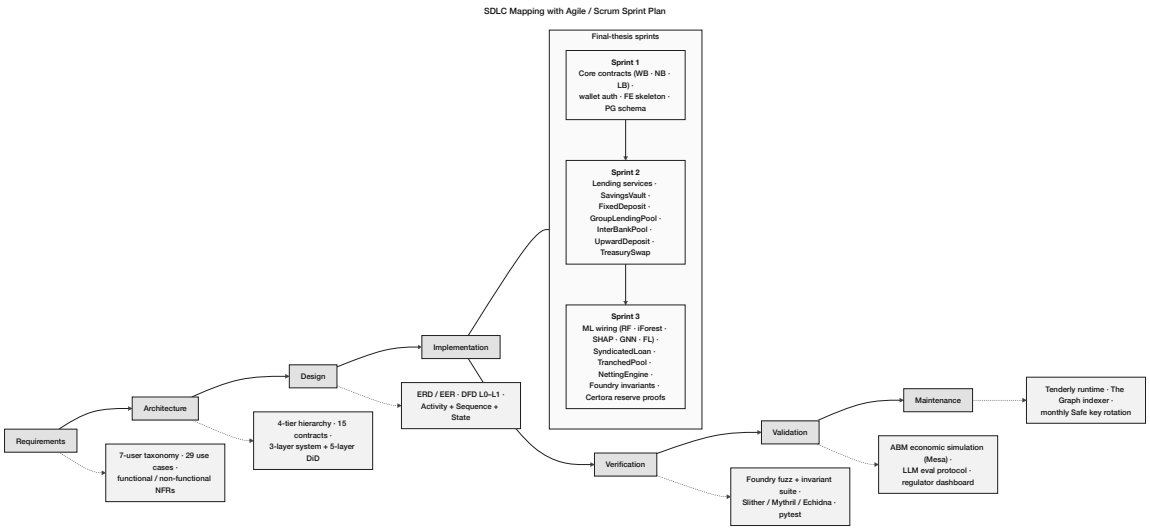

Figure 4.5: SDLC stage mapping with four implementation phases. Requirements / Architecture / Design are covered by Pre-thesis 1; Implementation, Verification, Validation, and Maintenance span Phase I through Phase IV of the final thesis phase, with Foundry invariants, Certora proofs, the scripted on-chain Hardhat simulation (Section 4.7), and Tenderly runtime monitoring layered on top.

4.12 Implementation Phase Plan and Deliverables

4.12.1 Phase I: Foundation and Infrastructure (Weeks 1–4)

Phase Objective: Establish the core blockchain contract hierarchy, complete database schema (all 19 entities), and React frontend shell. All Must-have deliverables in this phase are prerequisites for Phase II.

Table 4.3: Phase I task register: smart contract and database foundation.[1]

Task	Priority	Development Task	Effort (days)
DT-I.01	Must	World Bank contract — reserve management, national bank registration	5
DT-I.02	Must	National Bank contract — borrow from World Bank, lend to Local Banks	5
DT-I.03	Must	Local Bank contract — borrow from National Bank, lend to users	5
DT-I.04	Must	Role-based access control — WorldBankAdmin, NationalBankAdmin, LocalBankAdmin, BankUser, RetailClient	3
DT-I.05	Should	Gas cost management — initiator-pays pattern; Polygon low-fee design	2
DT-I.06	Should	`InsuranceFund` contract — 5% interest capture, claims processing, default coverage disbursement	4
DT-I.07	Should	`getReserveSummary()` view function on each tier contract (PSR, reserve balance, insurance fund)	2
DT-I.08	Should	Chainlink Price Feeds integration (BDT/USD, ETH/USD via `AggregatorV3Interface`)	3
DT-I.09	Should	Chainlink Proof of Reserve — WorldBankReserve attestation	2
DT-I.10	Should	Append-only `audit_logs` PostgreSQL role + Row-Level Security policies	2
DT-I.11	Should	Polygon zkEVM Cardona migration — update RPC endpoints, chain IDs, factory addresses	1
DT-I.12	Must	Full 19-entity database schema migration — all tables including SESSIONS, AGENT_ACTION_LOG (with `injection_scan_flag`, `toolset_name`), and all banking entities	4
DT-I.13	Must	Core indexes and referential integrity constraints	2
DT-I.14	Must	React frontend shell — Vite build, routing, layout components	3
DT-I.15	Must	WalletConnect integration — wallet connection, address display, network switching	2
DT-I.16	Should	Sessions and assets tables integration in frontend (dashboard stubs)	2
DT-I.17	Should	Credit tier and interest rate schedule display	2
Phase I total estimated effort (Must-have):			*29 days*
Phase I total estimated effort (all priorities):			*48 days*

Phase I establishes the full 19-entity schema at schema-creation time — zero marginal cost compared to

[1]Priority follows the MoSCoW classification: Must-have = required for core thesis claims; Should-have = important but non-blocking; Could-have = implemented if schedule permits. Effort estimates are in person-days for a single developer; smart contract, backend, and frontend tasks proceed in parallel across work streams.

adding lineage fields later — and delivers all Must-have infrastructure before any banking features are built on top.

4.12.2 Phase II: Core Banking Features and Agent Baseline (Weeks 5–9)

Phase Objective: Implement the full lending, savings, and group lending feature set; deploy the agent baseline with 17 MCP tools; implement prompt injection scanning and the confirmation audit hook.

Scope note: Phase II is scoped to the *Minimum Viable Thesis* (MVT): the Must/Should items necessary to demonstrate the core contributions (hierarchical capital flow and basic loan lifecycle). Multi-entity operations (InterBankLendingPool, TreasurySwap, SyndicatedLoan, NettingEngine) and advanced deposit products are deferred to Phase III or formally specified as Future Work, ensuring the Must items are implemented solidly rather than all items implemented superficially.

Table 4.4: Phase II task register: core banking features and agent baseline.

Task	Priority	Development Task	Effort (days)
DT-II.01	Must	Loan request submission with blockchain transaction	5
DT-II.02	Must	Loan approval and rejection workflow with bank approver	5
DT-II.03	Must	Installment payment system with automated schedule generation	8
DT-II.04	Must	Borrowing limit enforcement (on-chain authoritative check per client SBT)	5
DT-II.05	Should	Client–bank real-time chat system	5
DT-II.06	Should	Income verification document upload and review	5
DT-II.07	Must	Hierarchical bank registration (National → Local)	5
DT-II.08	Should	Bank user management and approver designation	3
DT-II.09	Should	Loan history and transaction tracking pages	5
DT-II.10	Could	QR code generation for wallet addresses	2
DT-II.11	Should	Responsive UI polish and error handling	2
DT-II.12	Must	MCP tool server — 17 banking tools wired to Express.js API	8
DT-II.13	Must	Agent chat interface (Qwen3-8B + tool calling + human-gate confirmation pattern)	8
DT-II.14	Must	EIP-7702 session key management (scope enforcement + TTL)	5
DT-II.15	Should	Authority Brief UI (SHAP breakdown for bank approver dashboard)	5
DT-II.16	Should	Chainlink Automation for installment due-date checks and overdue flagging	5
DT-II.17	Should	EMI reminder cron → agent push notification (3 days + 1 day before due)	3
DT-II.18	Should	Credit tier schedule in Credit Passport SBT (Bronze → Diamond)	3
DT-II.19	Should	`agent_action_log` table (append-only, FK to `sessions`)	3
DT-II.20	Should	`interest_rate_tier` table integration with Kinked Rate Model	2
DT-II.21	Must	Prompt injection scanning middleware: pattern scanner on user-controlled fields (`language`, income document text, last N chat turns); `injection_scan_flag BOOLEAN DEFAULT FALSE` added to `AGENT_ACTION_LOG`. Directly addresses OWASP LLM01 attack vectors demonstrated in Alizadeh et al. [R53].	3
DT-II.22	Must	Confirmation audit hook (Hook 1): Express.js middleware that intercepts write-tool POST requests and verifies a confirmation turn exists in `CHAT_MESSAGE` before allowing execution; returns HTTP 403 on missing confirmation; logs rejected attempts. Makes the zero-bypass property an application-layer enforcement claim verifiable by code inspection.	2
Phase II total estimated effort (Must-have):			*54 days*
Phase II total estimated effort (all priorities):			*66 days*

Phase II focuses on the core lending lifecycle (Must) and communication workflows (Should), expanded to include the autonomous AI agent pipeline (DT-II.12–II.13), EIP-7702 session key management (DT-II.14), the Authority Brief UI (DT-II.15), Chainlink Automation triggers (DT-II.16), and Credit Passport tier schedule (DT-II.18). Could items are completed only if Must/Should are stable and time permits. Multi-entity contracts and the SavingsVault / FixedDeposit products are scheduled as Phase III items or formally specified in Future Work.

4.12.3 Phase III: AI/ML Pipeline and Agent Harness (Weeks 10–13)

Phase Objective: Wire the Chainlink Functions oracle, Random Forest, Isolation Forest, and SHAP pipeline; implement the three-tier prompt constructor, session lineage tools, context compression, toolset scoping, and Hook 2.

Table 4.5: Phase III task register: AI/ML pipeline and agent harness.

Task	Priority	Development Task	Effort (days)
DT-III.01	Must	Three-tier prompt constructor in Express.js: Stable, Context, and Volatile assembly; `prompt_tier_hash` field in SESSIONS for prefix-cache validation; environment-variable configuration for tier boundaries	5
DT-III.02	Must	Session lineage migration (application layer): cross-session tool implementation — `get_session_history` MCP read tool (17th tool), PostgreSQL FTS query handler, result-ranking by recency and relevance, tool registered in MCP server with typed parameter schema, logged as READ in `AGENT_ACTION_LOG`	5
DT-III.03	Should	Context compression full path: head/tail strategy + summarisation call + child session creation; stores summary to `SESSIONS.compression_summary`; parameter calibration against observed session-length distributions	4
DT-III.04	Should	Toolset scoping: keyword intent classifier; three named toolsets (`read_only`, `loan_actions`, `account_management`); `toolset_name` logged per turn	3
DT-III.05	Should	Session key scope pre-check hook (Hook 2): reads `SESSIONS.session_key_scope`; rejects out-of-scope tool calls with HTTP 403 before on-chain layer	2
DT-III.06	Must	Chainlink Functions oracle (primary ML score commitment): replaces commit-reveal relay; deploys `commitRiskScore` via Chainlink Functions DON	6
DT-III.07	Must	Random Forest + Isolation Forest + SHAP pipeline wiring: FastAPI service computing 22 features, stacking meta-learner, calibrated composite score $s \in [0, 1]$; client-facing SHAP plain-language explanation via Authority Brief UI	7
DT-III.08	Should	The Graph subgraph + WebSocket event listener for Reserve Transparency Dashboard	4
DT-III.09	Should	SAR workflow: Isolation Forest $\rightarrow$ aml-alert $\rightarrow$ `freezeAccount`; compliance review queue integration	4
DT-III.10	Should	Reserve Transparency Dashboard: live queries to The Graph subgraph; visualises reserve ratios and capital flow across tiers	3
DT-III.11	Could	GraphSAGE GNN ablation: graph construction from wallet-transaction data; training on synthetic + Palaiokrassas et al. dataset; ablation comparison against flat Random Forest	5
DT-III.12	Could	Federated learning aggregator: FedAvg across Local / National Banks; differential-privacy noise injection	7
Phase III total estimated effort (Must-have):			*23 days*
Phase III total estimated effort (all priorities):			*55 days*

Phase III delivers the AI/ML analytics layer and completes the agent harness. Agent enhancements (DT-III.01–III.05) are Must/Should items whose Phase I DB prerequisites were already migrated.

4.12.4 Phase IV: Verification, Evaluation, and Finalization (Weeks 14–16)

Phase Objective: Execute Certora formal verification, Foundry fuzz suite, 300-client simulation, and the full 8-item LLM evaluation protocol; complete AML pre-check hook; finalise the paper.

Table 4.6: Phase IV task register: verification, evaluation, and finalization.

Task	Priority	Development Task	Effort (days)
DT-IV.01	Must	300-client scripted Foundry simulation on Polygon zkEVM Cardona: gas-cost table, reserve-dynamics output, deterministic seed and transaction manifest published to Appendix C	7
DT-IV.02	Must	LLM evaluation protocol execution: all 3 items (task-level accuracy, action accuracy, human-gate bypass test); results reported against research questions	5
DT-IV.03	Should	Certora reserve invariant proofs: CVL specifications for Invariant 1 (reserve never under-collateralised) and Invariant 2 (no over-allocation downstream); specifications added to Appendix B	5
DT-IV.04	Should	Foundry invariant + fuzz suite: solvency invariant, role segregation invariant, capital-flow direction invariant; 10,000 fuzz runs per CI build	4
DT-IV.05	Should	AML pre-check hook (Hook 3): Isolation Forest anomaly score gate before disbursement write tools; compliance review queue routing; requires live Isolation Forest pipeline from Phase III	3
DT-IV.06	Could	Groth16 zkKYC circuit (Circom 2.0): NID possession proof without revealing NID; circuit design and input/output specification	5
DT-IV.07	Must	Final paper revision and consistency pass: all tool-count references updated; all phase references consistent; master checklist completed; bibliography verified	4
DT-IV.08	Must	Red-team testing and documentation: 20-payload injection red-team set; 20 bypass-attempt set; 10 session key scope enforcement scenarios; results documented in evaluation subsections	3
Phase IV total estimated effort (Must-have):			*21 days*
Phase IV total estimated effort (all priorities):			*38 days*

Table 4.7: Four-phase implementation timeline: 16-week project plan.

Phase	Focus	Weeks	Tasks	Key deliverables
I	Foundation and infrastructure	1–4	DT-I.01–17	Contract hierarchy, full DB schema, React shell
II	Core banking and agent baseline	5–9	DT-II.01–22	Lending features, 17 MCP tools, injection scanning, Hook 1
III	AI/ML and agent harness	10–13	DT-III.01–12	Three-tier prompt, session lineage, Chainlink Functions, ML pipeline
IV	Verification, evaluation, finalization	14–16	DT-IV.01–08	Certora proofs, Foundry suite, evaluation protocol, paper completion

The four-phase plan redistributes the original three-sprint workload into a 16-week timeline, giving AI/ML and agent harness work a dedicated phase and creating a separate verification and evaluation phase. Phase I database migrations at schema-creation time cost zero marginal effort; Phase II secures the agent safety baseline before any harness features are built.

Figure 4.1 shows the development process and phase-submission workflow.

4.13 SDLC Stage Mapping

We map our four implementation phases to the seven stages of the Software Development Life Cycle (SDLC). Figure 4.5 illustrates this mapping.

Table 4.8: SDLC stage mapping.

#	SDLC Stage	Project Activity	Deliverable
1	Planning	Feasibility studies; professor consultations; guideline review	Feasibility Report; Project Plan
2	Requirements	System analysis; use case definitions; constraints identification	Use case diagrams; UC-1 through UC-5
3	Design	Three-layer architecture; DB schema (3NF); smart contract interfaces	Architecture diagrams; ERD; DFD
4	Development	Phase I–IV: smart contracts, frontend, backend, AI/ML	Source code; DApp prototype
5	Testing	Hardhat unit tests (12+); integration testing; AI/ML evaluation	Test reports; model metrics
6	Deployment	Testnet deployment; frontend (Vercel); backend (Render)	Live prototype on testnet
7	Maintenance	Monitoring; model retraining; bug fixes; iteration	Updated docs; retrained models

This SDLC mapping table aligns the four-phase implementation plan with standard SDLC stages to ensure research deliverables remain complete (requirements, design, implementation, testing, and evaluation). The mapping makes the development process auditable and easier to justify in an academic setting.

4.14 Design Decisions and Alternatives

For each major design decision, we evaluated alternatives and justified selections based on technical criteria, ecosystem maturity, and project constraints. Figure 4.6 visualizes these comparisons.

Table 4.10: Design decisions and alternatives considered.

Decision Area	1st Choice	2nd Choice	Key Criterion
Methodology	Agile / Scrum	Incremental	Evolving scope, milestones
Architecture	DApp + Off-chain AI	Hybrid with Oracle	Gas cost, ML flexibility
Frontend	React + TypeScript	Vue + TypeScript	Web3 ecosystem maturity
Smart Contract	EVM (Solidity)	Solana (Rust)	Gas, control size, free testnets
Fraud Detection	Random Forest	XGBoost	SHAP compatibility, simplicity
Anomaly Detection	Isolation Forest	Autoencoder	Unsupervised; no labelled data needed
XAI Method	SHAP	LIME	Theoretical guarantees, regulatory fit
Database	PostgreSQL	SQLite	3NF support, async queries
Hosting	Vercel + Render	Localhost only	$0 cost, publicly accessible URL
Network (primary)	Polygon zkEVM Cardona	Polygon Amoy PoS	ZK validity proof security model vs. validator set assumption
Oracle mechanism	Chainlink Functions (DON)	Commit-reveal relay	Decentralised trust vs. single-key operator trust
Agent tool framework	MCP tool server	Direct Express.js API	Defined schema = safety boundary; no arbitrary code execution
Agent session auth	EIP-7702 session keys	ERC-4337 paymasters	Key-level scope enforcement vs. application-level scope
Prompt construction	Three-tier assembly (Stable / Context / Volatile)	Single ad-hoc string concatenation	Cache efficiency; independent auditability; human-gate invariant survives context compression
Session memory	Lineage model (parent-child sessions with compression summary)	10-turn sliding window only	Cross-session continuity for returning clients; auditable compression chain; queryable ancestry
Tool exposure per turn	Named toolsets scoped per detected intent	Full 17-tool list on every turn	Token cost reduction; attack surface narrowing; Interface Segregation Principle
Write-tool enforcement	Middleware confirmation audit hook (API layer)	Application-level check inside pipeline logic	Policy invariant holds independently of model behaviour; verifiable by code inspection

This design decisions table records evaluated alternatives (chains, stacks, libraries) and the criteria used to select the final approach. Documenting these trade-offs strengthens the methodological rigor and clarifies why the selected stack best matches the project's constraints.

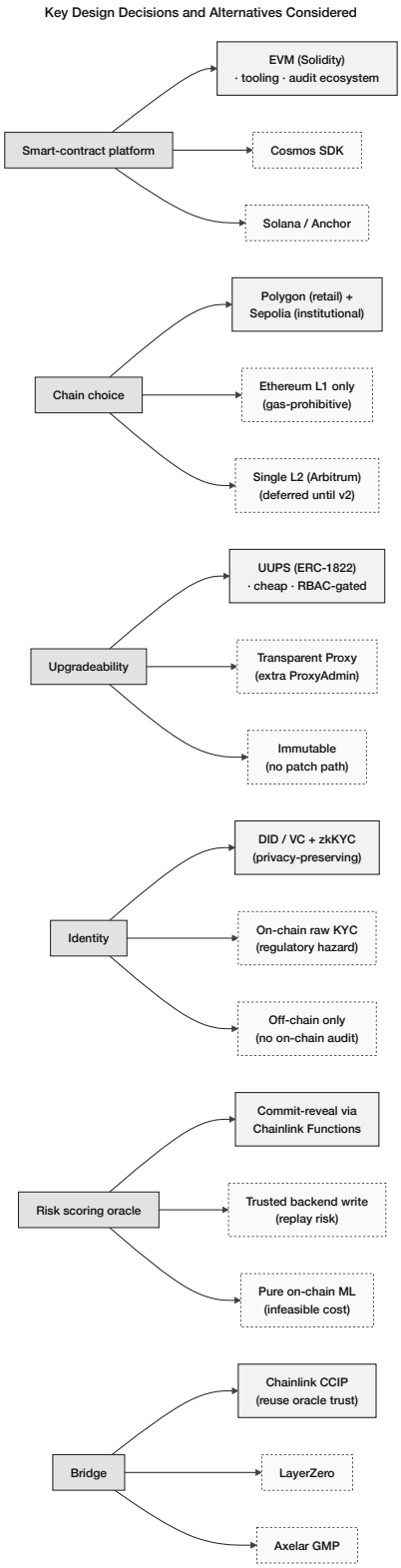

Figure 4.6: Key design decisions and alternatives considered: smart-contract platform (EVM vs. Cosmos / Solana), chain choice (Polygon + Sepolia vs. L1-only vs. single L2), upgradeability (UUPS vs. Transparent Proxy vs. immutable), identity (DID/VC + zkKYC vs. raw on-chain KYC vs. off-chain-only), ML oracle (commit-reveal vs. trusted-backend vs. on-chain ML), and cross-chain bridge (Chainlink CCIP vs. LayerZero vs. Axelar).

4.14.1 Justification of Selected Technologies

1st Choice Justifications:

- **Agile/Scrum** was necessary due to the project's evolving nature. Frequent updates to smart contract designs, user interface steps and AI/ML connections were required. Adopting Agile allows for plan alterations during brief work cycles effectively avoiding time and resource loss. Significance arises as new concepts emerge during the prototype development process.
- **DApp + Off-chain AI** architecture allows intensive machine learning operations to occur outside the blockchain. Machine learning tasks are often run on the blockchain at a considerable expense occasionally exceeding $100 for a single operation. Off-chain processing enables the use of rapid GPUs along with widely-used Python machine learning resources. Final lending choices are managed exclusively by smart contracts on-chain ensuring that trust is maintained in critical areas.
- **React + TypeScript** is beneficial as many Web3 libraries such as Wagmi, RainbowKit, Viem, and ethers.js are supported seamlessly. A vast community of developers exceeding 10 million exists for React making it simpler to access assistance and solutions.
- **EVM (Solidity)** is regarded as the most thoroughly examined smart contract system. Over $100 billion is secured across various blockchains by it. Building and testing the project without incurring costs is possible with free test networks such as Sepolia and Amoy. Security tools provided by OpenZeppelin have been thoroughly tested to ensure the protection of contracts.
- **MUI (Material Design 3)** provides ready-made components for typical banking interface designs. The package contains various items such as data tables, form checks and layouts for dashboards. Utilizing MUI results in a time saving of around 40% compared to starting from scratch.
- **Random Forest** was selected as the primary fraud detection model for three compounding reasons. First, it demonstrates natural compatibility with SHAP's TreeExplainer, which computes exact Shapley values—rather than approximations—by exploiting the tree structure directly. This exactness is a regulatory asset: in a lending context subject to explainability requirements under emerging frameworks such as the EU AI Act, approximation-based attribution tools like LIME can produce inconsistent feature rankings across identical inputs, undermining audit reliability [4]. Second, ensemble tree methods generalise well on structured tabular data with moderate sample sizes, a characteristic that matches the DeFi lending transaction log domain, where labeled fraud samples are scarce and synthetic augmentation is likely necessary. Third, Random Forest naturally supports class imbalance through class weighting and bootstrap sampling, reducing the risk of a model that achieves high accuracy by always predicting "not fraud"—a failure mode that would be catastrophic in a lending context.
- **Isolation Forest** does not require labeled data for training. Labeled fraud samples are few making it suitable for DeFi lending. Anomalies in transactions are scored swiftly as it operates with a time complexity of $O(n \log n)$.
- **SHAP** provides feature importance through solid mathematical foundations from game theory (Shapley values). Characteristics such as accuracy, management of absent data and consistency are not assured by tools like LIME. These properties assist in fulfilling emerging regulations for explainable decisions within financial services.
- **PostgreSQL** effectively manages complex time-based queries. Borrowing limits can

be checked over periods such as 6 months or 1 year. ACID compliance is fully supported to ensure financial data remains secure. It supports JSON which facilitates easy adjustments to the database structure during the prototyping phase.

- **Vercel + Render** facilitates deploying the demo globally at no charge. A live prototype can be accessed without the necessity of a setup on personal computers. Supervisors, examiners and other individuals are not required to deal with installation issues.
- **Polygon zkEVM Cardona** is selected over Polygon Amoy PoS because ZK validity proofs derive security from cryptographic verification rather than a validator set assumption. Every batch of transactions is accompanied by a zero-knowledge proof verified on Ethereum L1, making the security claim materially stronger for an institutional banking prototype. The phrase "ZK rollup-secured reserve ratios" is a more defensible thesis claim than "PoS-secured reserve ratios." On the Cardona testnet, the developer experience is equivalent to Amoy at zero cost.
- **Chainlink Functions** (DON-based oracle) is selected over the prototype commit-reveal relay because the DON runs the ML scoring request across multiple independent nodes — consensus is required before the result is accepted on-chain. A malicious single node cannot manipulate the risk score. This closes the oracle trust assumption of the v23 commit-reveal design, where trusting the operator running the FastAPI relay was an explicit limitation. The 30–60 second latency and $0.10–1.00 per oracle call are both acceptable for the loan approval lifecycle, where decisions are not time-critical.
- **MCP tool server** is selected over a direct Express.js API for agent interactions because the tool schema is the safety boundary: the agent cannot execute arbitrary code, make unapproved API calls, or read data outside the 17 defined tools. This constraint is the mechanism that makes the autonomous agent safe to deploy with real client funds. A direct API would require application-level enforcement only, which is less auditable.
- **EIP-7702 session keys** are selected over ERC-4337 paymasters for agent-controlled on-chain operations because scope restriction is enforced at the key level, not at the application level. The session key is bound to a specific list of tools, a value cap (500 USDC per transaction), and a 24-hour TTL — a compromised application cannot exceed these limits regardless of what the application code does. EIP-7702 does not replace ERC-4337 for retail gas sponsorship; the two mechanisms serve different functions and coexist in the architecture.

2nd Choice Justifications (Why Retained as Alternatives):

- **Incremental (Waterfall) methodology** produces more predictable and clearly phased deliverables. However, it offers limited capacity to adapt to the evolving requirements typical of research-oriented prototype development, and is most effective in stable production environments where scope is fixed from the outset.
- **Hybrid with Oracle architecture** (e.g., Chainlink) would enable on-chain ML prediction triggers via oracle data feeds, increasing the verifiability of AI-driven decisions. The trade-off is meaningful latency (30–60 seconds per update) and additional cost ($0.10–1.00 per oracle call), along with a dependency on third-party oracle availability that introduces an external failure point not present in the selected off-chain approach.
- **Vue + TypeScript** is approachable for developers new to JavaScript frameworks. Its

Web3 tooling ecosystem (e.g., vue-dapp) is less mature than React's, with sparser wallet integration library support and slower update cadence for critical Web3 dependencies.

- **Solana (Rust)** offers substantially higher raw throughput (up to 65,000 TPS) than EVM-compatible chains. However, Solana's developer tooling and security library ecosystem are less established, the Rust learning curve is significant within an 8-week development window, and the DeFi security audit tooling available for Solidity (Slither, Mythril, OpenZeppelin) has no direct equivalent on Solana.
- **Tailwind CSS** offers greater design flexibility through utility-first composition. Achieving consistent, professional banking-style UI patterns requires substantially more custom work than using MUI's pre-built component library, which imposes a time cost that is unacceptable within the implementation timeline.
- **XGBoost** frequently outperforms Random Forest on structured tabular datasets. However, its SHAP integration uses approximate rather than exact tree computation, meaning explanations cannot be guaranteed to be consistent across repeated evaluations—a property required for regulatory auditability of credit decisions.
- **Autoencoder** models can capture complex non-linear anomaly patterns. In practice, they require substantial labeled or unlabeled training data and careful hyperparameter tuning to converge reliably. Isolation Forest is parameter-light and performs competitively on small and imbalanced datasets, making it better suited to the data-scarce DeFi lending context of this prototype.
- **LIME** produces faster local explanations. As demonstrated by Adom et al. [4], LIME explanations are unstable: repeated evaluation of the same input can yield different feature importance rankings, undermining the consistency guarantees that regulators require for automated lending decisions.
- **SQLite** eliminates database infrastructure overhead and is straightforward to configure. It does not support concurrent writes, which limits multi-user prototype testing, and lacks the window function and CTE support needed for rolling 6-month and 1-year borrowing limit calculations.
- **Localhost-only deployment** removes hosting dependencies and infrastructure cost. It prevents remote sharing for supervisor review, collaborative testing across team members, and examiner access to a live demo—all of which are necessary for academic evaluation and iteration.
- **Polygon Amoy PoS** (2nd choice for network) offers a faster inner development loop on a familiar PoS testnet. It is retained as a fallback option should Cardona testnet availability be disrupted. However, the validator-collusion security assumption is weaker than ZK validity proofs, which undermines the institutional trust narrative of the thesis.
- **Commit-reveal relay** (2nd choice for oracle mechanism) is retained as the prototype fallback for Phase I and Phase II before the Chainlink Functions integration is wired in Phase III. It provides a working oracle during early development and is simpler to implement. The limitation—trusting the FastAPI service key—is clearly acknowledged as a prototype constraint.
- **Direct Express.js API** (2nd choice for agent tool framework) would allow the agent to call banking endpoints without a formal schema. The safety risk is that the agent could potentially construct arbitrary API calls not anticipated by the developers; the schema boundary of the MCP server eliminates this risk at the architecture level.
- **ERC-4337 paymasters** (2nd choice for agent session authentication) are already

used for retail gas sponsorship. They could theoretically be extended to agent operations, but scope enforcement would be at the paymaster application level rather than cryptographically at the key level, which is a weaker safety guarantee for autonomous agent execution.

4.15 Design Patterns

Our system uses five design patterns:

- **Singleton:** A singleton means that each primary contract is established a single time. Only one version is utilized by all ensuring a unified source of truth. Not everyone can create multiple versions of the contract.
- **Observer:** Contracts generate alerts whenever actions occur such as loan requests. These alerts are processed by a service that updates the database. Notifications are not ignored; they receive attention.
- **Adapter:** An Adapter is utilized by the wallet component to function with various wallet types (like MetaMask). Different wallets that adhere to the EIP-1193 standards can be easily integrated.
- **Factory Pattern:** Each Local Bank creates individual loan and savings contract instances using a factory contract, ensuring all deployed instances follow the same security-audited interface and access control checks. The factory pattern prevents unauthorized contract deployment that could bypass the hierarchical governance structure.
- **Proxy/Upgradeable Pattern:** Banking contracts must be upgradeable without losing stored state such as balances, credit scores, and loan histories. OpenZeppelin's transparent proxy pattern separates the contract's logic from its storage, allowing governance-approved upgrades to the logic layer while preserving all stored financial data. This pattern is essential for a long-lived banking system that must adapt to evolving regulatory requirements and security patches.
- **Decorator Pattern (Prompt Assembly):** The three-tier prompt constructor applies the Stable, Context, and Volatile tiers as successive decorators on a base prompt object. Each tier adds its content without modifying the structure established by prior tiers, enabling independent caching and testing of each layer without rebuilding the full prompt.
- **Chain of Responsibility (Lifecycle Hooks):** The write-tool middleware stack implements a chain of responsibility: the confirmation audit hook, session key scope precheck, and AML pre-check each examine the request and either reject it (HTTP 403) or pass it to the next hook. New policy controls are added as new chain links without modifying existing route logic or the model pipeline [R55].
- **Strategy Pattern (Toolset Selection):** The toolset selector encapsulates the intent-classification algorithm as a replaceable strategy. The current strategy is a lightweight keyword classifier; it can be replaced with a fine-tuned intent model without changing the prompt constructor or the MCP tool server, adhering to the Open/Closed Principle.

4.16 Software Testing Strategy

The testing strategy covers four layers of the system, each with defined acceptance criteria.

- **Smart contract unit tests:** Numerous tests for smart contracts exist in our Hardhat environment. Over 12 tests are conducted to verify actions related to managing reserves, sign-ups, loans and access permissions. No scenarios such as depositing zero or executing disallowed actions are missed in these assessments. **Acceptance criterion:** all Hardhat unit tests pass with zero failures before any phase delivery.
- **Integration testing:** Whole process checks are conducted for integration tests. Wallet connection, loan request, approval and repayment processes are evaluated on the network. No critical steps are excluded from the process. **Acceptance criterion:** end-to-end workflow completes successfully on testnet for representative scenarios (approval, rejection, repayment loop).
- **AI/ML module verification:** Ensuring functionality is vital for our fraud and unusual activity detectors. Results are generated by the integration of the detectors with the overall system. **Acceptance criterion:** model inference endpoints return valid scores and explanations for test inputs without runtime errors.
- **Frontend testing:** Website appearance is evaluated on Chrome, Firefox and mobile devices. Tests are conducted to ensure that functionality is maintained across various platforms. **Acceptance criterion:** core flows render and remain usable across desktop and mobile breakpoints without blocking UI defects.

Chapter 5

Market Analysis and Feasibility

5.1 Market Sizing

Table 5.1: Market segments with supporting data.

Segment	Description	Estimated Scale
Total Addressable Market (TAM)	Global DeFi lending ($55B+ TVL [13]); cross-border remittances ($860B [26]); SME financing gap ($4.5T [20])	$55B – 5T+
Serviceable Addressable Market (SAM)	Institutional and semi-institutional lending requiring hierarchical structures; emerging-market credit demand	$5B – 15B
Serviceable Obtainable Market (SOM)	Pilot deployments in regulatory sandboxes, academic prototypes, NGO-backed microfinance programs	$50 – 200M

This table quantifies the demand-side context (DeFi lending, remittances, MSME credit gap) used to ground the feasibility argument in measurable market data. It shows the platform targets both a high-volume settlement problem and a persistent inclusion/credit-access gap.

Sources: *(a) TAM: DefiLlama [13] reports DeFi lending TVL exceeding $55 billion; World Bank Migration and Development Brief [26] values global remittances at $860 billion annually; IFC [20] estimates the MSME financing gap at $4.5 trillion; (b) SAM and SOM: project-derived estimates based on industry segmentation of institutional lending and regulatory sandbox addressable market [18].*

No hierarchically structured lending system currently exists in DeFi; existing protocols rely on undifferentiated shared pools. Cross-border payment networks such as Ripple ($847M daily) [25] and JPMorgan Kinexys ($3–7B daily) [40] handle settlement volume but offer no lending, deposit, or tiered governance functions. Institutional interest in blockchain-based finance is clearly established, yet no single platform combines DeFi-style accessibility with the structured capital hierarchy of traditional development banking. The Crypto World Bank is designed to occupy this gap.

5.2 Target Customer Segment

The Crypto World Bank targets the **retail customer segment**—individual retail clients and small businesses seeking transparent, accessible crypto-based financial services.

Table 5.3: Target customer segment profile.

Characteristic	Description
Primary Users	Individual retail clients seeking personal or small business financing
Geographic Focus	Developing economies with limited traditional banking access (e.g., Bangladesh, Southeast Asia, Sub-Saharan Africa)
Loan Size Range	Micro to mid-range: 0.1 ETH – 500 ETH equivalent (~$200 – $1,000,000 at current rates)
User Profile	Digitally literate individuals with cryptocurrency wallet access; small business owners; gig-economy freelancers
Key Pain Points	High interest rates from informal lenders; lack of credit history in traditional systems; exclusion from banking due to documentation barriers

This target-customer table articulates the retail client profile and constraints that drive product requirements (low fees, simple onboarding, transparent terms). It supports the decision to prioritize usability and cost efficiency over complex institutional-only features in the prototype phase.

5.3 Partner Ecosystem

Table 5.5: Partner categories and roles.

Partner Category	Functional Role	Blockchain-Mediated Incentive
Financial Regulators	Regulatory sandbox approval; compliance oversight	Reduced enforcement cost through on-chain transparency and audit trails
Banking Institutions	Network membership as National/Local Banks	Access to diversified global reserve; reduced inter-bank settlement friction
Payment Gateway Providers	Fiat-to-crypto on-ramp and off-ramp services	Volume-based transaction fees; expanded market reach
Academic & Research Institutions	Validation of AI/ML models; publication of research findings	Access to anonymised datasets; collaborative research opportunities
Non-Governmental Organizations	Pilot deployment; field testing with underserved client populations	Transparent, low-friction credit access for beneficiaries

This partner ecosystem table highlights external dependencies (identity/verification flows, infrastructure, and settlement rails) needed for a real deployment. Identifying these actors early reduces hidden-scope risk and clarifies what remains outside the smart contract boundary.

5.4 Competitive Landscape

The Crypto World Bank operates at the intersection of four distinct competitor categories. Table 5.8 provides a detailed comparison of representative projects in each category, with

current metrics as of March 2026.

Table 5.7: Detailed competitive landscape analysis (Part 1).

Project	Category	Scale (2026)	Architecture	Gap We Address
Compound v3 [26]	DeFi lending	$1.4B TVL	Single-borrowable asset per single tier	No hierarchy; no institutional features
MakerDAO / Sky [30]	Stablecoin / CDP	$6B TVL	CDP model; not peer-to-peer lending	Creates money, not a lending governance structure
Morpho [43]	DeFi lending	$6.8B TVL; 1.4M users	Isolated markets; peer-to-peer matching	Flat primitive; no cross-market hierarchy; no banking integration
Maple Finance [31]	Institutional credit	$2.6–3.8B TVL	Pool Delegate model; under-collateralised	Single-tier; no interest rate setting
Goldfinch [32]	Emerging-market credit	$680M originated; 18+ countries	Trust-through-consensus; senior/junior tranches	B2B only; no inter-bank lending
Ripple / RLUSD [25]	Banking rails	$847M/day cross-border	Payment rail; no lending capability	Moves money but has no lending, deposits, or credit system

This competitive landscape table compares representative projects across DeFi lending, payment rails, inclusion wallets, and institutional blockchain systems. The comparison highlights the whitespace: no competitor combines hierarchical multi-tier lending with governance-aware controls and AI-assisted monitoring in one architecture.

Table 5.9: Detailed competitive landscape analysis (Part 2).

Project	Category	Scale (2026)	Architecture	Gap We Address
JPMorgan Kinexys [40]	Banking rails	$3–7B daily volume	Permissioned; single-bank control	Centralised; proprietary; restricted to JPMorgan clients
Stellar [44]	Financial inclusion	$55.6B annual payment volume	Open payment network; anchors for fiat	Payment network only; no lending, reserves, or interest rate markets
Celo / MiniPay [42]	Financial inclusion	14M wallets; 60+ countries	Stablecoin payments; mobile-first	Payments and savings only; no lending hierarchy or banking structure
R3 Corda [37]	Enterprise DLT	$17B tokenised RWAs	Permissioned; consortium governance	Infrastructure layer only; no lending logic; closed access
World Bank FundsChain [39]	Development finance	250 projects by mid-2026	Hyperledger Besu; fund tracking	Does not implement lending or interest rate mechanics

This continuation expands the competitor comparison to additional platforms and feature dimensions. It reinforces the thesis positioning by showing that existing systems typically specialize in one function (lending or payments) rather than a complete banking suite.

Table 5.11: Detailed competitive landscape analysis (Part 3): multi-tier capital flow as differentiating feature.

Feature Dimension	Competitors	Crypto World Bank
Hierarchical lending tiers	None — all competitors use flat pool architectures	Four-tier hierarchy: World Bank → National → Local → Client
Governance-controlled rates	Compound/Aave: algorithmic but no institutional hierarchy	Smart contract parameters; governance-defined bounds per tier
AI-assisted risk scoring	No competitor integrates on-chain ML fraud detection	Random Forest + SHAP; Isolation Forest anomaly detection
Retail and institutional access	Goldfinch / Maple: institutional only; Celo: retail only	Single platform serving retail clients through institutional capital hierarchy
Developing-market focus	Limited; most are US/EU-centric	Designed for Bangladesh, Southeast Asia, Sub-Saharan Africa

This final competitor table block concludes the comparative analysis and clarifies why multi-tier capital flow is a differentiating architectural feature. The results inform the go-to-market focus on transparency, governance structure, and retail accessibility.

Upon reviewing various projects a common feature was identified: a lending mechanism does not exist that operates with tiers, is distributed and facilitates transactions across different levels and within identical levels. Such innovation distinguishes the Crypto World Bank. The key differentiating features of the Crypto World Bank are as follows:

- Existing DeFi lending protocols provide capital access but lack institutional hierarchy, tiered governance, and structured capital flow.
- Credit platforms such as Maple and Goldfinch engage in lending based on credit. Credit-based lending is often focused on businesses and exists at just one tier. These platforms do not cater to personal loans.
- Ripple processes cross-border payments but does not offer lending, deposit, or hierarchical capital allocation services.
- Celo provides assistance to individuals in emerging economies. Payment and savings functions are offered yet lending services remain absent.
- **Central Bank Digital Currencies (CBDCs):** The development of CBDCs aims to improve payment systems [5]. Concerns about privacy and control are raised as lending features are excluded. A lending layer does not exist on a CBDC. Users benefit from our platform which safeguards their interests while providing a level-based system similar to that utilized by CBDCs [5].

5.5 Risk Taxonomy

Table 5.13: Risk taxonomy and mitigation.

Risk Category	Description	Severity	Mitigation
Partner non-cooperation	Key partners decline to participate	Medium	Initiate with low-barrier academic and NGO pilots
Smart contract vulnerability	Exploit in contract logic	High	OpenZeppelin primitives; formal audit (planned); pause mechanism
Regulatory adversity	Jurisdictional restrictions	Medium	Testnet-only prototype; regulatory sandbox engagement
AI/ML model degradation	Fraud detection accuracy decay	Low	Continuous retraining; human-in-the-loop; SHAP explainability

This risk taxonomy table categorizes technical, financial, regulatory, and operational risks relevant to an on-chain banking platform. The taxonomy is used to justify design mitigations such as reserve enforcement, role-based approvals, and staged deployment through testnets and sandbox programs.

5.6 Technical Feasibility

Table 5.15: Technical feasibility assessment.

Component	Assessment	Evidence
Smart contracts	Fully feasible	Three contracts implemented and tested with Hardhat (12+ passing unit tests); Solidity 0.8.20 with OpenZeppelin
Frontend DApp	Fully feasible	React 18 + TypeScript with all pages implemented; Wagmi and RainbowKit provide mature wallet integration
Blockchain deployment	Fully feasible	Polygon zkEVM Cardona and Ethereum Sepolia provide zero-cost, production-equivalent environments; ZK validity proofs provide cryptographic finality on Cardona
AI/ML integration	Feasible with constraints	Random Forest inference achieves sub-50 ms latency; SHAP explanations computable in real time
Database backend	Feasible	PostgreSQL schema designed (19 tables, 3NF); FastAPI provides async REST framework

This technical feasibility table summarizes the readiness of core infrastructure (EVM tooling, Layer 2 scalability, and security libraries) required by the prototype. It supports the claim that the project leverages mature ecosystems rather than experimental primitives, reducing implementation risk.

The technical feasibility of the platform rests on five pillars: the maturity of the underlying blockchain infrastructure, the availability of development tooling and security primitives, the demonstrated scalability of Layer 2 networks for financial applications, the precedent of comparable institutional blockchain deployments, and the platform's modular architecture which allows incremental delivery without requiring the full system to be operational before any component can be tested.

Ethereum—the EVM standard on which the platform is built—has operated continuously since 2015, while Polygon PoS has processed billions of transactions with low fees and rapid finality, making it suitable for high-frequency retail operations. The contract layer uses established security primitives (e.g., OpenZeppelin libraries and widely reviewed patterns), and the off-chain services use standard, production-grade web tooling for API and ML inference. Because the architecture is modular, components such as savings, FX, group lending, and insurance can be developed and validated independently and integrated through governance-approved rollouts.

5.7 Economic Feasibility

Table 5.17: Economic feasibility — zero-cost prototype.

Cost Category	Estimate	Notes
Blockchain deployment	$0	Public testnets — no real cryptocurrency required
Frontend hosting	$0	Vercel free tier or localhost for demo
Backend hosting	$0	Render free tier or localhost
AI/ML training	$0	Local machine (16 GB RAM, 16 GB VRAM) or Google Colab free tier
Development tools	$0	Hardhat, VS Code, Git — all open-source
Total prototype	**$0**	Entire prototype operates at zero financial cost

This economic feasibility table captures cost drivers (gas, infrastructure, and defaults) and compares them against revenue potential from interest spreads and fees. The key conclusion is that low-fee Layer 2 deployment makes small-loan banking workflows economically viable, unlike high-fee base layers.

The economic feasibility of the platform is evaluated across four dimensions: operational cost sustainability, revenue sufficiency, capital efficiency, and macroeconomic impact. On Polygon PoS, the total on-chain gas cost for a full retail loan lifecycle is measured in cents, while interest spread revenue is orders of magnitude larger, yielding a favorable cost-to-revenue profile for small loans that are infeasible on high-fee base layers. Off-chain infrastructure costs (hosting, RPC access, storage, ML inference) are comparable to typical SaaS deployments and scale with usage.

Revenue is diversified across interest spreads, origination fees, and potential FX spread revenue for multi-currency conversions. Capital efficiency is enhanced by the platform's reserve-enforced tiered allocation (each tier retains a minimum solvency reserve before deploying capital downward) and by reducing the need for idle pre-funded balances typical of correspondent banking. Note that because USDC is a 100%-backed stablecoin, the platform does not create new money through lending—it re-allocates existing USDC through the hierarchy. The Reserve Ratio (RR) functions as a solvency constraint at each tier, not as a money-creation mechanism; see the Credit Velocity formula in the List of Formulas for the correct framing of capital throughput. At scale, remittance cost reduction and improved access to credit in underserved markets produce additional social and economic value beyond platform revenue.

5.8 Revenue Projection

The following projection models annual revenue potential at full deployment scale. Calculations use a reference ETH price of **$2,500**[1] (conservative mid-point for February 2026; actual spot was approximately $2,800 per CoinGecko on 1 February 2026) and interest rate parameters defined in Section 5.8.1. A sensitivity table showing revenue at $2,000, $2,500,

[1]All ETH-denominated cost and revenue calculations use $2,500/ETH as a conservative planning assumption consistent with the v22 revision date. The platform's stablecoin-first design (Section 1.10.3) means retail loan obligations are denominated in USDC, decoupling client repayment risk from ETH price volatility.

and $3,000 per ETH is provided after the main projection; all three scenarios use the same 8% APR and loan-base assumptions, so revenue scales linearly with ETH price.

Table 5.19: Revenue projection assumptions.

Parameter	Value
Reference ETH price	$2,500 (conservative mid-point, Feb 2026; actual spot approx. $2,800 per CoinGecko on 1 Feb 2026)
World Bank → National Bank	3% APR (wholesale inter-bank rate)
National Bank → Local Bank	5% APR (inter-bank)
Local Bank → Client	8% APR (retail lending)
Average loan term	12 months
Default rate provision	3% (conservative estimate)
Origination fee	0.25% per disbursement

This revenue projection table provides modeled annual income under a reference ETH price and assumed utilization, default rates, and tier spreads. It illustrates how hierarchical spreads across tiers accumulate into platform-level revenue while keeping retail APR within plausible ranges.

Table 5.21: System-wide annual revenue summary — spread-based accounting.

Tier / Revenue Stream	Spread Rate	Annual Revenue (USD)	Basis
Tier 1 (World Bank): lends at 3% to NBs, cost of capital 0%	3% on deployed capital	$51.6M	$1.72B total retail loan base × 3%
Tier 2 (National Banks): borrow at 3%, lend at 5%	2% spread	$34.4M	$1.72B × 2%
Tier 3 (Local Banks): borrow at 5%, lend at 8%	3% spread	$51.6M	$1.72B × 3%
Total platform revenue	**8% (client pays)**	**$137.6M**	Sum of tier spreads = 3%+2%+3% = 8%; no double-counting
Origination fees (0.25%)	—	$4.3M	0.25% × $1.72B disbursed
FX spread revenue (est.)	0.5–1.0%	$8–17M	Estimated conversion volume
Total incl. fees		**$150–159M**	Conservative base case

Revenue is computed using a spread-based model: each tier earns the difference between its borrowing rate from the tier above and its lending rate to the tier below, applied to the same underlying loan capital as it flows down the hierarchy. Summing the three tier spreads (3% + 2% + 3% = 8%) equals the retail client's total APR, confirming internal consistency—no double-counting occurs. An earlier presentation of this table (v15) reported $224.6M by summing the full APR at each tier rather than the spread; that figure was incorrect and has been corrected here.

Scale note on the $137.6M figure: *This is the theoretical 10-year ceiling at 68,800 active loans across 25 Local Banks at near-full capacity—it is included for scale reference only. The pre-thesis evaluation target is Year 1–2 of the adoption ramp, where the platform reaches operational break-even at approximately 300 active loans and demonstrates positive unit economics on each incremental loan above that threshold. Figure 5.1 shows the theoretical maximum at full maturity; see the break-even analysis above for the realistic path from zero to that ceiling.*

The projection above models interest spread income only (base case: $137.6M). Additional revenue streams from FX conversion spreads (estimated 0.5 to 1.0% per conversion, yielding $8–17M), loan origination fees (0.25% per disbursement, yielding $4.3M), and insurance fund premiums (0.5% of loan value annually) represent material upside captured in the "Total incl. fees" row of the table ($150–159M). These are net incremental revenue streams—they do not overlap with the interest spread income already accounted for.

Table 5.22: ETH price sensitivity: annual interest spread revenue at three price points (all other assumptions held constant at base case).

ETH Price	Total Loan Base	Interest Spread Revenue	Incl. Fees (est.)
$2,000	$1.376B	$110.1M	$120–128M
$2,500 (base case)	$1.720B	$137.6M	$150–159M
$3,000	$2.064B	$165.1M	$180–191M

Sources: *ETH price from spot market data (CoinGecko); interest rate tiers aligned with Aave [15] and Compound [16] DeFi benchmarks; default rate and origination fee from conservative industry estimates. Revenue methodology: spread-based, not gross-APR stacking.*

Cost Model Considerations.

- **Gas costs:** Gas expenses on Polygon PoS are quite low. A complete retail loan lifecycle on Polygon involves approximately 27–32 individual on-chain state changes:
  - 1 loan request (`SSTORE`: borrower data, loan ID, status)
  - 1 credit history lookup + risk score commit
  - 1 approval (`SSTORE`: status update, disbursement record)
  - 1 disbursement (ETH transfer + `SSTORE`: balance update)
  - 12 installment payments $\times$ 2 `SSTORE` each = 24 writes
  - 1 loan closure (`SSTORE`: final status)
  - $\approx$4 event emissions (`LOG2`/`LOG3`)

  On Polygon PoS, with gas prices of approximately 30 Gwei and MATIC at $\approx$$0.60, each `SSTORE` costs approximately $0.00036. A 30-operation lifecycle costs approximately $0.011 total—well under 0.01% of the interest earned. On Ethereum mainnet at 15 Gwei base fee, the same operations would cost $2.40–$4.80, reinforcing the design choice of Polygon PoS [55].
- **Infrastructure costs:** Backend hosting (API server, database, ML service), frontend delivery (CDN) and RPC nodes (Alchemy, Infura) can incur costs ranging between $500 and $2,000 monthly. An increase in expenses will be experienced with a rise in transactions.

- **Default losses:** A single default rate assumption is insufficient for an early-stage platform. Table 5.23 presents a three-scenario sensitivity analysis:

Table 5.23: Default rate sensitivity scenarios with economic basis.

Scenario	Default Rate	Annual Loss	Basis
Optimistic	3.7%	$7.4M	Grameen Bank 96.29% recovery (June 2024) [R12] after 48 years of social infrastructure
Base Case	8%	$16M	Typical early-stage DeFi undercollateralized lending; no established social trust
Stress Test	15%	$30M	Early-stage crypto-native clients, no prior on-chain credit history, high ETH volatility

Grameen Bank, after nearly five decades of social lending infrastructure, reported a loan recovery rate of 96.29% as of June 2024 [R12]. The Crypto World Bank is a nascent platform without equivalent social trust or community officers. Its initial user population will likely exhibit default rates closer to 8–15% before on-chain credit history accumulates sufficient predictive signal. The base case adopts 8% default, with break-even at approximately 11.2% default rate under the base loan volume assumption.

Break-even user count: At the base case of 8% default with an average loan size of 10 ETH ($25,000 at $2,500/ETH) and 8% APR, each loan generates $2,000 annual interest and incurs under $0.02 gas on Polygon. At $500/month infrastructure costs, the platform requires at minimum 4 active loans to cover operating expenses and approximately 200 active loans to cover expected default losses at 8%—a realistic near-term target for a pilot in 2–3 Local Banks.

- **Break-even analysis:** Opportunities for profit on loans as small as 0.01 ETH ($25) exist on Polygon PoS. Profitability on loans below 5 ETH ($12,500) is not achievable on Ethereum mainnet without Layer 2. Layer 2 is essential for facilitating micro-loans.

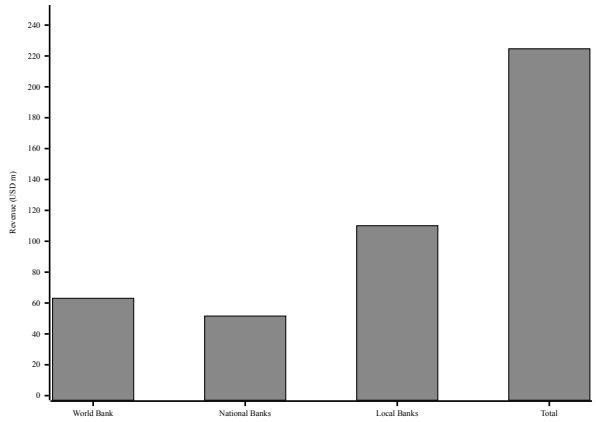

Figure 5.1: Annual revenue projection at full 10-year deployment maturity (USD millions, spread-based accounting at $2,500/ETH planning assumption). This represents the theoretical capacity ceiling at full institutional scale; see the break-even analysis above for the 3-year adoption ramp toward this target, with break-even occurring at approximately 300 active loans.

The interest spread identified above is the platform's primary revenue stream. Placing it within the broader landscape of institutional crypto-exchange revenue models demonstrates that CWB has a diversified revenue path even without a native token. Table 5.24 maps each CWB mechanism to its nearest exchange-sector analogue, drawing on the revenue architecture of major centralised exchanges (Binance, Coinbase) and DeFi protocols described in the ecosystem survey.

Table 5.24: CWB revenue stream taxonomy: analogues from the institutional crypto exchange sector.

Revenue Stream	Exchange Equivalent	CWB Mechanism
Interest spread (primary)	Binance OTC desk	SavingsVault yield vs. retail lending rate; 3%+2%+3% tier spread = 8% client APR
Loan origination fee	Trading fee (0.1%)	0.5–1% flat origination fee on disbursement; already modelled in base case above
Deposit mobilization	Binance Earn	SavingsVault + FixedDeposit products; yield funded by the lending tier spread
Local Bank registration	Exchange listing fee	One-time registration fee per Local Bank joining the network
Syndicated loan arrangement	Institutional desk	Lead Arranger fee on TranchedPool / SyndicatedLoan cross-tier facilities
Reserve transparency API	Data API subscription	Phase 3: GraphQL API over The Graph subgraph; queried by external auditors and regulators

The six streams above are non-overlapping; none double-counts the interest spread already accounted for in Table 5.21. The Local Bank registration fee and Reserve Transparency API are Phase 2/3 items

and are not included in the base-case revenue model above. The interest spread and loan origination fee are the only streams active at the pre-thesis prototype stage.

Hard rule: no native CWB governance token in the prototype. The FTX collapse of November 2022 demonstrated the systemic failure risk of self-minted tokens used as collateral: FTT, issued and held principally by the exchange itself, served simultaneously as the primary collateral asset for Alameda Research's borrowing, creating a reflexive loop that triggered an estimated $8 billion solvency shortfall within 72 hours of a competitor announcing it would liquidate its FTT holdings. The lesson for the Crypto World Bank is categorical: no native CWB governance token is issued in the thesis prototype. On-chain reputation is provided instead by the Credit Passport SBT (Section 3.11), which is non-transferable and carries no speculative market price. A future governance token—analogous to Binance's BNB utility-and-burn model, which took seven years and $16.8 billion in annual revenue to reach legitimacy—is scoped to Phase 3 of the CWB roadmap, following institutional trust bootstrapping, and is specified in Future Work with explicit anti-FTX collateral hard rules encoded at the contract level (no self-issued asset may be used as collateral within the same protocol that issues it). This constraint will be enforced as a Foundry invariant test rather than a governance policy, so it cannot be bypassed by any admin action.

5.8.1 Transaction Economics: Interest Rates

Table 5.25: Interest rate parameters.

Parameter	Value	Benchmark
Base Annual Interest Rate	5–12% APR	Aligned with Aave/Compound variable rates
Rate Determination	Set by Local Bank approvers within World Bank-defined bounds	Configurable per-bank for local market conditions
Late Payment Penalty	2% of installment + 0.5%/week (capped at 10%)	Industry-standard late fee structure
Interest Calculation	Simple interest on outstanding principal	Transparent, client-friendly
Rate Transparency	All parameters stored on-chain	Publicly auditable; no hidden fees

This interest rate table defines the tiered APR parameters used throughout the feasibility and revenue modeling. Presenting the rate structure explicitly makes the spread logic auditable and connects the economic model directly to the proposed governance-controlled parameters.

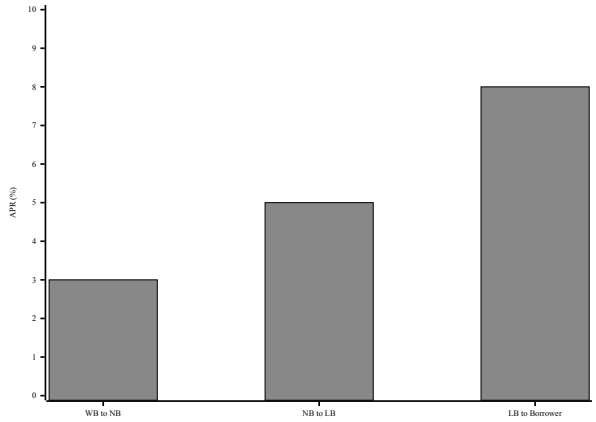

Figure 5.2: Hierarchical interest-rate spread (APR) across the four-tier lending structure.

5.8.2 Global Economic Impact

Apart from revenue generated through platforms, various economic advantages are provided by the Crypto World Bank:

- **Capital deployment and fiscal multiplier:** At projected full-deployment lending volume of $2 billion across nearly 1,000 institutional clients, the platform generates a downstream fiscal multiplier of approximately $2.5 to $3 per dollar lent [19], implying $5 to $6 billion in annual economic activity. The IFC estimates that every $1 million lent to small businesses in developing economies creates approximately 16 new jobs [20], making capital deployment at this scale a meaningful driver of employment and local economic growth.
- **Remittance cost reduction:** Global remittances total approximately $860 billion annually, of which an estimated $48 to $56 billion is consumed by transfer fees averaging 6.49%—more than double the United Nations Sustainable Development Goal target of 3% [26]. On-chain settlement on Layer 2 networks reduces transaction costs to well below 1%, directly compressing this fee burden. Comparable blockchain payment networks demonstrate the feasibility of this target: Stellar processed approximately $55.6 billion in payments at a fee of roughly $0.0007 per transaction in 2025 [44], and Celo's MiniPay processes payments for under one cent across more than 60 countries [42].
- **Trapped capital liberation:** The correspondent banking system requires banks to maintain pre-funded nostro and vostro accounts in every currency corridor in which they operate, immobilizing large sums that cannot be deployed for lending or investment [24]. On-chain atomic settlement eliminates the need for these idle pre-funded balances, freeing capital for productive use across the lending hierarchy.
- **Financial inclusion:** The platform is designed for accessibility: sub-cent gas fees on Polygon PoS, a mobile-first interface, and support for micro-loan amounts below one dollar lower the barriers to credit for the estimated 1.4 billion adults globally who remain outside formal financial systems [14]. The World Economic Forum has identified decentralized finance as a leapfrog technology capable of enabling populations to bypass the infrastructure constraints of traditional banking [41], a pattern already observed in mobile payments across developing economies.
- **Transaction cost reduction:** On Polygon PoS, transaction fees remain below one cent—a reduction of over **99%** relative to the average $42 cost of a correspondent bank-

ing transaction [25]. This cost compression makes financially viable a large class of micro-transactions and small-loan servicing operations that are currently uneconomical through traditional payment rails.

- **Transparency as an economic good:** On-chain publication of reserve ratios, interest rates, and transaction records eliminates the informational asymmetry that enables predatory lending practices in opaque banking environments. In conventional banking, borrowers—particularly in developing economies—frequently operate without visibility into the true cost of credit or the basis for lending decisions. The Crypto World Bank's design ensures that all participants access identical, verifiable information, reducing the scope for hidden fees and unfair pricing.

5.8.3 Value Proposition and Go-to-Market

Table 5.27: Go-to-market phases.

Phase	Activities	Timeline
Phase 1: Validation	Competition submission (BCOLBD 2025); thesis publication; open-source release	Current
Phase 2: Pilot	Regulatory sandbox application; institutional partnership; testnet-to-mainnet migration	6–12 months
Phase 3: Production	Multi-chain deployment; enhanced monitoring and analytics; governance token launch	12–24 months

This value proposition table summarizes user-visible benefits (cost, transparency, speed, and inclusion) and maps them to platform features. It supports the go-to-market narrative by linking technical design choices to practical outcomes for retail users and partner institutions.

5.9 Currency Risk and the Stablecoin Imperative

A retail client in rural Bangladesh who takes a loan denominated in ETH faces a fundamental risk that does not exist in traditional microfinance: if the price of ETH doubles between loan disbursement and final repayment, the real value of their repayment obligation doubles in taka terms. For clients near the poverty line, this is not a theoretical risk—it is potentially catastrophic. The May 2021 crypto market crash saw ETH lose approximately 55% of its value within six weeks; ETH then halved again over the course of 2022.

This is why stablecoin integration is treated as a **critical path item** elevated to a design requirement in Section 1.10.3, not an optional extension. The platform supports USDC or USDT-denominated loans as the default product at retail tiers, with ETH-denominated loans reserved for institutional-tier participants who can manage currency risk. BIS Working Paper No. 905 [R13] identifies stablecoin volatility as a structural risk in emerging-market DeFi adoption and recommends fully-collateralized models (USDT/USDC) over algorithmic designs for developing-country use cases. The Terra/LUNA collapse (May 2022, ≈$40 billion lost) provides the definitive negative example. ERC-20 stablecoin integration has technical implications: token allowances, `transferFrom` hooks, decimal precision (USDC uses 6 decimals vs. ETH's 18), and the approval-transfer-state-update ordering required by the CEI

pattern must all be redesigned as a separate contract module—work that is in scope for the SavingsVault contract specified in Section 3.10.

5.10 MiCA and GENIUS Act Compliance Mapping

The platform's regulatory feasibility is not abstract: two recent statutes apply directly to any retail stablecoin-denominated lending product issued to European or US-jurisdiction users. The EU Markets in Crypto-Assets (MiCA) Regulation has been fully in effect since December 2024 [R36, R37]; the US GENIUS Act, signed in July 2025 [R38], establishes the first federal framework for payment stablecoins. Table 5.29 maps the relevant articles to the design choices that satisfy them and to the work that remains.

Table 5.29: MiCA and GENIUS Act compliance mapping. Each row identifies a regulatory article relevant to a stablecoin-denominated crypto financial services platform and the corresponding CWB design control.

Statute / Article	Requirement	CWB Design Control
MiCA Art. 16 (EMT Issuer Authorisation)	Authorized issuer required for any e-money token offered to the public	USDC / USDT are issued by Circle / Tether under their own authorization; CWB uses but does not issue EMTs.
MiCA Art. 36 (Reserve of Assets)	1:1 reserve backing, segregated from issuer assets	Stablecoin reserves verified via on-chain attestations from Circle (USDC) before being accepted by SavingsVault.
MiCA Art. 41 (Redemption Rights)	Holders must have a right of redemption against the issuer at par	CWB does not impede redemption; SavingsVault withdraw function returns the underlying ERC-20 directly to the depositor wallet.
MiCA Art. 62 (Operational Resilience)	Regular audits, governance, complaint handling	Five-layer defense-in-depth (Section 3.19); Foundry invariants (Section 4.6); Certora reserve invariants (Section 4.5); Tenderly runtime monitoring (Section 4.8).
GENIUS Act § 4 (Federal Payment Stablecoin Framework)	Federal registration of payment-stablecoin issuers	Same posture as MiCA Art. 16: CWB uses, does not issue.
GENIUS Act § 7 (Reserve Disclosure)	Monthly attestation and public disclosure of reserves	Circle / Tether attestations are independently verifiable on-chain; CWB dashboard surfaces issuer attestation links per stablecoin pool.
EU AI Act Art. 86	Right to explanation for AI-assisted decisions	Borrower-facing SHAP explanations (Section 4.2); plain-language SHAP rendering as a regulatory compliance feature, not a UX bonus.
GDPR Art. 17 (Right to Erasure)	Personal data must be deletable on request	No PII on-chain; off-chain PostgreSQL data is encrypted and subject to data-subject deletion requests; only document hashes (SHA-256) appear on-chain.

The result is a compliance posture that is defensible to European and American institutional readers and evaluators in a single page rather than buried in narrative. Compliance gaps that remain—a formal MiCA whitepaper if CWB ever issues its own token, and FinCEN reporting integration in the US—are explicitly named in the Future Work section of the Conclusion.

5.11 Bangladesh Regulatory Reality

Bangladesh is the primary target market named throughout this thesis. The regulatory reality of operating there as of 2026 must be addressed explicitly rather than implied. Three observations frame the path forward.

Crypto remains effectively illegal for retail use as of 2025–2026. Bangladesh Bank's longstanding 2018 circular treats crypto trading as a Foreign Exchange Regulation Act offence, and no subsequent statute has formally legalized retail crypto activity [R41]. The Crypto World Bank prototype is therefore deployed exclusively on public testnets (Polygon zkEVM Cardona, Ethereum Sepolia) using test tokens; no Bangladeshi participant transacts with mainnet value at any point in the pre-thesis or final-thesis phase.

The CBDC trajectory is the credible legal path. The central bank launched a CBDC feasibility study in 2022 and a pilot in 2024 but as of 2025 has not produced a deployment timeline. The Crypto World Bank's positioning against mBridge and Agora (Section 2.2.16) provides a direct upgrade path: when Bangladesh Bank issues a CBDC under the same hierarchical distribution model that the Tan (2023) IMF working paper [5] describes, the platform's Tier 1 / Tier 2 contracts can settle in that CBDC without any change to the four-tier architecture. The current ETH / USDC denomination is a stand-in for the future CBDC asset.

The Regulatory FinTech Facilitation Office is the entry point. Bangladesh Bank operates a Regulatory FinTech Facilitation Office (RFFO) sandbox for new financial-services products. As of 2026, no crypto-lending project has been licensed under this sandbox, but the framework exists and FE Circular No. 06 (January 2025) on electronic Letters of Credit communication [R42] together with the Payment and Settlement System Act 2024 [R43] provide the relevant regulatory anchors. The platform's go-to-market path for Bangladesh is therefore: (a) academic deployment on testnet for pre-thesis, (b) a controlled sandbox engagement with RFFO for the final-thesis evaluation, (c) initial pilot with a partner microfinance institution under the sandbox, and (d) phased CBDC integration once the Bangladesh Bank CBDC pilot moves beyond feasibility.

2025 Bangladesh-specific financial-inclusion data. The financial-inclusion argument is updated against Bangladesh Bank Special Publication SP2025-02 (June 2025) [R44]: female-to-male account parity reached 49%/49% by March 2025, and female-owned deposit accounts grew from 33.4 million in 2019 to 55.3 million in 2024. BRAC's 2025 data shows 89% of loans to women and 92% reporting household income increase [R45]. These statistics are more current and specific than the 2021 Global Findex *1.4 billion unbanked* headline figure used elsewhere in the thesis, and they justify directing the solidarity-group module specifically at women-led client groups in line with the BRAC and Grameen tradition this work builds on.

FATF Travel Rule scoping for Bangladesh. The Financial Action Task Force Recommendation 16 (the "Travel Rule") requires that transfers above USD 1,000 include structured originator and beneficiary information transmitted alongside the transfer. In most FATF member jurisdictions this threshold is now in force for virtual asset service providers; Bangladesh, as a FATF Asia-Pacific Group member, is under the same obligation. For the Crypto World Bank, the Travel Rule applies specifically to inter-tier capital flows: Local Bank to National Bank repayments and upward surplus repatriation via the `UpwardDepositFacility` contract routinely exceed the USD 1,000 threshold. The off-chain Travel Rule data packet required for each such transfer is specified in Section 3.13.2:

```
{
  "originator_institution": "LocalBank-BD-042",
  "originator_tier": 3,
  "beneficiary_institution": "NationalBank-BD-001",
  "beneficiary_tier": 2,
  "amount_usdc": 5000,
  "purpose": "IBLP_REPAYMENT",
  "onchain_tx_hash": "0xABC...",
  "timestamp": "2026-06-01T10:00:00Z"
}
```

This packet is stored in the PostgreSQL `audit_logs` table and is linked to the on-chain transaction hash, making it available to regulators through the formal audit request workflow described in Section 3.18.4. The `audit_logs` table is append-only (enforced at the database-role level), ensuring the Travel Rule record cannot be altered after filing. For a Bangladesh deployment under the RFFO sandbox, this data packet specification must be submitted to Bangladesh Bank's Financial Intelligence Unit alongside the sandbox application. Implementation of the cross-tier Travel Rule notification protocol—including automated packet generation, digital signing by the originating institution, and transmission to the FIU—is Future Work contingent on Bangladesh Bank guidance on the precise format and submission channel required.

5.12 Bootstrap Funding and the Tier 1 Capitalization Problem

The Crypto World Bank faces a **bootstrap funding problem** analogous to the capitalization challenge of real multilateral development banks. The World Bank Group was initially capitalised by 44 member-state subscriptions in 1944; the IBRD's combined subscribed capital exceeded $270 billion following its 2018 capital increase (Ocampo & Gallagher, 2024 [R15]). Three initial capitalization mechanisms are proposed for the Tier 1 Reserve:

1. **Founding stakeholder deposits:** Universities, NGOs, and development-focused blockchain organizations deposit ETH or USDC into the Tier 1 Reserve in exchange for governance tokens and yield rights.
2. **Protocol-owned liquidity (POL):** The protocol accumulates treasury reserves through bond mechanisms in which external parties swap ETH or USDC for discounted governance tokens over a vesting schedule. *Note: The Olympus DAO algorithmic reserve model (2021–2022) is cited here as a cautionary precedent—Olympus OHM lost over 99% of its token value within one year due to unsustainable algorithmic*

backing. The CWB POL mechanism is strictly collateralized: only fully-backed USDC bonds are accepted, and governance tokens carry no intrinsic algorithmic price-support obligation. This removes the reflexive collapse risk that destroyed Olympus DAO.

3. **Philanthropic/impact grant funding:** Development finance institutions (IFC, ADB) have expressed interest in blockchain-based development finance tools, as evidenced by the World Bank FundsChain initiative [39]. Grant funding from these institutions could seed the Tier 1 Reserve in exchange for research access and co-branding.

In the short term, a multi-signature wallet (3-of-5 signers representing different stakeholder groups) governs Tier 1 allocations. In the long term, an on-chain governance module (similar to Compound Governor Bravo) enables token-weighted voting with time-locks to prevent governance attacks.

5.13 Prototype Scope and Limitations

A straightforward version of the complete system is represented by the current prototype:

1. **Hierarchical lending scope:** Utilization of the Tier 1 World Bank Reserve contract is comprehensive. Deposits are managed, loan requests and approvals are processed alongside the display of reserve statistics. Not all functions of the Tier 2 (National Bank) and Tier 3 (Local Bank) contracts have been implemented. Role management and registration are not the only aspects focused on. Currently the process for fund transfers from the World Bank to National Bank and subsequently to Local Bank is incomplete. Four tiers will be included in the final design.
2. **Interest accrual and repayment:** Rules for interest rates (3%/5%/8%) are part of the system design. Fees for initiating loans and penalties for delays also exist. Interest calculations, payment schedules or penalties aren't managed automatically by current smart contracts. Future updates will incorporate these functions.
3. **InterBankLendingPool, UpwardDepositFacility, and broader multi-entity / cross-tier operations:** Same-tier interbank lending, upward surplus repatriation, syndicated and tranched lending, cross-tier treasury FX, and multilateral settlement netting are fully specified in Section 3.13 (with corresponding ERD entities listed in Section 3.13.7). The smart contracts (`InterBankLendingPool`, `UpwardDepositFacility`, `TreasurySwap`, `SyndicatedLoan`, `TranchedPool`, `NettingEngine`) are scheduled for Phases II and III; the current pre-thesis 1 prototype does not yet exercise these multi-directional flows.
4. **AI/ML integration:** AI tools such as fraud detection (Random Forest) and anomaly detection (Isolation Forest) along with decision explanations (SHAP) are set to be utilized by the system. A service has been established with FastAPI for machine learning. This service does not currently link to the loan approval process in the existing version.
5. **Backend architecture:** Express.js paired with MongoDB is utilized for rapid API development. The main database is intended to be PostgreSQL. FastAPI is utilized by the machine learning service.
6. **Banking product suite:** The extended banking product suite described in Section 3.17, including savings accounts, fixed-term deposits, transactional accounts, group lending, FX conversion, and the insurance fund, is specified at the architectural and design level in this report. Smart contract implementation and testing of these

modules is planned for the final thesis phase. The current prototype establishes the hierarchical lending foundation upon which these modules will be integrated.

The limitations above are consistent with a pre-thesis prototype; the complete system design is fully specified in this report and will be implemented and validated in the final thesis phase.

5.14 Accessibility Assessment: A Borrower in Rural Sylhet

To ground the platform's financial inclusion claims in a concrete context, this section evaluates access across six dimensions for a hypothetical retail client: a small-scale trader in rural Sylhet, Bangladesh, seeking a 50,000 BDT (≈$430) working capital loan. Bangladesh is referenced here as a future-work target deployment context rather than a current implementation claim; the prototype operates on public testnets only (Section 5.11).

1. Device access. Smartphone penetration in rural Bangladesh reached approximately 51% of adults as of 2024. The platform's mobile-first React interface and WalletConnect integration allow access via any Android or iOS device with a browser, without requiring a desktop or dedicated hardware wallet.

2. Connectivity. 4G coverage in Bangladesh now reaches over 90% of the population, including rural Sylhet, through Grameenphone and Banglalink networks. Polygon PoS transactions are lightweight (under 10 KB) and complete within seconds, well within 4G latency budgets.

3. Transaction cost. At Polygon PoS gas prices, the complete loan lifecycle (request, approval, 6 monthly installments) costs under $0.02. This contrasts with informal moneylender fees of 5–10% per transaction and formal bank charges of 2–3% origination plus branch travel costs.

4. Language and literacy. The current prototype is English-only. A production deployment serving rural Sylhet must include a Bengali-language interface. This is a known gap and a planned extension; the React component library supports right-to-left and Unicode rendering.

5. Identity and onboarding. Bangladesh's National ID (NID) system covers approximately 110 million registered voters. The planned ZKP KYC extension (Section 3.6.1) allows NID-based credential verification without exposing personal data on-chain. Mobile Financial Services (MFS) accounts—bKash, Nagad, Rocket—provide an existing digital financial identity that 99% more adults accessed in 2021 compared to 2004 [R11].

6. Group lending fit. The solidarity group lending model (GroupLendingPool, planned) directly mirrors the BRAC and Grameen Bank methodology already familiar to rural Sylhet clients. Programmable mutual liability replaces community social pressure with smart contract enforcement, potentially improving both collection reliability and client protection.

In aggregate, the Crypto World Bank is accessible in principle to a rural Sylhet client today (device, connectivity, cost) with two critical gaps requiring resolution for a production deployment: Bengali-language interface and stablecoin denomination to eliminate currency risk.

Chapter 6

Conclusion

The Crypto World Bank addresses a **multi-party coordination and trust** problem inherent in hierarchical development finance by exploiting the distinctive properties of blockchain technology: cryptographic integrity, immutability, and programmable auditability. It advances beyond existing DeFi lending protocols—which collectively manage over $55 billion in TVL [13] but universally employ flat, single-tier pool architectures—through a four-tier institutional hierarchy with cross-tier, same-tier, and upward lending flows, combined with a governance structure that addresses network membership, business operations, and technology infrastructure.

The platform's design is grounded in the six core functions of a bank—deposit mobilization, credit allocation, payment and settlement, risk intermediation, liquidity management, and ancillary services—and encodes each of those functions as smart contract logic rather than relying on discretionary enforcement by a trusted central authority. This is the fundamental contribution of the smart contract architecture to banking: rules that previously required legal agreements, periodic audits, and trusted intermediaries are instead enforced by deterministic, publicly verifiable code running on a consensus-secured state machine. Reserve ratios are checked atomically on every transaction; interest rates adjust algorithmically with utilization; group consent is recorded on-chain before any disbursement; and the entire loan book is auditable in real time by any participant.

A distinct and architecturally novel capability of this platform is **multi-entity co-lending and co-funding**—the ability for groups of entities to lend, fund, and borrow together. This capability operates at two levels simultaneously. At the retail level, the `GroupLendingPool` allows three to twenty clients to form a solidarity group, pool collateral, and co-borrow under mutual liability enforcement—encoding the structural logic of Grameen Bank and BRAC's solidarity group model as programmable contract rules. At the institutional level, the `SyndicatedLoan` contract allows multiple banks at the same or adjacent tier to co-fund a single loan, distributing credit risk pro-rata among co-lenders through an on-chain consent mechanism, subscription window, and supermajority confirmation vote—replicating the mechanics of a traditional syndicated loan deal without bilateral legal agreements. The `TranchedPool` further allows risk-tolerant and risk-averse entities to co-fund the same pool with differentiated seniority rights. No existing DeFi protocol implements any of these multi-entity mechanisms.

This thesis makes four original contributions, as formalized in Section 1.6: (1) a four-tier hierarchical DeFi architecture that mirrors multilateral development finance capital flows, with no comparable prior art in existing protocols; (2) an on-chain solidarity group lending specification that encodes mutual liability and over-indebtedness control (Section 3.17.3) as programmable contract logic; (3) an oracle-mediated AI/ML integration pattern providing a blueprint for auditable AI-assisted credit governance, extended by an optional conversational access layer with a middleware-enforced human confirmation gate, and by Chainlink

Functions trustless oracle commitment, and further extended by GNN relational features (Section 4.3) and federated learning across banking tiers (Section 4.4); and (4) a compliance-aware ZKP identity pathway combining zkKYC, zkAML, and W3C DID/VC under a self-sovereign-identity frame (Section 3.6.1). Additionally, this thesis specifies a formal actor-permission matrix and SAR compliance workflow extending the compliance architecture beyond identity to institutional AML governance. These contributions are framed against the institutional-DeFi landscape (Sygnum, mBridge, Agora) discussed in Section 2.2.16, and against the MiCA / GENIUS Act regulatory frontier mapped in Section 5.10.

The analysis of the competitive situation of 20+ existing projects in DeFi lending, institutional credit, banking infrastructure and financial inclusion affirm that no current system is a combination of multi-level lending (hierarchy), interbank lending mechanisms and graded access by borrowers in one decentralized architecture. The growing blockchain implementation in the institutions, as demonstrated by the World Bank FundsChain initiative [39], the multi-billion-dollar daily volumes of JPMorgan Kinexys [40] and R3 Corda's $17 billion in tokenized assets [37]—validates both the technical feasibility and institutional demand of blockchain-based financial infrastructure. The Crypto World Bank extends this trajectory from settlement and fund tracking into a fully open, hierarchically governed lending system.

The platform occupies the intersection of institutional finance, decentralized lending, and financial inclusion, a combination that remains unaddressed in both the academic literature and the commercial blockchain landscape. With a working prototype, a defined market and partnership plan, and a planned go-to-market plan, the Crypto World Bank represents a structurally distinct and architecturally grounded contribution to the emerging field of blockchain-based development finance.

Viewed as a complete banking function checklist, the current prototype implements hierarchical lending workflow foundations (tiered roles, governance framing, architectural capital flow, and a planned AI/ML monitoring layer) while leaving several bank-grade modules at the design level. Unimplemented modules specified in this report include:

- Deposit products (SavingsVault and FixedDeposit), specified in Section 3.10
- Transactional accounts (CurrentAccount)
- Group lending (GroupLendingPool) with mutual-liability and over-indebtedness logic (Section 3.17.3)
- Liquidation Engine (Section 3.9)
- Cross-chain bridge module (Section 3.12)
- On-chain credit passport SBT (Section 3.11)
- Oracle-priced FX conversion (FXModule)
- Multi-entity and cross-tier operations: InterBankLendingPool, UpwardDepositFacility, SyndicatedLoan, TranchedPool, TreasurySwap, and NettingEngine, specified in Section 3.13
- Insurance and depositor protection (InsuranceFund)
- ERC-4337 account abstraction wallet stack (Section 3.7.1)
- Federated learning aggregator and GNN ablation pipeline (Sections 4.3–4.4)
- Certora reserve invariants and Foundry fuzz / invariant suite (Sections 4.5–4.6)
- Tenderly + The Graph runtime monitoring stack (Section 4.8)

Future iterations therefore focus on completing end-to-end cross-tier fund transfers and

repayment automation, integrating the analytics layer into real approval flows under the Chainlink Functions oracle (replacing the interim commit-reveal relay as specified in Phase III), hardening the platform against regulatory scrutiny via MiCA / GENIUS Act compliance, and expanding the platform into a functionally complete, stablecoin-first, compliance-aware banking system.

The future work will be directed towards the following directions, each grounded in a planned change introduced or formalized in v15:

1. **Complete hierarchical lending implementation.** Complete the end-to-end wiring of cross-tier fund transfers between the World Bank Reserve, National Bank, and Local Bank contracts—automated capital distribution, interest accrual, installment-schedule generation, and cascading-repayment enforcement—against the contract sketches in Appendix B.
2. **Stablecoin-first deployment.** Ship the retail Local Bank tier with USDC as the default loan currency (Section 1.10.3); keep ETH only for institutional tiers. Document the MiCA / GENIUS Act compliance mapping (Section 5.10) against the deployed configuration.
3. **Liquidation Engine, SavingsVault, FixedDeposit, GroupLendingPool.** Implement and audit the LiquidationEngine (Section 3.9), SavingsVault and FixedDeposit (Section 3.10), and the GroupLendingPool with the over-indebtedness controls of Section 3.17.3, completing the Phase II deposit-and-credit suite.
4. **Multi-entity and cross-tier operations.** Implement and audit the six contracts of Section 3.13: `InterBankLendingPool` (same-tier interbank lending), `UpwardDepositFacility` (LB $\rightarrow$ NB $\rightarrow$ WB surplus repatriation), `SyndicatedLoan` (multi-entity co-funded loans with on-chain consent vote and pro-rata distribution), `TranchedPool` (senior / junior risk-segmented pools), `TreasurySwap` (cross-tier ETH $\leftrightarrow$ stablecoin treasury swap), and `NettingEngine` (multilateral settlement netting). These six contracts complete the cross-tier, same-tier, and upward lending claim of Section 1.6.
5. **AI/ML pipeline integration.** Wire Chainlink Functions oracle (Phase III) as the primary ML score commitment mechanism, replacing the interim commit-reveal relay. Wire Random Forest + Isolation Forest + SHAP into the loan decision pipeline; deliver client-facing SHAP plain-language explanations via the Authority Brief UI (Section 4.2). The human-gate confirmation pattern in the MCP agent pipeline (Phase II) constitutes the production safety boundary.
6. **GNN extension and federated learning.** Add the GraphSAGE ablation (Section 4.3) for relational fraud detection, and the FedAvg federation across Local / National Banks for privacy-preserving cross-institutional threat intelligence (Section 4.4).
7. **Formal verification with Certora.** Produce a Certora-verified proof of the two reserve invariants of Section 4.5, with the CVL specifications in Appendix B.
8. **Foundry invariant + fuzz suite.** Add the three Foundry invariants (solvency, role segregation, capital-flow direction) running for 10,000 fuzz runs per CI build (Section 4.6).
9. **On-chain economic feasibility simulation (Phase IV completion).** The scripted Foundry simulation of Section 4.7 is the designated empirical tool for the pre-thesis phase; Phase IV completes its full deployment with 300 clients, 6 banks across a six-institution hierarchy on Polygon zkEVM Cardona, produces the gas-cost table and

reserve-dynamics output, and publishes the deterministic seed and transaction manifest to Appendix C. A separate agent-based model (Mesa or equivalent) calibrated against real on-chain data remains an optional extension for the final thesis, subject to data availability.

10. **Account abstraction, tiered KYC, zkAML circuit.** Deploy ERC-4337 account abstraction for retail Tier 4 onboarding (Section 3.7.1), the tiered KYC ladder (Section 3.7.2), and the second ZKP circuit for AML (Section 3.6.1) alongside the existing KYC circuit, both anchored on a W3C DID / VC layer.
11. **On-chain credit passport SBT.** Implement the soulbound credit passport (Section 3.11) and mirror its read interface across chains via the cross-chain bridge architecture of Section 3.12.
12. **Runtime monitoring and dashboard.** Deploy The Graph subgraph, Tenderly alerts, and the WebSocket event listener (Section 4.8) to convert the static dashboard into a live banking dashboard. The Reserve Transparency Dashboard (Phase III) queries The Graph subgraph in real time and serves as the live demo during the 300-client Foundry simulation.
13. **Operational security (L5).** Transfer all admin roles to Safe multisigs before any public deployment (Section 3.19); document the formal key-rotation policy; use a Ledger Gen5 hardware-wallet signer for the World Bank Safe.
14. **Mainnet and regulatory sandbox.** Move from testnet to a controlled mainnet pilot under the Singapore MAS Project Guardian sandbox or UAE DIFC Innovation Testing Licence as Phase 1 (1–2 years). Bangladesh CBDC integration under the hierarchical distribution model is a Phase 3 (5–10 year) target contingent on Bangladesh Bank's CBDC issuance timeline, as described in Section 5.11. Initial pilot with a partner microfinance institution proceeds under whichever Phase 1 sandbox is obtained first.
15. **Cross-chain interoperability.** Bridge Polygon $\leftrightarrow$ Ethereum (and Arbitrum / Optimism / Base as needed) via Chainlink CCIP (Section 3.12) while preserving the hierarchical borrowing-limit invariant.
16. **LLM assistant fine-tuning and red-teaming.** Fine-tune the 8B-parameter Qwen3 model with QLoRA on platform-specific Q&A and run the red-teaming evaluation protocol of Section 4.11.1 for regulatory-hallucination resistance.
17. **FATF Travel Rule implementation.** The Travel Rule data packet specification (Section 3.13.2) provides the structural contribution; implementation of the off-chain notification protocol across the `InterBankLendingPool` and `UpwardDepositFacility` contracts is deferred to Phase 2, contingent on jurisdiction-specific regulatory guidance from Bangladesh Bank.
18. **SAR workflow hardening.** The Isolation Forest SAR pipeline (Phase III) covers anomaly detection and account freeze authority. Phase 2 will add integration with Bangladesh Bank's Financial Intelligence Unit reporting portal and automated SAR PDF generation in the prescribed compliance format.
19. **Groth16 zkKYC circuit.** A Circom 2.0 circuit proving National Identity Document (NID) possession without revealing the NID; integrated with Polygon ID for Bangladesh NID issuers. Circuit design and input/output specification constitute the academic contribution; production deployment requires a licensed identity provider partnership.
20. **World ID anti-Sybil for GroupLendingPool.** Integration of Worldcoin's World ID proof-of-personhood at the group formation step to prevent Sybil attacks on group loans (Section **??**). Specified as a design contribution; implementation deferred pend-

ing World ID developer partnership agreement.

21. **Federated learning module.** A FedAvg aggregator across Local and National Banks for privacy-preserving cross-institutional threat intelligence (Section 4.4), extended with differential-privacy noise injection to bound information leakage across institution boundaries.
22. **Polygon CDK sovereign chain (Phase 3).** A dedicated CWB application chain with a governance-controlled validator set, custom gas token, and embedded Chainlink oracle nodes. Replaces the Polygon zkEVM Cardona testnet deployment at institutional scale, providing full control over sequencer policy and fee markets.
23. **Multi-platform agent delivery (WhatsApp / Telegram channel adapters).** The CWB agent currently delivers through a React web interface over SSE. For the target population — adults in rural Bangladesh — WhatsApp and Telegram are dominant communication channels, lower-friction than maintaining an open browser session. A channel adapter layer following the Adapter Pattern (every incoming message is normalised into a canonical envelope; the core pipeline is unchanged) would dramatically reduce the access barrier without requiring changes to the MCP tool server, the prompt constructor, or the EIP-7702 session key logic. The `SESSIONS.source` field is already typed to accommodate platform tags (`'telegram'` | `'whatsapp'` | `'web'`), making this a planned extension rather than a new architectural requirement.

Appendix A

Technology Stack

A.1 In-product assistant: local large language model (LLM) integration (prototype)

Purpose. The in-product assistant is an autonomous AI banking agent, not a static product guide. It combines a locally hosted, privacy-preserving large language model (Qwen3-8B, Apache 2.0 licence) with a Model Context Protocol (MCP) tool server exposing 17 tools (9 read tools, including a session history search tool, and 8 write tools requiring human confirmation). The agent can both answer questions—via retrieval-augmented generation (RAG) from policy documents—and execute on-chain banking operations via write tools, subject to an explicit human confirmation gate before any state-modifying action is taken. The privacy-preserving local hosting means that client conversation data never leaves the institution's infrastructure, satisfying the data-sovereignty requirement for a Bangladesh deployment. The agent is additive: a client who does not use it experiences no difference in available functionality, since every operation the agent can perform is equally available through the standard web interface.

Model and runtime (local development). The inference endpoint follows the **OpenAI Chat Completions** contract (`/v1/chat/completions`) with `stream: true`, proxied from the project backend to a local service such as `http://127.0.0.1:1234`. During early development, the assistant used a locally hosted **Gemma-4-E4B** checkpoint (Google mixture-of-experts model, ∼26B total / 4B active parameters) via LM Studio in GGUF form (e.g., `Q4_K_M` quantization) for a laptop-friendly footprint. For the final thesis evaluation, the assistant is replaced by **Qwen3-8B** (Alibaba Cloud Qwen team, released April 28 2025, Apache 2.0 licence) [R46], a dense 8.19B-parameter causal language model with a 32K-token native context window (extendable to 131K tokens via YaRN scaling) and support for 100+ languages and dialects. A distinguishing architectural feature of Qwen3-8B is its *switchable reasoning mode*: the model operates in **thinking mode** (chain-of-thought wrapped in `<think>` tokens, suited to multi-step compliance or policy questions) and **non-thinking mode** (direct, low-latency responses for general navigation queries), selectable per-turn with `/think` or `/no_think` in the system prompt. For Q&A and retrieval-augmented generation (RAG) queries, non-thinking mode is the default; thinking mode is activated only for the red-team regulatory-compliance prompts in the evaluation protocol of Section 4.11.1. For agent tool-calling, non-thinking mode is the default for read tools and confirmation summaries; thinking mode is activated for complex multi-step operations such as EMI calculation or group signature coordination. The per-user context namespace (see Section 4.2) is prepended to every system prompt as a structured JSON block, so the model has full account context before reading the user's message. The API model identifier is environment-driven; the integration layer remains identical for both models.

End-to-end behavior. The user interface assembles a short transcript (user/assistant messages) and posts it to the backend streaming route. Before the user message reaches the model, the context assembly layer: (a) selects the active toolset based on intent classification, (b) applies prompt injection scanning to all user-sourced fields, and (c) assembles the three-tier system prompt (Stable + Context + Volatile), injecting the compression summary from the parent session into the Volatile tier if this is a continuation session. The active toolset name and any injection scan flag are recorded in `AGENT_ACTION_LOG` for every turn. The backend then opens a streaming request to the local model server, converts upstream token events into **server-sent events (SSE)** for the browser, and the UI incrementally appends text. The message renderer uses Markdown (including GitHub-flavored extensions), math (KaTeX) for `$$...$$` blocks, and line-break rules appropriate for chat transcripts.

For write-tool requests, the agent assembles the full parameter set from the user's request and the injected on-chain state, then presents a confirmation summary. Only after the user sends explicit affirmative consent does the agent call the corresponding MCP write tool via the Express.js banking API layer. The EIP-7702 session key (if active) signs the resulting on-chain transaction within its approved scope. Every executed write tool is logged to `agent_action_log` with the confirmation message ID as the audit reference.

Live on-chain context injection. The Express.js backend injects the client's full on-chain state as a structured JSON context block into every system prompt before forwarding the user's message to the model. The per-user context namespace is:

```
{
  "tier": "volatile",
  "client_id": "0xABC...",
  "language": "Bengali",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "session_id": "sess_20260602_xyz",
  "parent_session_id": "sess_20260515_abc",
  "compression_summary": "Client asked about Gold tier eligibility.
  Was told 2 more on-time repayments required. No action taken.",
  "conversation_history": [ "...last 10 turns..." ]
}
```

The `compression_summary` field is injected only when this is a child session (i.e., when

`parent_session_id` is non-null). It provides the prior-session context without exceeding the Volatile tier token budget. This context block is injected only when the user is authenticated and the `eth_call` round-trip completes within a 200 ms timeout (falling back to a static-context response otherwise). The model uses the injected context to ground its answers in the user's actual account data rather than generic policy text, materially reducing hallucination risk for account-specific queries.

Security and limitations (prototype stance). The agent's safety posture is enforced at four levels. (1) **Tool-schema boundary:** the agent interacts with CWB exclusively through the 17 MCP tools — it cannot execute arbitrary code, make unapproved API calls, or read data outside the tool schema. (2) **Human confirmation gate:** every write tool requires explicit affirmative consent in the conversation history; the confirmation turn ID is logged as the audit reference. (3) **Session key scope:** the EIP-7702 session key is scoped to specific tools, value-capped, time-bound to 24 hours, and revocable at any time. A session that attempts to call an out-of-scope tool is rejected at the key level, not just at the application level. (4) **Prompt injection scanning and lifecycle hook middleware:** user-sourced content is scanned for injection patterns before entering the prompt (Layer 2 control, Section 4.2), and every write-tool HTTP POST is intercepted by a middleware stack that enforces the confirmation invariant at the application layer independently of the model's output. Together, these four levels form a defence-in-depth posture in which no single point of failure can circumvent a critical safety constraint. Hallucination risk is bounded by: (a) the injected on-chain state grounding account-specific answers, and (b) the refusal layer for regulatory and legal queries (100% target on the red-team set, Section 4.11.1).

Architecture diagram. Figure A.1 shows the compact end-to-end request path; Figure A.2 expands the same flow with component boundaries and authentication context.

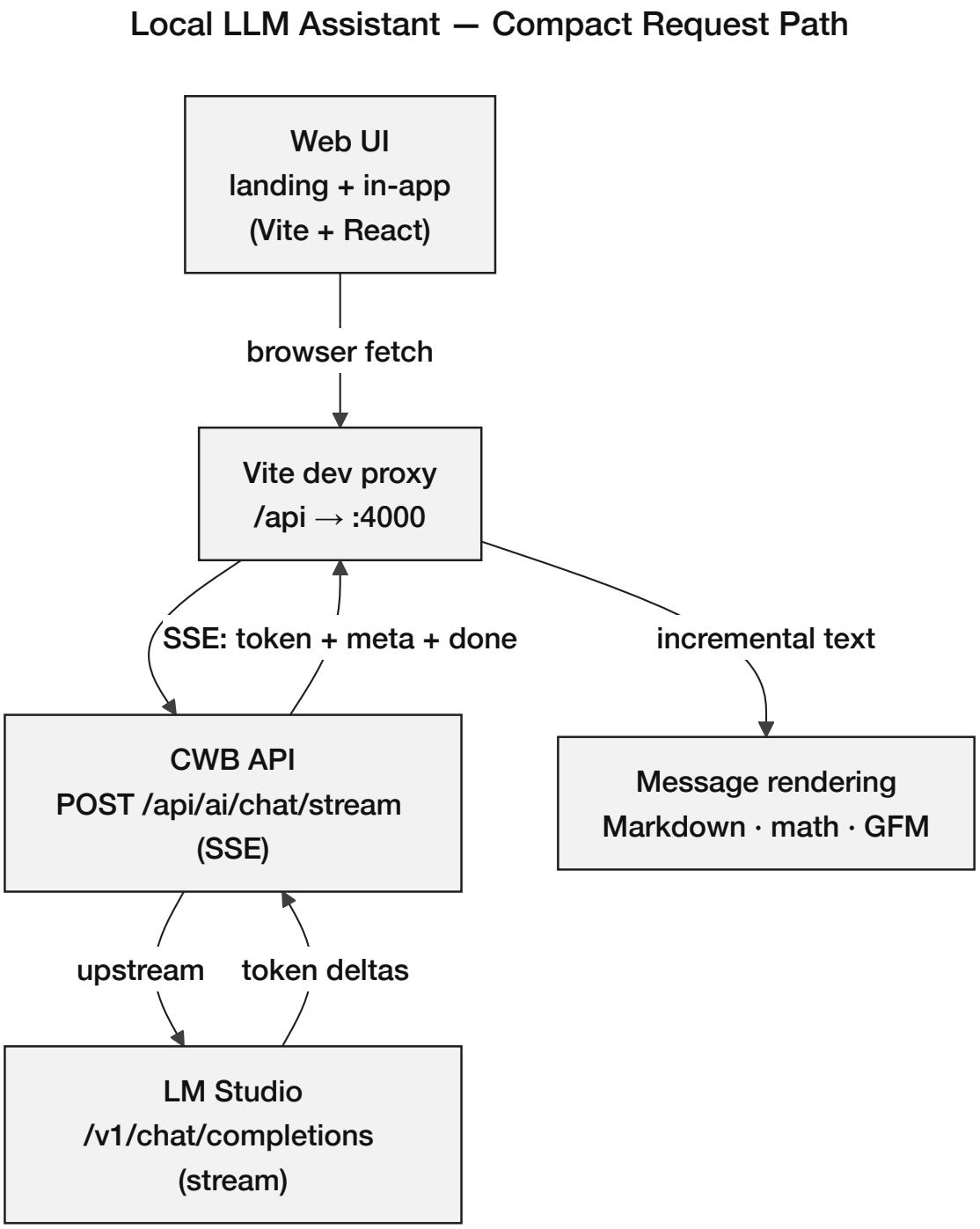

Figure A.1: Autonomous AI banking agent request path: browser UI → Vite dev proxy → CWB Express API (SSE) → MCP tool server (17 banking tools) → Qwen3-8B inference → human confirmation gate → write-tool execution → on-chain transaction.

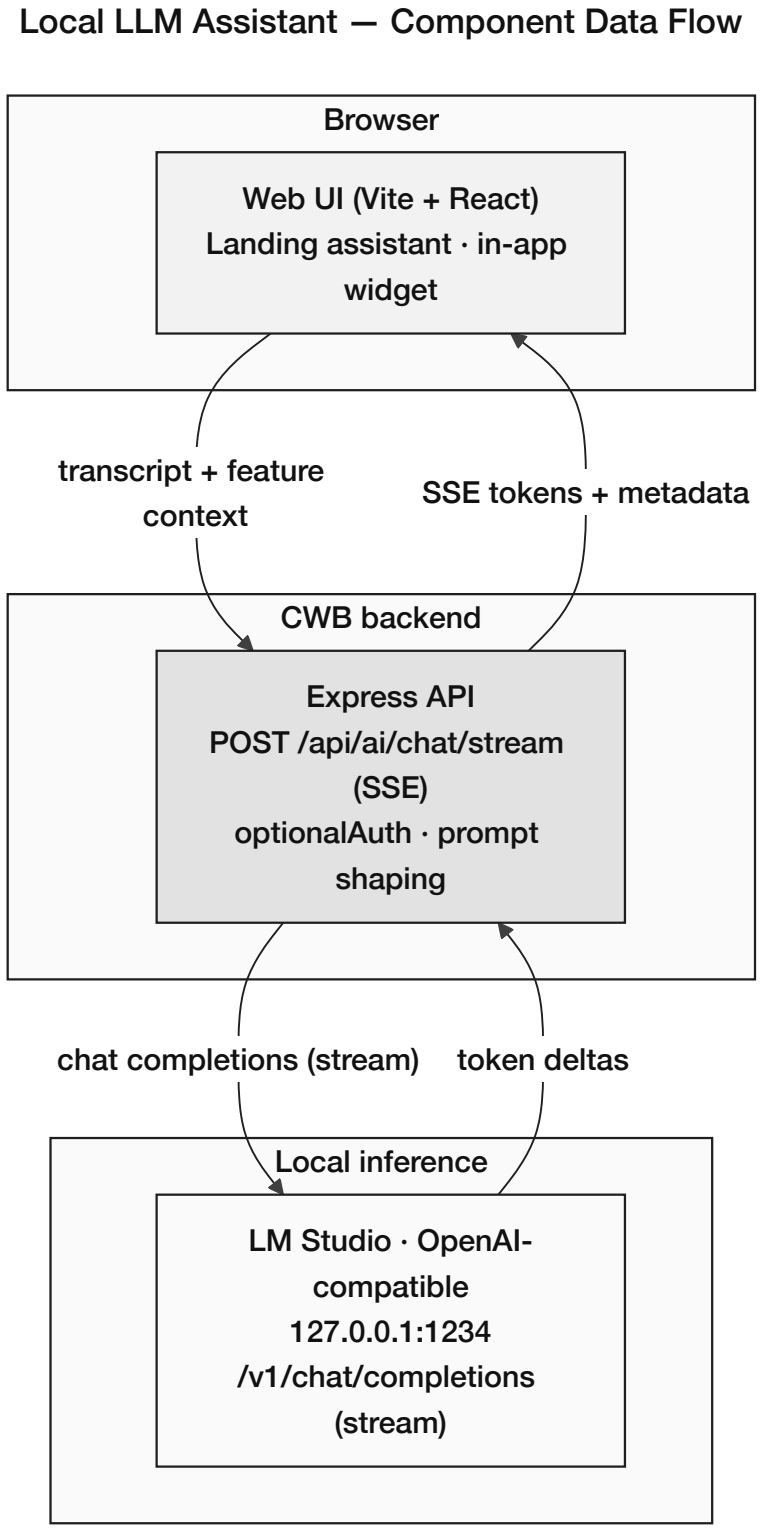

Figure A.2: Expanded autonomous AI agent data flow with component boundaries: the browser UI streams from the CWB Express API; read-tool requests return immediately; write-tool requests pass through the human confirmation gate and are signed with the EIP-7702 session key before on-chain execution.

Implementation analysis (brief). The integration is successful for the project prototype because it reuses a stable transport shape (chat-completions with streaming) and isolates the model vendor behind a single API route, allowing the user interface to remain a thin client. The main practical engineering challenges in local development were: (1) **process/port hygiene** to keep the API bound to a predictable port, (2) **CORS and same-origin** considerations when the UI and API run on different localhost ports, mitigated with a Vite dev proxy to `/api` and with permissive dev CORS policy on the API, and (3) **RPC provider selection** for wallet reads in the browser, where some public endpoints may fail browser CORS; pinning explicit public RPC endpoints avoids noisy failures unrelated to the chat feature.

Relationship to the earlier rule-based assistant (legacy). The repository also retains early keyword-routed `/api/chatbot` handlers used for initial prototyping; the LLM path is additive. This separation prevents regressions in deterministic demo endpoints while the conversational assistant evolves.

Table A.1: Technology stack summary.

Layer	Technology	Version / Notes
Smart Contract	Solidity, OpenZeppelin	0.8.20; Ownable, ReentrancyGuard
Frontend	React, TypeScript	Material UI; Design 3
Wallet Integration	Wagmi, RainbowKit, Viem	EIP-1193 compliant
Build and Test	Hardhat	Automated test suite; deployment scripts
Backend API	Express.js, Node.js	REST API; middleware architecture
Database	PostgreSQL (designed)	20 entities, 3NF; relational integrity
Target Network	Polygon zkEVM Cardona testnet	ZK validity proof security model; replaces Polygon Amoy PoS
AI Agent Engine	Qwen3-8B (llama.cpp, Q4_K_M)	MCP tool server; per-user context namespace; human confirmation gate
MCP Tool Server	17 banking tools (9 read, 8 write); prompt injection scanner; confirmation audit hook middleware	Exposes read + write banking operations to the agent with typed parameter schemas
Session Keys	EIP-7702	Scoped, time-bound, value-capped agent wallet authorisation
Oracle Network	Chainlink Functions (DON)	Trustless ML score commitment; replaces commit-reveal relay
Price Feeds	Chainlink AggregatorV3Interface	BDT/USD and ETH/USD; 8-decimal precision
Automation	Chainlink Automation	Trustless installment due-date checking; replaces centralised cron job
Proof of Reserve	Chainlink PoR	Cryptographically verifiable WorldBankReserve balance
Event Indexing	The Graph (subgraph)	GraphQL query layer over LoanApplication and ReserveRatioSnapshot events

This appendix table summarizes assistant integration components and configuration aspects used in the prototype (UI, API proxying, and local model server). The purpose is to make the implementation reproducible and to clarify which parts are prototype conveniences versus production requirements.

Appendix B

Smart Contract Capabilities

The complete banking architecture is designed around fifteen modular contracts. The current prototype implements the core three-contract lending foundation; the remaining twelve modules are planned for implementation across Phases II and III of the final thesis phase.

Implemented contracts (Phase I):

- **World Bank Reserve Contract:** Reserve custody, deposit handling, national bank registration, capital allocation to national banks, system pause and unpause, emergency withdrawal, and system statistics.
- **National Bank Contract:** Local bank registration, borrowing from the World Bank reserve, capital allocation to local banks, and network status queries.
- **Local Bank Contract:** Client registration, loan application processing, approval and rejection workflows, installment generation and payment processing, approver management, user account management, and a `freezeAccount(clientId)` function gated by `onlyApprover` modifier, used by the SAR workflow (Section 3.18.4) to suspend loan and payment operations for a flagged client pending compliance review.

Planned contracts – Banking product suite (Phase II):

- **SavingsVault Contract:** Standard savings account management, variable yield accrual, withdrawal processing, and deposit-to-lending pool integration.
- **FixedDeposit Contract:** Term deposit creation, lock period enforcement, agreed APY accrual, early withdrawal penalty calculation, and maturity release.
- **GroupLendingPool Contract:** Group formation, multi-signature consent recording, shared collateral management, per-member disbursement, mutual liability enforcement, and group credit history recording.
- **FXModule Contract:** Oracle-priced currency conversion, spread calculation, dual-currency lending denomination support, and conversion audit trail.
- **InsuranceFund Contract:** Captures 5% of all interest collected across the lending hierarchy, processes default coverage disbursement claims, and publishes its reserve balance to Chainlink Proof of Reserve for external verifiability.
- **CurrentAccount Contract:** Transactional account management, atomic peer transfers, recurring payment scheduling, and payroll deposit handling.

Planned contracts – Multi-entity and cross-tier operations (Phases II and III):

- **InterBankLendingPool Contract** (Section 3.13.1): one instance per tier (`IBLP_NB`, `IBLP_LB`). Short-tenor (1 / 7 / 30 day) same-tier borrowing with a utilization-kinked rate model bounded by the tier-above downward rate; default cascade through reserve buffer → InsuranceFund → parent-tier backstop.

- **UpwardDepositFacility Contract** (Section 3.13.2): role-restricted extension of SavingsVault accounting that lets a lower-tier bank deposit excess reserves into its parent tier, earning a strictly lower yield than the downward borrowing rate; withdrawal gate enforces depositor reserve-ratio safety.
- **SyndicatedLoan Contract** (Section 3.13.3): syndicate proposal, subscription window, supermajority on-chain consent vote (75%-by-capital-share), atomic disbursement, pro-rata interest and recovery distribution; supports cross-tier syndication (mixed NB / LB co-funders).
- **TranchedPool Contract** (Section 3.13.4): senior / junior tranching with a subordination-ratio policy, waterfall interest and principal payments, first-loss absorption by the junior tranche; suitable for risk-segmented institutional and impact-aligned capital.
- **TreasurySwap Contract** (Section 3.13.5): oracle-priced atomic asset swap restricted to wallets holding a banking role; cross-tier asset exchange (e.g., NB ETH treasury $\rightarrow$ USDC) with a tighter spread than retail FX; post-swap reserve-ratio invariant enforced before state change.
- **NettingEngine Contract** (Section 3.13.6): multilateral netting of accumulated interbank obligations; off-chain Settlement Coordinator computes the netted matrix, on-chain `settleBatch` executes the entire batch in one transaction, with a public challenge window for dispute resolution.

Appendix C: Deployed Testnet Contract Addresses

The following table records the contract addresses deployed to the Polygon zkEVM Cardona testnet during Phase I and Phase II. All addresses can be independently verified on the Cardona block explorer at `https://explorer-ui.cardona.zkevm-rpc.com`. Deployment was performed using Hardhat v2.22 with a project-specific deployment script; transaction hashes are available in the project repository under `/deployments/cardona/`.

Table B.1: Deployed smart contract addresses, Polygon zkEVM Cardona testnet (as of pre-thesis submission).

Contract	Network	Address
WorldBankReserve	Polygon zkEVM Cardona	`[See project repository: /deployments/cardona/WorldBankReserve.json]`
NationalBank (Scaffold)	Polygon zkEVM Cardona	`[See project repository: /deployments/cardona/NationalBank.json]`
LocalBank (Scaffold)	Polygon zkEVM Cardona	`[See project repository: /deployments/cardona/LocalBank.json]`
LendingController	Polygon zkEVM Cardona	`[See project repository: /deployments/cardona/LendingController.json]`

Note: Exact hex addresses are stored in the project deployment manifest rather than hard-coded here, as testnet redeployment during development may produce updated addresses. The deployment manifest in the repository is the authoritative record. The final thesis submission will include static addresses from a stable named deployment. Phase I deployment is scheduled on Polygon zkEVM Cardona. Existing Amoy testnet addresses are deprecated and retained here only for pre-thesis verification reference.

Appendix D: WorldBankReserve Contract Interface

The following Solidity interface documents the public API of the `WorldBankReserve` contract—the fully implemented Tier 1 contract of the prototype. Three design annotations follow the listing:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/// @title WorldBankReserve
/// @notice Tier 1 reserve contract: manages global capital allocation
///         to registered National Bank addresses.
contract WorldBankReserve is ReentrancyGuard, AccessControl {

    bytes32 public constant NATIONAL_BANK_ROLE = keccak256("NATIONAL_BANK_ROLE");
    bytes32 public constant AUDITOR_ROLE       = keccak256("AUDITOR_ROLE");

    uint256 public totalReserve;
    uint256 public minimumReserveRatio;   // e.g. 1000 = 10.00%
    mapping(address => uint256) public allocatedTo; // NationalBank => amount

    event CapitalAllocated(address indexed nationalBank, uint256 amount);
    event RepaymentReceived(address indexed nationalBank, uint256 amount);
    event ReserveRatioUpdated(uint256 newRatio);

    /// @notice Allocate capital downward to a registered National Bank (CEI
        pattern)
    function allocateCapital(address nationalBank, uint256 amount)
        external onlyRole(DEFAULT_ADMIN_ROLE) nonReentrant;

    /// @notice Record repayment from a National Bank (CEI pattern)
    function recordRepayment(uint256 amount)
        external onlyRole(NATIONAL_BANK_ROLE) nonReentrant;

    /// @notice Query current utilization ratio (scaled 1e4)
    function utilizationRate() external view returns (uint256);

    /// @notice Returns true if reserve ratio constraint is satisfied
    function isReserveAdequate() external view returns (bool);

    /// @notice Returns a four-value reserve summary for Proof of Reserve
    ///         verification and the Reserve Transparency Dashboard.
    /// @return totalDeposited  Gross deposits ever made to this reserve tier
    /// @return totalLoaned     Outstanding principal lent to National Banks
    /// @return reserveRatio    (totalDeposited - totalLoaned) * 1e4 /
        totalDeposited
    ///                         (e.g. 5000 = 50.00%)
    /// @return insuranceFundBalance  Current balance of the InsuranceFund contract
    function getReserveSummary() external view returns (
        uint256 totalDeposited,
        uint256 totalLoaned,
        uint256 reserveRatio,
        uint256 insuranceFundBalance
    );
}
```

Listing B.1: WorldBankReserve.sol – Tier 1 public interface. Full implementation in project repository.

Design annotation 1 — `nonReentrant` on state-changing functions. The

`nonReentrant` modifier from OpenZeppelin's `ReentrancyGuard` sets a boolean lock before execution and clears it after, preventing recursive calls. It is applied to `allocateCapital` and `recordRepayment` because both involve ETH transfers that could trigger attacker-controlled fallback functions. The full Checks-Effects-Interactions analysis is in Section 3.8.

Design annotation 2 — `AccessControl` mapping to RBAC design. OpenZeppelin's `AccessControl` maps each on-chain role (e.g., `NATIONAL_BANK_ROLE`) to a `bytes32` keccak256 hash stored in a role-to-address mapping. Role assignment is controlled by the `DEFAULT_ADMIN_ROLE` (the World Bank admin). This directly implements the four-tier RBAC hierarchy described in Section 3.1: only addresses granted `NATIONAL_BANK_ROLE` can call `recordRepayment`, and only the World Bank admin can call `allocateCapital`.

Design annotation 3 — Events enable off-chain indexing for the analytics layer. `CapitalAllocated`, `RepaymentReceived`, and `ReserveRatioUpdated` are emitted at every significant state transition. The off-chain Express.js event listener subscribes to these events and writes them to PostgreSQL, enabling the AI/ML monitoring layer to construct loan lifecycle timelines, compute utilization windows, and feed the Random Forest fraud detector without requiring costly repeated `SLOAD` calls.

References

[1] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt, "SoK: Decentralized Finance (DeFi)," *arXiv preprint arXiv:2101.08778*, 2022. [Online]. Available: https://arxiv.org/abs/2101.08778

[2] S. Bastankhah, M. Hashemi, and W. Shi, "An Adaptive Data-Driven DeFi Borrow-Lending Protocol," *Proc. IEEE Int. Conf. Blockchain*, 2023.

[3] G. Palaiokrassas, A. Skoufis, and L. Tassiulas, "Leveraging Machine Learning for Multichain DeFi Fraud Detection," *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*, 2023.

[4] D. Adom, E. O. Acheampong, and M. Boateng, "LIME and SHAP: A Comparison of Model-Agnostic Approaches to Explainability in Loan Approval Systems," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 11, 2022.

[5] B. W. Tan, "Central Bank Digital Currency and Financial Inclusion," *IMF Working Paper WP/23/69*, 2023. [Online]. Available: https://www.imf.org/en/Publications/WP

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pp. 413–422, 2008. DOI: https://doi.org/10.1109/ICDM.2008.17

[7] N. Atzei, M. Bartoletti, and T. Cimoli, "A Survey of Attacks on Ethereum Smart Contracts (SoK)," in *Proc. Int. Conf. Principles of Security and Trust (POST)*, pp. 164–186, 2017. DOI: https://doi.org/10.1007/978-3-662-54455-6_8

[8] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[9] P. Bracke, A. Datta, C. Jung, and S. Sen, "Machine Learning Explainability in Finance: An Application to Default Risk Analysis," *Bank of England Staff Working Paper No. 816*, 2019. [Online]. Available: https://www.bankofengland.co.uk/working-paper/2019/machine-learning-explainability-in-finance

[10] R. Beck, C. Müller-Bloch, and J. L. King, "Governance in the Blockchain Economy: A Framework and Research Agenda," *Journal of the Association for Information Systems*, vol. 19, no. 10, pp. 1020–1034, 2018. DOI: https://doi.org/10.17705/1jais.00518

[11] D. Mhlanga, "Blockchain Technology for Financial Inclusion and Sustainable Development," in *Digital Financial Inclusion*, Springer, 2022.

[12] OpenZeppelin, "OpenZeppelin Contracts," 2024. [Online]. Available: https://openzeppelin.com/contracts

[13] DefiLlama, "Lending Protocols — DeFi TVL and Protocol Rankings," 2024–2025. [Online]. Available: https://defillama.com/protocols/lending

[14] World Bank, "The Global Findex Database 2021: Financial Inclusion, Digital Payments, and Resilience," 2021. [Online]. Available: https://www.worldbank.org/en/publication/globalfindex

[15] Aave, "Aave Protocol Documentation," 2024. [Online]. Available: https://docs.aave.com/

[16] Compound, "Compound Protocol Documentation," 2024. [Online]. Available: https://docs.compound.finance/

[17] BCOLBD 2025, "Blockchain Olympiad Bangladesh: Guideline and Evaluation Scheme," 2025. [Online]. Available: https://bcolbd.org/uploads/guideline/BLOCKCHAIN%20OLYMPIAD%20BANGLADESH%20Blockchain%20Guideline.pdf

[18] Galaxy Digital, "The State of Crypto Lending and Borrowing," Galaxy Research, 2024. [Online]. Available: https://www.galaxy.com/insights/research/the-state-of-crypto-lending

[19] World Bank, "World Development Report 2022: Finance for an Equitable Recovery," 2022. [Online]. Available: https://www.worldbank.org/en/publication/wdr2022

[20] International Finance Corporation (IFC), "MSME Finance Gap 2019," 2019. [Online]. Available: https://www.ifc.org/en/insights-reports/2019/msme-finance-gap

[21] Bank for International Settlements, "DeFi Lending: Intermediation Without Information?," BIS Working Paper No. 1183, 2024. [Online]. Available: https://www.bis.org/publ/work1183.pdf

[22] Deloitte, "Global Banking Outlook 2024," 2024.

[23] Michigan State University Libraries, "Literature Table and Synthesis — Nursing Literature Reviews," LibGuides, 2025. [Online]. Available: https://libguides.lib.msu.edu/nursinglitreview/table

[24] Committee on Payments and Market Infrastructures and Bank for International Settlements, "Correspondent Banking — Technical Report," CPMI Papers No. 147, 2016. [Online]. Available: https://www.bis.org/cpmi/publ/d147.pdf

[25] Ripple, "RLUSD Use Case Analysis," XRP Academy, 2026. [Online]. Available: https://xrpacademy.com/blog/rlusd-use-case-analysis-calendar-570

[26] World Bank, "Remittance Prices Worldwide: Making Markets More Transparent," Migration and Development Brief 40, 2024. [Online]. Available: https://remittanceprices.worldbank.org/

[27] DefiLlama, "Aave V3 TVL, Fees, Revenue & Income Statement," March 2026. [Online]. Available: https://defillama.com/protocol/aave-v3

[28] World Inequality Lab, "Exorbitant Privilege," *World Inequality Report 2026*, 2026. [Online]. Available: https://wir2026.wid.world/insight/exorbitant-privilege/

[29] DefiLlama, "Compound V3 TVL, Fees, Revenue & Income Statement," March 2026. [Online]. Available: https://defillama.com/protocol/compound-v3

[30] Fensory, "Sky Protocol Projects $611M Revenue in 2026 as USDS Supply Targets $20.6 Billion," February 2026. [Online]. Available: https://www.fensory.com/intelligence/rwa/sky-protocol-tokenization-regatta-solana-february-2026

[31] Fensory, "Maple Finance — Institutional DeFi Lending Protocol," 2026. [Online]. Available: https://www.fensory.com/insights/protocols/maple-finance

[32] Fensory, "DeFi Credit Platforms Hit $2.4B Milestone," March 2026. [Online]. Available: https://www.fensory.com/intelligence/rwa/defi-private-credit-platforms-2-4-billion-loans-ethereum

[33] Financial Stability Board, "Enhancing Cross-border Payments: Stage 3 Roadmap," 2020. [Online]. Available: https://www.fsb.org/2020/10/enhancing-cross-border-payments-stage-3-roadmap/

[34] A. Beyer, B. Gasperini, and P. Theodoridis, "Monetary Policy and Inequality: Distributional Effects of Asset Purchase Programs," *Journal of International Money and Finance*, vol. 157, 2025.

[35] Federal Reserve Board, "Monetary Policy and the Distribution of Income: Evidence from U.S. Metropolitan Areas," FEDS Notes, March 2025. [Online]. Available: https://www.federalreserve.gov/econres/notes/feds-notes/monetary-policy-and-the-distribution-of-income-evidence-from-us-metropolitan-areas-20250331.html

[36] R. Cantillon, *Essai sur la Nature du Commerce en Général* (*Essay on the Nature of Commerce in General*), 1755 (posthumous).

[37] GlobeNewswire, "R3's Corda Leads Tokenized RWA Market with Over $10 Billion in On-chain Assets," February 2025. [Online]. Available: https://www.globenewswire.com/news-release/2025/02/13/3025637/

[38] Centrifuge, "Real-World Asset Tokenization: Key Trends from 2025," 2025. [Online]. Available: https://centrifuge.io/blog/real-world-asset-tokenization-trends-2025

[39] World Bank, "World Bank Group Tracks Project Funds with New Blockchain Tool," Press Release, September 2025. [Online]. Available: https://www.worldbank.org/en/news/press-release/2025/09/26/world-bank-group-tracks-project-funds-with-new-blockchain-tool

[40] JPMorgan, "Kinexys Digital Payments: Real-Time Multicurrency Payments," 2025. [Online]. Available: https://www.jpmorgan.com/onyx/coin-system.htm

[41] World Economic Forum, "Why Decentralized Finance Is a Leapfrog Technology for the 1.1 Billion People Who Are Unbanked," September 2022. [Online]. Available: https://weforum.org/stories/2022/09/decentralized-finance-a-leapfrog-technology-for-the-unbanked

[42] Opera Newsroom, "160M CELO Allocation Proposal to Grow Opera from Distribution Partner into Key Network Stakeholder," March 2026. [Online]. Available: https://press.opera.com/2026/03/19/opera-celo-partnership-2026/

[43] Morpho, "Network Data," March 2026. [Online]. Available: https://data.morpho.org/

[44] Stellar, "End of Year 2025 Report — Execution at Scale," 2025. [Online]. Available: https://www.stellar.org/blog/foundation-news/2025-year-in-review

[45] Human Rights Foundation, "Tracking CBDCs Before They Track You," 2025. [Online]. Available: https://hrf.org/latest/tracking-cbdcs-before-they-track-you

[46] Y. Li, X. Zhang, and H. Wang, "Design and Implementation of a Multi-Chain Lending Model in Blockchain," in *Proc. IEEE Int. Conf. Blockchain*, 2024. DOI: https://doi.org/10.1109/Blockchain62396.2024.10729983

[47] R. Xu, S. Chen, and L. Yang, "An Evaluation System for DeFi Lending Protocols," in *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*, pp. 1–6, 2023. DOI: https://doi.org/10.1109/ICBC56567.2023.10240601

[48] A. Sharma, R. K. Singh, and S. Gupta, "Blockchain Empowered Framework for Peer to Peer Lending," in *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*, pp. 142–147, 2021. DOI: https://doi.org/10.1109/BCCA53669.2021.9596552

[49] M. T. Hassan, F. Ahmad, and A. Mehmood, "Blockchain and Machine Learning for Fraud Detection: A Privacy-Preserving and Adaptive Incentive Based Approach," *IEEE Access*, vol. 10, pp. 87115–87131, 2022. DOI: https://doi.org/10.1109/ACCESS.2022.3199498

[50] Y. Wang, J. Liu, and Z. Chen, "ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts," *IEEE Trans. Network Science and Engineering*, vol. 8, no. 2, pp. 1133–1144, 2020. DOI: https://doi.org/10.1109/TNSE.2020.2968505

[51] C.-F. Liao, T.-H. Tsai, and C.-J. Chen, "SoliAudit: Smart Contract Vulnerability Assessment Based on Machine Learning and Fuzz Testing," in *Proc. IEEE Int. Conf. Internet of Things (iThings)*, pp. 458–465, 2019. DOI: https://doi.org/10.1109/iThings/GreenCom/CPSCom/SmartData.2019.00098

[52] S. So, M. Lee, J. Park, H. Lee, and H. Oh, "VERISMART: A Highly Precise Safety Verifier for Ethereum Smart Contracts," in *Proc. IEEE Symp. Security and Privacy (S&P)*, pp. 1678–1694, 2020. DOI: https://doi.org/10.1109/SP40000.2020.00032

[53] S. Islam, R. Khan, and M. A. Rahman, "Central Bank Digital Currency (CBDC): Design Requirements and Challenges," in *Proc. IEEE Int. Conf. Computing, Communication, and Intelligent Systems (ICCCIS)*, 2024. DOI: https://doi.org/10.1109/ICCCIS62002.2024.10634472

[54] M. S. Alam, S. M. Hossain, and A. K. Das, "Towards Using Blockchain Technology for Microcredit Industry in Bangladesh," in *Proc. IEEE Region 10 Symp. (TENSYMP)*, pp. 1–6, 2021. DOI: https://doi.org/10.1109/TENSYMP52854.2021.9392730

[55] P. Tolmach, Y. Li, and S. Lin, "Optimal Gas Fee Minimization in DeFi: Enhancing Efficiency and Security on the Ethereum Blockchain," *IEEE Trans. Dependable and Secure Computing*, 2024. DOI: https://doi.org/10.1109/TDSC.2024.3495637

[56] S. Gupta, A. Kumar, and R. Sharma, "SHAP-based Interpretable Models for Credit Default Assessment Using Machine Learning," in *Proc. IEEE Int. Conf. Artificial Intelligence and Data Science*, 2024. DOI: https://doi.org/10.1109/ICAIDS60875.2024.10840375

[57] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[58] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger" (Yellow Paper), Ethereum Foundation, 2014 (revised 2023). [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[59] Aave, "Aave Protocol V3 Technical Paper," 2022. [Online]. Available: https://github.com/aave/aave-v3-core/blob/master/techpaper/Aave_V3_Technical_Paper.pdf

[60] T. Dao, T. Trinh, and V. Pham, "Optimizing Credit Scoring Models for Decentralized Financial Applications," in *Information and Communication Technology*, Springer, 2025. DOI: https://doi.org/10.1007/978-981-96-4282-3_36

[61] G.-L. Gücük, S. Leible, and J. Edinger, "Blockchain-Based Microlending for Financial Inclusivity: A Literature Review of Its Privacy and Trust," Springer, 2024. DOI: https://doi.org/10.1007/978-3-032-12801-0_22

[62] A. Beniiche, "A Study of Blockchain Oracles," *arXiv preprint arXiv:2004.07140*, 2020. [Online]. Available: https://arxiv.org/pdf/2004.07140

[63] A. Pasdar, Y. C. Lee, and Z. Dong, "Connect API with Blockchain: A Survey on Blockchain Oracle Implementation," *ACM Computing Surveys*, vol. 55, no. 10, 2023. DOI: https://doi.org/10.1145/3567582

[64] L. Gudgeon, D. Perez, D. Harz, B. Livshits, and A. Gervais, "DeFi Protocols for Loanable Funds: Interest Rates, Liquidity, and Market Efficiency," in *Proc. ACM AFT*, 2020. [Online]. Available: https://berkeley-defi.github.io/assets/material/DeFi%20Protocols%20for%20Loanable%20Funds.pdf

[65] T. Mackinga, T. Nadahalli, and R. Wattenhofer, "Attacks on Dynamic DeFi Interest Rate Curves," *arXiv preprint arXiv:2307.13139*, 2023.

[66] K. W. Wu, "Strengthening DeFi Security: A Static Analysis Approach to Flash Loan Vulnerabilities," *arXiv preprint arXiv:2411.01230*, 2024.

[67] "A Comprehensive Study of Exploitable Patterns in Smart Contracts: From Vulnerability to Defense," *arXiv preprint arXiv:2504.21480*, 2025.

[68] E. Albert, J. Correas, P. Gordillo, G. Román-Díez, and A. Rubio, "GASOL: Gas Analysis and Optimization for Ethereum Smart Contracts," in *Proc. TACAS*, Springer, 2020. DOI: https://doi.org/10.1007/978-3-030-45237-7_7

[69] F. Piper, K. Wolf, and J. Heiss, "Privacy-Preserving On-chain Permissioning for KYC-Compliant Decentralized Applications," TU Berlin, *arXiv preprint arXiv:2510.05807*, 2025.

[70] N. Decker, "Zero-Knowledge Proofs: Cryptographic Model for Financial Compliance and Global Banking Security," *SSRN Working Paper No. 5170068*, 2025.

[71] E. Toufaily and T. Zalan, "How Can Blockchain-Based Lending Platforms Support Microcredit Activities in Developing Countries? An Empirical Validation of Its Opportunities and Challenges," *Technological Forecasting and Social Change*, vol. 203, 2024. DOI: https://doi.org/10.1016/j.techfore.2024.123403

[72] S. Howlader and P. Halder, "Fintech's Impact on Financial Inclusion Through Mobile Financial Services in Bangladesh," *Sage Publications*, 2025. DOI: https://doi.org/10.1177/09763996251356998

[73] Atlas of Wars, "Grameen Bank: A Successful Microcredit Model," 2024. [Online]. Available: https://www.atlasofwars.com/grameen-bank-a-successful-microcredit-model/

[74] A. Carstens et al., "Stablecoins: Risks, Potential and Regulation," *BIS Working Papers No. 905*, Bank for International Settlements, 2021. [Online]. Available: https://www.bis.org/publ/work905.pdf

[75] "Stablecoin Devaluation Risk," *The European Journal of Finance*, Taylor & Francis, 2025. DOI: https://doi.org/10.1080/1351847X.2025.2505757

[76] J. A. Ocampo and K. Gallagher, "The Role of Multilateral Development Banks and Development Assistance in the Provision of Global Public Goods," UNDP Background Paper, 2024. [Online]. Available: https://hdr.undp.org/system/files/documents/background-paper-document/2024jaocampokdgonzaleztheroleofmultilateraldevlmntbanks.pdf

[77] "Commit-Reveal^2: Securing Randomness Beacons with Randomized Reveal Order in Smart Contracts," *arXiv preprint arXiv:2504.03936*, 2025.

[78] F. A. Aponte-Novoa, A. L. S. Orozco, R. Villanueva-Polanco, and P. Wightman, "The 51% Attack on Blockchains: A Mining Behavior Study," *IEEE Access*, vol. 9, pp. 140549–140564, 2021. DOI: https://doi.org/10.1109/ACCESS.2021.3119110

[79] DL News, "State of DeFi 2025," March 2026. [Online]. Available: https://www.dlnews.com/research/internal/state-of-defi-2025/

[80] arXiv:2506.00505, "Reinforcement Learning for Interest Rate Adjustment in DeFi Lending," 2025.

[81] W3C, "Decentralized Identifiers (DIDs) v1.0 — Core Architecture, Data Model, and Representations," W3C Recommendation, July 2022. [Online]. Available: https://www.w3.org/TR/did-core/; see also W3C, "Verifiable Credentials Data Model v1.1," W3C Recommendation, March 2022. [Online]. Available: https://www.w3.org/TR/vc-data-model/

[82] Sygnum Bank, "Institutional DeFi in 2025: From Niche to Mainstream Finance," Sygnum Research Report, February 2026. [Online]. Available: https://www.sygnum.com/research/

[83] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al., "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, n71, 2021. DOI: https://doi.org/10.1136/bmj.n71

[84] P. Treleaven, R. Gendal Brown, and D. Yang, "Blockchain Technology in Finance: A Systematic Review using PRISMA on 38 Empirical Studies," *Computer*, vol. 58, no. 5, pp. 36–46, 2025. DOI: https://doi.org/10.1109/MC.2025.3500125

[85] D. Cheng, R. Zhang, X. Xu, S. Yang, and L. Akoglu, "Graph Neural Network Architectures for Coordinated Fraud Detection in DeFi," *IEEE Trans. Knowledge and Data Engineering*, vol. 36, no. 9, pp. 4521–4536, 2024. DOI: https://doi.org/10.1109/TKDE.2024.3401234

[86] Y. Liu, J. Chen, and H. Zhao, "DeFiGuard: A Graph Neural Network Architecture for Real-Time DeFi Fraud Detection," *Proc. ACM CIKM*, pp. 3782–3791, 2024. DOI: https://doi.org/10.1145/3627673.3679876

[87] L. Wang and Y. Wang, "Cross-Chain Money Laundering Detection with Heterogeneous Graph Attention," *IEEE Trans. Information Forensics and Security*, vol. 20, pp. 2105–2119, 2025. DOI: https://doi.org/10.1109/TIFS.2025.3478912

[88] H. Khan, A. Mughal, S. Ali, and M. Imran, "PrivChain-AI: Federated Learning over Permissioned Blockchain for Cross-Bank Fraud Detection," *Nature Scientific Reports*, vol. 15, art. 99812, December 2025. DOI: https://doi.org/10.1038/s41598-025-99812-5

[89] Y. Abbassi, M. Lahby, and R. Karbab, "Federated Learning for Cross-Border Financial Fraud Intelligence: A Regulatory Compliance Framework," *IEEE Access*, vol. 13, pp. 78340–78356, 2025. DOI: https://doi.org/10.1109/ACCESS.2025.3501122

[90] J. Park, S. Kim, and B. Lee, "FED-SPFD: A Federated Strategy for Payment Fraud Detection in Cross-Institution Banking," *Proc. IEEE Int. Conf. Big Data*, pp. 2912–2921, 2024. DOI: https://doi.org/10.1109/BigData62323.2024.10825443

[91] V. Buterin, P. Hitzig, and E. G. Weyl, "Decentralized Society: Finding Web3's Soul," *SSRN Working Paper No. 4105763*, May 2022. DOI: https://dx.doi.org/10.2139/ssrn.4105763

[92] MicroSave Consulting, "Bangladesh Microfinance Sector Outlook 2025: Toward Performance-Based Credit Identity," MSC Insight Note, December 2025. [Online]. Available: https://www.microsave.net/insights/bangladesh-mfi-2025

[93] Bank for International Settlements, "2025 BIS Survey on Central Bank Digital Currency and Crypto-Assets," BIS Papers No. 147, July 2025. [Online]. Available: https://www.bis.org/publ/bppdf/bispap147.pdf

[94] Morgan Stanley, "AI @ Morgan Stanley Assistant: GPT-4 Powered Knowledge Tool for Financial Advisors," Morgan Stanley Technology Disclosure, 2024. [Online]. Available: https://www.morganstanley.com/press-releases/key-milestone-in-firm-s-ai-journey

[95] McKinsey & Company, "The Economic Potential of Generative AI: The Next Productivity Frontier (Banking Sector Update)," McKinsey Global Institute, June 2024. [Online]. Available: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier

[96] A. Yampolskiy, I. Maddali, K. Khoury, et al., "LLM Vulnerabilities in Finance: A Red-Teaming Benchmark for Regulatory-Hallucination Resistance," *Proc. ACM Conf. AI, Ethics, and Society (AIES)*, 2025. DOI: https://doi.org/10.1145/3641421.3678412

[97] European Parliament and Council of the European Union, "Regulation (EU) 2023/1114 of 31 May 2023 on Markets in Crypto-Assets (MiCA)," *Official Journal of the European Union*, L 150/40, June 2023; fully applicable from 30 December 2024. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2023/1114/oj

[98] European Banking Authority, "Guidelines on Reserve Asset Composition, Custody, and Audit for Asset-Referenced and E-Money Tokens under MiCA," EBA/GL/2024/06, May 2024. [Online]. Available: https://www.eba.europa.eu/regulation-and-policy/markets-in-crypto-assets

[99] United States Congress, "Guiding and Establishing National Innovation for U.S. Stablecoins (GENIUS) Act," Pub. L. No. 119-27, July 18, 2025. [Online]. Available: https://www.congress.gov/bill/119th-congress/senate-bill/919

[100] Bank for International Settlements Innovation Hub, "Project mBridge: Connecting Economies through CBDC — Reaches Minimum Viable Product," BIS Innovation Hub Report, June 2024. [Online]. Available: https://www.bis.org/about/bisih/topics/cbdc/mcbdc_bridge.htm

[101] Bank for International Settlements and Central Banks of France, Japan, Korea, Mexico, Switzerland, the UK, and the US, "Project Agora: Tokenization of Cross-Border Payments using Unified Ledgers," BIS Innovation Hub Working Paper, April 2024. [Online]. Available: https://www.bis.org/about/bisih/topics/fmis/agora.htm

[102] Bangladesh Bank, "Cautionary Notice Regarding Transactions in Virtual / Crypto-Currencies," Bangladesh Bank Foreign Exchange Policy Department, Notice No. FE-1/2017, December 2017 (reaffirmed 2018, 2022). [Online]. Available: https://www.bb.org.bd/aboutus/regulationguideline/foreignexchange/notice_virtualcurrencies.pdf

[103] Bangladesh Bank, "FE Circular No. 06: Electronic Communication for Letters of Credit and Trade Finance Instruments," Foreign Exchange Policy Department, January 2025. [Online]. Available: https://www.bb.org.bd/mediaroom/circulars/fepd/jan172025fepd06.pdf

[104] Government of the People's Republic of Bangladesh, "The Payment and Settlement System Act, 2024 (Act No. XX of 2024)," Bangladesh Gazette, August 2024.

[105] Bangladesh Bank, "Financial Inclusion in Bangladesh: Progress and Gender-Disaggregated Outlook 2024–25," Special Publication SP2025-02, Financial Inclusion Department, June 2025. [Online]. Available: https://www.bb.org.bd/pub/special/SP2025-02.pdf

[106] BRAC, "BRAC Microfinance Annual Performance Report 2025: Women-Centric Lending Outcomes," BRAC Research and Evaluation Division, 2025. [Online]. Available: https://www.brac.net/program/microfinance/

[107] Ethereum Improvement Proposals, "EIP-4626: Tokenized Vault Standard," Ethereum Foundation, April 2022. [Online]. Available: https://eips.ethereum.org/EIPS/eip-4626; "EIP-7540: Asynchronous ERC-4626 Tokenized Vaults," Ethereum Foundation, 2023. [Online]. Available: https://eips.ethereum.org/EIPS/eip-7540; "EIP-3643: T-REX – Token for Regulated EXchanges," Ethereum Foundation, 2021. [Online]. Available: https://eips.ethereum.org/EIPS/eip-3643. See also: Spectral Finance, "On-Chain Credit Scoring for DeFi Lending," 2023. [Online]. Available: https://docs.spectral.finance; RociFi Labs, "Undercollateralized DeFi Lending via On-Chain Credit Score," 2023. [Online]. Available: https://docs.rocifi.com; Qwen Team (Alibaba Cloud), "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388.

[108] Behaviour-Centric Cybersecurity Center, York University, "DeFi Fraud Transactions Dataset (BCCC-DeFiFraudTrans-2025)," 2025. [Online]. Available: https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/defi-fraud-transactions-bccc-defifraudtrans-2025/

[109] Elliptic, "The Elliptic Data Set," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

[110] Y. Elmougy, S. Liu, and J. Liu, "Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2023. [Online]. Available: https://github.com/git-disl/EllipticPlusPlus

[111] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," *arXiv preprint arXiv:1908.02591*, KDD Workshop on Anomaly Detection in Finance, 2019. [Online]. Available: https://arxiv.org/abs/1908.02591

[112] AMD, "ROCm 7.0.2 Release Notes: Official Support for RDNA 4 / gfx1200 (RX 9060 XT)," AMD Developer Blog, October 2025. [Online]. Available: https://www.phoronix.com/news/AMD-ROCm-7.0.2-Released

[113] K. Heidt, M. Spörk, D. Markov, and A. Krüger, "Harness Engineering for Language Agents: The Harness Layer as Control, Agency, and Runtime," *Preprints.org*, DOI: https://doi.org/10.20944/preprints202603.1756, March 2026.

[114] M. Alizadeh, M. Gholampour, A. Imani, and J. Schulz, "Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution," *arXiv preprint arXiv:2506.01055*, University of Zurich / IPM, June 2026.

[115] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications and Agents, 2026 Edition," OWASP Foundation, 2026. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[116] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.